UNIVERSITÉ POLYTECHNIQUE DE GITEGA

FACULTÉ DES TECHNOLOGIES DE L'INFORMATION ET DE LA COMMUNICATION

Classe de BAC+3 (Génie Logiciel)

COURS DE

DEVELOPPEMENT WEB AVEC ASP.NET CORE

Support de cours et travaux pratiques

DISPENSÉ PAR

MA NIRAGIRA Ferdinand

Année Académique 2025 – 2026

TABLE DES MATIERES

PARTIE I — FONDATIONS 1
CHAPITRE 0 — CAHIER DES CHARGES DU PROJET AKARANGA BANK 1
0.1. Objectifs du chapitre 1
0.2. Présentation du projet 1
0.3. Les acteurs du système 2
0.4. Le réseau d'agences 2
0.5. Principes techniques du secteur bancaire 3
0.6. Modèle conceptuel de données (MCD) 4
0.7. Dictionnaire de données 6
0.8. Script de référence (T-SQL, SQL Server) 7
0.9. Jeu de données de test 9
0.10. Conventions de code adoptées dans le cours 10
0.11. Questions de compréhension 11
CHAPITRE 1 — ENVIRONNEMENT DE TRAVAIL ET PREMIER PROJET 12
1.1. Qu'est-ce qu' ASP.NET Core ? 12
1.2. Installer les outils 12
1.3. Créer le premier projet 13
1.4. Comprendre Program.cs 14
1.5. Le cycle d'une requête ASP.NET Core 14
1.6. Exécuter le projet 15
1.7. Auto-évaluation — questions courtes 15
1.8. Mini-TP 1 — Mettre en place votre environnement Akaranga Bank 16
CHAPITRE 2 — C# ET RAZOR POUR LES PREMIÈRES PAGES 17
2.1. Les bases du langage C# utiles pour Razor 17
2.2. Qu'est-ce que Razor ? 18
2.3. Créer un projet avec des Razor Pages 18
2.4. Le code-behind : PageModel 19
2.5. Afficher les données dans la vue 20
2.6. Structures de contrôle dans Razor 20
2.7. La mise en page partagée (_Layout.cshtml) 21
2.8. Formulaires HTML et liaison de données 22
2.9. Auto-évaluation — questions courtes 23
2.10. Mini-TP 2 — Pages d'agence et formulaire de contact 23
PARTIE II — ARCHITECTURE MVC ET DONNÉES 24
CHAPITRE 3 — ASP.NET CORE MVC 24
3.1. Qu'est-ce que MVC ? 24
3.2. Le cycle d'une requête MVC 24
3.3. Créer un projet MVC 25
3.4. Première compilation 26
3.5. Lancer l'application 26
3.6. Le routage par convention 26
3.7. Créer le modèle Agence 27
3.8. Créer le ViewModel AgenceListViewModel 27
3.9. Créer le contrôleur AgencesController 27
3.10. Créer la vue Index 28
3.11. Mettre à jour le menu 29

3.12. Auto-évaluation — questions courtes 29
3.13. Mini-TP 3 — Page des agences en MVC 30
CHAPITRE 4 —ENTITY FRAMEWORK CORE 31
4.1. Vérifier l'installation 31
4.2. Structure du modèle de données (rappel du chapitre 0) 31
4.3. Installer Entity Framework Core 32
4.4. Configurer la chaîne de connexion (avec secrets) 32
4.5. Créer le DbContext 34
4.6. Créer les 16 entités C# mappées au script 36
4.7. Enregistrer le DbContext dans Program.cs 44
4.8. Première requête LINQ sur la base 45
4.9. Tests 47
4.10. Exercices 48
4.11. Questions de compréhension 48
4.12. Mini-TP 4 — Première page connectée à la base 49
CHAPITRE 5 — CRUD AGENCES 50
5.1. Fichiers du chapitre 50
5.2. Les ViewModels 50
5.3. Le Service Agence 53
5.3. Le Contrôleur Agences 57
5.4. Les Vues 61
5.6. Tests manuels 67
5.7. Exercices 69
5.8. Questions de compréhension 70
5.9. Mini-TP 5 — CRUD complet des agences 70
CHAPITRE 6 — CRUD CLIENTS, KYC ET COMPTES 71
6.1. Les relations du modèle 71
6.2. Architecture cible 71
6.3. Le module Clients 72
6.4. Le module Vérifications KYC 95
6.5. Le module Comptes 102
6.6. Mise à jour du menu 121
6.7. Tests manuels 122
6.9. Exercices 124
6.10. Questions de compréhension 125
6.11. Mini-TP 6 — Module complet Clients + KYC + Comptes 125
PARTIE III — SECURITE ET CŒUR METIER BANCAIRE 126
CHAPITRE 7 — AUTHENTIFICATION FORTE (2FA, SESSIONS) 126
Objectifs du chapitre 126
7.1. Pourquoi une authentification forte en banque ? 126
7.2. Options de conception 127
7.3. Installation des packages 127
7.4. Créer ApplicationUser 128
7.5. Créer ApplicationRole 128
7.6. Modifier le DbContext 129
7.7. Configurer Identity dans Program.cs 141
7.8. Gestion des migrations 144
7.9. Politique MFA 145
7.10. Initialisation des rôles et de l'administrateur 145

7.11. Scaffolding des pages Identity (sans Register) 147
7.12. QR code pour la 2FA 147
7.13. Lier un utilisateur Identity à un Client 148
7.14. Back-office admin des utilisateurs 149
7.15. Protéger les opérations sensibles avec MFA 157
7.16. Menu et layout 157
7.17. Tests manuels 159
7.18. Mini-TP 7 — Authentification forte complète 161
CHAPITRE 8 — OPÉRATIONS BANCAIRES, PARTIE DOUBLE ET CONTRÔLES ANTI-BLANCHIMENT 162
Objectifs du chapitre 162
Partie A — Comptabilité en partie double 162
Partie B — Cadre réglementaire LBC/FT 163
Partie C — Modélisation et services 163
Partie D — Contrôleurs et vues 192
Partie E — Tests manuels 205
Exercices 206
Questions de compréhension 206
Mini-TP 8 — Cœur comptable et conformité 206
CHAPITRE 9 — VIREMENTS INTERBANCAIRES, SEUILS AML AVANCÉS ET DÉCLARATIONS À LA CNRF 207
Objectifs du chapitre 207
Partie A — Typologie des virements 207
Partie B — Modélisation étendue 208
Partie C — Services 212
Partie D — Contrôleurs et vues 232
Partie E — Tests manuels 240
Exercices 241
Questions de compréhension 242
Mini-TP 9 — Virements interbancaires et conformité avancée 242
CHAPITRE 10 — INTRODUCTION AUX API POUR APPLICATION MOBILE 243
Objectifs du chapitre 243
Partie A — Qu'est-ce qu'une API ? 243
Partie B — Sécurité des API 246
Partie C — Cas d'usage pour Akaranga Bank 249
CHAPITRE 11 — TESTS AUTOMATISÉS, PISTE D'AUDIT ET DÉPLOIEMENT 251
Objectifs du chapitre 251
Partie A — Pourquoi tester ? 251
Partie B — Tests unitaires avec xUnit et Moq 252
Partie C — Tests d'intégration 268
Partie D — Piste d'audit 271
Partie E — Déploiement 278
Partie F — CI/CD avec GitHub Actions 281
Partie G — Health checks 283
Partie H — Tests manuels 283
Exercices 284
Questions de compréhension 284
Mini-TP 11 — Tests, audit et déploiement 285
CHAPITRE 12 — PRÊTS, REMBOURSEMENTS ET CARTES BANCAIRES 286

Énoncé du travail à réaliser 286
1. Objectif du chapitre 286
2. Contexte 286
3. Livrables attendus 286
4. Exigences fonctionnelles 286
5. Exigences non-fonctionnelles 288
6. Tests à réaliser 289
CHAPITRE 13 — ACCESSIBILITÉ ET INTERNATIONALISATION 291
Énoncé du travail à réaliser 291
1. Objectif du chapitre 291
2. Contexte 291
3. Livrables attendus 291
4. Exigences fonctionnelles 291
5. Exigences non-fonctionnelles 292
6. Contraintes techniques 292
7. Tests à réaliser 293
Énoncé du travail à réaliser 294
1. Objectif du chapitre 294
2. Contexte 294
3. Livrables attendus 294
4. Exigences fonctionnelles 294
5. Exigences non-fonctionnelles 295
6. Contraintes techniques 295
7. Tests à réaliser 296
CHAPITRE 15 — GÉNÉRATION DE DOCUMENTS ET REPORTING 297
Énoncé du travail à réaliser 297
1. Objectif du chapitre 297
2. Contexte 297
3. Livrables attendus 297
4. Exigences fonctionnelles 297
5. Exigences non-fonctionnelles 299
6. Contraintes techniques 299
7. Tests à réaliser 299
CHAPITRE 16 — PROJET FINAL : SYNTHÈSE ET SOUTENANCE 300
Énoncé du projet intégrateur 300
1. Objectif du chapitre 300
2. Contexte du projet 300
3. Périmètre du projet final 300
4. Livrables attendus 301
5. Exigences fonctionnelles du projet final 301
6. Exigences non-fonctionnelles 303
7. Tests à réaliser 304
8. Documentation attendue 305
9. Bonus possibles 305
10. Conclusion du cours 305

PARTIE I — FONDATIONS

CHAPITRE 0 — CAHIER DES CHARGES DU PROJET AKARANGA BANK

Ce chapitre sert de **référence commune** à tout le cours. Il fixe le contexte métier, les acteurs, les principes techniques du secteur bancaire, le modèle de données (16 tables) et les conventions de code — afin que chaque chapitre s'appuie sur les mêmes fondations plutôt que d'improviser une variante à chaque fois.

0.1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- Comprendre le **contexte métier** d'Akaranga Bank.
- Identifier les **cinq acteurs** du système et leurs rôles.
- Situer le **réseau d'agences** (Bujumbura, Gitega, Ngozi, Rumonge).
- Comprendre les **principes techniques** propres au secteur bancaire.
- Lire le **modèle conceptuel de données** (16 tables).
- Utiliser le **script SQL de référence** comme source de vérité.
- Respecter les **conventions de code** adoptées dans tout le cours.

0.2. Présentation du projet

Akaranga Bank est une **banque commerciale fictive** dont le siège se trouve à **Bujumbura**, avec des agences à **Gitega**, **Ngozi** et **Rumonge**. Le nom fait référence à l'**akaranga**, un mot kirundi qui signifie **culture**, **intégrité** et **courage**. Il évoque la rigueur, l'éthique et le sérieux.

Comme toute banque opérant au Burundi, Akaranga Bank est soumise à la supervision de la **BRB** (Banque de la République du Burundi), la banque centrale et autorité de régulation du secteur bancaire burundais. La monnaie utilisée dans tout le projet est le **Franc Burundais (BIF)**.

Tout au long du cours, l'application est développée progressivement :

Chapitres	Contenu
1-2	Environnement (VS Code, SDK .NET), bases du langage C#, premières pages Razor
3-6	ASP.NET Core MVC, gestion des clients et des comptes (KYC), Entity Framework Core
7-8	Authentification forte (2FA, sessions), opérations et écritures en partie double, virements et contrôles anti-blanchiment
9-10	API Web, sécurité avancée et piste d'audit, tests automatisés, déploiement
11	Prêts et échéanciers, cartes, accessibilité et internationalisation
12-13	Gestion documentaire et notifications, génération de documents et reporting

0.3. Les acteurs du système

Cinq rôles sont utilisés de façon cohérente dans tous les exemples et exercices. Ce sont exactement les cinq valeurs autorisées par la contrainte `CK_Utilisateurs_Role` du script :

Rôle	Code	Permissions principales
Client	client	Consulte ses propres comptes, initie des virements depuis son espace personnel
Guichetier	guichetier	Effectue des dépôts/retraits en agence, consulte les comptes de son agence
Gestionnaire	gestionnaire	Valide les opérations importantes, gère les prêts, consulte les clients de son agence
Auditeur	auditeur	Consulte les journaux, les audits, les alertes AML — lecture seule
Administrateur	admin	Gère les utilisateurs, les agences, les paramètres — accès complet

0.3.1. Matrice des permissions

Action	client	guichetier	gestionnaire	auditeur	admin
Consulter ses comptes	✓	—	—	—	✓
Consulter les comptes de son agence	—	✓	✓	✓	✓
Effectuer un dépôt	—	✓	✓	—	✓
Effectuer un retrait	—	✓	✓	—	✓
Initier un virement	✓	✓	✓	—	✓
Valider un prêt	—	—	✓	—	✓
Consulter les alertes AML	—	—	✓	✓	✓
Consulter le journal d'audit	—	—	—	✓	✓
Gérer les utilisateurs	—	—	—	—	✓
Modifier les paramètres	—	—	—	—	✓

0.4. Le réseau d'agences

Chaque agence est identifiée par un **code unique** au format `AKR-<VILLE>` (colonne `Agences.Code`) :

Code agence	Ville	Statut
AKR-BJM	Bujumbura	Siège du réseau
AKR-GIT	Gitega	Agence
AKR-NGO	Ngozi	Agence
AKR-RUM	Rumonge	Agence

0.4.1. Règle métier

- Un client, un compte, un utilisateur appartiennent **obligatoirement** à une agence.
- Le siège (EstSiege = true) ne peut pas être supprimé.
- Une agence ne peut être supprimée que si elle n'a **aucun client**, **aucun compte**, **aucun utilisateur** rattaché.

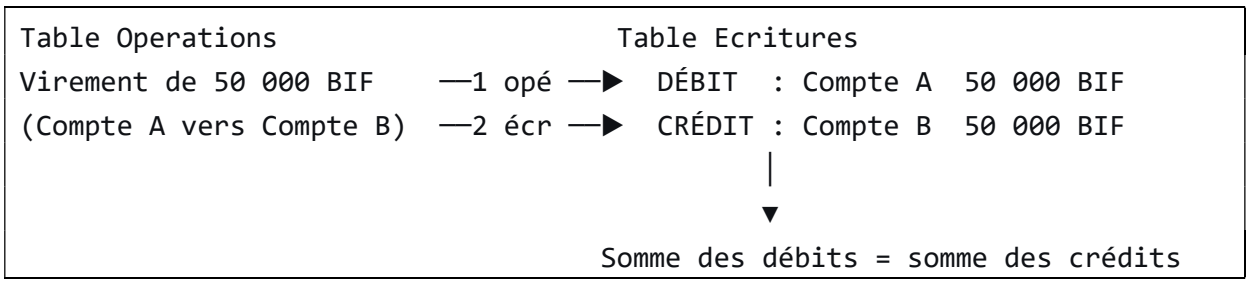
0.5. Principes techniques du secteur bancaire

Une application bancaire ne se résume pas à un CRUD habillé de termes financiers : le secteur impose des **exigences techniques précises**. Chacune est portée par des **tables précises** du modèle, et non simplement mentionnée en théorie.

0.5.1. La comptabilité en partie double

Aucun montant ne « disparaît » ni n'« apparaît » jamais seul. Le modèle distingue **deux niveaux** :

- la table Operations enregistre l'**événement métier** (un dépôt, un virement, un remboursement de prêt) ;
- la table Ecritures enregistre ses **lignes comptables**.
- Toute opération produit **au moins deux écritures**, l'une au **débit** et l'autre au **crédit**, dont les totaux doivent être **égaux**.



Le script fournit même la **requête de contrôle** : elle ne doit retourner **aucune ligne** si la comptabilité est équilibrée.

```sql
SELECT OperationId,
    SUM(CASE WHEN Sens = 'Debit'  THEN Montant ELSE 0 END) AS TotalDebit,
    SUM(CASE WHEN Sens = 'Credit' THEN Montant ELSE 0 END) AS TotalCredit
FROM Ecritures
GROUP BY OperationId
HAVING SUM(CASE WHEN Sens = 'Debit' THEN Montant ELSE -Montant END) <> 0;
-- Aucune ligne retournée = comptabilité équilibrée
```

Ce principe est mis en œuvre au **chapitre 8**.

0.5.2. L'atomicité des opérations (ACID)

Un virement qui débiterait un compte sans jamais créditer l'autre (coupure réseau, erreur applicative) serait **catastrophique**. Toute opération monétaire est donc **atomique** : l'opération, ses écritures et la mise à jour des soldes réussissent ensemble, ou **aucune n'est appliquée**.

Entity Framework Core fournit les **transactions de base de données** nécessaires (BeginTransactionAsync, CommitAsync, RollbackAsync) — voir **chapitre 8**.

0.5.3. Une piste d'audit détaillée

La table AuditLog conserve, pour chaque action sensible :

- **qui** (UtilisateurId) ;
- **où** (AgenceId, AdresseIP, UserAgent) ;
- **quoi** (Action, TableConcernee, EnregistrementId) ;
- surtout l'**état avant et après** (ValeurAvant, ValeurApres).

En cas de litige ou de contrôle, l'historique doit rester **intact** et **consultable** par l'auditeur : le **chapitre 10** montre comment le rendre réellement **immuable**.

0.5.4. KYC et vigilance anti-blanchiment (AML)

KYC (*Know Your Customer*) impose de **vérifier l'identité d'un client** : la table VerificationsKYC suit chaque document (statut EnAttente, Valide, Rejete ou Expire, vérificateur, date).

AML (*Anti-Money Laundering*) impose une **vigilance sur les mouvements inhabituels** : le paramètre SEUIL_AML de la table Parametres fixe le seuil (**5 000 000 BIF**) au-delà duquel un virement est marqué ControleAML et peut générer une ligne dans AlertesAML (voir **chapitre 8**).

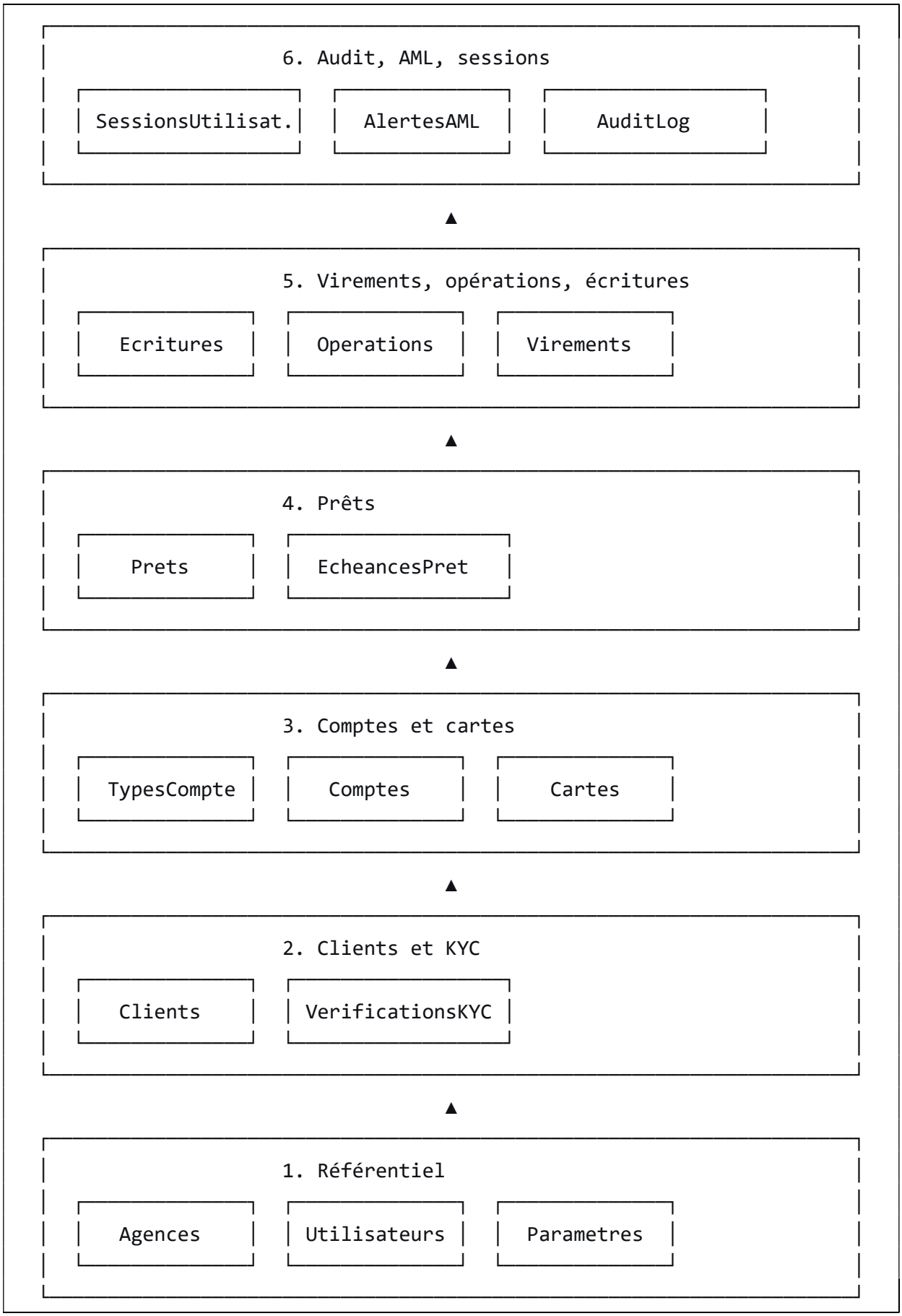
0.5.5. Une sécurité renforcée

Au-delà d'une application web classique :

- **authentification à deux facteurs** (colonnes DeuxFA_Active et DeuxFA_Secret de la table Utilisateurs) ;
- **sessions tracées et limitées** dans le temps (SessionsUtilisateur, paramètres DUREE_SESSION_MIN et MAX_TENTATIVES_CONNEXION) ;
- **protection des données sensibles** (chapitres **7** et **10**).

0.6. Modèle conceptuel de données (MCD)

Le modèle compte **16 tables**, organisées en **six blocs cohérents**. Chaque flèche du schéma est une clé étrangère (FOREIGN KEY) du script :


6. Audit, AML, sessions
SessionsUtilisat.
AlertesAML
AuditLog
5. Virements, opérations, écritures
Ecritures
Operations
Virements
4. Prêts
Prets
EcheancesPret
3. Comptes et cartes
TypesCompte
Comptes
Cartes
2. Clients et KYC
Clients
VerificationsKYC
1. Référentiel
Agences
Utilisateurs
Parametres

Lecture du schéma : Agences est la table la plus « en bas » (rien n'en dépend en amont) ; Ecritures, AlertesAML et AuditLog sont au sommet, car elles s'appuient sur presque tout le reste. C'est aussi **l'ordre de création des tables** dans le script.

0.7. Dictionnaire de données

Pour chaque bloc, les colonnes les plus significatives et les **règles que la base elle-même impose** (contraintes CHECK, UNIQUE, FOREIGN KEY) :

Bloc 1 — Référentiel

Table	Colonnes clés	Règles portées par la base
Agences	Code, Nom, Ville, EstSiege	Code unique
Utilisateurs	Email, MotDePasseHash, Role, AgenceId, DeuxFA_Active, DeuxFA_Secret, DerniereConnexion, EstActif	Email unique ; Role limité aux 5 rôles (0.3)
Parametres	Cle, Valeur, Description, AgenceId	Couple (Cle, AgenceId) unique ; AgenceId NULL = paramètre global

Bloc 2 — Clients et KYC

Table	Colonnes clés	Règles portées par la base
Clients	NumeroClient, Nom, Prenom, DateNaissance, NumeroCNI, Telephone, Email, Adresse, AgenceId, Statut	NumeroClient et NumeroCNI uniques ; Statut ∈ {Actif, Suspendu, Cloture}
VerificationsKYC	ClientId, TypeDocument, NumeroDocument, CheminDocument, Statut, VerifieParId, DateVerification	Statut ∈ {EnAttente, Valide, Rejete, Expire}

Bloc 3 — Comptes et cartes

Table	Colonnes clés	Règles portées par la base
TypesCompte	Code, Libelle, TauxInteret, SoldeMinimum, AutoriseDecouvert	Taux entre 0 et 100 ; SoldeMinimum ≥ 0
Comptes	NumeroCompte, ClientId, TypeCompteId, AgenceId, Solde, Devise, Statut, DateOuverture, DateFermeture	NumeroCompte unique ; Devise BIF par défaut ; Statut ∈ {Actif, Bloque, Cloture}
Cartes	NumeroCarte, CompteId, ClientId, TypeCarte, DateExpiration, Statut	TypeCarte ∈ {Visa, MasterCard, Locale} ; Statut ∈ {Active, Bloquee, Expiree}

Bloc 4 — Prêts

Table	Colonnes clés	Règles portées par la base
Prets	Reference, ClientId, CompteDebitId, ValideParId, MontantPrincipal, TauxInteret, DureeMois, Mensualite, Statut	Montant, durée et mensualité > 0 ; Statut ∈ {EnAttente, Valide, Rejete, EnCours, Solde, Impaye}
EcheancesPret	PretId, NumeroEcheance, Capital, Interet, MontantTotal, DateEcheance, DatePaiement, Statut	Couple (PretId, NumeroEcheance) unique ; Statut ∈ {EnAttente, Payee, EnRetard, Annulee}

Bloc 5 — Virements, opérations et écritures

Table	Colonnes clés	Règles portées par la base
Virements	Reference, CompteSourceId, CompteDestinationId, UtilisateurId, Montant, Devise, Statut, Motif, ControleAML	Montant > 0 ; compte source ≠ compte destination ; Statut ∈ {EnAttente, Valide, Rejete, Annule}
Operations	Reference, TypeOperation, CompteId, UtilisateurId, AgenceId, VirementId, PretId, Montant, Devise, Statut	Montant > 0 ; TypeOperation ∈ {Depot, Retrait, Virement, DeblocagePret, RemboursementPret, Frais, Interet}
Ecritures	OperationId, CompteId, Sens, Montant, Devise, DateEcriture, Libelle	Sens ∈ {Debit, Credit} ; Montant > 0

Bloc 6 — Audit, AML et sessions

Table	Colonnes clés	Règles portées par la base
AuditLog	DateEvenement, UtilisateurId, AgenceId, Action, TableConcernee, EnregistrementId, ValeurAvant, ValeurApres, AdresseIP, UserAgent	Utilisateur et agence facultatifs (événements système)
AlertesAML	ClientId, OperationId, TypeAlerte, Niveau, Description, Statut, TraiteParId	Niveau ∈ {Faible, Moyen, Eleve, Critique} ; Statut ∈ {Ouverte, EnCours, Traitee, Classee}
SessionsUtilisateur	UtilisateurId, Token, AdresseIP, UserAgent, DeuxFA_Valide	Token unique

0.8. Script de référence (T-SQL, SQL Server)

Le script AkarangaBank-Script-DB.sql est la **source de vérité** du modèle de données. Il :

- supprime puis recrée la base AkarangaBank ;
- crée les **16 tables** et leurs index ;

- insère le **jeu de données de test** (0.9) ;
- affiche des **contrôles de vérification**.

0.8.1. Installation de SQL Server

1. Téléchargez **SQL Server 2022 Developer Edition** (gratuit) depuis le site officiel de Microsoft : microsoft.com/sql-server/sql-server-downloads.
2. Lancez l'installateur, choisissez le type **« Basic »**.
3. Acceptez les conditions, choisissez le dossier d'installation.
4. À la fin, notez le **nom de l'instance** (par défaut MSSQLSERVER).

0.8.2. Activer TCP/IP (une seule fois)

Par défaut, SQL Server Developer désactive le protocole TCP/IP, ce qui empêche la connexion depuis sqlcmd ou une application .NET.

1. Ouvrez le **Gestionnaire de configuration SQL Server** (menu Démarrer → rechercher « SQL Server Configuration Manager »).
2. Développez **« Configuration réseau SQL Server »** → **« Protocoles pour MSSQLSERVER »**.
3. Clic droit sur **TCP/IP** → **Activer**.
4. Double-cliquez sur **TCP/IP** → onglet **« Adresses IP »** → descendez jusqu'à **« IPAll »** → vérifiez que **« Port TCP »** = **1433**.
5. Validez.
6. Dans **« Services SQL Server »**, clic droit sur l'instance → **Redémarrer**.

0.8.3. Se connecter à SQL Server

Ouvrez une **invite de commandes** et tapez :

```
sqlcmd -S localhost -E
```

- -S localhost : nom du serveur (instance par défaut).
- -E : authentification Windows (votre compte utilisateur).

Si la connexion réussit, vous voyez l'invite 1>. Tapez :

sql

```
SELECT @@VERSION;
GO
```

Le numéro de version de SQL Server s'affiche. Pour quitter : EXIT.

0.8.4. Exécuter le script AkarangaBank-Script-DB.sql

Placez-vous dans le dossier contenant le script, puis exécutez :
cmd

```
sqlcmd -S localhost -E -f 65001 -i AkarangaBank-Script-DB.sql
```

- -f 65001 : encodage UTF-8 (indispensable pour les accents).
- -i : fichier script à exécuter.

Le script :

1. supprime puis recrée la base AkarangaBank ;
2. crée les **16 tables** et leurs index ;
3. insère le **jeu de données de test** ;
4. affiche les contrôles de vérification.

0.8.5. Vérifier l'installation

cmd

```
sqlcmd -S localhost -E -d AkarangaBank -Q "SELECT name FROM
sys.tables ORDER BY name"
```

Vous devez voir les **16 tables**

0.8.6. Vérifier les données

- **Se connecter directement à la base**
  cmd

```
sqlcmd -S localhost -E -d AkarangaBank
```

L'option `-d AkarangaBank` indique à `sqlcmd` d'ouvrir directement cette base. Vous entrez dans l'invite 1>, puis :

sql

```
SELECT * FROM Agences;

GO
```

0.8.7. Chaîne de connexion pour le projet .NET

Une fois SQL Server installé et le script exécuté, la chaîne de connexion à utiliser dans `appsettings.json` ou via User Secrets est :

```
Server=localhost;Database=AkarangaBank;Trusted_Connection=True
;TrustServerCertificate=True;
```

- Trusted_Connection=True : authentification Windows (pas de mot de passe).
- TrustServerCertificate=True : accepte le certificat auto-signé en développement.

0.9. Jeu de données de test

Le script insère un jeu de données réaliste pour permettre les tests dès le premier chapitre :

Table	Lignes	Contenu
Agences	4	AKR-BJM (siège), AKR-GIT, AKR-NGO, AKR-RUM
TypesCompte	3	COURANT (0 %), EPARGNE (3,50 %, solde minimum 10 000), CREDIT (12 %, découvert autorisé)

Parametres	5	SEUIL_AML = 5 000 000, taux d'épargne, durée de session (30 min), tentatives de connexion (3), devise (BIF)
Utilisateurs	5	Un admin, un gestionnaire et un auditeur à Bujumbura ; un guichetier à Gitega et un à Ngozi (mots de passe fictifs HASH_...)
Clients	5	CLI-000001 à CLI-000005, un ou deux par agence
Comptes	6	De 150 000 BIF (compte courant) à 25 000 000 BIF (compte crédit, Ngozi)
Cartes / VerificationsKYC	3 / 5	Trois cartes ; cinq vérifications de CNI, dont une seule EnAttente (client 5)
Virements / Operations / Ecritures	1 / 1 / 2	Un virement de 50 000 BIF entre les deux comptes du client 1, avec son débit et son crédit
Prets, EcheancesPret, AuditLog, AlertesAML, SessionsUtilisateur	0	Vides : remplies par l'application au fil des chapitres

0.9.1. Clients de test

Numéro	Nom	Prénom	Agence	Statut
CLI-000001	Nshimirimana	Emmanuel	AKR-BJM	Actif
CLI-000002	Nduwimana	Marie	AKR-BJM	Actif
CLI-000003	Hategekimana	Joseph	AKR-GIT	Actif
CLI-000004	Nkeshimana	Béatrice	AKR-NGO	Actif
CLI-000005	Irakoze	Patrick	AKR-RUM	Actif

0.10. Conventions de code adoptées dans le cours

Pour que le code reste **lisible et cohérent** sur l'ensemble du projet, les conventions suivantes sont appliquées **partout** :

0.10.1. Nommage C#

- **Classes, méthodes, propriétés publiques** : PascalCase (ex. Compte, CalculerInteret(), NumeroCompte).
- **Variables locales et paramètres** : camelCase (ex. montantVirement, compteSource).
- **Champs privés** : _camelCase (ex. private readonly IClientService _clientService;).
- **Interfaces** : préfixe I (ex. IClientService).

0.10.2. Nommage base de données

- **Tables et colonnes** : PascalCase, tables au **pluriel**, exactement comme dans le script.

- Les propriétés C# portent **le même nom** que les colonnes (ex. `Client.NumeroClient` ↔ `Clients.NumeroClient`).

0.10.3. Colonnes de statut

- Une **enum C#** par colonne, aux membres **identiques** aux valeurs autorisées (ex. `StatutClient.Cloture`).

0.10.4. Montants

- Toujours `decimal` en C# ;
- `DECIMAL(18,2)` en base ;
- **jamais** `double` ni `float` ;
- **devise affichée** dans tous les exemples : **Franc Burundais (BIF)**.

0.11. Questions de compréhension

1. Quels sont les **cinq rôles** du système ? Quelle est la différence entre `gestionnaire` et `auditeur` ?
2. Quelle est la **monnaie** utilisée dans tout le projet ?
3. Quels sont les **codes d'agence** ? Lequel correspond au siège ?
4. Qu'est-ce que la **partie double** ? Pourquoi est-elle essentielle ?
5. Qu'est-ce que l'**atomicité** ? Pourquoi est-elle critique ?
6. À quoi sert la table `AuditLog` ? Que contient-elle ?
7. Qu'est-ce que le **KYC** ? Et l'**AML** ?
8. Quel est le **seuil AML** par défaut ?
9. Pourquoi les montants sont-ils en `decimal` et jamais en `float` ?
10. Combien de tables compte le modèle ? Citez les 6 blocs.
11. Pourquoi le script SQL est-il la « source de vérité » ?

CHAPITRE 1 — ENVIRONNEMENT DE TRAVAIL ET PREMIER PROJET

Objectifs du chapitre :

- Comprendre ce qu'est .NET, C# et ASP.NET Core.
- Installer le SDK .NET 10 et VS Code.
- Créer, exécuter et modifier un premier projet ASP.NET Core.
- Comprendre le cycle d'une requête HTTP dans ASP.NET Core.
- Afficher une première page Akaranga Bank dans le navigateur.

1.1. Qu'est-ce qu'ASP.NET Core ?

.NET est une plateforme de développement créée par Microsoft, aujourd'hui open source et multiplateforme (Windows, macOS, Linux). ASP.NET Core est le framework web bâti sur .NET, utilisé pour créer des sites, des API et des applications d'entreprise — y compris, très largement, des systèmes bancaires réels.

Le langage utilisé tout au long de ce cours est **C#** (prononcé « C sharp »), un langage orienté objet, fortement typé, conçu par Microsoft.

Terme	Définition
.NET	La plateforme d'exécution et les bibliothèques de base
C#	Le langage de programmation utilisé
ASP.NET Core	Le framework web construit sur .NET
SDK .NET	L'ensemble d'outils (compilateur, CLI) nécessaire pour développer

1.2. Installer les outils

1.2.1. Le SDK .NET

Le SDK (Software Development Kit) contient tout ce qu'il faut pour compiler et exécuter du code .NET. Il se télécharge sur https://dotnet.microsoft.com/ — ce cours utilise la version **10.0**, la dernière LTS (Long Term Support) au moment de sa rédaction, prise en charge jusqu'en novembre 2028.

Vérification de l'installation, dans un terminal :

```bash
dotnet --version
# Doit afficher quelque chose comme : 10.0.100
```

1.2.2. Visual Studio Code

VS Code est un éditeur de code léger et multiplateforme, **différent de Visual Studio** (l'environnement complet, réservé à Windows). Pour développer en C#, on installe deux extensions indispensables, depuis l'onglet Extensions de VS Code :

- **C# Dev Kit** (édité par Microsoft) : coloration syntaxique, autocomplétion, débogueur.
- **NuGet Gallery** (facultatif mais utile) : parcourir les bibliothèques externes directement depuis VS Code.

1.2.3. Le certificat HTTPS de développement

Au premier lancement, .NET peut proposer d'installer un certificat de développement auto-signé pour activer HTTPS en local :

```bash
dotnet dev-certs https --trust
```

Ce n'est pas un détail secondaire, HTTPS est absolument non négociable pour une application bancaire, y compris en développement.

1.3. Créer le premier projet

La CLI (Command Line Interface) `dotnet` permet de créer, compiler et exécuter un projet sans jamais quitter le terminal :

```bash
mkdir AkarangaBank
cd AkarangaBank
dotnet new web -o AkarangaBank.Web
cd AkarangaBank.Web
```

1.3.1. Comprendre la commande `dotnet new web -o AkarangaBank.Web`

Élément	Signification
`dotnet`	L'outil en ligne de commande .NET
`new`	Sous-commande : « créer un nouveau projet »
`web`	Le **template** : projet ASP.NET Core minimal
`-o`	Abréviation de `--output` : dossier de sortie
`AkarangaBank.Web`	Nom du dossier (et du projet) à créer

Pourquoi `-o` et pas `-n` ?

- `-n` (ou `--name`) sert à **nommer le projet**.
- `-o` (ou `--output`) sert à **choisir le dossier de sortie**.

Ici, les deux coïncident, donc on pourrait écrire `dotnet new web -n AkarangaBank.Web` — le résultat serait identique. Utiliser `-o` est plus explicite pour un débutant : « le projet sera créé dans ce dossier ».

1.3.2. Les autres templates disponibles

Le SDK .NET propose plusieurs templates :

Template	**Usage**
`web`	Projet ASP.NET Core minimal (utilisé ici)
`mvc`	Projet ASP.NET Core MVC complet
`webapp`	Projet Razor Pages
`webapi`	Projet Web API
`blazor`	Projet Blazor (interactif)

1.3.3. Structure du projet généré

```
AkarangaBank.Web/
├── Program.cs                    <- Point d'entrée de l'application
├── appsettings.json              <- Configuration (chaînes de connexion, etc.)
├── appsettings.Development.json
├── wwwroot/                      <- Fichiers statiques (CSS, JS, images)
├── Properties/
│   └── launchSettings.json       <- Profils de lancement (port, environnement)
└── AkarangaBank.Web.csproj       <- Fichier de projet (dépendances, version .NET)
```

1.4. Comprendre Program.cs

Depuis .NET 6, ASP.NET Core utilise un modèle « minimal » : plus besoin de classes Startup séparées, tout tient dans Program.cs :

Fichier : Program.cs

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/", () => "Bienvenue sur Akaranga Bank");

app.Run();
```

Ligne	Rôle
`WebApplication.CreateBuilder(args)`	Prépare l'application : configuration, journalisation, services
`builder.Build()`	Construit l'application à partir de la configuration précédente
`app.MapGet("/", ...)`	Définit ce qui doit se passer pour une requête GET sur /
`app.Run()`	Démarre le serveur et écoute les requêtes entrantes

1.5. Le cycle d'une requête ASP.NET Core

Contrairement à PHP où chaque page est un fichier interprété à chaque requête, **ASP.NET** Core démarre un **vrai serveur web** (Kestrel) qui reste en écoute et traite chaque requête à travers un pipeline configurable de **middlewares** :

text

Navigateur → Kestrel → Pipeline de middlewares → Votre code → Réponse HTTP

Kestrel est le serveur web intégré directement à l'application .NET. Il reste en écoute en permanence ; Apache, dans un projet PHP, est un serveur externe séparé qui invoque PHP à chaque requête. C'est une différence fondamentale de modèle d'exécution.

1.6. Exécuter le projet

Dans le terminal, à la racine du projet :

bash

```
dotnet run
```

VS Code affiche la sortie dans son terminal intégré, avec l'adresse d'écoute :

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
```

On ouvre ensuite l'adresse indiquée (https://localhost:5001 par défaut) dans le navigateur. La page affiche :

> **Bienvenue sur Akaranga Bank**
> Réseau bancaire : Bujumbura (siège), Gitega, Ngozi, Rumonge.

1.6.1. Le rechargement à chaud (hot reload)

Pour éviter de redémarrer le serveur à chaque modification, on utilise :

bash

```
dotnet watch run
```

Les modifications du code C# ou des vues sont alors automatiquement recompilées et rechargées dans le navigateur.

1.7. Auto-évaluation — questions courtes

1. Quelle est la différence entre .NET, C# et ASP.NET Core ?
2. À quoi sert la commande `dotnet new web -o AkarangaBank.Web` ?
3. Que fait exactement `app.MapGet("/", ...)` dans `Program.cs` ?
4. Qu'est-ce que Kestrel, et en quoi son rôle diffère-t-il de celui d'Apache dans un projet PHP ?
5. Quel est l'intérêt de `dotnet watch run` par rapport à `dotnet run` ?

Corrigé indicatif :

1. .NET est la plateforme d'exécution, C# le langage de programmation utilisé, ASP.NET Core le framework web construit sur .NET.
2. Elle crée un nouveau projet ASP.NET Core minimal nommé `AkarangaBank.Web`, dans un dossier du même nom.
3. Elle définit la réponse à renvoyer lorsqu'une requête GET arrive sur l'URL racine (`/`).
4. Kestrel est le serveur web intégré directement à l'application .NET, qui reste en écoute en permanence ; Apache est un serveur externe séparé qui invoque PHP à chaque requête.

5. dotnet watch run surveille les fichiers du projet et recompile/recharge automatiquement l'application à chaque modification, sans redémarrage manuel.

1.8. Mini-TP 1 — Mettre en place votre environnement Akaranga Bank

Objectif : installer votre environnement et faire tourner une première page personnalisée.

1. Installez le SDK .NET 10 et VS Code avec l'extension C# Dev Kit.
2. Créez un projet AkarangaBank.Web avec dotnet new web.
3. Modifiez Program.cs pour que la page d'accueil affiche le nom de **votre agence assignée** (Bujumbura, Gitega, Ngozi ou Rumonge) et son code (AKR-...).
4. Lancez le projet avec dotnet watch run et vérifiez le résultat dans le navigateur.
5. Modifiez le texte affiché sans arrêter le serveur, et constatez que le navigateur se met à jour automatiquement.

Rendu attendu : une capture d'écran de VS Code (terminal visible) et une capture du résultat dans le navigateur, avec le nom de votre agence visible.

CHAPITRE 2 — C# ET RAZOR POUR LES PREMIÈRES PAGES

Objectifs

- Écrire du C# de base (types, classes, propriétés, méthodes).
- Utiliser les structures de contrôle (if, foreach).
- Comprendre le moteur Razor et le symbole @.
- Créer une Razor Page avec son code-behind (PageModel).
- Utiliser un layout partagé (_Layout.cshtml).
- Lier un formulaire à des propriétés C# avec [BindProperty].
- Afficher la liste des agences d'Akaranga Bank.

2.1. Les bases du langage C# utiles pour Razor

2.1.1. Types de base

csharp

```csharp
string nomBanque = "Akaranga Bank";

int nombreAgences = 4;

decimal seuilAML = 5_000_000m;   // <- jamais float ni double pour l'argent

bool estSiege = true;

DateTime dateOuverture = DateTime.Now;
```

2.1.2. Classes et propriétés

csharp

```csharp
public class Agence
{
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Nom { get; set; } = string.Empty;
    public string Ville { get; set; } = string.Empty;
    public bool EstSiege { get; set; }
}
```

- public : accessible depuis l'extérieur de la classe.
- int, string, bool : les types.
- { get; set; } : propriété auto-implémentée (lecture + écriture).
- = string.Empty : valeur par défaut pour éviter les null.

2.1.3. Structures de contrôle

csharp

```csharp
if (agence.EstSiege) {
    // ...
}
else {
    // ...
}
```

```csharp
foreach (var agence in agences) {
    // ...
}
```

Ces structures seront utilisées **directement dans Razor**, précédées de `@`.

2.2. Qu'est-ce que Razor ?

Dans un fichier `.cshtml`, tout est interprété comme du HTML par défaut. Le symbole `@` signale au moteur Razor qu'une **expression ou un bloc C#** commence :

cshtml

```cshtml
<h1>Bienvenue chez @nomBanque</h1>
<p>Nous sommes le @DateTime.Now.ToString("dd/MM/yyyy").</p>
```

Pas besoin de fermer le « mode C# » explicitement pour une simple expression : Razor détecte automatiquement où elle s'arrête (à la fin du mot, ou à la première balise HTML rencontrée).

2.3. Créer un projet avec des Razor Pages

Le modèle **Razor Pages** organise l'application autour de pages, chacune associée à son propre code C# (le « code-behind ») — une approche plus simple que le MVC complet du chapitre 3, idéale pour des pages orientées contenu.

Terminal :

```
dotnet new webapp -o AkarangaBank.Web
```

2.3.1. Structure générée

```
AkarangaBank.Web/
├── Pages/
│   ├── Index.cshtml              <- La vue (HTML + Razor)
│   ├── Index.cshtml.cs           <- Le code-behind (C#)
│   ├── Shared/
│   │   ├── _Layout.cshtml        <- Mise en page commune
│   │   └── _ViewImports.cshtml
│   └── _ViewStart.cshtml
├── wwwroot/
└── Program.cs
```

Chaque page .cshtml est automatiquement associée à l'URL correspondant à son chemin dans le dossier Pages/ : Pages/Agences.cshtml répond à /Agences, sans configuration de route à écrire.

2.4. Le code-behind : PageModel

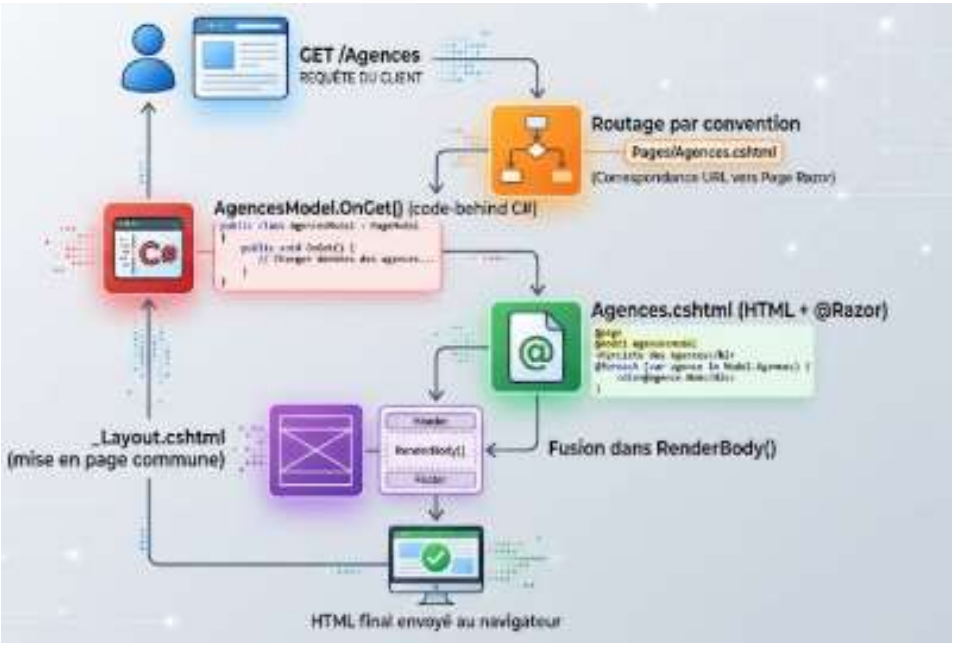
Le fichier .cshtml.cs contient la classe qui **prépare les données** avant l'affichage. C'est l'équivalent du contrôleur, mais réservé à une seule page.

Fichier à créer : Pages/Agences.cshtml.cs

```
using Microsoft.AspNetCore.Mvc.RazorPages;
namespace AkarangaBank.Web.Pages;
public class AgencesModel : PageModel {
    public List<string> NomsAgences { get; set; } = new();
    public void OnGet() {
        // Pour l'instant, des données en dur ; la base de données
        // arrive au chapitre 6.
        NomsAgences = new List<string> {"Bujumbura (Siège)",
                      "Gitega","Ngozi","Rumonge"};
    }
}
```

OnGet() s'exécute automatiquement pour toute requête **GET** vers la page. Il existe de même OnPost() pour les soumissions de formulaire (section 2.8).

2.4.1. Le cycle d'une requête Razor Pages



2.5. Afficher les données dans la vue

Fichier à créer : Pages/Agences.cshtml

```
@page
@model AkarangaBank.Web.Pages.AgencesModel
<h1>Nos agences</h1>
<ul>
   @foreach (var nom in Model.NomsAgences) {
      <li>@nom</li>
   }
</ul>
```

Ligne	Rôle
`@page`	Indique que ce fichier est une Razor Page routable (obligatoire en première ligne)
`@model ...`	Relie la vue à sa classe PageModel, rendant Model accessible
`Model.NomsAgences`	Accède à la propriété publique remplie dans OnGet()

2.6. Structures de contrôle dans Razor

Les mêmes structures C# du chapitre précédent s'utilisent directement dans le HTML, précédées de `@` :

cshtml

```
@if (Model.NomsAgences.Count == 0) {
    <p>Aucune agence enregistrée.</p>
}
else{
    <table>
        <thead>
            <tr><th>Agence</th><th>Statut</th></tr>
        </thead>
        <tbody>
        @foreach (var nom in Model.NomsAgences){
            <tr>
                <td>@nom</td>
                <td>
                    @if (nom.Contains("Siège")){
                        <strong>Siège</strong>
                    }
                    else{ span>Agence</span> }
                </td>
            </tr>
        }
        </tbody>
```

```
    </table>
}
```

2.7. La mise en page partagée (_Layout.cshtml)

Plutôt que de répéter l'en-tête et le pied de page sur chaque vue, Razor centralise la structure commune dans un **layout unique**.

Fichier à créer : Pages/Shared/_Layout.cshtml

cshtml

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="utf-8" />
    <title>@ViewData["Title"] - Akaranga Bank</title>
    <link rel="stylesheet" href="/css/site.css" />
</head>
<body>
    <header>
        <h1>Akaranga Bank</h1>
        <nav>
            <a href="/">Accueil</a>
            <a href="/Agences">Agences</a>
        </nav>
    </header>
    <main>
        @RenderBody()
    </main>
    <footer>
        <p>&copy; @DateTime.Now.Year Akaranga Bank</p>
    </footer>
</body>
</html>
```

@RenderBody() est remplacé, à l'exécution, par le contenu spécifique de chaque page (Agences.cshtml, Index.cshtml…). Le lien entre une page et son layout se fait dans _ViewStart.cshtml :

Fichier à créer : Pages/_ViewStart.cshtml

```
@{
    Layout = "_Layout"; }
```

2.8. Formulaires HTML et liaison de données

Razor fournit des **Tag Helpers** — des attributs HTML préfixés `asp-` — qui relient directement un formulaire aux propriétés C# du `PageModel`, sans manipuler `$_POST` à la main.

Fichier à créer : `Pages/Contact.cshtml.cs`

```
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace AkarangaBank.Web.Pages;

public class ContactModel : PageModel{
    [BindProperty]
    public string NomClient { get; set; } = string.Empty;

    [BindProperty]
    public string Message { get; set; } = string.Empty;

    public string? Confirmation { get; set; }

    public void OnPost(){
        Confirmation = $"Merci {NomClient}, votre message a bien été reçu.";
    }
}
```

Fichier à créer : `Pages/Contact.cshtml`

```
@page
@model AkarangaBank.Web.Pages.ContactModel

<h1>Contact</h1>

<form method="post">
    <label asp-for="NomClient">Votre nom :</label>
    <input asp-for="NomClient" />

    <label asp-for="Message">Message :</label>
    <textarea asp-for="Message"></textarea>

    <button type="submit">Envoyer</button>
</form>

@if (Model.Confirmation != null) {
    <p>@Model.Confirmation</p>
}
```

`[BindProperty]` indique à ASP.NET Core de remplir automatiquement cette propriété à partir des données soumises par le formulaire, à condition que son `name` corresponde (`asp-for` s'en charge).

2.9. Auto-évaluation — questions courtes

1. À quoi sert le symbole @ dans un fichier .cshtml ?
2. Quelle est la différence de responsabilité entre Agences.cshtml et Agences.cshtml.cs ?
3. Que fait @RenderBody() dans _Layout.cshtml ?
4. Quelle est la différence entre OnGet() et OnPost() dans une classe PageModel ?
5. À quoi sert l'attribut [BindProperty]?
6. Pourquoi ne jamais utiliser float ou double pour un montant en banque ?

Corrigé indicatif :

1. Il signale au moteur Razor qu'une expression ou un bloc de code C# commence, au milieu de code HTML.
2. Agences.cshtml.cs (le code-behind) prépare les données en C# ; Agences.cshtml (la vue) les affiche en HTML/Razor, sans logique de calcul.
3. Il insère, à l'exécution, le contenu spécifique de la page appelante à cet endroit précis du layout commun.
4. OnGet() s'exécute pour une requête GET (afficher la page) ; OnPost() s'exécute pour une requête POST (soumission d'un formulaire).
5. Il relie automatiquement une propriété C# typée aux données soumises par le formulaire.
6. Parce que float et double introduisent des erreurs d'arrondi inacceptables en finance. En banque, on utilise decimal (et DECIMAL(18,2) en base).

2.10. Mini-TP 2 — Pages d'agence et formulaire de contact

Objectif : construire deux pages Razor pour votre agence, l'une affichant des données, l'autre traitant un formulaire simple.

1. Créez Pages/Agences.cshtml et son code-behind : affichez la liste des 4 agences du chapitre 0 dans un tableau, avec le siège mis en évidence.
2. Créez _Layout.cshtml avec un en-tête (nom de la banque) et un menu de navigation, et reliez-le à vos pages via _ViewStart.cshtml.
3. Créez Pages/Contact.cshtml avec un formulaire (nom, message) utilisant asp-for et [BindProperty].
4. Implémentez OnPost() pour afficher un message de confirmation personnalisé après soumission.
5. **Bonus** : ajoutez une condition @if pour n'afficher le message de confirmation qu'après un envoi réussi, jamais au premier chargement de la page.
6. **Rendu attendu** : les fichiers Agences.cshtml(.cs), Contact.cshtml(.cs), _Layout.cshtml, et une capture d'écran de chaque page.

PARTIE II — ARCHITECTURE MVC ET DONNÉES

CHAPITRE 3 — ASP.NET CORE MVC

Objectifs du chapitre

- Comprendre le pattern MVC (Model — View — Controller).
- Créer un projet MVC (`dotnet new mvc`).
- Comprendre le routage par convention.
- Utiliser les ViewModels pour découpler la vue du modèle.
- Utiliser l'injection de dépendances.
- Afficher une première liste d'agences en mémoire.

3.1. Qu'est-ce que MVC ?

MVC signifie **Model — View — Controller**. C'est un pattern d'architecture qui sépare une application en trois responsabilités :

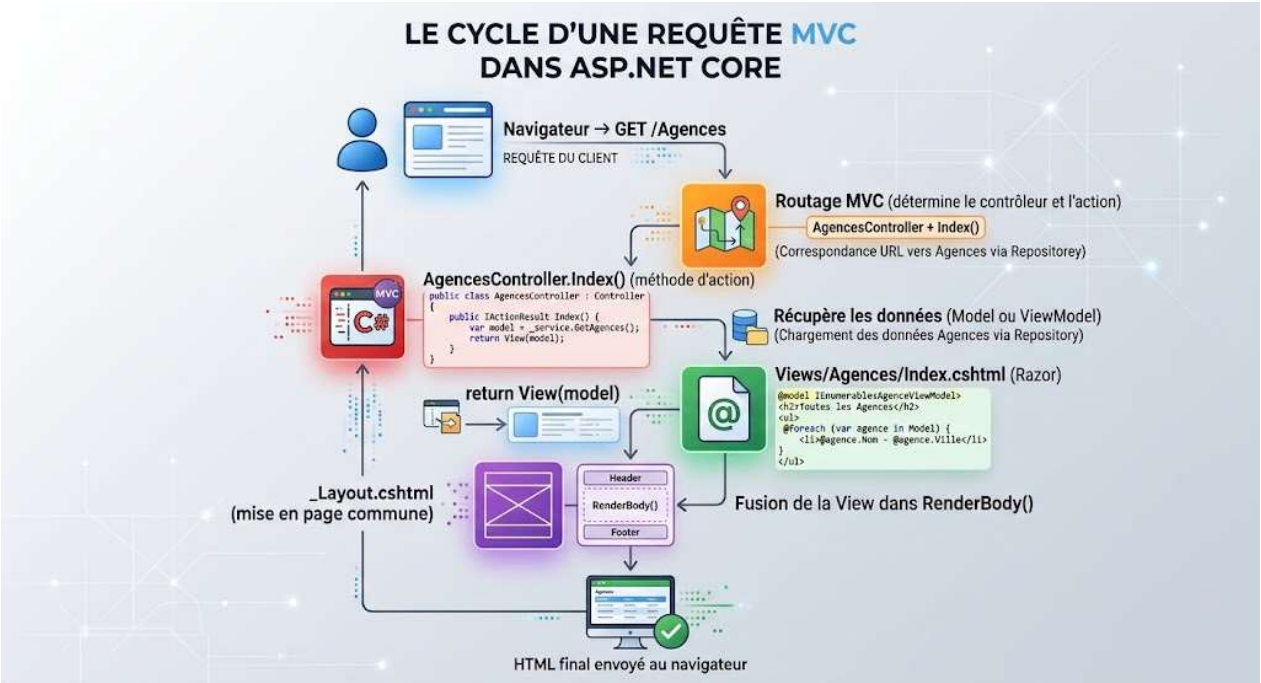
Composant	Rôle
Model	Représente les données et les règles métier
View	Affiche les données à l'utilisateur (interface)
Controller	Reçoit les requêtes, orchestre le traitement, choisit la vue

3.1.1. Pourquoi MVC?

La séparation stricte des responsabilités permet :

- de **tester** chaque couche indépendamment ;
- de **sécuriser** l'accès aux données (le Model ne s'expose pas directement à la vue) ;
- de **faire évoluer** l'interface sans casser la logique métier..

3.2. Le cycle d'une requête MVC



3.3. Créer un projet MVC

Terminal :

```
dotnet new mvc -n AkarangaBank.Web

cd AkarangaBank.Web
```

3.3.1. Comprendre la commande `dotnet new mvc -n AkarangaBank.Web`

Élément	Signification
`dotnet`	L'outil en ligne de commande .NET
`new`	Sous-commande : « créer un nouveau projet »
`mvc`	Le **template** : projet ASP.NET Core MVC complet
`-n`	Abréviation de –name
`AkarangaBank.Web`	Nom du projet (et du dossier, et du namespace racine)

3.3.2. Autres options utiles de `dotnet new mvc`

Option	Abréviation	Rôle
`--name`	`-n`	Nom du projet
`--output`	`-o`	Dossier de sortie (si différent du nom)
`--framework`	`-f`	Version de .NET cible (net10.0)
`--auth`	–	Type d'authentification (None, Individual, etc.)

Sur --auth :

- `None` (défaut) : aucun mécanisme d'authentification n'est généré. **C'est notre choix** pour le cours, car nous construisons l'authentification à la main au chapitre 7.
- `Individual` : génère automatiquement ASP.NET Core Identity avec **inscription publique ouverte**. **Non adapté à une banque** où les comptes doivent être créés par un administrateur.
- SingleOrg / MultiOrg : authentification Microsoft Entra ID (entreprise).
- Windows : authentification Windows (Active Directory).

Pourquoi --auth None **pour Akaranga Bank ?**

1. **Contrôle total** : dans une banque, personne ne s'inscrit tout seul. Un administrateur crée les comptes.
2. **Compréhension** : on construit Identity pas à pas au chapitre 7, on comprend chaque brique.
3. **Personnalisation** : on utilise un ApplicationUser personnalisé dès le départ.

3.3.3. Structure du projet MVC Créé

```
AkarangaBank.Web/
├── Controllers/
│   └── HomeController.cs
├── Models/
│   └── ErrorViewModel.cs
├── Views/
```

```
│   ├── Home/
│   │   ├── Index.cshtml
│   │   └── Privacy.cshtml
│   └── Shared/
│       ├── _Layout.cshtml
│       ├── _ValidationScriptsPartial.cshtml
│       └── Error.cshtml
├── wwwroot/
├── appsettings.json
├── Program.cs
└── AkarangaBank.Web.csproj
```

Les trois dossiers essentiels : `Controllers/`, `Models/`, `Views/`.

3.4. Première compilation

Avant de modifier quoi que ce soit :

bash

```
dotnet build
```

Résultat attendu :

```
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

Règle d'or : toujours vérifier que le projet de départ compile **avant** de le modifier.

3.5. Lancer l'application

bash

```
dotnet watch run
```

Le terminal affiche :

text

```
Now listening on: https://localhost:5001
```

Ouvrez cette adresse dans le navigateur. La page d'accueil ASP.NET Core MVC apparaît.

Pour arrêter : `Ctrl + C`.

3.6. Le routage par convention

Dans ASP.NET Core MVC, une URL est interprétée selon un **motif** défini dans `Program.cs` :

➡ **Fichier :** `Program.cs`

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

URL	Controller	Action	Paramètre
`/`	`HomeController`	`Index`	—
`/Agences`	`AgencesController`	`Index`	—
`/Agences/Details/5`	`AgencesController`	`Details`	`id = 5`

Convention : le nom du contrôleur se termine par `Controller`, mais l'URL ne l'inclut pas.

3.7. Créer le modèle `Agence`

📄 **Fichier à créer :** `Models/Agence.cs`

```
namespace AkarangaBank.Web.Models;

public class Agence{
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Nom { get; set; } = string.Empty;
    public string Ville { get; set; } = string.Empty;
    public bool EstSiege { get; set; }
}
```

3.8. Créer le ViewModel `AgenceListViewModel`

Pourquoi un ViewModel ? Pour ne pas exposer directement le modèle à la vue, et pour ajouter des champs spécifiques à l'affichage.

📄 **Fichier à créer :** `ViewModels/AgenceListViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class AgenceListViewModel {
    public IEnumerable<Agence> Agences { get; set; } = new
            List<Agence>();
    public int Total => Agences.Count();
}
```

3.9. Créer le contrôleur `AgencesController`

📄 **Fichier à créer :** `Controllers/AgencesController.cs`

```
using Microsoft.AspNetCore.Mvc;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;
```

```
public class AgencesController : Controller {
    public IActionResult Index() {
      // Données en dur pour l'instant ; la base de données arrive au chapitre 4.
        var agences = new List<Agence> {
            new() { Id = 1, Code ="AKR-BJM", Nom= "Akaranga Bank
              Bujumbura (Siège)", Ville="Bujumbura",EstSiege=true },
            new() { Id = 2, Code = "AKR-GIT", Nom = "Akaranga Bank
               Gitega", Ville = "Gitega", EstSiege = false },
            new() { Id = 3, Code = "AKR-NGO", Nom = "Akaranga Bank
               Ngozi", Ville = "Ngozi", EstSiege = false },
            new() { Id = 4, Code = "AKR-RUM", Nom = "Akaranga Bank
               Rumonge", Ville = "Rumonge", EstSiege = false }
        };

        var vm = new AgenceListViewModel { Agences = agences };
        return View(vm);
    }

    public IActionResult Details(int id) {
        // Simplifié pour l'instant
        if (id < 1 || id > 4) return NotFound();
        return View();
    }
}
```

3.10. Créer la vue Index

Fichier à créer : `Views/Agences/Index.cshtml`

```
@model AkarangaBank.Web.ViewModels.AgenceListViewModel
@{
    ViewData["Title"] = "Agences";
}

<h1>Nos agences</h1>

<p>@Model.Total agence(s) au total.</p>

<table class="table table-striped">
    <thead>
        <tr>
            <th>Code</th>
            <th>Nom</th>
            <th>Ville</th>
```

```
            <th>Statut</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
    @foreach (var agence in Model.Agences) {
        <tr>
            <td>@agence.Code</td>
            <td>@agence.Nom</td>
            <td>@agence.Ville</td>
            <td>
                @if (agence.EstSiege)
                {
                    <span class="badge bg-primary">Siège</span>
                }
                else
                {
                    <span class="badge bg-secondary">Agence</span>
                }
            </td>
            <td>
                <a asp-action="Details" asp-route-id="@agence.Id"
                   class="btn btn-sm btn-primary">Détails</a>
            </td>
        </tr>
    }
    </tbody>
</table>
```

3.11. Mettre à jour le menu

➡ **Fichier :** `Views/Shared/_Layout.cshtml` — dans la barre de navigation :

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-controller="Agences" asp-
action="Index">Agences</a>
</li>
```

3.12. Auto-évaluation — questions courtes

1. Que signifie MVC ? Quel est le rôle de chaque composant ?
2. Quelle est la différence entre un Model et un ViewModel ?
3. Que fait `return View(vm)` dans une action de contrôleur ?
4. Comment `l'URL /Agences/Details/5` est-elle interprétée par le routage MVC ?

5. Pourquoi préférer `--auth None` à `--auth Individual` pour Akaranga Bank ?

Corrigé indicatif :

1. Model — données et règles métier ; View — affichage ; Controller — orchestration.
2. Le Model représente une entité métier (souvent mappée à une table). Le ViewModel est conçu **spécifiquement pour une vue** : il peut contenir des champs calculés, des listes, des filtres, etc.
3. Elle rend la vue associée à l'action (par convention `Views/<Controller>/<Action>.cshtml`) en lui passant `vm` comme modèle.
4. `Controller = AgencesController, Action = Details, paramètre id = 5`.
5. Parce qu'une banque ne doit pas laisser les utilisateurs s'inscrire librement. Les comptes doivent être créés par un administrateur.

3.13. Mini-TP 3 — Page des agences en MVC

Objectif : construire la première page MVC d'Akaranga Bank.

1. Créez un projet MVC : `dotnet new mvc -n AkarangaBank.Web`.
2. Créez `Models/Agence.cs`, `ViewModels/AgenceListViewModel.cs`.
3. Créez `Controllers/AgencesController.cs` avec une action `Index` retournant les 4 agences.
4. Créez `Views/Agences/Index.cshtml` affichant le tableau.
5. Ajoutez le lien « Agences » dans `_Layout.cshtml`.
6. Testez `/Agences` dans le navigateur.

Rendu attendu : capture d'écran de la page `/Agences` avec les 4 agences.

CHAPITRE 4 —ENTITY FRAMEWORK CORE

Objectifs du chapitre

- Installer Entity Framework Core 10.
- Créer le DbContext et les 16 entités C# mappées au script.
- Configurer la chaîne de connexion (avec secrets utilisateur).
- Écrire la première requête LINQ sur la base.

4.1. Vérifier l'installation

4.1.1. Se connecter à la base

bash

```
sqlcmd -S localhost -E -d AkarangaBank
```

Puis, dans l'invite `1>` :

sql

```
SELECT name FROM sys.tables ORDER BY name;
GO
```

Vous devez voir les **16 tables** :

4.1.2. Vérifier les données de test

sql

```
SELECT Code, Nom, Ville, EstSiege FROM Agences;
GO
```

Résultat attendu :

```
AKR-BJM  Akaranga Bank Bujumbura (Siège)  Bujumbura  1
AKR-GIT  Akaranga Bank Gitega             Gitega     0
AKR-NGO  Akaranga Bank Ngozi              Ngozi      0
AKR-RUM  Akaranga Bank Rumonge            Rumonge    0
```

4.1.3. Quitter

sql

```
exit
```

4.2. Structure du modèle de données (rappel du chapitre 0)

Le script définit **16 tables** organisées en 6 blocs :

Bloc	Tables
1. Référentiel	`Agences`, `Utilisateurs`, `Parametres`
2. Clients et KYC	`Clients`, `VerificationsKYC`
3. Comptes et cartes	`TypesCompte`, `Comptes`, `Cartes`
4. Prêts	`Prets`, `EcheancesPret`
5. Virements et opérations	`Virements`, `Operations`, `Ecritures`
6. Audit et AML	`AuditLog`, `AlertesAML`, `SessionsUtilisateur`

4.3. Installer Entity Framework Core

4.3.1. Packages à installer

Terminal (dans le projet) :

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

4.3.2. Outil `dotnet-ef`

bash

```
dotnet tool install --global dotnet-ef
```

Vérification :

bash

```
dotnet ef --version
```

4.3.3. Restaurer et compiler

bash

```
dotnet restore
dotnet build
```

Résultat attendu : `Build succeeded. 0 Error(s)`.

4.4. Configurer la chaîne de connexion (avec secrets)

4.4.1. Pourquoi les secrets utilisateur ?

En développement, mettre un mot de passe en clair dans `appsettings.json` est une mauvaise pratique :

- le fichier peut être versionné par erreur ;
- le mot de passe peut fuiter sur un dépôt public ;
- ce n'est pas conforme aux exigences bancaires (akaranga : rigueur, éthique).

.NET fournit **User Secrets** : un stockage local, hors du projet, jamais versionné.

4.4.2. Initialiser les secrets

1. **Se connecter avec l'authentification Windows**

bash

```
sqlcmd -S localhost –E
```

2. **Activer le compte `sa` et définir un mot de passe**

```
ALTER LOGIN sa ENABLE;
GO
ALTER LOGIN sa WITH PASSWORD = 'NouveauMotDePasseFort!';
GO
```

3. **Vérifier que le mode d'authentification SQL Server est activé**

```
-- Vérifier le mode d'authentification actuel
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly');
GO
```

Si le résultat est `1`, le mode SQL Server est désactivé. Pour l'activer :

```
USE master;
GO
EXEC xp_instance_regwrite N'HKEY_LOCAL_MACHINE', N'Software\Microsoft\MSSQLServer\MSSQLServer',
N'LoginMode', REG_DWORD, 2;
GO
```

Note : Un redémarrage du service SQL Server est nécessaire après cette modification.

4. **Tester la connexion avec `sa`**

Essayez de vous reconnecter avec le nouveau mot de passe :

```
 sqlcmd -S localhost -U sa -P "NouveauMotDePasseFort!"
```

5. **Initialiser les secrets**

```
dotnet user-secrets init

dotnet user-secrets set "ConnectionStrings:AkarangaBank"
"Server=localhost,1433;Database=AkarangaBank;User
Id=sa;Password=Akaranga@2025!;TrustServerCertificate=True;Encrypt=False;"
```

4.4.3. Vérifier

bash

```
dotnet user-secrets list
```

Résultat attendu :

```
ConnectionStrings:AkarangaBank =
Server=localhost,1433;Database=AkarangaBank;User
Id=sa;Password=Akaranga@2025!;TrustServerCertificate=True;Encrypt=False;
```

En production, on utilisera des variables d'environnement ou un coffre-fort (Azure Key Vault, etc.). Voir chapitre 11 (sécurité avancée).

4.4.4. `appsettings.json` reste propre

➖ **Fichier :** `appsettings.json` — la chaîne de connexion peut rester vide (les secrets la fournissent en développement) :

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

4.5. Créer le DbContext

4.5.1. Le rôle du DbContext

Le DbContext est **le cœur d'EF Core** :

- il représente une **session de travail** avec la base ;
- il expose des DbSet<T> (tables logiques) ;
- il suit les modifications (Change Tracker) ;
- il génère le SQL à partir de LINQ.

4.5.2. Créer le dossier Data/

📁 **Dossier à créer :** Data/

4.5.3. Créer le contexte

📄 **Fichier à créer :** `Data/AkarangaBankContext.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Data;
// ------------------------------------------------------------------
// AkarangaBankContext — session de travail avec la base AkarangaBank.
// les 16 DbSet correspondent exactement aux 16 tables du script
// AkarangaBank-Script-DB.sql.
// ------------------------------------------------------------------
public class AkarangaBankContext : DbContext{
    public AkarangaBankContext(DbContextOptions<AkarangaBankContext>
       options) : base(options) {
    }
```

```
    // Bloc 1 — Référentiel
    public DbSet<Agence> Agences => Set<Agence>();
    public DbSet<Utilisateur> Utilisateurs => Set<Utilisateur>();
    public DbSet<Parametre> Parametres => Set<Parametre>();

    // Bloc 2 — Clients et KYC
    public DbSet<Client> Clients => Set<Client>();
    public DbSet<VerificationKYC> VerificationsKYC =>
            Set<VerificationKYC>();

    // Bloc 3 — Comptes et cartes
    public DbSet<TypeCompte> TypesCompte => Set<TypeCompte>();
    public DbSet<Compte> Comptes => Set<Compte>();
    public DbSet<Carte> Cartes => Set<Carte>();

    // Bloc 4 — Prêts
    public DbSet<Pret> Prets => Set<Pret>();
    public DbSet<EcheancePret> EcheancesPret => Set<EcheancePret>();

    // Bloc 5 — Virements, opérations, écritures
    public DbSet<Virement> Virements => Set<Virement>();
    public DbSet<Operation> Operations => Set<Operation>();
    public DbSet<Ecriture> Ecritures => Set<Ecriture>();

    // Bloc 6 — Audit, AML, sessions
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();
    public DbSet<AlerteAML> AlertesAML => Set<AlerteAML>();
    public DbSet<SessionUtilisateur> SessionsUtilisateur =>
           Set<SessionUtilisateur>();

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // La configuration détaillée (relations, contraintes, index)
       // est faite dans le chapitre 5 (CRUD Agences) puis complétée
        // dans le chapitre 6 (CRUD Clients, KYC, Comptes).
    }
}
```

4.6. Créer les 16 entités C# mappées au script

Chaque table du script devient une classe C#. Les propriétés portent **exactement** le nom des colonnes.

4.6.1. Bloc 1 — Référentiel

➖ **Fichier :** `Models/Agence.cs` (complété par rapport au chapitre 3)

```
namespace AkarangaBank.Web.Models;


public class Agence {
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Nom { get; set; } = string.Empty;
    public string Ville { get; set; } = string.Empty;
    public bool EstSiege { get; set; }
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateModification { get; set; }
}
```

📄 **Fichier à créer :** `Models/Utilisateur.cs`

```
namespace AkarangaBank.Web.Models;

public class Utilisateur {
    public int Id { get; set; }
    public string NomComplet { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string MotDePasseHash { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public int AgenceId { get; set; }
    public Agence Agence { get; set; } = null!;
    public bool DeuxFA_Active { get; set; }
    public string? DeuxFA_Secret { get; set; }
    public DateTime? DerniereConnexion { get; set; }
    public bool EstActif { get; set; } = true;
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateModification { get; set; }
}
```

Fichier à créer : `Models/Parametre.cs`

```
namespace AkarangaBank.Web.Models;

public class Parametre {
    public int Id { get; set; }
    public string Cle { get; set; } = string.Empty;
    public string Valeur { get; set; } = string.Empty;
    public string? Description { get; set; }
    public int? AgenceId { get; set; }
    public Agence? Agence { get; set; }
    public DateTime DateModification { get; set; } = DateTime.UtcNow;
}
```

4.6.2. Bloc 2 — Clients et KYC

Fichier à créer : `Models/Client.cs`

```
namespace AkarangaBank.Web.Models;

public class Client {
    public int Id { get; set; }
    public string NumeroClient { get; set; } = string.Empty;
    public string Nom { get; set; } = string.Empty;
    public string Prenom { get; set; } = string.Empty;
    public DateTime DateNaissance { get; set; }
    public string NumeroCNI { get; set; } = string.Empty;
    public string? Telephone { get; set; }
    public string? Email { get; set; }
    public string? Adresse { get; set; }
    public int AgenceId { get; set; }
    public Agence Agence { get; set; } = null!;
    public string Statut { get; set; } = "Actif";
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateModification { get; set; }

    // Navigation
    public ICollection<Compte> Comptes { get; set; } = new
           List<Compte>();
    public ICollection<VerificationKYC> VerificationsKYC {get;set;}
           = new List<VerificationKYC>();
}
```

Fichier à créer : `Models/VerificationKYC.cs`

```
namespace AkarangaBank.Web.Models;

public class VerificationKYC {
    public int Id { get; set; }
    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;
    public string TypeDocument { get; set; } = string.Empty;
    public string NumeroDocument { get; set; } = string.Empty;
    public string? CheminDocument { get; set; }
    public string Statut { get; set; } = "EnAttente";
    public int? VerifieParId { get; set; }
    public Utilisateur? VerifiePar { get; set; }
    public DateTime? DateVerification { get; set; }
    public string? Commentaire { get; set; }
}
```

4.6.3. Bloc 3 — Comptes et cartes

Fichier à créer : `Models/TypeCompte.cs`

```
namespace AkarangaBank.Web.Models;

public class TypeCompte {
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Libelle { get; set; } = string.Empty;
    public decimal TauxInteret { get; set; }
    public decimal SoldeMinimum { get; set; }
    public bool AutoriseDecouvert { get; set; }
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
}
```

Fichier à créer : `Models/Compte.cs`

```
namespace AkarangaBank.Web.Models;

public class Compte {
    public int Id { get; set; }
    public string NumeroCompte { get; set; } = string.Empty;
    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;
    public int TypeCompteId { get; set; }
```

```
    public TypeCompte TypeCompte { get; set; } = null!;
    public int AgenceId { get; set; }
    public Agence Agence { get; set; } = null!;
    public decimal Solde { get; set; }
    public string Devise { get; set; } = "BIF";
    public string Statut { get; set; } = "Actif";
    public DateTime DateOuverture { get; set; } = DateTime.UtcNow;
    public DateTime? DateFermeture { get; set; }
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateModification { get; set; }
    // Navigation
    public ICollection<Ecriture> Ecritures { get; set; } = new
           List<Ecriture>();
    public ICollection<Carte> Cartes { get; set; } = new
           List<Carte>();
}
```

Fichier à créer : `Models/Carte.cs`

```
namespace AkarangaBank.Web.Models;

public class Carte {
    public int Id { get; set; }
    public string NumeroCarte { get; set; } = string.Empty;
    public int CompteId { get; set; }
    public Compte Compte { get; set; } = null!;
    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;
    public string TypeCarte { get; set; } = string.Empty;
    public DateTime DateExpiration { get; set; }
    public string Statut { get; set; } = "Active";
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
}
```

4.6.4. Bloc 4 — Prêts

Fichier à créer : `Models/Pret.cs`

```
namespace AkarangaBank.Web.Models;

public class Pret {
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;
    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;
```

```
    public int CompteDebitId { get; set; }
    public Compte CompteDebit { get; set; } = null!;
    public int? ValideParId { get; set; }
    public Utilisateur? ValidePar { get; set; }
    public decimal MontantPrincipal { get; set; }
    public decimal TauxInteret { get; set; }
    public int DureeMois { get; set; }
    public decimal Mensualite { get; set; }
    public string Statut { get; set; } = "EnAttente";
    public DateTime DateDemande { get; set; } = DateTime.UtcNow;
    public DateTime? DateValidation { get; set; }
    public DateTime? DateDebut { get; set; }
    public DateTime? DateFin { get; set; }

    // Navigation
    public ICollection<EcheancePret> Echeances { get; set; } = new
           List<EcheancePret>();
}
```

Fichier à créer : `Models/EcheancePret.cs`

```
namespace AkarangaBank.Web.Models;


public class EcheancePret {

    public int Id { get; set; }
    public int PretId { get; set; }
    public Pret Pret { get; set; } = null!;
    public int NumeroEcheance { get; set; }
    public decimal Capital { get; set; }
    public decimal Interet { get; set; }
    public decimal MontantTotal { get; set; }
    public DateTime DateEcheance { get; set; }
    public DateTime? DatePaiement { get; set; }
    public string Statut { get; set; } = "EnAttente";

}
```

4.6.5. Bloc 5 — Virements, opérations, écritures

Fichier à créer : `Models/Virement.cs`

```
namespace AkarangaBank.Web.Models;

public class Virement {
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;
    public int CompteSourceId { get; set; }
    public Compte CompteSource { get; set; } = null!;
    public int CompteDestinationId { get; set; }
    public Compte CompteDestination { get; set; } = null!;
    public int UtilisateurId { get; set; }
    public Utilisateur Utilisateur { get; set; } = null!;
    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
    public string Statut { get; set; } = "EnAttente";
    public string? Motif { get; set; }
    public bool ControleAML { get; set; }
    public DateTime DateVirement { get; set; } = DateTime.UtcNow;
    public DateTime? DateValidation { get; set; }
}
```

Fichier à créer : `Models/Operation.cs`

```
namespace AkarangaBank.Web.Models;

public class Operation {
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;
    public string TypeOperation { get; set; } = string.Empty;
    public int CompteId { get; set; }
    public Compte Compte { get; set; } = null!;
    public int UtilisateurId { get; set; }
    public Utilisateur Utilisateur { get; set; } = null!;
    public int AgenceId { get; set; }
    public Agence Agence { get; set; } = null!;
    public int? VirementId { get; set; }
    public Virement? Virement { get; set; }
    public int? PretId { get; set; }
    public Pret? Pret { get; set; }
    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
```

```
    public string Statut { get; set; } = "EnAttente";
    public DateTime DateOperation { get; set; } = DateTime.UtcNow;
    public DateTime? DateValidation { get; set; }
    public string? Commentaire { get; set; }
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;

    // Navigation
    public ICollection<Ecriture> Ecritures { get; set; } = new
           List<Ecriture>();
}
```

Fichier à créer : `Models/Ecriture.cs`

```
namespace AkarangaBank.Web.Models;

public class Ecriture {
    public int Id { get; set; }
    public int OperationId { get; set; }
    public Operation Operation { get; set; } = null!;
    public int CompteId { get; set; }
    public Compte Compte { get; set; } = null!;
    public string Sens { get; set; } = string.Empty; // "Debit" ou "Credit"
    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
    public DateTime DateEcriture { get; set; } = DateTime.UtcNow;
    public string? Libelle { get; set; }
}
```

4.6.6. Bloc 6 — Audit, AML, sessions

Fichier à créer : `Models/AuditLog.cs`

```
namespace AkarangaBank.Web.Models;

public class AuditLog {
    public int Id { get; set; }
    public DateTime DateEvenement { get; set; } = DateTime.UtcNow;
    public int? UtilisateurId { get; set; }
    public Utilisateur? Utilisateur { get; set; }
    public int? AgenceId { get; set; }
    public Agence? Agence { get; set; }
    public string Action { get; set; } = string.Empty;
    public string? TableConcernee { get; set; }
    public int? EnregistrementId { get; set; }
    public string? ValeurAvant { get; set; }
```

```
    public string? ValeurApres { get; set; }
    public string? AdresseIP { get; set; }
    public string? UserAgent { get; set; }
    public string? Details { get; set; }
}
```

Fichier à créer : `Models/AlerteAML.cs`

```
namespace AkarangaBank.Web.Models;

public class AlerteAML {
    public int Id { get; set; }
    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;
    public int? OperationId { get; set; }
    public Operation? Operation { get; set; }
    public string TypeAlerte { get; set; } = string.Empty;
    public string Niveau { get; set; } = "Moyen";   // Faible,
Moyen, Eleve, Critique
    public string? Description { get; set; }
    public string Statut { get; set; } = "Ouverte"; // Ouverte,
EnCours, Traitee, Classee
    public int? TraiteParId { get; set; }
    public Utilisateur? TraitePar { get; set; }
    public DateTime DateAlerte { get; set; } = DateTime.UtcNow;
    public DateTime? DateTraitement { get; set; }
}
```

Fichier à créer : `Models/SessionUtilisateur.cs`

```
namespace AkarangaBank.Web.Models;

public class SessionUtilisateur {
    public int Id { get; set; }
    public int UtilisateurId { get; set; }
    public Utilisateur Utilisateur { get; set; } = null!;
    public string Token { get; set; } = string.Empty;
    public string? AdresseIP { get; set; }
    public string? UserAgent { get; set; }
    public bool DeuxFA_Valide { get; set; }
    public DateTime DateConnexion { get; set; } = DateTime.UtcNow;
    public DateTime DateExpiration { get; set; }
    public bool EstActive { get; set; } = true;
}
```

4.7. Enregistrer le DbContext dans Program.cs

➖ **Fichier :** `Program.cs`

```
using AkarangaBank.Web.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Services MVC
builder.Services.AddControllersWithViews();

// DbContext — la chaîne de connexion vient des User Secrets en
développement.
builder.Services.AddDbContext<AkarangaBankContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("AkarangaBank")));

var app = builder.Build();

// Pipeline HTTP
if (!app.Environment.IsDevelopment()) {
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

4.8. Première requête LINQ sur la base

4.8.1. Injecter le DbContext dans un contrôleur

➡ **Fichier :** Controllers/AgencesController.cs (remplacer le contenu)

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

public class AgencesController : Controller {
    private readonly AkarangaBankContext _context;

    public AgencesController(AkarangaBankContext context) {
        _context = context;
    }

    public async Task<IActionResult> Index() {
        // Première requête LINQ : récupère les agences depuis la base.
        var agences = await _context.Agences
            .AsNoTracking()
            .OrderBy(a => a.Ville)
            .ToListAsync();

        var vm = new AgenceListViewModel { Agences = agences };
        return View(vm);
    }

    public async Task<IActionResult> Details(int id) {
        var agence = await _context.Agences
            .AsNoTracking()
            .FirstOrDefaultAsync(a => a.Id == id);

        if (agence is null) return NotFound();
        return View(agence);
    }
}
```

4.8.2. Créer la vue `Details`

Fichier à créer : `Views/Agences/Details.cshtml`

```
@model AkarangaBank.Web.Models.Agence
@{
    ViewData["Title"] = "Détails de l'agence";
}

<h1>Détails de l'agence</h1>

<dl class="row">
    <dt class="col-sm-3">Code</dt>
    <dd class="col-sm-9">@Model.Code</dd>

    <dt class="col-sm-3">Nom</dt>
    <dd class="col-sm-9">@Model.Nom</dd>

    <dt class="col-sm-3">Ville</dt>
    <dd class="col-sm-9">@Model.Ville</dd>

    <dt class="col-sm-3">Statut</dt>
    <dd class="col-sm-9">
        @if (Model.EstSiege) {
            <span class="badge bg-primary">Siège</span>
        }
        else {
            <span class="badge bg-secondary">Agence</span>
        }
    </dd>

    <dt class="col-sm-3">Date de création</dt>
    <dd class="col-sm-9">@Model.DateCreation.ToString("dd/MM/yyyy
        HH:mm")</dd>
</dl>

<a asp-action="Index" class="btn btn-secondary">Retour à la liste</a>
```

4.9. Tests

Test 1 — Compilation

bash

```
dotnet build
```

Résultat attendu : `Build succeeded. 0 Error(s)`.

Test 2 — Démarrage

bash

```
dotnet watch run
```

L'application doit démarrer sans exception. L'adresse est affichée dans le terminal.

Test 3 — Page Agences

Ouvrir `http://localhost:5001/Agences`.

Résultat attendu : les 4 agences du script SQL s'affichent, triées par ville.

Test 4 — Détails d'une agence

Cliquer sur « Détails » à côté de `AKR-BJM`.

Résultat attendu : la page affiche les informations de l'agence, avec le badge « Siège ».

Test 5 — Agence inexistante

Ouvrir `http://localhost:5001/Agences/Details/9999`.

Résultat attendu : page **404 Not Found**.

Test 6 — Contrôle de l'équilibre comptable

Depuis `sqlcmd` :

bash

```
sqlcmd -S localhost -U sa -P "Akaranga@2025!" -C -d AkarangaBank -Q
"SELECT OperationId, SUM(CASE WHEN Sens='Debit' THEN Montant ELSE 0
END) AS TotalDebit, SUM(CASE WHEN Sens='Credit' THEN Montant ELSE 0
END) AS TotalCredit FROM Ecritures GROUP BY OperationId HAVING
SUM(CASE WHEN Sens='Debit' THEN Montant ELSE -Montant END) <> 0;"
```

Résultat attendu : aucune ligne retournée.

4.10. Exercices

Exercice 1 — Afficher le nombre de clients par agence

Dans la page /Agences, ajouter une colonne « Nombre de clients » qui compte les clients de chaque agence.

Fichiers concernés :

- `ViewModels/AgenceListViewModel.cs` (ajouter `NombreClients`)
- `Controllers/AgencesController.cs` (utiliser `Include` ou `CountAsync`)

Exercice 2 — Afficher les agences du siège uniquement

Ajouter une case à cocher « Siège uniquement » sur la page /Agences.

Fichiers concernés :

- `ViewModels/AgenceListViewModel.cs`
- `Controllers/AgencesController.cs`
- `Views/Agences/Index.cshtml`

Exercice 3 — Vérifier les données de test

Via sqlcmd, vérifier que :

- les 4 agences sont présentes ;
- les 5 clients sont répartis entre Bujumbura, Gitega et Ngozi ;
- les 6 comptes sont cohérents avec les types de compte ;
- le virement de 50 000 BIF a bien 1 débit + 1 crédit.

Rédiger les requêtes SQL correspondantes.

Exercice 4 — Explorer le modèle avec LINQ

Dans un contrôleur temporaire, écrire des requêtes LINQ pour :

1. lister les clients de Bujumbura ;
2. lister les comptes dont le solde est supérieur à 1 000 000 BIF ;
3. lister les virements avec leurs comptes source et destination.

4.11. Questions de compréhension

1. Quelle est la différence entre IDENTITY(1,1) et une clé primaire classique ?
2. Pourquoi tous les montants sont-ils en DECIMAL(18,2) et jamais en FLOAT ?
3. Que fait AddDbContext dans Program.cs ?
4. Quelle est la différence entre ToListAsync() et ToList() ?
5. Que fait AsNoTracking() ? Quand l'utiliser ?
6. Pourquoi utiliser dotnet user-secrets plutôt que appsettings.json ?
7. Que fait FirstOrDefaultAsync() par rapport à FindAsync() ?
8. Comment EF Core traduit-il une requête LINQ en SQL ?

4.12. Mini-TP 4 — Première page connectée à la base

Objectif : connecter la page Agences à la base AkarangaBank.

1. Installez EF Core 10.
2. Configurez les User Secrets avec la chaîne de connexion.
3. Créez le DbContext et les 16 entités.
4. Enregistrez le DbContext dans Program.cs.
5. Modifiez AgencesController pour lire depuis la base.
6. Testez /Agences et /Agences/Details/1.

Rendu attendu :

- capture d'écran de /Agences avec les 4 agences issues de la base ;
- capture d'écran de /Agences/Details/1 ;
- capture d'écran du contrôle d'équilibre comptable (aucune ligne).

CHAPITRE 5 — CRUD AGENCES

Objectifs du chapitre

- Comprendre le pattern Controller → Service → ViewModel → View.
- Implémenter un CRUD complet (Create, Read, Update, Delete) sur une entité simple.
- Utiliser les Data Annotations pour la validation.
- Gérer les erreurs : `NotFound`, contraintes d'unicité, intégrité référentielle.
- Implémenter la pagination et la recherche.
- Utiliser `TempData` pour les messages flash.
- Protéger les formulaires contre les attaques CSRF.

5.1. Fichiers du chapitre

```
AkarangaBank.Web/
├── Controllers/
│   └── AgencesController.cs          ← remplacé
├── Services/
│   ├── IAgenceService.cs             ← créé
│   └── AgenceService.cs              ← créé
├── ViewModels/
│   ├── AgenceListViewModel.cs        ← remplacé
│   ├── AgenceCreateViewModel.cs      ← créé
│   ├── AgenceEditViewModel.cs        ← créé
│   └── AgenceDetailsViewModel.cs     ← créé
├── Views/
│   └── Agences/
│       ├── Index.cshtml              ← remplacé
│       ├── Details.cshtml            ← remplacé
│       ├── Create.cshtml             ← créé
│       ├── Edit.cshtml               ← créé
│       └── Delete.cshtml             ← créé
└── Program.cs                        ← modifié (enregistrement du service)
```

5.2. Les ViewModels

5.2.1. ViewModel de liste (avec pagination et recherche)

🔁 **Fichier :** `ViewModels/AgenceListViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;
// ViewModel utilisé par la page Index des agences.
// Il contient la liste paginée + les paramètres de recherche.
public class AgenceListViewModel {
    public IEnumerable<Agence> Agences{get;set;}=new List<Agence>();
```

```
    public string? Recherche { get; set; }
    public int Page { get; set; } = 1;
    public int TaillePage { get; set; } = 10;
    public int Total { get; set; }

    // Propriété calculée : nombre total de pages.
    public int TotalPages =>(int)Math.Ceiling((double)Total / TaillePage);
}
```

5.2.2. ViewModel de création

Fichier à créer : `ViewModels/AgenceCreateViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;
// Données nécessaires pour créer une agence.
// Les Data Annotations définissent les règles de validation côté se
rveur ET côté client (via jQuery Validation).
public class AgenceCreateViewModel {

    [Required(ErrorMessage = "Le code de l'agence est obligatoire.")]
    [StringLength(20, MinimumLength = 3,
        ErrorMessage = "Le code doit contenir entre 3 et 20 caractères.")]
    [Display(Name = "Code agence")]
    [RegularExpression(@"^AKR-[A-Z]{3}$",
        ErrorMessage = "Le code doit respecter le format AKR-XXX (3 lettres majuscules).")]
    public string Code { get; set; } = string.Empty;

    [Required(ErrorMessage = "Le nom est obligatoire.")]
    [StringLength(100, ErrorMessage = "Le nom ne peut pas dépasser 100 caractères.")]
    public string Nom { get; set; } = string.Empty;

    [Required(ErrorMessage = "La ville est obligatoire.")]
    [StringLength(50)]
    public string Ville { get; set; } = string.Empty;

    [Display(Name = "Est le siège")]
    public bool EstSiege { get; set; }
}
```

5.2.3. ViewModel de modification

Fichier à créer : `ViewModels/AgenceEditViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class AgenceEditViewModel {
    public int Id { get; set; }

    [Required(ErrorMessage = "Le code de l'agence est obligatoire.")]
    [StringLength(20, MinimumLength = 3)]
    [Display(Name = "Code agence")]
    [RegularExpression(@"^AKR-[A-Z]{3}$",
        ErrorMessage = "Le code doit respecter le format AKR-XXX.")]
    public string Code { get; set; } = string.Empty;

    [Required(ErrorMessage = "Le nom est obligatoire.")]
    [StringLength(100)]
    public string Nom { get; set; } = string.Empty;

    [Required(ErrorMessage = "La ville est obligatoire.")]
    [StringLength(50)]
    public string Ville { get; set; } = string.Empty;

    [Display(Name = "Est le siège")]
    public bool EstSiege { get; set; }
}
```

5.2.4. ViewModel de détails

Fichier à créer : `ViewModels/AgenceDetailsViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;
// ViewModel pour la page Détails.
// Contient l'agence + des compteurs utiles (clients, comptes, utili
sateurs).
public class AgenceDetailsViewModel {
    public Agence Agence { get; set; } = null!;
    public int NombreClients { get; set; }
    public int NombreComptes { get; set; }
    public int NombreUtilisateurs { get; set; }
}
```

5.3. Le Service Agence

5.3.1. Interface

Fichier à créer : `Services/IAgenceService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public interface IAgenceService {
    // Liste paginée avec recherche
    Task<AgenceListViewModel> GetPagedAsync(string? recherche, int p
          age, int taille);

    // Détails avec compteurs
    Task<AgenceDetailsViewModel?> GetDetailsAsync(int id);

    // Récupération simple (pour Edit/Delete)
    Task<Agence?> GetByIdAsync(int id);

    // Création
    Task<Agence> CreateAsync(AgenceCreateViewModel vm);

    // Modification
    Task<bool> UpdateAsync(AgenceEditViewModel vm);

    // Suppression
    Task<bool> DeleteAsync(int id);

    // Vérifications métier
    Task<bool> CodeExisteAsync(string code, int? excludeId = null);
    Task<bool> APeuEtreSupprimeeAsync(int id);
}
```

5.2.2. Implémentation

Fichier à créer : `Services/AgenceService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

// Ce service encapsule la logique métier des agences, séparée du contrôleur.
public class AgenceService : IAgenceService {
    private readonly AkarangaBankContext _context;

    public AgenceService(AkarangaBankContext context) {
```

```
        _context = context;
    }

    // ------------------------------------------------------------
    // Liste paginée avec recherche
    // ------------------------------------------------------------
    public async Task<AgenceListViewModel> GetPagedAsync(string? rec
           herche, int page, int taille) {
        var query = _context.Agences.AsNoTracking().AsQueryable();

        // Recherche par code, nom ou ville
        if (!string.IsNullOrWhiteSpace(recherche)) {
            var r = recherche.Trim();
            query = query.Where(a =>
                a.Code.Contains(r) ||
                a.Nom.Contains(r) ||
                a.Ville.Contains(r));
        }
        var total = await query.CountAsync();

        var agences = await query
            .OrderByDescending(a => a.EstSiege)  // Le siège en premier
            .ThenBy(a => a.Ville)
            .Skip((page - 1) * taille)
            .Take(taille)
            .ToListAsync();

        return new AgenceListViewModel {
            Agences = agences,
            Recherche = recherche,
            Page = page,
            TaillePage = taille,
            Total = total
        };
    }

    // ------------------------------------------------------------
    // Détails avec compteurs (clients, comptes, utilisateurs)
    // ------------------------------------------------------------
    public async Task<AgenceDetailsViewModel?> GetDetailsAsync(int id) {
        var agence = await _context.Agences
            .AsNoTracking()
            .FirstOrDefaultAsync(a => a.Id == id);
```

```
        if (agence is null) return null;

        return new AgenceDetailsViewModel {
            Agence = agence,
            NombreClients = await _context.Clients
                .CountAsync(c => c.AgenceId == id),
            NombreComptes = await _context.Comptes
                .CountAsync(c => c.AgenceId == id),
            NombreUtilisateurs = await _context.Utilisateurs
                .CountAsync(u => u.AgenceId == id)
        };
    }
    // ------------------------------------------------------------
    // Récupération simple
    // ------------------------------------------------------------
    public async Task<Agence?> GetByIdAsync(int id)
    {
        return await _context.Agences
            .AsNoTracking()
            .FirstOrDefaultAsync(a => a.Id == id);
    }
    // ------------------------------------------------------------
    // Création
    // ------------------------------------------------------------
    public async Task<Agence> CreateAsync(AgenceCreateViewModel vm)
    {
        var agence = new Agence
        {
            Code = vm.Code.Trim().ToUpperInvariant(),
            Nom = vm.Nom.Trim(),
            Ville = vm.Ville.Trim(),
            EstSiege = vm.EstSiege,
            DateCreation = DateTime.UtcNow
        };

        _context.Agences.Add(agence);
        await _context.SaveChangesAsync();
        return agence;
    }


    // ------------------------------------------------------------
```

```
    // Modification
    // ------------------------------------------------------------
    public async Task<bool> UpdateAsync(AgenceEditViewModel vm) {
        var agence = await _context.Agences.FindAsync(vm.Id);
        if (agence is null) return false;

        agence.Code = vm.Code.Trim().ToUpperInvariant();
        agence.Nom = vm.Nom.Trim();
        agence.Ville = vm.Ville.Trim();
        agence.EstSiege = vm.EstSiege;
        agence.DateModification = DateTime.UtcNow;

        await _context.SaveChangesAsync();
        return true;
    }
    // ------------------------------------------------------------
    // Suppression — refusée si l'agence a des dépendances
    // ------------------------------------------------------------
    public async Task<bool> DeleteAsync(int id) {
        var agence = await _context.Agences.FindAsync(id);
        if (agence is null) return false;

        // Vérification d'intégrité référentielle
        if (!await APeuEtreSupprimeeAsync(id))
            throw new InvalidOperationException(
                "Impossible de supprimer cette agence : des clients, comptes " +
                "ou utilisateurs y sont rattachés.");
        _context.Agences.Remove(agence);
        await _context.SaveChangesAsync();
        return true;
    }
    // ------------------------------------------------------------
    // Vérifications métier
    // ------------------------------------------------------------
    public async Task<bool> CodeExisteAsync(string code, int? exclud
          eId = null){
        var query = _context.Agences.Where(a => a.Code == code);
        if (excludeId.HasValue)
            query = query.Where(a => a.Id != excludeId.Value);
        return await query.AnyAsync();
    }
    public async Task<bool> APeuEtreSupprimeeAsync(int id){
```

```
        var aDesClients = await _context.Clients.AnyAsync(c => c.Agen
            ceId == id);
        var aDesComptes = await _context.Comptes.AnyAsync(c => c.Age
            nceId == id);
        var aDesUtilisateurs = await _context.Utilisateurs.AnyAsync(
            u => u.AgenceId == id);

        return !aDesClients && !aDesComptes && !aDesUtilisateurs;
    }
}
```

5.2.3. Enregistrement du service

➖ **Fichier :** `Program.cs` — ajouter avant `var app = builder.Build();` :

```
using AkarangaBank.Web.Services;
// ...
builder.Services.AddScoped<IAgenceService, AgenceService>();
```

5.3. Le Contrôleur Agences

🔁 **Fichier :** `Controllers/AgencesController.cs`

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

public class AgencesController : Controller {
    private readonly IAgenceService _agenceService;

    public AgencesController(IAgenceService agenceService) {
        _agenceService = agenceService;
    }
    // ------------------------------------------------------------
    // GET: /Agences
    // ------------------------------------------------------------
    public async Task<IActionResult> Index(string? recherche, int page=1){
        const int taille = 10;
        var vm= await _agenceService.GetPagedAsync(recherche,page,taille);
        return View(vm);
    }
    // ------------------------------------------------------------
```

```
    // GET: /Agences/Details/5
    // ------------------------------------------------------------
    public async Task<IActionResult> Details(int id) {
        var vm = await _agenceService.GetDetailsAsync(id);
        if (vm is null) return NotFound();
        return View(vm);
    }
    // ------------------------------------------------------------
    // GET: /Agences/Create
    // ------------------------------------------------------------
    public IActionResult Create()
    {
        return View(new AgenceCreateViewModel());
    }

    // ------------------------------------------------------------
    // POST: /Agences/Create
    // ------------------------------------------------------------
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(AgenceCreateViewModel vm) {
        if (!ModelState.IsValid) return View(vm);

        // Vérification d'unicité du code
        if (await _agenceService.CodeExisteAsync(vm.Code.Trim().ToUp
         perInvariant()))
        {
            ModelState.AddModelError(nameof(vm.Code),
                "Ce code d'agence est déjà utilisé.");
            return View(vm);
        }

        try
        {
            await _agenceService.CreateAsync(vm);
            TempData["Success"] = "Agence créée avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (DbUpdateException ex)
            when (ex.InnerException is SqlException sql && sql.Numbe
            r == 2601)
        {
            // Contrainte UNIQUE violée côté base
```

```
            ModelState.AddModelError(nameof(vm.Code),
                "Ce code d'agence est déjà utilisé.");
            return View(vm);
        }
    }

    // -----------------------------------------------------------
    // GET: /Agences/Edit/5
    // -----------------------------------------------------------
    public async Task<IActionResult> Edit(int id)
    {
        var agence = await _agenceService.GetByIdAsync(id);
        if (agence is null) return NotFound();

        var vm = new AgenceEditViewModel
        {
            Id = agence.Id,
            Code = agence.Code,
            Nom = agence.Nom,
            Ville = agence.Ville,
            EstSiege = agence.EstSiege
        };
        return View(vm);
    }

    // -----------------------------------------------------------
    // POST: /Agences/Edit/5
    // -----------------------------------------------------------
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, AgenceEditViewModel vm){
        if (id != vm.Id) return BadRequest();
        if (!ModelState.IsValid) return View(vm);

        if (await _agenceService.CodeExisteAsync(vm.Code.Trim().ToUp
         perInvariant(), vm.Id)) {
            ModelState.AddModelError(nameof(vm.Code),
                "Ce code d'agence est déjà utilisé.");
            return View(vm);
        }
        try {
            var ok = await _agenceService.UpdateAsync(vm);
```

```
            if (!ok) return NotFound();

            TempData["Success"] = "Agence modifiée avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (DbUpdateException ex)
            when (ex.InnerException is SqlException sql && sql.Numbe
            r == 2601) {
            ModelState.AddModelError(nameof(vm.Code),
                "Ce code d'agence est déjà utilisé.");
            return View(vm);
        }
    }
    // ------------------------------------------------------------
    // GET: /Agences/Delete/5
    // ------------------------------------------------------------
    public async Task<IActionResult> Delete(int id) {
        var agence = await _agenceService.GetByIdAsync(id);
        if (agence is null) return NotFound();
        return View(agence);
    }

    // ------------------------------------------------------------
    // POST: /Agences/Delete/5
    // ------------------------------------------------------------
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id) {
        try {
            var ok = await _agenceService.DeleteAsync(id);
            if (!ok) return NotFound();

            TempData["Success"] = "Agence supprimée avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (InvalidOperationException ex)
        {
            TempData["Error"] = ex.Message;
            return RedirectToAction(nameof(Index));
        }
    }
}
```

5.4. Les Vues

5.4.1. Index — liste paginée avec recherche

🔁 **Fichier :** `Views/Agences/Index.cshtml`

```
@model AkarangaBank.Web.ViewModels.AgenceListViewModel
@{
    ViewData["Title"] = "Agences";
}

<h1>Nos agences</h1>

@* Messages flash *@
@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}
@if (TempData["Error"] is string err)
{
    <div class="alert alert-danger">@err</div>
}

@* Barre de recherche + bouton créer *@
<form method="get" class="row g-2 mb-3">
    <div class="col-md-6">
        <input type="text" name="recherche" value="@Model.Recherche"
               class="form-control"
               placeholder="Rechercher par code, nom ou ville..." />
    </div>
    <div class="col-md-2">
        <button type="submit" class="btn btn-primary">Rechercher</bu
tton>
    </div>
    <div class="col-md-4 text-end">
        <a asp-action="Create" class="btn btn-success">+ Nouvelle ag
ence</a>
    </div>
</form>

<p class="text-muted">@Model.Total agence(s) trouvée(s).</p>

@* Tableau *@
<table class="table table-striped">
```

```
    <thead>
        <tr>
            <th>Code</th>
            <th>Nom</th>
            <th>Ville</th>
            <th>Statut</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
    @foreach (var agence in Model.Agences)
    {
        <tr>
            <td>@agence.Code</td>
            <td>@agence.Nom</td>
            <td>@agence.Ville</td>
            <td>
                @if (agence.EstSiege)
                {
                    <span class="badge bg-primary">Siège</span>
                }
                else
                {
                    <span class="badge bg-secondary">Agence</span>
                }
            </td>
            <td>
                <a asp-action="Details" asp-route-id="@agence.Id"
                   class="btn btn-sm btn-primary">Détails</a>
                <a asp-action="Edit" asp-route-id="@agence.Id"
                   class="btn btn-sm btn-warning">Modifier</a>
                <a asp-action="Delete" asp-route-id="@agence.Id"
                   class="btn btn-sm btn-danger">Supprimer</a>
            </td>
        </tr>
    }
    </tbody>
</table>
@* Pagination *@
@if (Model.TotalPages > 1)
{
    <nav>
        <ul class="pagination">
```

```
            @for (int i = 1; i <= Model.TotalPages; i++)
            {
                <li class="page-item @(i == Model.Page ? "active" : "")">
                    <a class="page-link"
                       asp-action="Index"
                       asp-route-page="@i"
                       asp-route-recherche="@Model.Recherche">@i</a>
                </li>
            }
        </ul>
    </nav>
}
```

5.4.2. Details — fiche détaillée

Fichier : `Views/Agences/Details.cshtml`

```
@model AkarangaBank.Web.ViewModels.AgenceDetailsViewModel
@{
    ViewData["Title"] = "Détails de l'agence";
}

<h1>Détails de l'agence</h1>

<dl class="row">
    <dt class="col-sm-3">Code</dt>
    <dd class="col-sm-9">@Model.Agence.Code</dd>

    <dt class="col-sm-3">Nom</dt>
    <dd class="col-sm-9">@Model.Agence.Nom</dd>

    <dt class="col-sm-3">Ville</dt>
    <dd class="col-sm-9">@Model.Agence.Ville</dd>

    <dt class="col-sm-3">Statut</dt>
    <dd class="col-sm-9">
        @if (Model.Agence.EstSiege)
        {
            <span class="badge bg-primary">Siège</span>
        }
        else
        {
            <span class="badge bg-secondary">Agence</span>
        }
```

```
    </dd>

    <dt class="col-sm-3">Date de création</dt>
    <dd class="col-sm-9">@Model.Agence.DateCreation.ToString("dd/MM/
        yyyy HH:mm")</dd>
</dl>

<h2>Activité de l'agence</h2>

<div class="row">
    <div class="col-md-4">
        <div class="card text-center">
            <div class="card-body">
                <h5 class="card-title">@Model.NombreClients</h5>
                <p class="card-text">Client(s)</p>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card text-center">
            <div class="card-body">
                <h5 class="card-title">@Model.NombreComptes</h5>
                <p class="card-text">Compte(s)</p>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card text-center">
            <div class="card-body">
                <h5 class="card-title">@Model.NombreUtilisateurs</h5>
                <p class="card-text">Utilisateur(s)</p>
            </div>
        </div>
    </div>
</div>

<div class="mt-3">
    <a asp-action="Edit" asp-route-id="@Model.Agence.Id" class="btn
       btn-warning">Modifier</a>
    <a asp-action="Index" class="btn btn-secondary">Retour à la liste</a>
</div>
```

5.3.3. Create — formulaire de création

Fichier à créer : `Views/Agences/Create.cshtml`

```
@model AkarangaBank.Web.ViewModels.AgenceCreateViewModel

@{
    ViewData["Title"] = "Nouvelle agence";
}

<h1>Nouvelle agence</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="Code" class="form-label"></label>
        <input asp-for="Code" class="form-control" placeholder="AKR-BJM" />
        <span asp-validation-for="Code" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Nom" class="form-label"></label>
        <input asp-for="Nom" class="form-control" />
        <span asp-validation-for="Nom" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Ville" class="form-label"></label>
        <input asp-for="Ville" class="form-control" />
        <span asp-validation-for="Ville" class="text-danger"></span>
    </div>

    <div class="form-check mb-3">
        <input asp-for="EstSiege" class="form-check-input" />
        <label asp-for="EstSiege" class="form-check-label"></label>
    </div>

    <button type="submit" class="btn btn-primary">Enregistrer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

5.4.4. Edit — formulaire de modification

Fichier à créer : `Views/Agences/Edit.cshtml`

```
@model AkarangaBank.Web.ViewModels.AgenceEditViewModel

@{
    ViewData["Title"] = "Modifier l'agence";
}

<h1>Modifier l'agence</h1>

<form asp-action="Edit" method="post">
    <input type="hidden" asp-for="Id" />
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="Code" class="form-label"></label>
        <input asp-for="Code" class="form-control" />
        <span asp-validation-for="Code" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Nom" class="form-label"></label>
        <input asp-for="Nom" class="form-control" />
        <span asp-validation-for="Nom" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Ville" class="form-label"></label>
        <input asp-for="Ville" class="form-control" />
        <span asp-validation-for="Ville" class="text-danger"></span>
    </div>

    <div class="form-check mb-3">
        <input asp-for="EstSiege" class="form-check-input" />
        <label asp-for="EstSiege" class="form-check-label"></label>
    </div>

    <button type="submit" class="btn btn-primary">Enregistrer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

5.5.5. Delete — confirmation de suppression

Fichier à créer : `Views/Agences/Delete.cshtml`

```
@model AkarangaBank.Web.Models.Agence
@{
    ViewData["Title"] = "Supprimer l'agence";
}

<h1>Supprimer l'agence</h1>

<div class="alert alert-warning">
    Confirmez-vous la suppression de cette agence ?
</div>

<dl class="row">
    <dt class="col-sm-3">Code</dt>
    <dd class="col-sm-9">@Model.Code</dd>

    <dt class="col-sm-3">Nom</dt>
    <dd class="col-sm-9">@Model.Nom</dd>

    <dt class="col-sm-3">Ville</dt>
    <dd class="col-sm-9">@Model.Ville</dd>
</dl>

<form asp-action="Delete" method="post">
    <input type="hidden" asp-for="Id" />
    <button type="submit" class="btn btn-danger">Supprimer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>
```

5.6. Tests manuels

Test 1 — Compilation

bash

```
dotnet build
```

Résultat attendu : `Build succeeded. 0 Error(s)`.

Test 2 — Liste des agences

Ouvrir `http://localhost:5001/Agences`.

Résultat attendu : les 4 agences s'affichent, triées avec le siège en premier.

Test 3 — Recherche

Taper `Gitega` dans la barre de recherche, valider.

Résultat attendu : seule l'agence de Gitega s'affiche.

Test 4 — Création
Cliquer sur « + Nouvelle agence », remplir :

- Code : AKR-MUY
- Nom : Akaranga Bank Muyinga
- Ville : Muyinga
- Est le siège : décoché

Valider.

Résultat attendu : message « Agence créée avec succès. », la nouvelle agence apparaît.

Test 5 — Validation du format du code

Essayer de créer une agence avec le code TEST.

Résultat attendu : message d'erreur « Le code doit respecter le format AKR-XXX ».

Test 6 — Unicité du code

Essayer de créer une agence avec le code AKR-BJM (déjà utilisé).

Résultat attendu : message d'erreur « Ce code d'agence est déjà utilisé. »

Test 7 — Modification

Modifier l'agence AKR-MUY, changer la ville en Muyinga Centre, valider.

Résultat attendu : message « Agence modifiée avec succès. », la modification est visible.

Test 8 — Détails avec compteurs

Ouvrir les détails de AKR-BJM.

Résultat attendu : la fiche affiche le code, nom, ville, siège, et les compteurs :

- Clients : 2 (Nshimirimana, Nduwimana)
- Comptes : 3
- Utilisateurs : 3 (admin, gestionnaire, auditeur)

Test 9 — Suppression impossible (intégrité référentielle)

Tenter de supprimer AKR-BJM.

Résultat attendu : message d'erreur « Impossible de supprimer cette agence : des clients, comptes ou utilisateurs y sont rattachés. »

Test 10 — Suppression réussie

Créer une agence AKR-TEST (sans dépendance), puis la supprimer.

Résultat attendu : message « Agence supprimée avec succès. »

Test 11 — Agence inexistante

Ouvrir /Agences/Details/9999.
Résultat attendu : page **404 Not Found**.

Test 12 — Protection CSRF

Tenter de soumettre le formulaire de suppression sans le token anti-forgery (via un outil comme Postman).

Résultat attendu : HTTP 400 Bad Request.

5.7. Exercices

Exercice 1 — Ajouter un filtre « Siège uniquement »

Sur la page /Agences, ajouter une case à cocher « Siège uniquement » qui filtre les agences.

Fichiers concernés :

- `ViewModels/AgenceListViewModel.cs` (ajouter SiègeUniquement)
- `Services/IAgenceService.cs` et `AgenceService.cs` (paramètre supplémentaire)
- `Controllers/AgencesController.cs`
- `Views/Agences/Index.cshtml`

Exercice 2 — Trier par colonne

Permettre le tri par code, nom ou ville en cliquant sur les en-têtes de colonnes.

Fichiers concernés :

- `ViewModels/AgenceListViewModel.cs` (ajouter `TriPar`, `Ordre`)
- `Services/AgenceService.cs`
- `Views/Agences/Index.cshtml`

Exercice 3 — Export CSV de la liste

Ajouter un bouton « Exporter en CSV » sur la page Index.

Fichiers concernés :

- `Controllers/AgencesController.cs` (action `ExportCsv`)
- `Views/Agences/Index.cshtml`

Exercice 4 — Afficher les clients d'une agence

Sur la page Détails, ajouter un tableau listant les clients de l'agence.

Fichiers concernés :

- `ViewModels/AgenceDetailsViewModel.cs` (ajouter `List<Client>`)
- `Services/AgenceService.cs`
- `Views/Agences/Details.cshtml`

Exercice 5 — Empêcher la suppression du siège

Modifier la règle métier : le siège ne peut jamais être supprimé, même sans dépendance.

Fichiers concernés :

- `Services/AgenceService.cs` (méthode `APeuEtreSupprimeeAsync`)

5.8. Questions de compréhension

1. Pourquoi utiliser un Service plutôt que d'accéder directement au DbContext depuis le contrôleur ?
2. Que fait [ValidateAntiForgeryToken] ? Contre quelle attaque protège-t-il ?
3. Que fait ModelState.IsValid ? Quand est-il rempli ?
4. Quelle est la différence entre une validation côté client et côté serveur ?
5. Pourquoi vérifier l'unicité du code à la fois dans le service ET côté base de données ?
6. Que fait AsNoTracking() ? Pourquoi l'utiliser dans une liste paginée ?
7. Comment fonctionne la pagination avec Skip et Take ?
8. À quoi sert TempData ? En quoi diffère-t-il de ViewData ?
9. Que fait NotFound() ? Quel code HTTP renvoie-t-il ?
10. Pourquoi la suppression d'une agence avec des clients est-elle refusée ?

5.9. Mini-TP 5 — CRUD complet des agences

Objectif : construire le CRUD complet des agences d'Akaranga Bank.

1. Créez les 4 ViewModels (List, Create, Edit, Details).
2. Créez IAgenceService et AgenceService.
3. Enregistrez le service dans Program.cs (AddScoped).
4. Réécrivez AgencesController avec les 6 actions (Index, Details, Create GET/POST, Edit GET/POST, Delete GET/POST).
5. Créez les 5 vues.
6. Testez tous les cas de la section 5.6.

CHAPITRE 6 — CRUD CLIENTS, KYC ET COMPTES

Objectifs du chapitre

- Comprendre les relations entre Clients, VerificationsKYC, Comptes et TypesCompte.
- Implémenter un CRUD complet sur les clients avec liste déroulante des agences.
- Gérer les vérifications KYC (création, validation, rejet).
- Ouvrir des comptes bancaires en respectant les règles métier.
- Filtrer par agence, par statut, par type de compte.
- Gérer les erreurs d'intégrité référentielle à tous les niveaux.

6.1. Les relations du modèle

À ce stade, le modèle `AkarangaBank-Script-DB.sql` est clair :

```
Agences     (1) ──< (N) Clients
Clients     (1) ──< (N) VerificationsKYC
Clients     (1) ──< (N) Comptes
TypesCompte (1) ──< (N) Comptes
Agences     (1) ──< (N) Comptes
```

6.1.1. Règles portées par la base

Table	Contrainte	Conséquence
`Clients.NumeroClient`	`UNIQUE`	Deux clients ne peuvent pas avoir le même numéro
`Clients.NumeroCNI`	`UNIQUE`	Deux clients ne peuvent pas avoir le même CNI
`Clients.AgenceId`	`NOT NULL + FK`	Un client doit être rattaché à une agence existante
`Clients.Statut`	`CHECK`	Seulement Actif, Suspendu, Cloture
`VerificationsKYC.Statut`	`CHECK`	Seulement EnAttente, Valide, Rejete, Expire
`Comptes.NumeroCompte`	`UNIQUE`	Deux comptes ne peuvent pas avoir le même numéro
`Comptes.Statut`	`CHECK`	Seulement Actif, Bloque, Cloture
`Comptes.Solde`	`DECIMAL(18,2)`	Jamais FLOAT
`TypesCompte.TauxInteret`	`CHECK`	Entre 0 et 100
`TypesCompte.SoldeMinimum`	`CHECK`	≥ 0

6.2. Architecture cible

```
Controllers/
├── ClientsController.cs          ← créé
├── VerificationsKYCController.cs ← créé
└── ComptesController.cs          ← créé

Services/
```

```
├── IClientService.cs              ← créé
├── ClientService.cs               ← créé
├── IVerificationKYCService.cs     ← créé
├── VerificationKYCService.cs      ← créé
├── ICompteService.cs              ← créé
└── CompteService.cs               ← créé

ViewModels/
├── ClientListViewModel.cs         ← créé
├── ClientCreateViewModel.cs       ← créé
├── ClientEditViewModel.cs         ← créé
├── ClientDetailsViewModel.cs      ← créé
├── VerificationKYCCreateViewModel.cs ← créé
├── CompteListViewModel.cs         ← créé
├── CompteCreateViewModel.cs       ← créé
├── CompteEditViewModel.cs         ← créé
└── CompteDetailsViewModel.cs      ← créé

Views/
├── Clients/           ← 5 vues
├── VerificationsKYC/  ← 3 vues
└── Comptes/           ← 5 vues
```

6.3. Le module Clients

6.3.1. ViewModels Clients

Fichier à créer : `ViewModels/ClientListViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class ClientListViewModel{
    public IEnumerable<Client> Clients { get; set; } = new
           List<Client>();
    public string? Recherche { get; set; }
    public int? AgenceId { get; set; }
    public string? Statut { get; set; }
    public int Page { get; set; } = 1;
    public int TaillePage { get; set; } = 10;
    public int Total { get; set; }
    public int TotalPages => (int)Math.Ceiling((double)Total / TaillePage);
    // Pour les listes déroulantes de filtre
    public IEnumerable<Agence> Agences { get; set; } = new List<Agence>();
}
```

Fichier à créer : `ViewModels/ClientCreateViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class ClientCreateViewModel
{
    [Required(ErrorMessage = "Le numéro de client est
obligatoire.")]
    [StringLength(20)]
    [Display(Name = "Numéro client")]
    [RegularExpression(@"^CLI-\d{6}$",
        ErrorMessage = "Le numéro doit respecter le format CLI-
NNNNNN.")]
    public string NumeroClient { get; set; } = string.Empty;

    [Required(ErrorMessage = "Le nom est obligatoire.")]
    [StringLength(100)]
    public string Nom { get; set; } = string.Empty;

    [Required(ErrorMessage = "Le prénom est obligatoire.")]
    [StringLength(100)]
    public string Prenom { get; set; } = string.Empty;

    [Required(ErrorMessage = "La date de naissance est
obligatoire.")]
    [DataType(DataType.Date)]
    [Display(Name = "Date de naissance")]
    public DateTime DateNaissance { get; set; } =
DateTime.Today.AddYears(-18);

    [Required(ErrorMessage = "Le numéro de CNI est obligatoire.")]
    [StringLength(30)]
    [Display(Name = "Numéro CNI")]
    public string NumeroCNI { get; set; } = string.Empty;

    [Phone]
    [StringLength(20)]
    public string? Telephone { get; set; }

    [EmailAddress]
    [StringLength(100)]
```

```
    public string? Email { get; set; }

    [StringLength(255)]
    public string? Adresse { get; set; }

    [Required(ErrorMessage = "L'agence est obligatoire.")]
    [Display(Name = "Agence")]
    public int AgenceId { get; set; }
}
```

Fichier à créer : `ViewModels/ClientEditViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class ClientEditViewModel
{
    public int Id { get; set; }

    [Required]
    [StringLength(20)]
    [Display(Name = "Numéro client")]
    public string NumeroClient { get; set; } = string.Empty;

    [Required]
    [StringLength(100)]
    public string Nom { get; set; } = string.Empty;

    [Required]
    [StringLength(100)]
    public string Prenom { get; set; } = string.Empty;

    [DataType(DataType.Date)]
    [Display(Name = "Date de naissance")]
    public DateTime DateNaissance { get; set; }

    [Required]
    [StringLength(30)]
    [Display(Name = "Numéro CNI")]
    public string NumeroCNI { get; set; } = string.Empty;
```

```
    [Phone]
    [StringLength(20)]
    public string? Telephone { get; set; }

    [EmailAddress]
    [StringLength(100)]
    public string? Email { get; set; }

    [StringLength(255)]
    public string? Adresse { get; set; }

    [Required]
    [Display(Name = "Agence")]
    public int AgenceId { get; set; }

    [Required]
    [StringLength(20)]
    public string Statut { get; set; } = "Actif";
}
```

Fichier à créer : `ViewModels/ClientDetailsViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class ClientDetailsViewModel
{
    public Client Client { get; set; } = null!;
    public IEnumerable<Compte> Comptes { get; set; } = new
List<Compte>();
    public IEnumerable<VerificationKYC> VerificationsKYC { get; set;
} = new List<VerificationKYC>();
    public decimal SoldeTotal => Comptes.Sum(c => c.Solde);
}
```

6.3.2. Service Clients

Fichier à créer : `Services/IClientService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;
```

```
public interface IClientService
{
    Task<ClientListViewModel> GetPagedAsync(string? recherche, int?
agenceId, string? statut, int page, int taille);
    Task<ClientDetailsViewModel?> GetDetailsAsync(int id);
    Task<Client?> GetByIdAsync(int id);
    Task<Client> CreateAsync(ClientCreateViewModel vm);
    Task<bool> UpdateAsync(ClientEditViewModel vm);
    Task<bool> DeleteAsync(int id);
    Task<bool> NumeroClientExisteAsync(string numeroClient, int?
excludeId = null);
    Task<bool> NumeroCNIExisteAsync(string numeroCNI, int? excludeId
= null);
}
```

Fichier à créer : `Services/ClientService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public class ClientService : IClientService {
    private readonly AkarangaBankContext _context;

    public ClientService(AkarangaBankContext context) {
        _context = context;
    }

    public async Task<ClientListViewModel> GetPagedAsync(
     string? recherche, int? agenceId, string? statut,int page,int taille)
    {
        var query = _context.Clients
            .AsNoTracking()
            .Include(c => c.Agence)
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(recherche)){
            var r = recherche.Trim();
            query = query.Where(c =>
```

```
                c.NumeroClient.Contains(r) ||
                c.Nom.Contains(r) ||
                c.Prenom.Contains(r) ||
                c.NumeroCNI.Contains(r) ||
                (c.Email != null && c.Email.Contains(r)));
        }

        if (agenceId.HasValue)
            query = query.Where(c => c.AgenceId == agenceId.Value);

        if (!string.IsNullOrWhiteSpace(statut))
            query = query.Where(c => c.Statut == statut);

        var total = await query.CountAsync();

        var clients = await query
            .OrderBy(c => c.Nom)
            .ThenBy(c => c.Prenom)
            .Skip((page - 1) * taille)
            .Take(taille)
            .ToListAsync();

        // Liste des agences pour le filtre
        var agences = await _context.Agences
            .AsNoTracking()
            .OrderByDescending(a => a.EstSiege)
            .ThenBy(a => a.Ville)
            .ToListAsync();

        return new ClientListViewModel
        {
            Clients = clients,
            Recherche = recherche,
            AgenceId = agenceId,
            Statut = statut,
            Page = page,
            TaillePage = taille,
            Total = total,
            Agences = agences
        };
    }
```

```
    public async Task<ClientDetailsViewModel?> GetDetailsAsync(int id)
    {
        var client = await _context.Clients
            .AsNoTracking()
            .Include(c => c.Agence)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (client is null) return null;

        var comptes = await _context.Comptes
            .AsNoTracking()
            .Include(c => c.TypeCompte)
            .Where(c => c.ClientId == id)
            .OrderBy(c => c.NumeroCompte)
            .ToListAsync();

        var kyc = await _context.VerificationsKYC
            .AsNoTracking()
            .Include(v => v.VerifiePar)
            .Where(v => v.ClientId == id)
            .OrderByDescending(v => v.DateVerification)
            .ToListAsync();

        return new ClientDetailsViewModel
        {
            Client = client,
            Comptes = comptes,
            VerificationsKYC = kyc
        };
    }

    public async Task<Client?> GetByIdAsync(int id)
    {
        return await _context.Clients
            .AsNoTracking()
            .FirstOrDefaultAsync(c => c.Id == id);
    }

    public async Task<Client> CreateAsync(ClientCreateViewModel vm)
    {
        var client = new Client
        {
```

```
            NumeroClient = vm.NumeroClient.Trim().ToUpperInvariant(),
            Nom = vm.Nom.Trim(),
            Prenom = vm.Prenom.Trim(),
            DateNaissance = vm.DateNaissance,
            NumeroCNI = vm.NumeroCNI.Trim().ToUpperInvariant(),
            Telephone = vm.Telephone?.Trim(),
            Email = vm.Email?.Trim(),
            Adresse = vm.Adresse?.Trim(),
            AgenceId = vm.AgenceId,
            Statut = "Actif",
            DateCreation = DateTime.UtcNow
        };

        _context.Clients.Add(client);
        await _context.SaveChangesAsync();
        return client;
    }

    public async Task<bool> UpdateAsync(ClientEditViewModel vm)
    {
        var client = await _context.Clients.FindAsync(vm.Id);
        if (client is null) return false;

        client.NumeroClient = vm.NumeroClient.Trim().ToUpperInvariant();
        client.Nom = vm.Nom.Trim();
        client.Prenom = vm.Prenom.Trim();
        client.DateNaissance = vm.DateNaissance;
        client.NumeroCNI = vm.NumeroCNI.Trim().ToUpperInvariant();
        client.Telephone = vm.Telephone?.Trim();
        client.Email = vm.Email?.Trim();
        client.Adresse = vm.Adresse?.Trim();
        client.AgenceId = vm.AgenceId;
        client.Statut = vm.Statut;
        client.DateModification = DateTime.UtcNow;

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var client = await _context.Clients
            .Include(c => c.Comptes)
```

```
            .Include(c => c.VerificationsKYC)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (client is null) return false;

        if (client.Comptes.Any())
            throw new InvalidOperationException(
              "Impossible de supprimer ce client : il possède des comptes bancaires.");

        // Suppression des vérifications KYC rattachées
        if (client.VerificationsKYC.Any())
           _context.VerificationsKYC.RemoveRange(client.VerificationsKYC);

        _context.Clients.Remove(client);
        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> NumeroClientExisteAsync(string
         numeroClient, int? excludeId = null)
    {
        var query = _context.Clients.Where(c => c.NumeroClient ==
                    numeroClient);
        if (excludeId.HasValue)
            query = query.Where(c => c.Id != excludeId.Value);
        return await query.AnyAsync();
    }

    public async Task<bool> NumeroCNIExisteAsync(string numeroCNI,
         int? excludeId = null)
    {
        var query = _context.Clients.Where(c => c.NumeroCNI == numeroCNI);
        if (excludeId.HasValue)
            query = query.Where(c => c.Id != excludeId.Value);
        return await query.AnyAsync();
    }
}
```

6.3.3. Contrôleur Clients

Fichier à créer : `Controllers/ClientsController.cs`

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
```

```
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

public class ClientsController : Controller
{
    private readonly IClientService _clientService;
    private readonly IAgenceService _agenceService;

    public ClientsController(IClientService clientService,
        IAgenceService agenceService)
    {
        _clientService = clientService;
        _agenceService = agenceService;
    }

    // GET: /Clients
    public async Task<IActionResult> Index(
        string? recherche, int? agenceId, string? statut, int page = 1)
    {
        const int taille = 10;
        var vm = await _clientService.GetPagedAsync(recherche,
                  agenceId, statut, page, taille);
        return View(vm);
    }

    // GET: /Clients/Details/5
    public async Task<IActionResult> Details(int id)
    {
        var vm = await _clientService.GetDetailsAsync(id);
        if (vm is null) return NotFound();
        return View(vm);
    }

    // GET: /Clients/Create
    public async Task<IActionResult> Create()
    {
        await ChargerAgencesAsync();
        return View(new ClientCreateViewModel());
    }
```

```
    // POST: /Clients/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(ClientCreateViewModel vm)
    {
        if (!ModelState.IsValid)
        {
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }

        if (await _clientService.NumeroClientExisteAsync(
           vm.NumeroClient.Trim().ToUpperInvariant())) {
            ModelState.AddModelError(nameof(vm.NumeroClient), "Ce
                numéro de client est déjà utilisé.");
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }

        if (await _clientService.NumeroCNIExisteAsync(
         vm.NumeroCNI.Trim().ToUpperInvariant()))
        {
            ModelState.AddModelError(nameof(vm.NumeroCNI), "Ce
                numéro de CNI est déjà utilisé.");
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }

        try {
            await _clientService.CreateAsync(vm);
            TempData["Success"] = "Client créé avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (DbUpdateException ex)
            when (ex.InnerException is SqlException sql &&
            sql.Number == 2601)
        {
            ModelState.AddModelError("", "Un client avec ce numéro
                ou ce CNI existe déjà.");
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }
    }
```

```
    // GET: /Clients/Edit/5
    public async Task<IActionResult> Edit(int id) {
        var client = await _clientService.GetByIdAsync(id);
        if (client is null) return NotFound();

        var vm = new ClientEditViewModel {
            Id = client.Id,
            NumeroClient = client.NumeroClient,
            Nom = client.Nom,
            Prenom = client.Prenom,
            DateNaissance = client.DateNaissance,
            NumeroCNI = client.NumeroCNI,
            Telephone = client.Telephone,
            Email = client.Email,
            Adresse = client.Adresse,
            AgenceId = client.AgenceId,
            Statut = client.Statut
        };

        await ChargerAgencesAsync(vm.AgenceId);
        return View(vm);
    }

    // POST: /Clients/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id,
    ClientEditViewModel vm) {
        if (id != vm.Id) return BadRequest();
        if (!ModelState.IsValid) {
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }

        if (await _clientService.NumeroClientExisteAsync(
         vm.NumeroClient.Trim().ToUpperInvariant(), vm.Id)) {
            ModelState.AddModelError(nameof(vm.NumeroClient), "Ce
              numéro de client est déjà utilisé.");
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }
```

```
        if (await _clientService.NumeroCNIExisteAsync(
          vm.NumeroCNI.Trim().ToUpperInvariant(), vm.Id)) {
            ModelState.AddModelError(nameof(vm.NumeroCNI), "Ce
              numéro de CNI est déjà utilisé.");
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }

        var ok = await _clientService.UpdateAsync(vm);
        if (!ok) return NotFound();

        TempData["Success"] = "Client modifié avec succès.";
        return RedirectToAction(nameof(Index));
    }

    // GET: /Clients/Delete/5
    public async Task<IActionResult> Delete(int id) {
        var vm = await _clientService.GetDetailsAsync(id);
        if (vm is null) return NotFound();
        return View(vm);
    }

    // POST: /Clients/Delete/5
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id) {
        try{
            var ok = await _clientService.DeleteAsync(id);
            if (!ok) return NotFound();

            TempData["Success"] = "Client supprimé avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (InvalidOperationException ex) {
            TempData["Error"] = ex.Message;
            return RedirectToAction(nameof(Index));
        }
    }

    private async Task ChargerAgencesAsync(int? selectedId = null){
        var agencesVm = await _agenceService.GetPagedAsync(null, 1,
                        int.MaxValue);
        ViewBag.Agences = new SelectList(
```

```
            agencesVm.Agences.Select(a => new
            {
                a.Id,
                Libelle = $"{a.Code} — {a.Nom}"
            }),
            "Id", "Libelle", selectedId);
    }
}
```

6.3.4. Vues Clients

Fichier à créer : `Views/Clients/Index.cshtml`

```
@model AkarangaBank.Web.ViewModels.ClientListViewModel
@{
    ViewData["Title"] = "Clients";
}

<h1>Clients</h1>

@if (TempData["Success"] is string msg) {
    <div class="alert alert-success">@msg</div>
}
@if (TempData["Error"] is string err) {
    <div class="alert alert-danger">@err</div>
}

<form method="get" class="row g-2 mb-3">
    <div class="col-md-4">
        <input type="text" name="recherche" value="@Model.Recherche"
               class="form-control" placeholder="Rechercher par nom,
                     numéro, CNI..." />
    </div>
    <div class="col-md-3">
        <select name="agenceId" class="form-select">
            <option value="">-- Toutes les agences --</option>
            @foreach (var agence in Model.Agences) {
                <option value="@agence.Id"
                  selected="@(Model.AgenceId == agence.Id)">
                    @agence.Code — @agence.Ville
                </option>
            }
        </select>
    </div>
```

```
    <div class="col-md-2">
        <select name="statut" class="form-select">
            <option value="">-- Tous statuts --</option>
            <option value="Actif" selected="@(Model.Statut ==
                   "Actif")">Actif</option>
            <option value="Suspendu" selected="@(Model.Statut ==
                   "Suspendu")">Suspendu</option>
            <option value="Cloture" selected="@(Model.Statut ==
                   "Cloture")">Clôturé</option>
        </select>
    </div>
    <div class="col-md-1">
        <button type="submit" class="btn btn-primary w-
                  100">Filtrer</button>
    </div>
    <div class="col-md-2 text-end">
        <a asp-action="Create" class="btn btn-success">+ Nouveau
           client</a>
    </div>
</form>

<p class="text-muted">@Model.Total client(s) trouvé(s).</p>

<table class="table table-striped">
    <thead>
        <tr>
            <th>Numéro</th>
            <th>Nom</th>
            <th>Prénom</th>
            <th>CNI</th>
            <th>Agence</th>
            <th>Statut</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
    @foreach (var client in Model.Clients)
    {
        <tr>
            <td>@client.NumeroClient</td>
            <td>@client.Nom</td>
            <td>@client.Prenom</td>
            <td>@client.NumeroCNI</td>
```

```
            <td>@client.Agence.Code</td>
            <td>
                <span class="badge @(client.Statut == "Actif" ? "bg-
                  success" : client.Statut == "Suspendu" ? "bg-
                  warning" : "bg-secondary")">
                    @client.Statut
                </span>
            </td>
            <td>
                <a asp-action="Details" asp-route-id="@client.Id"
                   class="btn btn-sm btn-primary">Détails</a>
                <a asp-action="Edit" asp-route-id="@client.Id"
                   class="btn btn-sm btn-warning">Modifier</a>
                <a asp-action="Delete" asp-route-id="@client.Id"
                   class="btn btn-sm btn-danger">Supprimer</a>
            </td>
        </tr>
    }
    </tbody>
</table>

@if (Model.TotalPages > 1)
{
    <nav>
        <ul class="pagination">
            @for (int i = 1; i <= Model.TotalPages; i++)
            {
               <li class="page-item @(i == Model.Page ? "active" : "")">
                     <a class="page-link"
                        asp-action="Index"
                        asp-route-page="@i"
                        asp-route-recherche="@Model.Recherche"
                        asp-route-agenceId="@Model.AgenceId"
                        asp-route-statut="@Model.Statut">@i</a>
                </li>
            }
        </ul>
    </nav>
}
```

Fichier à créer : `Views/Clients/Create.cshtml`

```
@model AkarangaBank.Web.ViewModels.ClientCreateViewModel
@{
    ViewData["Title"] = "Nouveau client";
}

<h1>Nouveau client</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="NumeroClient" class="form-label"></label>
        <input asp-for="NumeroClient" class="form-control"
             placeholder="CLI-000006" />
       <span asp-validation-for="NumeroClient" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Nom" class="form-label"></label>
        <input asp-for="Nom" class="form-control" />
        <span asp-validation-for="Nom" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Prenom" class="form-label"></label>
        <input asp-for="Prenom" class="form-control" />
        <span asp-validation-for="Prenom" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="DateNaissance" class="form-label"></label>
       <input asp-for="DateNaissance" type="date" class="form-control" />
       <span asp-validation-for="DateNaissance" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="NumeroCNI" class="form-label"></label>
        <input asp-for="NumeroCNI" class="form-control"
               placeholder="BJM-1234567" />
        <span asp-validation-for="NumeroCNI" class="text-
              danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Telephone" class="form-label"></label>
        <input asp-for="Telephone" class="form-control" />
        <span asp-validation-for="Telephone" class="text-danger"></span>
```

```
    </div>
    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Adresse" class="form-label"></label>
        <input asp-for="Adresse" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="AgenceId" class="form-label"></label>
        <select asp-for="AgenceId" asp-items="ViewBag.Agences"
           class="form-select">
            <option value="">-- Sélectionner une agence --</option>
        </select>
        <span asp-validation-for="AgenceId" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-
             primary">Enregistrer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Fichier à créer : `Views/Clients/Edit.cshtml`

```
@model AkarangaBank.Web.ViewModels.ClientEditViewModel
@{
    ViewData["Title"] = "Modifier le client";
}

<h1>Modifier le client</h1>

<form asp-action="Edit" method="post">
    <input type="hidden" asp-for="Id" />
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="NumeroClient" class="form-label"></label>
        <input asp-for="NumeroClient" class="form-control" />
```

```
        <span asp-validation-for="NumeroClient" class="text-
              danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Nom" class="form-label"></label>
        <input asp-for="Nom" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Prenom" class="form-label"></label>
        <input asp-for="Prenom" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="DateNaissance" class="form-label"></label>
       <input asp-for="DateNaissance" type="date" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="NumeroCNI" class="form-label"></label>
        <input asp-for="NumeroCNI" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Telephone" class="form-label"></label>
        <input asp-for="Telephone" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Adresse" class="form-label"></label>
        <input asp-for="Adresse" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="AgenceId" class="form-label"></label>
        <select asp-for="AgenceId" asp-items="ViewBag.Agences"
              class="form-select"></select>
    </div>
    <div class="mb-3">
        <label asp-for="Statut" class="form-label"></label>
        <select asp-for="Statut" class="form-select">
            <option value="Actif">Actif</option>
            <option value="Suspendu">Suspendu</option>
            <option value="Cloture">Clôturé</option>
        </select>
```

```
    </div>

    <button type="submit" class="btn btn-
             primary">Enregistrer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Fichier à créer : `Views/Clients/Details.cshtml`

```
@model AkarangaBank.Web.ViewModels.ClientDetailsViewModel
@{
    ViewData["Title"] = "Détails du client";
}

<h1>@Model.Client.Nom @Model.Client.Prenom</h1>

<dl class="row">
    <dt class="col-sm-3">Numéro client</dt>
    <dd class="col-sm-9">@Model.Client.NumeroClient</dd>

    <dt class="col-sm-3">Date de naissance</dt>
    <dd class="col-sm-9">
        @Model.Client.DateNaissance.ToString("dd/MM/yyyy")</dd>

    <dt class="col-sm-3">CNI</dt>
    <dd class="col-sm-9">@Model.Client.NumeroCNI</dd>

    <dt class="col-sm-3">Téléphone</dt>
    <dd class="col-sm-9">@Model.Client.Telephone</dd>

    <dt class="col-sm-3">Email</dt>
    <dd class="col-sm-9">@Model.Client.Email</dd>

    <dt class="col-sm-3">Adresse</dt>
    <dd class="col-sm-9">@Model.Client.Adresse</dd>

    <dt class="col-sm-3">Agence</dt>
    <dd class="col-sm-9">
```

```
        <a asp-controller="Agences" asp-action="Details" asp-route-
           id="@Model.Client.AgenceId">
               @Model.Client.Agence.Code — @Model.Client.Agence.Nom
        </a>
    </dd>

    <dt class="col-sm-3">Statut</dt>
    <dd class="col-sm-9">
        <span class="badge @(Model.Client.Statut == "Actif" ? "bg-
              success" : "bg-warning")">
            @Model.Client.Statut
        </span>
    </dd>
</dl>

<h2>Comptes bancaires</h2>

@if (!Model.Comptes.Any())
{
    <p class="text-muted">Ce client ne possède aucun compte.</p>
}
else
{
    <table class="table table-sm">
        <thead>
            <tr>
                <th>Numéro</th>
                <th>Type</th>
                <th>Solde</th>
                <th>Devise</th>
                <th>Statut</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
        @foreach (var compte in Model.Comptes)
        {
            <tr>
                <td>@compte.NumeroCompte</td>
                <td>@compte.TypeCompte.Libelle</td>
                <td>@compte.Solde.ToString("N2")</td>
                <td>@compte.Devise</td>
                <td>@compte.Statut</td>
```

```
                <td>
                    <a asp-controller="Comptes" asp-action="Details"
                       asp-route-id="@compte.Id" class="btn btn-sm
                        btn-primary">Détails</a>
                </td>
            </tr>
        }
        </tbody>
    </table>

    <p><strong>Solde total :</strong>
           @Model.SoldeTotal.ToString("N2") BIF</p>
}

<h2>Vérifications KYC</h2>

@if (!Model.VerificationsKYC.Any())
{
    <p class="text-muted">Aucune vérification KYC enregistrée.</p>
}
else
{
    <table class="table table-sm">
        <thead>
            <tr>
                <th>Type</th>
                <th>Numéro</th>
                <th>Statut</th>
                <th>Vérifié par</th>
                <th>Date</th>
            </tr>
        </thead>
        <tbody>
        @foreach (var kyc in Model.VerificationsKYC)
        {
            <tr>
                <td>@kyc.TypeDocument</td>
                <td>@kyc.NumeroDocument</td>
                <td>
                    <span class="badge @(kyc.Statut == "Valide" ?
                        "bg-success" : kyc.Statut == "Rejete" ? "bg-
                        danger" : kyc.Statut == "Expire" ? "bg-
                        secondary" : "bg-warning")">
```

```
                        @kyc.Statut
                    </span>
                </td>
                <td>@(kyc.VerifiePar?.NomComplet ?? "—")</td>
                <td>@(kyc.DateVerification?.ToString("dd/MM/yyyy")
                    ?? "—")</td>
            </tr>
        }
        </tbody>
    </table>
}

<div class="mt-3">
    <a asp-controller="VerificationsKYC" asp-action="Create"
       asp-route-clientId="@Model.Client.Id" class="btn btn-info">+
          Nouvelle vérification KYC</a>
    <a asp-controller="Comptes" asp-action="Create"
       asp-route-clientId="@Model.Client.Id" class="btn btn-
          success">+ Ouvrir un compte</a>
    <a asp-action="Edit" asp-route-id="@Model.Client.Id" class="btn
          btn-warning">Modifier</a>
    <a asp-action="Index" class="btn btn-secondary">Retour à la liste</a>
</div>
```

Fichier à créer : `Views/Clients/Delete.cshtml`

```
@model AkarangaBank.Web.ViewModels.ClientDetailsViewModel
@{
    ViewData["Title"] = "Supprimer le client";
}

<h1>Supprimer le client</h1>

<div class="alert alert-warning">
    Confirmez-vous la suppression de ce client ?
</div>

<dl class="row">
    <dt class="col-sm-3">Numéro client</dt>
    <dd class="col-sm-9">@Model.Client.NumeroClient</dd>

    <dt class="col-sm-3">Nom complet</dt>
    <dd class="col-sm-9">@Model.Client.Nom @Model.Client.Prenom</dd>
```

```
    <dt class="col-sm-3">CNI</dt>
    <dd class="col-sm-9">@Model.Client.NumeroCNI</dd>

    <dt class="col-sm-3">Comptes</dt>
    <dd class="col-sm-9">@Model.Comptes.Count()</dd>
</dl>

@if (Model.Comptes.Any())
{
    <div class="alert alert-danger">
        Ce client possède @Model.Comptes.Count() compte(s). La
suppression sera refusée.
    </div>
}

<form asp-action="Delete" method="post">
    <input type="hidden" asp-for="Client.Id" name="id" />
    <button type="submit" class="btn btn-danger">Supprimer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>
```

6.3.5. Enregistrement du service

➡ **Fichier :** `Program.cs` — ajouter :

```
builder.Services.AddScoped<IClientService, ClientService>();
```

6.4. Le module Vérifications KYC

6.4.1. ViewModel

📄 **Fichier à créer :** `ViewModels/VerificationKYCCreateViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class VerificationKYCCreateViewModel
{
    [Required]
    public int ClientId { get; set; }

    [Required(ErrorMessage = "Le type de document est
obligatoire.")]
    [StringLength(50)]
```

```
    [Display(Name = "Type de document")]
    public string TypeDocument { get; set; } = "CNI";

    [Required(ErrorMessage = "Le numéro de document est
obligatoire.")]
    [StringLength(50)]
    [Display(Name = "Numéro de document")]
    public string NumeroDocument { get; set; } = string.Empty;

    [StringLength(500)]
    [Display(Name = "Chemin du document")]
    public string? CheminDocument { get; set; }

    [StringLength(500)]
    public string? Commentaire { get; set; }

    // Pour l'affichage
    public string? ClientNom { get; set; }
    public string? ClientNumero { get; set; }
}
```

6.4.2. Service

Fichier à créer : `Services/IVerificationKYCService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public interface IVerificationKYCService
{
    Task<VerificationKYC> CreateAsync(VerificationKYCCreateViewModel
vm, int? verifieParId);
    Task<bool> ValiderAsync(int id, int verifieParId);
    Task<bool> RejeterAsync(int id, int verifieParId, string
commentaire);
    Task<VerificationKYC?> GetByIdAsync(int id);
}
```

Fichier à créer : `Services/VerificationKYCService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public class VerificationKYCService : IVerificationKYCService
{
    private readonly AkarangaBankContext _context;

    public VerificationKYCService(AkarangaBankContext context)
    {
        _context = context;
    }

    public async Task<VerificationKYC> CreateAsync(
        VerificationKYCCreateViewModel vm, int? verifieParId)
    {
        var kyc = new VerificationKYC
        {
            ClientId = vm.ClientId,
            TypeDocument = vm.TypeDocument.Trim(),
            NumeroDocument =
vm.NumeroDocument.Trim().ToUpperInvariant(),
            CheminDocument = vm.CheminDocument?.Trim(),
            Commentaire = vm.Commentaire?.Trim(),
            Statut = "EnAttente"
        };

        _context.VerificationsKYC.Add(kyc);
        await _context.SaveChangesAsync();
        return kyc;
    }

    public async Task<bool> ValiderAsync(int id, int verifieParId)
    {
        var kyc = await _context.VerificationsKYC.FindAsync(id);
        if (kyc is null) return false;

        kyc.Statut = "Valide";
        kyc.VerifieParId = verifieParId;
```

```
        kyc.DateVerification = DateTime.UtcNow;

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> RejeterAsync(int id, int verifieParId,
string commentaire)
    {
        var kyc = await _context.VerificationsKYC.FindAsync(id);
        if (kyc is null) return false;

        kyc.Statut = "Rejete";
        kyc.VerifieParId = verifieParId;
        kyc.DateVerification = DateTime.UtcNow;
        kyc.Commentaire = commentaire;

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<VerificationKYC?> GetByIdAsync(int id)
    {
        return await _context.VerificationsKYC
            .Include(v => v.Client)
            .Include(v => v.VerifiePar)
            .AsNoTracking()
            .FirstOrDefaultAsync(v => v.Id == id);
    }
}
```

6.4.3. Contrôleur

Fichier à créer : `Controllers/VerificationsKYCController.cs`

```
using Microsoft.AspNetCore.Mvc;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

public class VerificationsKYCController : Controller {
    private readonly IVerificationKYCService _kycService;
    private readonly IClientService _clientService;
```

```
public VerificationsKYCController(
    IVerificationKYCService kycService,
    IClientService clientService) {
    _kycService = kycService;
    _clientService = clientService;
}

// GET: /VerificationsKYC/Create?clientId=5
public async Task<IActionResult> Create(int clientId) {
    var client = await _clientService.GetByIdAsync(clientId);
    if (client is null) return NotFound();

    return View(new VerificationKYCCreateViewModel {
        ClientId = client.Id,
        ClientNom = $"{client.Nom} {client.Prenom}",
        ClientNumero = client.NumeroClient
    });
}

// POST: /VerificationsKYC/Create
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult>
     Create(VerificationKYCCreateViewModel vm) {
    if (!ModelState.IsValid) return View(vm);

    // L'utilisateur vérificateur est fictif pour l'instant
       (id=2 : gestionnaire).
    // L'identité réelle sera branchée au chapitre 7.
    const int verifieParId = 2;

    await _kycService.CreateAsync(vm, verifieParId);
    TempData["Success"] = "Vérification KYC créée. En attente de validation.";
    return RedirectToAction("Details", "Clients", new { id =
           vm.ClientId });
}

// POST: /VerificationsKYC/Valider/5
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Valider(int id) {
    const int verifieParId = 2;
```

```
        var ok = await _kycService.ValiderAsync(id, verifieParId);
        if (!ok) return NotFound();

        TempData["Success"] = "Vérification KYC validée.";
        return RedirectToAction("Index", "Clients");
    }

    // POST: /VerificationsKYC/Rejeter/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Rejeter(int id, string commentaire)
    {
        const int verifieParId = 2;
        var ok = await _kycService.RejeterAsync(id, verifieParId,
                 commentaire ?? "");
        if (!ok) return NotFound();

        TempData["Success"] = "Vérification KYC rejetée.";
        return RedirectToAction("Index", "Clients");
    }
}
```

6.4.4. Vue

Fichier à créer : `Views/VerificationsKYC/Create.cshtml`

```
@model AkarangaBank.Web.ViewModels.VerificationKYCCreateViewModel
@{
    ViewData["Title"] = "Nouvelle vérification KYC";
}

<h1>Nouvelle vérification KYC</h1>

<p class="text-muted">
    Client : <strong>@Model.ClientNumero — @Model.ClientNom</strong>
</p>

<form asp-action="Create" method="post">
    <input type="hidden" asp-for="ClientId" />
    <input type="hidden" asp-for="ClientNom" />
    <input type="hidden" asp-for="ClientNumero" />
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

```
    <div class="mb-3">
        <label asp-for="TypeDocument" class="form-label"></label>
        <select asp-for="TypeDocument" class="form-select">
            <option value="CNI">CNI</option>
            <option value="Passeport">Passeport</option>
            <option value="Permis">Permis de conduire</option>
        </select>
    </div>
    <div class="mb-3">
        <label asp-for="NumeroDocument" class="form-label"></label>
        <input asp-for="NumeroDocument" class="form-control" />
        <span asp-validation-for="NumeroDocument" class="text-
              danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="CheminDocument" class="form-label"></label>
        <input asp-for="CheminDocument" class="form-control"
              placeholder="/uploads/kyc/..." />
    </div>
    <div class="mb-3">
        <label asp-for="Commentaire" class="form-label"></label>
        <textarea asp-for="Commentaire" class="form-
              control"></textarea>
    </div>

    <button type="submit" class="btn btn-
              primary">Enregistrer</button>
    <a asp-controller="Clients" asp-action="Details" asp-route-
              id="@Model.ClientId"
       class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

6.4.5. Enregistrement

➖ **Fichier :** `Program.cs` — ajouter :

```
builder.Services.AddScoped<IVerificationKYCService,
VerificationKYCService>();
```

6.5. Le module Comptes

6.5.1. ViewModels Comptes

Fichier à créer : `ViewModels/CompteListViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class CompteListViewModel
{
    public IEnumerable<Compte> Comptes { get; set; } = new
List<Compte>();
    public string? Recherche { get; set; }
    public int? AgenceId { get; set; }
    public int? TypeCompteId { get; set; }
    public string? Statut { get; set; }
    public int Page { get; set; } = 1;
    public int TaillePage { get; set; } = 10;
    public int Total { get; set; }
    public int TotalPages => (int)Math.Ceiling((double)Total /
           TaillePage);

    public IEnumerable<Agence> Agences { get; set; } = new
           List<Agence>();
    public IEnumerable<TypeCompte> TypesCompte { get; set; } = new
           List<TypeCompte>();
}
```

Fichier à créer : `ViewModels/CompteCreateViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class CompteCreateViewModel
{
    [Required(ErrorMessage = "Le numéro de compte est obligatoire.")]
    [StringLength(30)]
    [Display(Name = "Numéro de compte")]
    [RegularExpression(@"^[A-Z]{3}-\d{4}-\d{4}$",
        ErrorMessage = "Le numéro doit respecter le format XXX-NNNN-NNNN.")]
    public string NumeroCompte { get; set; } = string.Empty;

    [Required(ErrorMessage = "Le client est obligatoire.")]
```

```
    [Display(Name = "Client")]
    public int ClientId { get; set; }

    [Required(ErrorMessage = "Le type de compte est obligatoire.")]
    [Display(Name = "Type de compte")]
    public int TypeCompteId { get; set; }

    [Required(ErrorMessage = "L'agence est obligatoire.")]
    [Display(Name = "Agence")]
    public int AgenceId { get; set; }

    [Range(0, double.MaxValue, ErrorMessage = "Le solde initial doit
              être positif ou nul.")]
    [Display(Name = "Solde initial (BIF)")]
    public decimal SoldeInitial { get; set; }
}
```

Fichier à créer : `ViewModels/CompteEditViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class CompteEditViewModel
{
    public int Id { get; set; }

    [Required]
    [StringLength(30)]
    [Display(Name = "Numéro de compte")]
    public string NumeroCompte { get; set; } = string.Empty;

    [Required]
    [Display(Name = "Type de compte")]
    public int TypeCompteId { get; set; }

    [Required]
    [Display(Name = "Agence")]
    public int AgenceId { get; set; }

    [Required]
    [StringLength(20)]
    public string Statut { get; set; } = "Actif";
```

```
    // Affichage seul
    public string? ClientNom { get; set; }
    public decimal Solde { get; set; }
    public string Devise { get; set; } = "BIF";
}
```

Fichier à créer : `ViewModels/CompteDetailsViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class CompteDetailsViewModel
{
    public Compte Compte { get; set; } = null!;
    public IEnumerable<Ecriture> Ecritures { get; set; } = new
           List<Ecriture>();
}
```

6.5.2. Service Comptes

Fichier à créer : `Services/ICompteService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public interface ICompteService
{
    Task<CompteListViewModel> GetPagedAsync(
        string? recherche, int? agenceId, int? typeCompteId, string?
statut, int page, int taille);
    Task<CompteDetailsViewModel?> GetDetailsAsync(int id);
    Task<Compte?> GetByIdAsync(int id);
    Task<Compte> CreateAsync(CompteCreateViewModel vm);
    Task<bool> UpdateAsync(CompteEditViewModel vm);
    Task<bool> DeleteAsync(int id);
    Task<bool> NumeroCompteExisteAsync(string numeroCompte, int?
excludeId = null);
}
```

Fichier à créer : `Services/CompteService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public class CompteService : ICompteService {
    private readonly AkarangaBankContext _context;

    public CompteService(AkarangaBankContext context) {
        _context = context;
    }

    public async Task<CompteListViewModel> GetPagedAsync(
        string? recherche, int? agenceId, int? typeCompteId, string?
        statut, int page, int taille) {
        var query = _context.Comptes
            .AsNoTracking()
            .Include(c => c.Client)
            .Include(c => c.TypeCompte)
            .Include(c => c.Agence)
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(recherche)) {
            var r = recherche.Trim();
            query = query.Where(c =>
                c.NumeroCompte.Contains(r) ||
                c.Client.Nom.Contains(r) ||
                c.Client.Prenom.Contains(r) ||
                c.Client.NumeroClient.Contains(r));
        }

        if (agenceId.HasValue)
            query = query.Where(c => c.AgenceId == agenceId.Value);

        if (typeCompteId.HasValue)
            query = query.Where(c => c.TypeCompteId ==
                    typeCompteId.Value);

        if (!string.IsNullOrWhiteSpace(statut))
```

```
            query = query.Where(c => c.Statut == statut);

        var total = await query.CountAsync();

        var comptes = await query
            .OrderBy(c => c.NumeroCompte)
            .Skip((page - 1) * taille)
            .Take(taille)
            .ToListAsync();

        var agences = await _context.Agences
            .AsNoTracking()
            .OrderBy(a => a.Ville)
            .ToListAsync();

        var types = await _context.TypesCompte
            .AsNoTracking()
            .OrderBy(t => t.Code)
            .ToListAsync();

        return new CompteListViewModel {
            Comptes = comptes,
            Recherche = recherche,
            AgenceId = agenceId,
            TypeCompteId = typeCompteId,
            Statut = statut,
            Page = page,
            TaillePage = taille,
            Total = total,
            Agences = agences,
            TypesCompte = types
        };
    }

    public async Task<CompteDetailsViewModel?> GetDetailsAsync(int id){
        var compte = await _context.Comptes
            .AsNoTracking()
            .Include(c => c.Client)
            .Include(c => c.TypeCompte)
            .Include(c => c.Agence)
            .FirstOrDefaultAsync(c => c.Id == id);
```

```
        if (compte is null) return null;

        var ecritures = await _context.Ecritures
            .AsNoTracking()
            .Include(e => e.Operation)
            .Where(e => e.CompteId == id)
            .OrderByDescending(e => e.DateEcriture)
            .Take(50)
            .ToListAsync();

        return new CompteDetailsViewModel {
            Compte = compte,
            Ecritures = ecritures
        };
    }

    public async Task<Compte?> GetByIdAsync(int id) {
        return await _context.Comptes
            .AsNoTracking()
            .FirstOrDefaultAsync(c => c.Id == id);
    }

    public async Task<Compte> CreateAsync(CompteCreateViewModel vm){
        var compte = new Compte
        {
            NumeroCompte = vm.NumeroCompte.Trim().ToUpperInvariant(),
            ClientId = vm.ClientId,
            TypeCompteId = vm.TypeCompteId,
            AgenceId = vm.AgenceId,
            Solde = vm.SoldeInitial,
            Devise = "BIF",
            Statut = "Actif",
            DateOuverture = DateTime.UtcNow,
            DateCreation = DateTime.UtcNow
        };

        _context.Comptes.Add(compte);
        await _context.SaveChangesAsync();
        return compte;
    }
```

```
    public async Task<bool> UpdateAsync(CompteEditViewModel vm) {
        var compte = await _context.Comptes.FindAsync(vm.Id);
        if (compte is null) return false;

        compte.NumeroCompte = vm.NumeroCompte.Trim().ToUpperInvariant();
        compte.TypeCompteId = vm.TypeCompteId;
        compte.AgenceId = vm.AgenceId;
        compte.Statut = vm.Statut;
        compte.DateModification = DateTime.UtcNow;

        if (vm.Statut == "Cloture")
            compte.DateFermeture = DateTime.UtcNow;

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> DeleteAsync(int id){
        var compte = await _context.Comptes
            .Include(c => c.Ecritures)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (compte is null) return false;

        if (compte.Ecritures.Any())
            throw new InvalidOperationException(
                "Impossible de supprimer ce compte : des écritures
                 comptables y sont rattachées.");

        _context.Comptes.Remove(compte);
        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> NumeroCompteExisteAsync(string
        numeroCompte, int? excludeId = null) {
        var query = _context.Comptes.Where(c => c.NumeroCompte ==
                    numeroCompte);
        if (excludeId.HasValue)
            query = query.Where(c => c.Id != excludeId.Value);
        return await query.AnyAsync();
    }
}
```

6.5.3. Contrôleur Comptes

Fichier à créer : `Controllers/ComptesController.cs`

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

public class ComptesController : Controller{
    private readonly ICompteService _compteService;
    private readonly IClientService _clientService;
    private readonly IAgenceService _agenceService;

    public ComptesController(
        ICompteService compteService,
        IClientService clientService,
        IAgenceService agenceService) {
        _compteService = compteService;
        _clientService = clientService;
        _agenceService = agenceService;
    }

    // GET: /Comptes
    public async Task<IActionResult> Index(
        string? recherche, int? agenceId, int? typeCompteId, string?
         statut, int page = 1) {
        const int taille = 10;
        var vm = await _compteService.GetPagedAsync(recherche,
                  agenceId, typeCompteId, statut, page, taille);
        return View(vm);
    }

    // GET: /Comptes/Details/5
    public async Task<IActionResult> Details(int id) {
        var vm = await _compteService.GetDetailsAsync(id);
        if (vm is null) return NotFound();
        return View(vm);
    }
```

```
    // GET: /Comptes/Create?clientId=5
    public async Task<IActionResult> Create(int? clientId){
        await ChargerListesAsync(clientId);
        return View(new CompteCreateViewModel { ClientId = clientId
                        ?? 0 });
    }

    // POST: /Comptes/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(CompteCreateViewModel vm) {
        if (!ModelState.IsValid)
        {
            await ChargerListesAsync(vm.ClientId, vm.TypeCompteId,
                  vm.AgenceId);
            return View(vm);
        }

        if (await _compteService.NumeroCompteExisteAsync(
           vm.NumeroCompte.Trim().ToUpperInvariant())) {
            ModelState.AddModelError(nameof(vm.NumeroCompte),
                "Ce numéro de compte est déjà utilisé.");
            await ChargerListesAsync(vm.ClientId, vm.TypeCompteId,
                 vm.AgenceId);
            return View(vm);
        }

        try{
            await _compteService.CreateAsync(vm);
            TempData["Success"] = "Compte ouvert avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (DbUpdateException ex)
            when (ex.InnerException is SqlException sql &&
                 sql.Number == 2601){
            ModelState.AddModelError(nameof(vm.NumeroCompte),
                "Ce numéro de compte est déjà utilisé.");
            await ChargerListesAsync(vm.ClientId, vm.TypeCompteId,
                vm.AgenceId);
            return View(vm);
        }
    }
```

```
    // GET: /Comptes/Edit/5
    public async Task<IActionResult> Edit(int id) {
        var compte = await _compteService.GetByIdAsync(id);
        if (compte is null) return NotFound();

        var client = await _clientService.GetByIdAsync(
                    compte.ClientId);

        var vm = new CompteEditViewModel {
            Id = compte.Id,
            NumeroCompte = compte.NumeroCompte,
            TypeCompteId = compte.TypeCompteId,
            AgenceId = compte.AgenceId,
            Statut = compte.Statut,
            Solde = compte.Solde,
            Devise = compte.Devise,
            ClientNom = client is null ? null : $"{client.Nom}
                        {client.Prenom}"
        };

        await ChargerListesAsync(compte.ClientId, vm.TypeCompteId,
              vm.AgenceId);
        return View(vm);
    }

    // POST: /Comptes/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, CompteEditViewModel vm)
    {
        if (id != vm.Id) return BadRequest();
        if (!ModelState.IsValid) {
            await ChargerListesAsync(null, vm.TypeCompteId,
                  vm.AgenceId);
            return View(vm);
        }

        var ok = await _compteService.UpdateAsync(vm);
        if (!ok) return NotFound();

        TempData["Success"] = "Compte modifié avec succès.";
        return RedirectToAction(nameof(Index));
    }
```

```
    // GET: /Comptes/Delete/5
    public async Task<IActionResult> Delete(int id) {
        var vm = await _compteService.GetDetailsAsync(id);
        if (vm is null) return NotFound();
        return View(vm);
    }

    // POST: /Comptes/Delete/5
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id) {
        try {
            var ok = await _compteService.DeleteAsync(id);
            if (!ok) return NotFound();

            TempData["Success"] = "Compte supprimé avec succès.";
            return RedirectToAction(nameof(Index));
        }
        catch (InvalidOperationException ex) {
            TempData["Error"] = ex.Message;
            return RedirectToAction(nameof(Index));
        }
    }

    private async Task ChargerListesAsync(int? clientId = null,int?
           typeCompteId = null, int? agenceId = null) {
        // Clients
        var clientsVm = await _clientService.GetPagedAsync(null,
                        null, null, 1, int.MaxValue);
        ViewBag.Clients = new SelectList(
            clientsVm.Clients.Select(c => new {
                c.Id,
                Libelle = $"{c.NumeroClient} — {c.Nom} {c.Prenom}"
            }),
            "Id", "Libelle", clientId);

        // Agences
        var agencesVm = await _agenceService.GetPagedAsync(null, 1,
                        int.MaxValue);
        ViewBag.Agences = new SelectList(
            agencesVm.Agences.Select(a => new {
                a.Id,
                Libelle = $"{a.Code} — {a.Ville}"
```

```
            }),
            "Id", "Libelle", agenceId);

        // Types de compte
        var typesVm = await _compteService.GetPagedAsync(null, null,
                      null, null, 1, int.MaxValue);
        ViewBag.TypesCompte = new SelectList(
            typesVm.TypesCompte.Select(t => new {
                t.Id,
                Libelle = $"{t.Code} — {t.Libelle}"
            }),
            "Id", "Libelle", typeCompteId);
    }
}
```

6.5.4. Vues Comptes

Fichier à créer : `Views/Comptes/Index.cshtml`

```
@model AkarangaBank.Web.ViewModels.CompteListViewModel
@{
    ViewData["Title"] = "Comptes";
}

<h1>Comptes bancaires</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}
@if (TempData["Error"] is string err)
{
    <div class="alert alert-danger">@err</div>
}

<form method="get" class="row g-2 mb-3">
    <div class="col-md-3">
        <input type="text" name="recherche" value="@Model.Recherche"
               class="form-control" placeholder="Numéro, nom du
               client..." />
    </div>
    <div class="col-md-2">
        <select name="agenceId" class="form-select">
            <option value="">-- Agence --</option>
```

```
            @foreach (var a in Model.Agences)
            {
                <option value="@a.Id" selected="@(Model.AgenceId ==
                        a.Id)">@a.Code</option>
            }
        </select>
    </div>
    <div class="col-md-2">
        <select name="typeCompteId" class="form-select">
            <option value="">-- Type --</option>
            @foreach (var t in Model.TypesCompte)
            {
                <option value="@t.Id" selected="@(Model.TypeCompteId
                        == t.Id)">@t.Libelle</option>
            }
        </select>
    </div>
    <div class="col-md-2">
        <select name="statut" class="form-select">
            <option value="">-- Statut --</option>
            <option value="Actif" selected="@(Model.Statut ==
                  "Actif")">Actif</option>
            <option value="Bloque" selected="@(Model.Statut ==
                  "Bloque")">Bloqué</option>
            <option value="Cloture" selected="@(Model.Statut ==
                  "Cloture")">Clôturé</option>
        </select>
    </div>
    <div class="col-md-1">
        <button type="submit" class="btn btn-primary w-
               100">Filtrer</button>
    </div>
    <div class="col-md-2 text-end">
        <a asp-action="Create" class="btn btn-success">+ Ouvrir un
                compte</a>
    </div>
</form>

<p class="text-muted">@Model.Total compte(s) trouvé(s).</p>

<table class="table table-striped">
    <thead>
        <tr>
```

```
            <th>Numéro</th>
            <th>Client</th>
            <th>Type</th>
            <th>Agence</th>
            <th>Solde</th>
            <th>Statut</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
    @foreach (var compte in Model.Comptes) {
        <tr>
            <td>@compte.NumeroCompte</td>
            <td>@compte.Client.Nom @compte.Client.Prenom</td>
            <td>@compte.TypeCompte.Libelle</td>
            <td>@compte.Agence.Code</td>
            <td class="text-end">@compte.Solde.ToString("N2")
                @compte.Devise</td>
            <td>
                <span class="badge @(compte.Statut == "Actif" ? "bg-
                  success" : compte.Statut == "Bloque" ? "bg-
                  warning" : "bg-secondary")">@compte.Statut
                </span>
            </td>
            <td>
                <a asp-action="Details" asp-route-id="@compte.Id"
                    class="btn btn-sm btn-primary">Détails</a>
                <a asp-action="Edit" asp-route-id="@compte.Id"
                    class="btn btn-sm btn-warning">Modifier</a>
                <a asp-action="Delete" asp-route-id="@compte.Id"
                    class="btn btn-sm btn-danger">Supprimer</a>
            </td>
        </tr>
    }
    </tbody>
</table>

@if (Model.TotalPages > 1) {
    <nav>
        <ul class="pagination">
            @for (int i = 1; i <= Model.TotalPages; i++) {
               <li class="page-item @(i == Model.Page ? "active" : "")">
                    <a class="page-link"
```

```
                        asp-action="Index"
                        asp-route-page="@i"
                        asp-route-recherche="@Model.Recherche"
                        asp-route-agenceId="@Model.AgenceId"
                        asp-route-typeCompteId="@Model.TypeCompteId"
                        asp-route-statut="@Model.Statut">@i</a>
                </li>
            }
        </ul>
    </nav>
}
```

Fichier à créer : `Views/Comptes/Create.cshtml`

```
@model AkarangaBank.Web.ViewModels.CompteCreateViewModel
@{
    ViewData["Title"] = "Ouvrir un compte";
}

<h1>Ouvrir un compte</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="NumeroCompte" class="form-label"></label>
        <input asp-for="NumeroCompte" class="form-control"
               placeholder="BJM-0006-0001" />
        <span asp-validation-for="NumeroCompte" class="text-
               danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="ClientId" class="form-label"></label>
        <select asp-for="ClientId" asp-items="ViewBag.Clients"
               class="form-select">
            <option value="">-- Sélectionner un client --</option>
        </select>
       <span asp-validation-for="ClientId" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="TypeCompteId" class="form-label"></label>
        <select asp-for="TypeCompteId" asp-
            items="ViewBag.TypesCompte" class="form-select">
```

```
            <option value="">-- Sélectionner un type --</option>
        </select>
        <span asp-validation-for="TypeCompteId" class="text-
              danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="AgenceId" class="form-label"></label>
        <select asp-for="AgenceId" asp-items="ViewBag.Agences"
              class="form-select">
            <option value="">-- Sélectionner une agence --</option>
        </select>
        <span asp-validation-for="AgenceId" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="SoldeInitial" class="form-label"></label>
       <input asp-for="SoldeInitial" class="form-control" value="0" />
       <span asp-validation-for="SoldeInitial" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Ouvrir le
            compte</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Fichier à créer : `Views/Comptes/Edit.cshtml`

```
@model AkarangaBank.Web.ViewModels.CompteEditViewModel
@{
    ViewData["Title"] = "Modifier le compte";
}

<h1>Modifier le compte</h1>

<p class="text-muted">
    Client : <strong>@Model.ClientNom</strong> — Solde actuel :
    <strong>@Model.Solde.ToString("N2") @Model.Devise</strong>
</p>

<form asp-action="Edit" method="post">
```

```
    <input type="hidden" asp-for="Id" />
    <input type="hidden" asp-for="ClientNom" />
    <input type="hidden" asp-for="Solde" />
    <input type="hidden" asp-for="Devise" />
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="NumeroCompte" class="form-label"></label>
        <input asp-for="NumeroCompte" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="TypeCompteId" class="form-label"></label>
        <select asp-for="TypeCompteId" asp-
           items="ViewBag.TypesCompte" class="form-select"></select>
    </div>
    <div class="mb-3">
        <label asp-for="AgenceId" class="form-label"></label>
        <select asp-for="AgenceId" asp-items="ViewBag.Agences"
               class="form-select"></select>
    </div>
    <div class="mb-3">
        <label asp-for="Statut" class="form-label"></label>
        <select asp-for="Statut" class="form-select">
            <option value="Actif">Actif</option>
            <option value="Bloque">Bloqué</option>
            <option value="Cloture">Clôturé</option>
        </select>
    </div>

    <button type="submit" class="btn btn-
            primary">Enregistrer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Fichier à créer : `Views/Comptes/Details.cshtml`

```
@model AkarangaBank.Web.ViewModels.CompteDetailsViewModel
@{
    ViewData["Title"] = "Détails du compte";
}

<h1>Compte @Model.Compte.NumeroCompte</h1>

<dl class="row">
    <dt class="col-sm-3">Client</dt>
    <dd class="col-sm-9">
        <a asp-controller="Clients" asp-action="Details" asp-route-
id="@Model.Compte.ClientId">
            @Model.Compte.Client.Nom @Model.Compte.Client.Prenom
        </a>
    </dd>

    <dt class="col-sm-3">Type</dt>
    <dd class="col-sm-9">@Model.Compte.TypeCompte.Libelle</dd>

    <dt class="col-sm-3">Agence</dt>
    <dd class="col-sm-9">@Model.Compte.Agence.Code —
@Model.Compte.Agence.Ville</dd>

    <dt class="col-sm-3">Solde</dt>
    <dd class="col-sm-9"><strong>@Model.Compte.Solde.ToString("N2")
@Model.Compte.Devise</strong></dd>

    <dt class="col-sm-3">Statut</dt>
    <dd class="col-sm-9">@Model.Compte.Statut</dd>

    <dt class="col-sm-3">Date d'ouverture</dt>
    <dd class="col-sm-
9">@Model.Compte.DateOuverture.ToString("dd/MM/yyyy")</dd>
</dl>

<h2>Dernières écritures</h2>

@if (!Model.Ecritures.Any())
{
    <p class="text-muted">Aucune écriture enregistrée.</p>
}
```

```
else {
    <table class="table table-sm">
        <thead>
            <tr>
                <th>Date</th>
                <th>Opération</th>
                <th>Sens</th>
                <th class="text-end">Montant</th>
                <th>Libellé</th>
            </tr>
        </thead>
        <tbody>
        @foreach (var e in Model.Ecritures)
        {
            <tr>
               <td>@e.DateEcriture.ToString("dd/MM/yyyy HH:mm")</td>
                <td>@e.Operation.Reference</td>
                <td>@e.Sens</td>
                <td class="text-end">@e.Montant.ToString("N2")
                    @e.Devise</td>
                <td>@e.Libelle</td>
            </tr>
        }
        </tbody>
    </table>
}

<a asp-action="Index" class="btn btn-secondary">Retour à la
liste</a>
```

Fichier à créer : `Views/Comptes/Delete.cshtml`

```
@model AkarangaBank.Web.ViewModels.CompteDetailsViewModel
@{
    ViewData["Title"] = "Supprimer le compte";
}

<h1>Supprimer le compte</h1>

<div class="alert alert-warning">
    Confirmez-vous la suppression de ce compte ?
</div>
```

```
<dl class="row">
    <dt class="col-sm-3">Numéro</dt>
    <dd class="col-sm-9">@Model.Compte.NumeroCompte</dd>

    <dt class="col-sm-3">Client</dt>
    <dd class="col-sm-9">@Model.Compte.Client.Nom
                         @Model.Compte.Client.Prenom</dd>

    <dt class="col-sm-3">Solde</dt>
    <dd class="col-sm-9">@Model.Compte.Solde.ToString("N2")
                         @Model.Compte.Devise</dd>

    <dt class="col-sm-3">Écritures</dt>
    <dd class="col-sm-9">@Model.Ecritures.Count()</dd>
</dl>

@if (Model.Ecritures.Any())
{
    <div class="alert alert-danger">
        Ce compte a des écritures comptables. La suppression sera refusée.
    </div>
}

<form asp-action="Delete" method="post">
    <input type="hidden" name="id" value="@Model.Compte.Id" />
    <button type="submit" class="btn btn-danger">Supprimer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>
```

6.5.5. Enregistrement

➡ **Fichier :** `Program.cs` — ajouter :

```
builder.Services.AddScoped<ICompteService, CompteService>();
```

6.6. Mise à jour du menu

➡ **Fichier :** `Views/Shared/_Layout.cshtml` — ajouter :

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-controller="Clients" asp-
action="Index">Clients</a>
</li>
<li class="nav-item">
    <a class="nav-link text-dark" asp-controller="Comptes" asp-
action="Index">Comptes</a>
</li>
```

6.7. Tests manuels

Test 1 — Compilation

```
dotnet build
```

Résultat attendu : Build succeeded. 0 Error(s).

Test 2 — Liste des clients

`/Clients` : la liste des 5 clients s'affiche avec leur agence.

Test 3 — Filtre par agence

Sélectionner AKR-BJM dans le filtre agence.

Résultat attendu : seuls les clients de Bujumbura s'affichent (Nshimirimana, Nduwimana).

Test 4 — Recherche

Taper `Hategekimana.`

Résultat attendu : seul le client de Gitega s'affiche.

Test 5 — Création d'un client

Créer un client :

- Numéro : `CLI-000006`
- Nom : `Ndayizeye`
- Prénom : `Alice`
- Date de naissance : `1992-04-15`
- CNI : `BJM-6789012`
- Agence : `AKR-BJM`

Résultat attendu : message « `Client créé avec succès.` »

Test 6 — Unicité du CNI

Créer un client avec le CNI BJM-1234567 (déjà utilisé).

Résultat attendu : message « Ce numéro de CNI est déjà utilisé. »

Test 7 — Détails d'un client

Ouvrir les détails de Nshimirimana Emmanuel.

Résultat attendu :

- coordonnées complètes ;
- liste de ses comptes (2 comptes) ;
- liste de ses vérifications KYC (1 valide) ;
- solde total.

Test 8 — Création d'une vérification KYC

Depuis la fiche du client `Irakoze Patrick` (client 5, KYC en attente), cliquer sur « + Nouvelle vérification KYC ».

Résultat attendu : création en statut `EnAttente.`

Test 9 — Validation KYC

Cliquer sur « Valider » sur la fiche KYC.

Résultat attendu : statut `Valide`, vérificateur renseigné, date enregistrée.

Test 10 — Ouverture d'un compte

Ouvrir un compte pour `Ndayizeye Alice :`

- Numéro : `BJM-0006-0001`
- Type : `COURANT`
- Agence : `AKR-BJM`
- Solde initial : `50000`

Résultat attendu : message « `Compte ouvert avec succès.` »

Test 11 — Unicité du numéro de compte
Créer un compte avec le numéro `BJM-0001-0001` (déjà utilisé).

Résultat attendu : message « `Ce numéro de compte est déjà utilisé.` »

Test 12 — Filtres des comptes
Filtrer par type `EPARGNE`.

Résultat attendu : seuls les comptes épargne s'affichent.

Test 13 — Suppression d'un compte avec écritures
Tenter de supprimer le compte `BJM-0001-0001` (compte 1, qui a une écriture de débit).

Résultat attendu : message d'erreur « `Impossible de supprimer ce compte : des écritures comptables y sont rattachées.` »

Test 14 — Suppression d'un compte sans écriture
Créer un compte sans écriture, puis le supprimer.
Résultat attendu : suppression réussie.

Test 15 — Suppression d'un client avec comptes

Tenter de supprimer `Nshimirimana Emmanuel` (client 1, qui a 2 comptes).

Résultat attendu : message « Impossible de supprimer ce client : il possède des comptes bancaires. »

Test 16 — Suppression d'un client sans comptes

Créer un client sans compte, puis le supprimer.

Résultat attendu : suppression réussie.

6.9. Exercices

Exercice 1 — Vérifier le KYC avant d'ouvrir un compte

Objectif : empêcher l'ouverture d'un compte pour un client dont le KYC n'est pas validé.

Fichiers concernés :

- `Services/CompteService.cs` (vérification avant création)
- `Controllers/ComptesController.cs` (message d'erreur)

Exercice 2 — Suspendre un client

Objectif : ajouter un bouton « Suspendre » sur la fiche client qui passe le statut à `Suspendu`.

Fichiers concernés :

- `Services/IClientService.cs` et `ClientService.cs`
- `Controllers/ClientsController.cs`
- `Views/Clients/Details.cshtml`

Exercice 3 — Bloquer un compte

Objectif : ajouter un bouton « Bloquer » sur la fiche compte.

Fichiers concernés :

- `Services/ICompteService.cs` et `CompteService.cs`
- `Controllers/ComptesController.cs`
- `Views/Comptes/Details.cshtml`

Exercice 4 — Export CSV des clients

Objectif : ajouter un bouton « Exporter en CSV » sur la liste des clients.

Fichiers concernés :

- `Controllers/ClientsController.cs`
- `Views/Clients/Index.cshtml`

Exercice 5 — Nombre de comptes par type

Objectif : sur la page d'accueil des comptes, afficher un résumé : nombre de comptes courants, épargne, crédit.

Fichiers concernés :

- `ViewModels/CompteListViewModel.cs`
- `Services/CompteService.cs`
- `Views/Comptes/Index.cshtml`

6.10. Questions de compréhension

1. Pourquoi un client ne peut-il pas exister sans agence ?
2. Quelle est la différence entre `Include` et `ThenInclude` ?
3. Comment remplir une liste déroulante avec `SelectList` ?
4. Pourquoi vérifier l'unicité du CNI à la fois dans le service et côté base ?
5. Que se passe-t-il si on tente de supprimer un client qui a des comptes ?
6. Que se passe-t-il si on tente de supprimer un compte qui a des écritures ?
7. Quelle est la différence entre `Statut = "Suspendu"` et `Statut = "Cloture"` pour un client ?
8. Pourquoi une vérification KYC a-t-elle un statut `EnAttente` avant d'être `Valide` ?
9. Comment empêcher l'ouverture d'un compte pour un client non vérifié ?
10. Pourquoi les dates `DateCreation` et `DateModification` sont-elles importantes pour l'audit ?

6.11. Mini-TP 6 — Module complet Clients + KYC + Comptes

Objectif : construire le module complet de gestion des clients, de leurs documents KYC et de leurs comptes.

1. Créez les ViewModels (Clients, KYC, Comptes).
2. Créez les services (`IClientService, IVerificationKYCService, ICompteService` + implémentations).
3. Enregistrez les services dans `Program.cs.`
4. Créez les 3 contrôleurs (`ClientsController, VerificationsKYCController, ComptesController).`
5. Créez les vues (5 + 1 + 5).
6. Ajoutez les liens dans `_Layout.cshtml.`
7. Testez tous les cas de la section 6.7.

PARTIE III — SECURITE ET CŒUR METIER BANCAIRE

CHAPITRE 7 — AUTHENTIFICATION FORTE (2FA, SESSIONS)

Objectifs du chapitre

- Comprendre la différence entre authentification et autorisation.
- Intégrer ASP.NET Core Identity **sans inscription publique**.
- Créer un back-office d'administration des utilisateurs (rôles).
- Mettre en place la double authentification TOTP (Google Authenticator, Microsoft Authenticator).
- Générer et gérer les codes de récupération.
- Configurer les sessions : durée, expiration glissante, révocation.
- Exiger la MFA pour les opérations sensibles.
- Mapper un utilisateur Identity à un Client de la base AkarangaBank.

7.1. Pourquoi une authentification forte en banque ?

7.1.1. Le problème

À ce stade, Akaranga Bank est une application ouverte : n'importe qui peut accéder aux pages `/Clients`, `/Comptes`, `/Agences`. C'est inacceptable pour une banque.

7.1.2. Les exigences

Exigence	Réponse technique
Identifier les utilisateurs	ASP.NET Core Identity
Contrôler ce qu'ils peuvent faire	Rôles + politiques d'autorisation
Protéger contre le vol de mot de passe	Double authentification (2FA)
Limiter la durée d'exposition	Sessions avec expiration
Tracer les connexions	Table `SessionsUtilisateur`
Créer les comptes de façon contrôlée	Back-office admin, pas d'inscription publique

7.1.3. Pourquoi `--auth Individual` n'est pas adapté

Le template `dotnet new mvc --auth Individual` génère automatiquement Identity avec une page d'inscription publique. Dans une banque :

- personne ne s'inscrit tout seul ;
- un administrateur crée les comptes du personnel et des clients ;
- les rôles sont attribués explicitement ;
- la 2FA est obligatoire pour les opérations sensibles.

Décision : on ajoute Identity manuellement, sans page d'inscription publique, avec un back-office d'administration.

7.2. Options de conception

Option	Description	Avantages	Inconvénients
A	`--auth Individual` + suppression de la page Register	Rapide à mettre en place	Mélange code généré et code maison ; difficile à comprendre
B	Ajout manuel d'Identity, sans Register	Contrôle total, pédagogie	Plus de code à écrire
C	Identity + table `Utilisateurs` du script	Double système	Complexité inutile

Choix retenu : Option B

Pourquoi ?

1. **Pédagogie** : on construit chaque brique, on comprend comment Identity fonctionne.
2. **Cohérence avec le modèle** : le script a déjà une table `Utilisateurs` avec les colonnes `Role, DeuxFA_Active, DeuxFA_Secret, AgenceId.` On va s'y connecter.
3. **Contrôle** : pas d'inscription publique ; back-office admin dédié.

7.2.1. Comment cohabiter avec la table Utilisateurs du script ?

Approche	Description	Choix
A	Utiliser uniquement Identity (AspNetUsers) et ignorer Utilisateurs	Simple mais incohérent avec le script
B	Faire hériter ApplicationUser d'IdentityUser<int> et mapper les colonnes du script	Cohérent, respecte le modèle

Choix retenu : Option B — `ApplicationUser` hérite d'`IdentityUser<int>` et contient aussi les colonnes du script (`NomComplet`, `Role`, `AgenceId`, `DeuxFA_Active`, `DeuxFA_Secret`…). On utilise une seule table Utilisateurs qui est à la fois table Identity et table métier.

Conséquence : on va devoir renommer les tables Identity pour qu'elles utilisent nos noms (`Utilisateurs` au lieu d'`AspNetUsers`). C'est une configuration `OnModelCreating`.

7.3. Installation des packages

bash

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.AspNetCore.Identity.UI
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design
dotnet add package QRCoder
```

- `Microsoft.AspNetCore.Identity.EntityFrameworkCore` : Identity + EF Core.
- `Microsoft.AspNetCore.Identity.UI` : pages Identity par défaut (Login, 2FA, etc.).

- `Microsoft.VisualStudio.Web.CodeGeneration.Design` : scaffolding des pages Identity.
- `QRCoder` : génération des QR codes pour TOTP.

7.4. Créer `ApplicationUser`

Fichier à créer : `Models/ApplicationUser.cs`

```
using Microsoft.AspNetCore.Identity;

namespace AkarangaBank.Web.Models;

public class ApplicationUser : IdentityUser<int>
{
    public string NomComplet { get; set; } = string.Empty;
    public string Role { get; set; } = "client";
    public int AgenceId { get; set; }
    public Agence Agence { get; set; } = null!;
    public bool DeuxFA_Active { get; set; }
    public string? DeuxFA_Secret { get; set; }
    public DateTime? DerniereConnexion { get; set; }
    public bool EstActif { get; set; } = true;

    // Lien facultatif vers un Client (rôle client)
    public int? ClientId { get; set; }
    public Client? Client { get; set; }

    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateModification { get; set; }
}
```

Point clé : `IdentityUser<int>` (et non `IdentityUser`) pour que la clé primaire soit un `int`, compatible avec les FK des autres tables (`Virements`, `Operations`, `AuditLog`, etc.).

7.5. Créer `ApplicationRole`

Fichier à créer : `Models/ApplicationRole.cs`

```
using Microsoft.AspNetCore.Identity;

namespace AkarangaBank.Web.Models;

public class ApplicationRole : IdentityRole<int>
{
}
```

7.6. Modifier le `DbContext`

7.6.1. Le piège de la table `Utilisateurs` du script

Le problème : votre script SQL initial contenait :

```
CREATE TABLE Utilisateurs (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    ...
);
```

Or, `IdentityDbContext` expose par défaut une propriété `Users` (en anglais). Si vous voulez utiliser `_context.Utilisateurs` dans votre code (plus lisible en français), vous devez **ajouter un alias** dans le `DbContext`.

La solution retenue : ajouter un alias `Utilisateurs` qui pointe vers `Users` :

Fichier : `Data/AkarangaBankContext.cs`

```
public DbSet<ApplicationUser> Utilisateurs
{
    get => Users;
    set => Users = value;
}
```

⚠ Ce qu'il faut savoir :

- Cette propriété est un **alias** vers `Users` (défini dans `IdentityDbContext`).
- Elle **ne crée pas** une nouvelle table : la table reste `Utilisateurs` grâce à `b.ToTable("Utilisateurs")`.
- Elle permet d'écrire `_context.Utilisateurs` au lieu de `_context.Users` dans vos services.
- Le compilateur peut émettre un avertissement (`new` keyword) car `Users` existe déjà. Pour l'éviter, on peut utiliser `new` :

```
public new DbSet<ApplicationUser> Utilisateurs
{
    get => Users;
    set => Users = value;
}
```

7.6.2. Fichier complet `Data/AkarangaBankContext.cs`

Fichier : `Data/AkarangaBankContext.cs`

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Models;
```

```
namespace AkarangaBank.Web.Data;

public class AkarangaBankContext : IdentityDbContext<ApplicationUser,
ApplicationRole, int> {
    public AkarangaBankContext(DbContextOptions<AkarangaBankContext> options)
        : base(options) {
    }

    // ============================================================
    // DbSet métier
    // ============================================================
    public DbSet<Agence> Agences => Set<Agence>();
    public DbSet<Client> Clients => Set<Client>();
    public DbSet<Compte> Comptes => Set<Compte>();
    public DbSet<Virement> Virements => Set<Virement>();
    public DbSet<Operation> Operations => Set<Operation>();
    public DbSet<Ecriture> Ecritures => Set<Ecriture>();
    public DbSet<Pret> Prets => Set<Pret>();
    public DbSet<EcheancePret> EcheancesPret => Set<EcheancePret>();
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();
    public DbSet<AlerteAML> AlertesAML => Set<AlerteAML>();
    public DbSet<SessionUtilisateur> SessionsUtilisateur =>
            Set<SessionUtilisateur>();
    public DbSet<VerificationKYC> VerificationsKYC => Set<VerificationKYC>();
    public DbSet<Carte> Cartes => Set<Carte>();
    public DbSet<Parametre> Parametres => Set<Parametre>();
    public DbSet<TypeCompte> TypesCompte => Set<TypeCompte>();
    public DbSet<ApplicationUser> Utilisateurs { get => Users; set => Users =
            value; }

    // ============================================================
    // Configuration Fluent API
    // ============================================================
    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        base.OnModelCreating(modelBuilder); // IMPORTANT : Identity d'abord

        // ------------------------------------------------------
        // 1. IDENTITY — Renommage des tables
        // ------------------------------------------------------
        modelBuilder.Entity<ApplicationUser>(b =>
        {
            b.ToTable("Utilisateurs");
            b.Property(u => u.NomComplet).HasMaxLength(100).IsRequired();
            b.Property(u => u.Role).HasMaxLength(20).IsRequired();
            b.Property(u => u.DeuxFA_Secret).HasMaxLength(255);

            b.HasOne(u => u.Agence)
             .WithMany()
```

```
            .HasForeignKey(u => u.AgenceId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasIndex(u => u.AgenceId);
        b.HasIndex(u => u.Role);
    });

    modelBuilder.Entity<ApplicationRole>(b => b.ToTable("Roles"));
    modelBuilder.Entity<IdentityUserRole<int>>(b =>
        b.ToTable("UtilisateursRoles"));
    modelBuilder.Entity<IdentityUserClaim<int>>(b =>
        b.ToTable("UtilisateursClaims"));
    modelBuilder.Entity<IdentityUserLogin<int>>(b =>
        b.ToTable("UtilisateursLogins"));
    modelBuilder.Entity<IdentityRoleClaim<int>>(b =>
        b.ToTable("RolesClaims"));
    modelBuilder.Entity<IdentityUserToken<int>>(b =>
        b.ToTable("UtilisateursTokens"));

    // ------------------------------------------------------
    // 2. AGENCE
    // ------------------------------------------------------
    modelBuilder.Entity<Agence>(b =>
    {
        b.ToTable("Agences");
        b.HasKey(a => a.Id);

        b.Property(a => a.Code).HasMaxLength(20).IsRequired();
        b.Property(a => a.Nom).HasMaxLength(100).IsRequired();
        b.Property(a => a.Ville).HasMaxLength(50).IsRequired();
        b.Property(a => a.EstSiege).HasDefaultValue(false);
        b.Property(a =>
            a.DateCreation).HasDefaultValueSql("SYSDATETIME()");

        b.HasIndex(a => a.Code).IsUnique();
    });

    // ------------------------------------------------------
    // 3. PARAMETRE
    // ------------------------------------------------------
    modelBuilder.Entity<Parametre>(b =>
    {
        b.ToTable("Parametres");
        b.HasKey(p => p.Id);

        b.Property(p => p.Cle).HasMaxLength(50).IsRequired();
        b.Property(p => p.Valeur).HasMaxLength(255).IsRequired();
        b.Property(p => p.Description).HasMaxLength(255);
```

```
            b.Property(p =>
                p.DateModification).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(p => new { p.Cle, p.AgenceId }).IsUnique();

            b.HasOne<Agence>()
             .WithMany()
             .HasForeignKey(p => p.AgenceId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 4. CLIENT
        // ------------------------------------------------------------
        modelBuilder.Entity<Client>(b =>
        {
            b.ToTable("Clients");
            b.HasKey(c => c.Id);

            b.Property(c => c.NumeroClient).HasMaxLength(20).IsRequired();
            b.Property(c => c.Nom).HasMaxLength(100).IsRequired();
            b.Property(c => c.Prenom).HasMaxLength(100).IsRequired();
            b.Property(c => c.NumeroCNI).HasMaxLength(30).IsRequired();
            b.Property(c => c.Telephone).HasMaxLength(20);
            b.Property(c => c.Email).HasMaxLength(100);
            b.Property(c => c.Adresse).HasMaxLength(255);
            b.Property(c =>
c.Statut).HasMaxLength(20).HasDefaultValue("Actif");
            b.Property(c =>
c.DateCreation).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(c => c.NumeroClient).IsUnique();
            b.HasIndex(c => c.NumeroCNI).IsUnique();
            b.HasIndex(c => c.AgenceId);

            b.HasOne(c => c.Agence)
             .WithMany()
             .HasForeignKey(c => c.AgenceId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 5. VERIFICATION KYC
        // ------------------------------------------------------------
        modelBuilder.Entity<VerificationKYC>(b =>
        {
            b.ToTable("VerificationsKYC");
            b.HasKey(k => k.Id);
```

```
            b.Property(k => k.TypeDocument).HasMaxLength(50).IsRequired();
            b.Property(k => k.NumeroDocument).HasMaxLength(50).IsRequired();
            b.Property(k => k.CheminDocument).HasMaxLength(500);
            b.Property(k =>
k.Statut).HasMaxLength(20).HasDefaultValue("EnAttente");
            b.Property(k => k.Commentaire).HasMaxLength(500);

            b.HasOne(k => k.Client)
             .WithMany()
             .HasForeignKey(k => k.ClientId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(k => k.VerifieParId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------
        // 6. TYPE DE COMPTE
        // ------------------------------------------------------
        modelBuilder.Entity<TypeCompte>(b =>
        {
            b.ToTable("TypesCompte");
            b.HasKey(t => t.Id);

            b.Property(t => t.Code).HasMaxLength(20).IsRequired();
            b.Property(t => t.Libelle).HasMaxLength(50).IsRequired();
            b.Property(t =>
t.TauxInteret).HasColumnType("decimal(5,2)").HasDefaultValue(0m);
            b.Property(t =>
t.SoldeMinimum).HasColumnType("decimal(18,2)").HasDefaultValue(0m);
            b.Property(t => t.AutoriseDecouvert).HasDefaultValue(false);
            b.Property(t =>
t.DateCreation).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(t => t.Code).IsUnique();
        });

        // ------------------------------------------------------
        // 7. COMPTE
        // ------------------------------------------------------
        modelBuilder.Entity<Compte>(b =>
        {
            b.ToTable("Comptes");
            b.HasKey(c => c.Id);
```

```
            b.Property(c => c.NumeroCompte).HasMaxLength(30).IsRequired();
            b.Property(c =>
c.Solde).HasColumnType("decimal(18,2)").HasDefaultValue(0m);
            b.Property(c => c.Devise).HasMaxLength(3).HasDefaultValue("BIF");
            b.Property(c =>
c.Statut).HasMaxLength(20).HasDefaultValue("Actif");
            b.Property(c =>
c.DateOuverture).HasDefaultValueSql("SYSDATETIME()");
            b.Property(c =>
c.DateCreation).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(c => c.NumeroCompte).IsUnique();
            b.HasIndex(c => c.ClientId);
            b.HasIndex(c => c.AgenceId);
            b.HasIndex(c => c.TypeCompteId);

            b.HasOne(c => c.Client)
             .WithMany()
             .HasForeignKey(c => c.ClientId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(c => c.TypeCompte)
             .WithMany()
             .HasForeignKey(c => c.TypeCompteId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(c => c.Agence)
             .WithMany()
             .HasForeignKey(c => c.AgenceId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 8. CARTE
        // ------------------------------------------------------------
        modelBuilder.Entity<Carte>(b =>
        {
            b.ToTable("Cartes");
            b.HasKey(c => c.Id);

            b.Property(c => c.NumeroCarte).HasMaxLength(20).IsRequired();
            b.Property(c => c.TypeCarte).HasMaxLength(20).IsRequired();
            b.Property(c =>
c.Statut).HasMaxLength(20).HasDefaultValue("Active");
            b.Property(c =>
c.DateCreation).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(c => c.NumeroCarte).IsUnique();
```

```
            b.HasIndex(c => c.CompteId);
            b.HasIndex(c => c.ClientId);

            b.HasOne(c => c.Compte)
             .WithMany()
             .HasForeignKey(c => c.CompteId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(c => c.Client)
             .WithMany()
             .HasForeignKey(c => c.ClientId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 9. PRET
        // ------------------------------------------------------------
        modelBuilder.Entity<Pret>(b =>
        {
            b.ToTable("Prets");
            b.HasKey(p => p.Id);

            b.Property(p => p.Reference).HasMaxLength(30).IsRequired();
            b.Property(p =>
p.MontantPrincipal).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(p =>
p.TauxInteret).HasColumnType("decimal(5,2)").IsRequired();
            b.Property(p =>
p.Mensualite).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(p =>
p.Statut).HasMaxLength(20).HasDefaultValue("EnAttente");
            b.Property(p =>
p.DateDemande).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(p => p.Reference).IsUnique();
            b.HasIndex(p => p.ClientId);
            b.HasIndex(p => p.Statut);

            b.HasOne(p => p.Client)
             .WithMany()
             .HasForeignKey(p => p.ClientId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<Compte>()
             .WithMany()
             .HasForeignKey(p => p.CompteDebitId)
             .OnDelete(DeleteBehavior.Restrict);
```

```
            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(p => p.ValideParId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------
        // 10. ECHEANCE PRET
        // ------------------------------------------------------
        modelBuilder.Entity<EcheancePret>(b =>
        {
            b.ToTable("EcheancesPret");
            b.HasKey(e => e.Id);

            b.Property(e =>
e.Capital).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(e =>
e.Interet).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(e =>
e.MontantTotal).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(e =>
e.Statut).HasMaxLength(20).HasDefaultValue("EnAttente");

            b.HasIndex(e => new { e.PretId, e.NumeroEcheance }).IsUnique();
            b.HasIndex(e => e.DateEcheance);

            b.HasOne(e => e.Pret)
             .WithMany(p => p.Echeances)
             .HasForeignKey(e => e.PretId)
             .OnDelete(DeleteBehavior.Cascade);
        });

        // ------------------------------------------------------
        // 11. VIREMENT
        // ------------------------------------------------------
        modelBuilder.Entity<Virement>(b =>
        {
            b.ToTable("Virements");
            b.HasKey(v => v.Id);

            b.Property(v => v.Reference).HasMaxLength(30).IsRequired();
            b.Property(v =>
v.Montant).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(v => v.Devise).HasMaxLength(3).HasDefaultValue("BIF");
            b.Property(v =>
v.Statut).HasMaxLength(20).HasDefaultValue("EnAttente");
            b.Property(v => v.Motif).HasMaxLength(255);
            b.Property(v => v.ControleAML).HasDefaultValue(false);
```

```
            b.Property(v =>
v.DateVirement).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(v => v.Reference).IsUnique();
            b.HasIndex(v => v.CompteSourceId);
            b.HasIndex(v => v.CompteDestinationId);
            b.HasIndex(v => v.Statut);

            b.HasOne(v => v.CompteSource)
             .WithMany()
             .HasForeignKey(v => v.CompteSourceId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(v => v.CompteDestination)
             .WithMany()
             .HasForeignKey(v => v.CompteDestinationId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(v => v.UtilisateurId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 12. OPERATION
        // ------------------------------------------------------------
        modelBuilder.Entity<Operation>(b =>
        {
            b.ToTable("Operations");
            b.HasKey(o => o.Id);

            b.Property(o => o.Reference).HasMaxLength(30).IsRequired();
            b.Property(o => o.TypeOperation).HasMaxLength(30).IsRequired();
            b.Property(o =>
o.Montant).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(o => o.Devise).HasMaxLength(3).HasDefaultValue("BIF");
            b.Property(o =>
o.Statut).HasMaxLength(20).HasDefaultValue("EnAttente");
            b.Property(o => o.Commentaire).HasMaxLength(255);
            b.Property(o =>
o.DateOperation).HasDefaultValueSql("SYSDATETIME()");
            b.Property(o =>
o.DateCreation).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(o => o.Reference).IsUnique();
            b.HasIndex(o => o.CompteId);
            b.HasIndex(o => o.UtilisateurId);
```

```
            b.HasIndex(o => o.AgenceId);
            b.HasIndex(o => o.DateOperation);
            b.HasIndex(o => o.Statut);

            b.HasOne(o => o.Compte)
             .WithMany()
             .HasForeignKey(o => o.CompteId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(o => o.UtilisateurId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(o => o.Agence)
             .WithMany()
             .HasForeignKey(o => o.AgenceId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(o => o.Virement)
             .WithMany()
             .HasForeignKey(o => o.VirementId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(o => o.Pret)
             .WithMany()
             .HasForeignKey(o => o.PretId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 13. ECRITURE (partie double)
        // ------------------------------------------------------------
        modelBuilder.Entity<Ecriture>(b =>
        {
            b.ToTable("Ecritures");
            b.HasKey(e => e.Id);

            b.Property(e => e.Sens).HasMaxLength(10).IsRequired();
            b.Property(e =>
e.Montant).HasColumnType("decimal(18,2)").IsRequired();
            b.Property(e => e.Devise).HasMaxLength(3).HasDefaultValue("BIF");
            b.Property(e =>
e.DateEcriture).HasDefaultValueSql("SYSDATETIME()");
            b.Property(e => e.Libelle).HasMaxLength(255);

            b.HasIndex(e => e.OperationId);
            b.HasIndex(e => e.CompteId);
```

```
            b.HasOne(e => e.Operation)
             .WithMany(o => o.Ecritures)
             .HasForeignKey(e => e.OperationId)
             .OnDelete(DeleteBehavior.Cascade);

            b.HasOne(e => e.Compte)
             .WithMany()
             .HasForeignKey(e => e.CompteId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 14. AUDIT LOG
        // ------------------------------------------------------------
        modelBuilder.Entity<AuditLog>(b =>
        {
            b.ToTable("AuditLog");
            b.HasKey(a => a.Id);

            b.Property(a =>
a.DateEvenement).HasDefaultValueSql("SYSDATETIME()");
            b.Property(a => a.Action).HasMaxLength(50).IsRequired();
            b.Property(a => a.TableConcernee).HasMaxLength(50);
            b.Property(a => a.ValeurAvant).HasColumnType("nvarchar(max)");
            b.Property(a => a.ValeurApres).HasColumnType("nvarchar(max)");
            b.Property(a => a.AdresseIP).HasMaxLength(45);
            b.Property(a => a.UserAgent).HasMaxLength(255);
            b.Property(a => a.Details).HasColumnType("nvarchar(max)");

            b.HasIndex(a => a.DateEvenement);
            b.HasIndex(a => a.UtilisateurId);

            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(a => a.UtilisateurId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<Agence>()
             .WithMany()
             .HasForeignKey(a => a.AgenceId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 15. ALERTE AML
        // ------------------------------------------------------------
        modelBuilder.Entity<AlerteAML>(b =>
```

```
        {
            b.ToTable("AlertesAML");
            b.HasKey(a => a.Id);

            b.Property(a => a.TypeAlerte).HasMaxLength(50).IsRequired();
            b.Property(a => a.Niveau).HasMaxLength(20).IsRequired();
            b.Property(a => a.Description).HasMaxLength(500);
            b.Property(a =>
a.Statut).HasMaxLength(20).HasDefaultValue("Ouverte");
            b.Property(a => a.DateAlerte).HasDefaultValueSql("SYSDATETIME()");

            b.HasIndex(a => a.Statut);
            b.HasIndex(a => a.ClientId);

            b.HasOne(a => a.Client)
             .WithMany()
             .HasForeignKey(a => a.ClientId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<Operation>()
             .WithMany()
             .HasForeignKey(a => a.OperationId)
             .OnDelete(DeleteBehavior.Restrict);

            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(a => a.TraiteParId)
             .OnDelete(DeleteBehavior.Restrict);
        });

        // ------------------------------------------------------------
        // 16. SESSION UTILISATEUR
        // ------------------------------------------------------------
        modelBuilder.Entity<SessionUtilisateur>(b =>
        {
            b.ToTable("SessionsUtilisateur");
            b.HasKey(s => s.Id);

            b.Property(s => s.Token).HasMaxLength(255).IsRequired();
            b.Property(s => s.AdresseIP).HasMaxLength(45);
            b.Property(s => s.UserAgent).HasMaxLength(255);
            b.Property(s => s.DeuxFA_Valide).HasDefaultValue(false);
            b.Property(s =>
s.DateConnexion).HasDefaultValueSql("SYSDATETIME()");
            b.Property(s => s.EstActive).HasDefaultValue(true);

            b.HasIndex(s => s.Token).IsUnique();
            b.HasIndex(s => s.UtilisateurId);
```

```
            b.HasOne<ApplicationUser>()
             .WithMany()
             .HasForeignKey(s => s.UtilisateurId)
             .OnDelete(DeleteBehavior.Cascade);
        });
    }
}
```

7.6.3. Suppression de la table `Utilisateurs` du script

Action à faire dans le script `AkarangaBank-Script-DB.sql` :

1. **Supprimer** le bloc `CREATE TABLE Utilisateurs (...).`
2. **Supprimer** le bloc `INSERT INTO Utilisateurs (...).`
3. **Conserver** les FK des autres tables vers `Utilisateurs(Id)` : elles resteront valides car Identity créera la table avec un `Id INT.`

Raison : Identity va créer la table `Utilisateurs` avec ses propres colonnes (`Id, UserName, Email, PasswordHash,` etc.) plus les colonnes métier (`NomComplet, Role, AgenceId, DeuxFA_Active, DeuxFA_Secret).` Si le script la crée aussi, il y aura conflit.

7.7. Configurer Identity dans `Program.cs`

🔁 **Fichier :** `Program.cs`

```
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc.Authorization;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// --- Base de données ---
// La chaîne de connexion est dans user-secrets sous la clé "AkarangaBank"
builder.Services.AddDbContext<AkarangaBankContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("AkarangaBank")));

// --- Identity ---
builder.Services.AddIdentity<ApplicationUser, ApplicationRole>(options =>
{
    // Politique de mot de passe bancaire
    options.Password.RequiredLength = 8;
    options.Password.RequireDigit = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireLowercase = true;
```

```
    options.Password.RequireNonAlphanumeric = true;

    // Verrouillage après tentatives échouées
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    options.Lockout.MaxFailedAccessAttempts = 3;
    options.Lockout.AllowedForNewUsers = true;

    // Confirmation email (désactivée pour le cours)
    options.SignIn.RequireConfirmedEmail = false;
    options.SignIn.RequireConfirmedAccount = false;

    // Token TOTP pour la 2FA
    options.Tokens.AuthenticatorTokenProvider =
TokenOptions.DefaultAuthenticatorProvider;
    options.Tokens.AuthenticatorIssuer = "AkarangaBank";
})
.AddEntityFrameworkStores<AkarangaBankContext>()
.AddDefaultTokenProviders();
```

Suite de `Program.cs`

```
// --- Cookie d'authentification (sessions) ---
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Identity/Account/Login";
    options.LogoutPath = "/Identity/Account/Logout";
    options.AccessDeniedPath = "/Identity/Account/AccessDenied";

    // DUREE_SESSION_MIN = 30 minutes (paramètre du script)
    // On prend une marge : 60 minutes avec expiration glissante.
    options.ExpireTimeSpan = TimeSpan.FromMinutes(60);
    options.SlidingExpiration = true;

    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
    options.Cookie.SameSite = SameSiteMode.Lax;
});

// --- Services métier ---
builder.Services.AddScoped<IAgenceService, AgenceService>();
builder.Services.AddScoped<IClientService, ClientService>();
builder.Services.AddScoped<IVerificationKYCService, VerificationKYCService>();
builder.Services.AddScoped<ICompteService, CompteService>();
builder.Services.AddScoped<IUtilisateurService, UtilisateurService>();
builder.Services.AddSingleton<IQrCodeService, QrCodeService>();

// --- Politiques d'autorisation ---
builder.Services.AddSingleton<IAuthorizationHandler, RequireMfaHandler>();
builder.Services.AddAuthorization(options =>
{
```

```
    options.AddPolicy("RequireMfa", policy =>
        policy.Requirements.Add(new RequireMfa()));

    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("admin"));

    options.AddPolicy("GestionnaireOrAdmin", policy =>
        policy.RequireRole("gestionnaire", "admin"));
});

// --- Protection globale : tout exige l'authentification ---
builder.Services.AddControllersWithViews(options =>
{
    var policy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
    options.Filters.Add(new AuthorizeFilter(policy));
});

builder.Services.AddRazorPages(options =>
{
    options.Conventions.AllowAnonymousToAreaPage("Identity",
"/Account/Login");
    options.Conventions.AllowAnonymousToAreaPage("Identity",
"/Account/Logout");
    options.Conventions.AllowAnonymousToAreaPage("Identity",
"/Account/AccessDenied");
    options.Conventions.AllowAnonymousToAreaPage("Identity",
"/Account/LoginWith2fa");
    options.Conventions.AllowAnonymousToAreaPage("Identity",
"/Account/LoginWithRecoveryCode");
});

var app = builder.Build();

// --- Initialisation (rôles + admin) ---
using (var scope = app.Services.CreateScope())
{
    await AdminInitializer.InitializeAsync(scope.ServiceProvider);
}

// --- Pipeline HTTP ---
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
```

```
app.UseStaticFiles();
app.UseRouting();

app.UseAuthentication();    // AVANT UseAuthorization
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.MapRazorPages();

app.Run();
```

7.8. Gestion des migrations

7.8.1. Supprimer la base

sql

```
USE master;
GO
ALTER DATABASE AkarangaBank SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
DROP DATABASE AkarangaBank;
GO
```

Ou via `sqlcmd` :

```
bash

sqlcmd -S localhost,1433 -U sa -P "VotreMotDePasse" -Q "DROP
DATABASE IF EXISTS AkarangaBank"
```

7.8.2. Supprimer les migrations existantes

bash

```
dotnet ef migrations remove
```

Répétez si plusieurs migrations existent.

7.8.3. Vérifier que le dossier `Migrations/` est vide

bash

```
dir Migrations
```

Il ne doit rester que `AkarangaBankContextModelSnapshot.cs` (ou rien).

7.8.4. Recréer la migration initiale

bash

```
dotnet ef migrations add InitialIdentityWithInt
```

7.8.5. Appliquer à la base

bash

```
dotnet ef database update
```

7.9. Politique MFA

Fichier à créer : `Services/RequireMfaHandler.cs`

```
using Microsoft.AspNetCore.Authorization;

namespace AkarangaBank.Web.Services;

public class RequireMfa : IAuthorizationRequirement
{
}

public class RequireMfaHandler : AuthorizationHandler<RequireMfa>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        RequireMfa requirement)
    {
        if (context.User.Identity?.IsAuthenticated != true)
            return Task.CompletedTask;

        var amrClaim = context.User.FindFirst(c => c.Type == "amr");
        if (amrClaim?.Value == "mfa")
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

7.10. Initialisation des rôles et de l'administrateur

Fichier à créer : `Services/AdminInitializer.cs`

```
using Microsoft.AspNetCore.Identity;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.Data;
using Microsoft.EntityFrameworkCore;

namespace AkarangaBank.Web.Services;

public static class AdminInitializer
{
    public static async Task InitializeAsync(IServiceProvider services)
```

```
    {
        var userManager =
services.GetRequiredService<UserManager<ApplicationUser>>();
        var roleManager =
services.GetRequiredService<RoleManager<ApplicationRole>>();
        var context = services.GetRequiredService<AkarangaBankContext>();

        // Créer les rôles
        string[] roles = { "admin", "gestionnaire", "auditeur", "guichetier",
"client" };
        foreach (var role in roles)
        {
            if (!await roleManager.RoleExistsAsync(role))
                await roleManager.CreateAsync(new ApplicationRole { Name =
role });
        }

        // Créer le premier admin
        const string adminEmail = "mettez_votre_email_ici";
        var admin = await userManager.FindByEmailAsync(adminEmail);

        if (admin is null)
        {
            var siege = await context.Agences.FirstOrDefaultAsync(a =>
a.EstSiege);
            if (siege is null) return;

            admin = new ApplicationUser
            {
                UserName = adminEmail,
                Email = adminEmail,
                EmailConfirmed = true,
                NomComplet = "Administrateur Système",
                Role = "admin",
                AgenceId = siege.Id,
                EstActif = true
            };

            var result = await userManager.CreateAsync(admin,
"Administrateur@2025!");
            if (result.Succeeded)
                await userManager.AddToRoleAsync(admin, "admin");
        }
    }
}
```

7.11. Scaffolding des pages Identity (sans Register)

bash

```
dotnet aspnet-codegenerator identity --dbContext AkarangaBankContext --files "
Account.Login;Account.Logout;Account.AccessDenied;Account.Manage.Index;Account
.Manage.EnableAuthenticator;Account.Manage.TwoFactorAuthentication;Account.Man
age.GenerateRecoveryCodes;Account.Manage.ResetAuthenticator;Account.Manage.Dis
able2fa;Account.LoginWith2fa;Account.LoginWithRecoveryCode"
```

Ne pas inclure `Account.Register` : on ne veut pas d'inscription publique. Les pages générées se trouvent dans `Areas/Identity/Pages/Account/`.

7.12. QR code pour la 2FA

7.12.1. Configuration du générateur

Bash

```
dotnet add package QRCoder
```

Fichier à créer : `Services/IQrCodeService.cs`

```
namespace AkarangaBank.Web.Services;

public interface IQrCodeService
{
    string GenerateQrCodeBase64(string text);
}
```

Fichier à créer : `Services/QrCodeService.cs`

```
using QRCoder;

namespace AkarangaBank.Web.Services;

public class QrCodeService : IQrCodeService
{
    public string GenerateQrCodeBase64(string text)
    {
        using var qrGenerator = new QRCodeGenerator();
        using var qrCodeData = qrGenerator.CreateQrCode(text,
QRCodeGenerator.ECCLevel.Q);
        using var qrCode = new PngByteQRCode(qrCodeData);
        var bytes = qrCode.GetGraphic(10);
        return Convert.ToBase64String(bytes);
    }
}
```

➥ **Fichier :** `Program.cs` — **ajouter :**

```
using AkarangaBank.Web.Services;
```

```
builder.Services.AddSingleton<IQrCodeService, QrCodeService>();
```

7.12.2. Afficher le QR code dans la page `EnableAuthenticator`

➥ **Fichier :**
`Areas/Identity/Pages/Account/Manage/EnableAuthenticator.cshtml`

```
@inject AkarangaBank.Web.Services.IQrCodeService QrCodeService
@{
    var qrBase64 =
QrCodeService.GenerateQrCodeBase64(Model.AuthenticatorUri);
}

<div>
    <img src="@qrBase64" alt="QR code 2FA" />
</div>
```

7.13. Lier un utilisateur Identity à un Client

Le script ne relie pas `Utilisateurs` à `Clients`. On ajoute une colonne `ClientId` facultative sur `ApplicationUser`.

➥ **Fichier :** `Models/ApplicationUser.cs` — **ajouter :**

csharp

```
public int? ClientId { get; set; }
public Client? Client { get; set; }
```

➥ **Fichier :** `Data/AkarangaBankContext.cs` — **dans la config** `ApplicationUser` **:**

```
b.HasOne(u => u.Client)
 .WithMany()
 .HasForeignKey(u => u.ClientId)
 .OnDelete(DeleteBehavior.SetNull);


b.HasIndex(u => u.ClientId);
```

Terminal :

```
dotnet ef migrations add AjoutIdentityEt2FA
dotnet ef database update
```

7.14. Back-office admin des utilisateurs

7.14.1. Service

Fichier à créer : `Services/IUtilisateurService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public interface IUtilisateurService
{
    Task<IEnumerable<ApplicationUser>> GetAllAsync();
    Task<ApplicationUser?> GetByIdAsync(int id);
    Task<(bool Success, string? Error)>
CreateAsync(UtilisateurCreateViewModel vm);
    Task<(bool Success, string? Error)> ToggleActifAsync(int id);
    Task<(bool Success, string? Error)> ResetPasswordAsync(int id, string
newPassword);
    Task<(bool Success, string? Error)> AssignRoleAsync(int id, string
role);
}
```

Fichier à créer : `Services/UtilisateurService.cs`

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public class UtilisateurService : IUtilisateurService
{
    private readonly UserManager<ApplicationUser> _userManager;

    public UtilisateurService(UserManager<ApplicationUser> userManager)
    {
        _userManager = userManager;
    }

    public async Task<IEnumerable<ApplicationUser>> GetAllAsync()
    {
        // On utilise _userManager.Users (table Utilisateurs)
        return await _userManager.Users
            .Include(u => u.Agence)
            .OrderBy(u => u.NomComplet)
```

```
            .ToListAsync();
    }

    public async Task<ApplicationUser?> GetByIdAsync(int id)
    {
        return await _userManager.Users
            .Include(u => u.Agence)
            .FirstOrDefaultAsync(u => u.Id == id);
    }

    public async Task<(bool Success, string? Error)>
CreateAsync(UtilisateurCreateViewModel vm)
    {
        var user = new ApplicationUser
        {
            UserName = vm.Email,
            Email = vm.Email,
            EmailConfirmed = true,
            NomComplet = vm.NomComplet,
            Role = vm.Role,
            AgenceId = vm.AgenceId,
            EstActif = true
        };

        var result = await _userManager.CreateAsync(user, vm.Password);
        if (!result.Succeeded)
            return (false, string.Join(", ", result.Errors.Select(e =>
e.Description)));

        await _userManager.AddToRoleAsync(user, vm.Role);
        return (true, null);
    }

    public async Task<(bool Success, string? Error)> ToggleActifAsync(int
id)
    {
        var user = await _userManager.FindByIdAsync(id.ToString());
        if (user is null) return (false, "Utilisateur introuvable.");

        user.EstActif = !user.EstActif;
        await _userManager.UpdateAsync(user);
        return (true, null);
    }

    public async Task<(bool Success, string? Error)> ResetPasswordAsync(int
id, string newPassword)
    {
```

```
        var user = await _userManager.FindByIdAsync(id.ToString());
        if (user is null) return (false, "Utilisateur introuvable.");

        var token = await
_userManager.GeneratePasswordResetTokenAsync(user);
        var result = await _userManager.ResetPasswordAsync(user, token,
newPassword);

        return result.Succeeded
            ? (true, null)
            : (false, string.Join(", ", result.Errors.Select(e =>
e.Description)));
    }

    public async Task<(bool Success, string? Error)> AssignRoleAsync(int
id, string role)
    {
        var user = await _userManager.FindByIdAsync(id.ToString());
        if (user is null) return (false, "Utilisateur introuvable.");

        var currentRoles = await _userManager.GetRolesAsync(user);
        await _userManager.RemoveFromRolesAsync(user, currentRoles);
        await _userManager.AddToRoleAsync(user, role);
        user.Role = role;
        await _userManager.UpdateAsync(user);

        return (true, null);
    }
}
```

Point important : dans `GetAllAsync` et `GetByIdAsync`, on utilise `_userManager.Users` (propriété héritée d'`IdentityDbContext`). Si vous voulez utiliser `_context.Utilisateurs`, injectez le `DbContext` dans le service. Voir la note ci-dessous.

7.14.2. Utiliser `_context.Utilisateurs` dans les services

Si vous préférez `_context.Utilisateurs` (plus lisible), injectez `AkarangaBankContext` :

```
public class UtilisateurService : IUtilisateurService {
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly AkarangaBankContext _context;

    public UtilisateurService(
        UserManager<ApplicationUser> userManager,
        AkarangaBankContext context) {
        _userManager = userManager;
```

```
        _context = context;
    }

    public async Task<IEnumerable<ApplicationUser>> GetAllAsync()
    {
        // Utilise l'alias "Utilisateurs" (défini dans le DbContext)
        return await _context.Utilisateurs
            .Include(u => u.Agence)
            .OrderBy(u => u.NomComplet)
            .ToListAsync();
    }
    // ...
}
```

Rappel : `_context.Utilisateurs` est un alias vers `_context.Users`, donc les deux fonctionnent de manière identique.

7.14.3. ViewModel

Fichier à créer : `ViewModels/UtilisateurCreateViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class UtilisateurCreateViewModel
{
    [Required]
    [StringLength(100)]
    [Display(Name = "Nom complet")]
    public string NomComplet { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    [StringLength(100, MinimumLength = 8)]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;

    [Required]
    public string Role { get; set; } = "guichetier";

    [Required]
    [Display(Name = "Agence")]
    public int AgenceId { get; set; }
```

```
}
```

7.14.4. Contrôleur

Fichier à créer : `Controllers/AdminUtilisateursController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

[Authorize(Policy = "AdminOnly")]
public class AdminUtilisateursController : Controller {
    private readonly IUtilisateurService _utilisateurService;
    private readonly IAgenceService _agenceService;

    public AdminUtilisateursController(
        IUtilisateurService utilisateurService,
        IAgenceService agenceService)
    {
        _utilisateurService = utilisateurService;
        _agenceService = agenceService;
    }

    public async Task<IActionResult> Index()
    {
        var users = await _utilisateurService.GetAllAsync();
        return View(users);
    }

    public async Task<IActionResult> Create()
    {
        await ChargerAgencesAsync();
        return View(new UtilisateurCreateViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(UtilisateurCreateViewModel vm)
    {
        if (!ModelState.IsValid)
        {
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }
```

```
        var (success, error) = await _utilisateurService.CreateAsync(vm);
        if (!success)
        {
            ModelState.AddModelError("", error ?? "Erreur inconnue.");
            await ChargerAgencesAsync(vm.AgenceId);
            return View(vm);
        }

        TempData["Success"] = "Utilisateur créé avec succès.";
        return RedirectToAction(nameof(Index));
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> ToggleActif(int id)
    {
        await _utilisateurService.ToggleActifAsync(id);
        TempData["Success"] = "Statut modifié.";
        return RedirectToAction(nameof(Index));
    }

    private async Task ChargerAgencesAsync(int? selectedId = null)
    {
        var agencesVm = await _agenceService.GetPagedAsync(null, 1,
int.MaxValue);
        ViewBag.Agences = new SelectList(
            agencesVm.Agences.Select(a => new { a.Id, Libelle = $"{a.Code}
— {a.Ville}" }), "Id", "Libelle", selectedId);

        ViewBag.Roles = new SelectList(new[]
        {
            "admin", "gestionnaire", "auditeur", "guichetier", "client"
        });
    }
}
```

7.14.5. Vues

Fichier à créer : `Views/AdminUtilisateurs/Index.cshtml`

```
@model IEnumerable<AkarangaBank.Web.Models.ApplicationUser>
@{
    ViewData["Title"] = "Administration des utilisateurs";
}

<h1>Utilisateurs</h1>
```

```
@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<div class="mb-3 text-end">
    <a asp-action="Create" class="btn btn-success">+ Nouvel utilisateur</a>
</div>

<table class="table table-striped">
    <thead>
        <tr>
            <th>Nom</th>
            <th>Email</th>
            <th>Rôle</th>
            <th>Agence</th>
            <th>2FA</th>
            <th>Actif</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
    @foreach (var u in Model)
    {
        <tr>
            <td>@u.NomComplet</td>
            <td>@u.Email</td>
            <td>@u.Role</td>
            <td>@u.Agence.Code</td>
            <td>
                @if (u.TwoFactorEnabled)
                {
                    <span class="badge bg-success">Activée</span>
                }
                else
                {
                    <span class="badge bg-warning text-
dark">Inactive</span>
                }
            </td>
            <td>
                <span class="badge @(u.EstActif ? "bg-success" : "bg-
secondary")">
                    @(u.EstActif ? "Oui" : "Non")
                </span>
            </td>
            <td>
```

```
                <form asp-action="ToggleActif" asp-route-id="@u.Id"
method="post" class="d-inline">
                    <button type="submit" class="btn btn-sm btn-warning">
                        @(u.EstActif ? "Désactiver" : "Activer")
                    </button>
                </form>
            </td>
        </tr>
    }
    </tbody>
</table>
```

Fichier à créer : `Views/AdminUtilisateurs/Create.cshtml`

```
@model AkarangaBank.Web.ViewModels.UtilisateurCreateViewModel
@{
    ViewData["Title"] = "Nouvel utilisateur";
}

<h1>Nouvel utilisateur</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="NomComplet" class="form-label"></label>
        <input asp-for="NomComplet" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Password" class="form-label"></label>
        <input asp-for="Password" class="form-control" />
    </div>
    <div class="mb-3">
        <label asp-for="Role" class="form-label"></label>
        <select asp-for="Role" asp-items="ViewBag.Roles" class="form-
select"></select>
    </div>
    <div class="mb-3">
        <label asp-for="AgenceId" class="form-label"></label>
        <select asp-for="AgenceId" asp-items="ViewBag.Agences" class="form-
select">
            <option value="">-- Sélectionner une agence --</option>
        </select>
    </div>
```

```
    <button type="submit" class="btn btn-primary">Créer</button>
    <a asp-action="Index" class="btn btn-secondary">Annuler</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

7.15. Protéger les opérations sensibles avec MFA

➖ **Fichier :** `Controllers/ComptesController.cs` **— sur la suppression :**

```
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
[Authorize(Policy = "RequireMfa")]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    // ...
}
```

➖ **Fichier :** `Controllers/ClientsController.cs` **— idem sur** `DeleteConfirmed`**.**

➖ **Fichier :** `Controllers/AgencesController.cs` **— sur** `Create`**,** `Edit`**,** `DeleteConfirmed` **:**

```
 [Authorize(Policy = "AdminOnly")]
public async Task<IActionResult> Create() { ... }
```

7.16. Menu et layout

➖ **Fichier :** `Views/Shared/_Layout.cshtml` **— remplacer la nav par :**

```
<nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-
white border-bottom box-shadow mb-3">
    <div class="container-fluid">
        <a class="navbar-brand" asp-controller="Home" asp-action="Index">
            AkarangaBank
        </a>
        <ul class="navbar-nav flex-grow-1">
            <li class="nav-item">
                <a class="nav-link text-dark" asp-controller="Agences" asp-
action="Index">Agences</a>
            </li>
            @if (User.Identity?.IsAuthenticated == true)
            {
                <li class="nav-item">
```

```
                    <a class="nav-link text-dark" asp-controller="Clients"
asp-action="Index">Clients</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link text-dark" asp-controller="Comptes"
asp-action="Index">Comptes</a>
                </li>
                @if (User.IsInRole("admin"))
                {
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-
controller="AdminUtilisateurs" asp-action="Index">
                            Utilisateurs
                        </a>
                    </li>
                }
            }
        </ul>
        <partial name="_LoginPartial" />
    </div>
</nav>
```

Fichier à créer : `Views/Shared/_LoginPartial.cshtml`

```
@using Microsoft.AspNetCore.Identity
@using AkarangaBank.Web.Models
@inject SignInManager<ApplicationUser> SignInManager
@inject UserManager<ApplicationUser> UserManager

<ul class="navbar-nav">
@if (SignInManager.IsSignedIn(User))
{
    var currentUser = await UserManager.GetUserAsync(User);
    <li class="nav-item">
        <span class="nav-link text-dark">
            @currentUser?.NomComplet
            @if (currentUser?.TwoFactorEnabled == true)
            {
                <span class="badge bg-success">2FA</span>
            }
        </span>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Identity" asp-
page="/Account/Manage/Index">Mon compte</a>
    </li>
    <li class="nav-item">
```

```
        <form class="form-inline" asp-area="Identity" asp-
page="/Account/Logout" method="post">
            <button type="submit" class="nav-link btn btn-link text-
dark">Déconnexion</button>
        </form>
    </li>
}
else
{
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Identity" asp-
page="/Account/Login">Connexion</a>
    </li>
}
</ul>
```

7.17. Tests manuels

Test 1 — Compilation et migration

bash

```
dotnet build
dotnet ef migrations add CorrectionRelationsEtShadowProperties
dotnet ef database update
```

Test 2 — Démarrage

bash

```
dotnet watch run
```

Vous êtes redirigé vers `/Identity/Account/Login`.

Test 3 — Connexion administrateur

Se connecter avec :

- Email : `admin@akaranga.bi (remplacer par votre email admin)`
- Mot de passe : `Administrateur@2025!`

Résultat attendu : connexion réussie, le badge « 2FA » n'apparaît pas (pas encore activée).

Test 4 — Activer la 2FA

1. Cliquer sur **Mon compte**.
2. Cliquer sur **Authentification à deux facteurs**.
3. Cliquer sur **Activer l'authentificateur**.
4. Scanner le QR code avec Google Authenticator ou Microsoft Authenticator.
5. Saisir le code à 6 chiffres.
6. Sauvegarder les codes de récupération (affichés une seule fois).

Résultat attendu : 2FA activée. Le badge « 2FA » apparaît à côté du nom.

Test 5 — Se reconnecter avec 2FA

1. Se déconnecter.
2. Se reconnecter.
3. Après le mot de passe, saisir le code TOTP.
4. Résultat attendu : connexion réussie.

Test 6 — Code de récupération

1. Se déconnecter.
2. Se reconnecter.
3. Sur la page TOTP, cliquer sur **Utiliser un code de récupération**.
4. Saisir un code sauvegardé.

Résultat attendu : connexion réussie.

Test 7 — Protection globale

Sans être connecté, essayer `/Clients.`
Résultat attendu : redirection vers /Identity/Account/Login.

Test 8 — MFA pour la suppression

1. Se connecter avec un utilisateur sans 2FA.
2. Tenter de supprimer un compte.

Résultat attendu : accès refusé (AccessDenied).

Test 9 — Rôle admin

1. Se connecter avec un utilisateur non-admin.
2. Tenter d'accéder à `/AdminUtilisateurs.`

Résultat attendu : accès refusé.

Test 10 — Créer un utilisateur

Se connecter en admin, cliquer sur **Utilisateurs → + Nouvel utilisateur**.
Créer :

- Nom : Niyonzima Jean
- Email : `jean.niyonzima@akaranga.bi`
- Mot de passe : `Jean@2025!`
- Rôle : guichetier
- Agence : `AKR-GIT`

Résultat attendu : utilisateur créé, apparaît dans la liste.

7.18. Mini-TP 7 — Authentification forte complète

Objectif : sécuriser Akaranga Bank avec Identity, 2FA et rôles.

1. Ajoutez les packages Identity.
2. Créez `ApplicationUser et ApplicationRole` (avec `int`).
3. Adaptez `AkarangaBankContext (IdentityDbContext<ApplicationUser, ApplicationRole, int> + alias Utilisateurs +` renommage des tables).
4. Supprimez la table `Utilisateurs` du script SQL.
5. Configurez Identity dans `Program.cs.`
6. Créez `RequireMfaHandler` et les politiques.
7. Créez `AdminInitializer` (rôles + premier admin).
8. Scaffoldez les pages Identity (sans Register).
9. Intégrez `QRCoder` pour le QR code.
10. Créez le back-office admin des utilisateurs.
11. Ajoutez `_LoginPartial.cshtml` et mettez à jour `_Layout.cshtml.`
12. Testez tous les cas de la section 7.17.

CHAPITRE 8 — OPÉRATIONS BANCAIRES, PARTIE DOUBLE ET CONTRÔLES ANTI-BLANCHIMENT

Objectifs du chapitre

- Comprendre le principe de la **partie double** et l'articulation Operations / Ecritures.
- Implémenter **dépôt, retrait et virement** en respectant l'atomicité (ACID).
- Utiliser les **transactions explicites** d'EF Core.
- Calculer un **solde** à partir des écritures.
- Produire un **journal comptable** et un **relevé de compte**.
- Comprendre le cadre **LBC/FT burundais** (loi 1/02 de 2008, révisée en 2025).
- Mettre en place un **moteur de surveillance** des transactions.
- Générer des **alertes AML**.

Partie A — Comptabilité en partie double

8.1. Le principe

Toute opération affecte **au moins deux comptes** :

```
Total des débits = Total des crédits
```

8.1.1. Deux niveaux dans le modèle

Niveau	Table	Rôle
Métier	`Operations`	L'événement : dépôt, retrait, virement
Comptable	`Ecritures`	Les lignes : débit d'un compte, crédit d'un autre

Règle absolue : une opération a au moins deux écritures, dont les totaux débit/crédit sont égaux.

8.1.2. Exemple — virement de 50 000 BIF

Table	Ligne
`Operations`	Réf. AKR-20261015-000001, type Virement, compte 1, montant 50 000
`Ecritures` (débit)	Compte 1 — Débit — 50 000 BIF
`Ecritures` (crédit)	Compte 2 — Crédit — 50 000 BIF

Total débit = Total crédit = 50 000 BIF ✓

8.1.3. Requête de contrôle

```sql
SELECT OperationId,
    SUM(CASE WHEN Sens = 'Debit'  THEN Montant ELSE 0 END) AS TotalDebit,
    SUM(CASE WHEN Sens = 'Credit' THEN Montant ELSE 0 END) AS TotalCredit
FROM Ecritures
GROUP BY OperationId
HAVING SUM(CASE WHEN Sens = 'Debit' THEN Montant ELSE -Montant END) <> 0;
-- Aucune ligne retournée = comptabilité équilibrée
```

Partie B — Cadre réglementaire LBC/FT

8.2. Le contexte burundais

Textes de référence :

- Loi n° 1/02 du 4 février 2008 (LBC/FT)
- Loi n° 1/08 du 27 mars 2025 (modification)
- Instructions de la BRB
- Recommandations du GAFI

Autorité : la Cellule Nationale de Renseignement Financier (CNRF) reçoit et analyse les déclarations d'opérations suspectes.

8.2.1. Obligations des banques

Obligation	Implémentation
Vigilance client (KYC)	Table `VerificationsKYC` (chapitre 6)
Vigilance renforcée	Seuil AML + `AlertesAML`
Surveillance des transactions	Moteur de règles
Conservation des documents	AuditLog

8.2.2. Paramètre `SEUIL_AML`

```
INSERT INTO Parametres (Cle, Valeur, Description) VALUES
('SEUIL_AML', '5000000', 'Seuil de vigilance anti-blanchiment (BIF)');
```

Règle : toute opération ≥ 5 000 000 BIF est marquée `ControleAML = 1` et génère une alerte.

Partie C — Modélisation et services

8.3. Les entités du modèle

8.3.1. Operation

Fichier : `Models/Operation.cs`

```
namespace AkarangaBank.Web.Models;

public class Operation
{
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;
    public string TypeOperation { get; set; } = string.Empty;

    public int CompteId { get; set; }
    public Compte Compte { get; set; } = null!;

    public int UtilisateurId { get; set; }
    public ApplicationUser Utilisateur { get; set; } = null!;

    public int AgenceId { get; set; }
    public Agence Agence { get; set; } = null!;

    public int? VirementId { get; set; }
    public Virement? Virement { get; set; }
```

```
    public int? PretId { get; set; }
    public Pret? Pret { get; set; }

    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
    public string Statut { get; set; } = "EnAttente";
    public DateTime DateOperation { get; set; } = DateTime.UtcNow;
    public DateTime? DateValidation { get; set; }
    public string? Commentaire { get; set; }
    public DateTime DateCreation { get; set; } = DateTime.UtcNow;

    public ICollection<Ecriture> Ecritures { get; set; } = new List<Ecriture>();
    public ICollection<AlerteAML> Alertes { get; set; } = new List<AlerteAML>();
}
```

8.3.2. Ecriture

Fichier : `Models/Ecriture.cs`

```
namespace AkarangaBank.Web.Models;

public class Ecriture {
    public int Id { get; set; }

    public int OperationId { get; set; }
    public Operation Operation { get; set; } = null!;

    public int CompteId { get; set; }
    public Compte Compte { get; set; } = null!;

    public string Sens { get; set; } = string.Empty;   // "Debit" ou "Credit"
    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
    public DateTime DateEcriture { get; set; } = DateTime.UtcNow;
    public string? Libelle { get; set; }
}
```

8.3.3. AlerteAML

Fichier : `Models/AlerteAML.cs`

```
namespace AkarangaBank.Web.Models;

public class AlerteAML {
    public int Id { get; set; }

    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;

    public int? OperationId { get; set; }
```

```
    public Operation? Operation { get; set; }

    public string TypeAlerte { get; set; } = string.Empty;
    public string Niveau { get; set; } = "Moyen";      // Faible, Moyen, Eleve, Critique
    public string? Description { get; set; }
    public string Statut { get; set; } = "Ouverte";    // Ouverte, EnCours, Traitee, Classee

    public int? TraiteParId { get; set; }
    public ApplicationUser? TraitePar { get; set; }

    public DateTime DateAlerte { get; set; } = DateTime.UtcNow;
    public DateTime? DateTraitement { get; set; }
}
```

8.3.4. Virement

Fichier : `Models/Virement.cs`

```
namespace AkarangaBank.Web.Models;
public class Virement {
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;

    public int CompteSourceId { get; set; }
    public Compte CompteSource { get; set; } = null!;

    public int CompteDestinationId { get; set; }
    public Compte CompteDestination { get; set; } = null!;

    public int UtilisateurId { get; set; }
    public ApplicationUser Utilisateur { get; set; } = null!;

    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
    public string Statut { get; set; } = "EnAttente";
    public string? Motif { get; set; }
    public bool ControleAML { get; set; }
    public DateTime DateVirement { get; set; } = DateTime.UtcNow;
    public DateTime? DateValidation { get; set; }
}
```

8.3.5. Compte — ajout de la collection `Ecritures`

➖ **Fichier :** `Models/Compte.cs` — ajoutez :

```
public ICollection<Ecriture> Ecritures { get; set; } = new List<Ecriture>();
```

8.4. Configuration du DbContext

⚠ Point critique : les shadow properties

Pour éviter les colonnes fantômes (`UtilisateurId1`, `CompteId1`, etc.), utilisez **toujours** la syntaxe avec **lambda vers la propriété de navigation** :

```
// ✗ MAUVAIS : crée une shadow property
b.HasOne<ApplicationUser>()
 .WithMany()
 .HasForeignKey(o => o.UtilisateurId);

// ✓ BON : utilise la propriété de navigation
b.HasOne(o => o.Utilisateur)
 .WithMany()
 .HasForeignKey(o => o.UtilisateurId);
```

Règle : la lambda `o => o.Utilisateur` doit pointer vers une propriété **qui existe** dans le modèle `Operation`.

➖ **Fichier :** `Data/AkarangaBankContext.cs`

Ajoutez les DbSet :

```
public DbSet<Operation> Operations => Set<Operation>();
public DbSet<Ecriture> Ecritures => Set<Ecriture>();
public DbSet<Virement> Virements => Set<Virement>();
public DbSet<AlerteAML> AlertesAML => Set<AlerteAML>();
```

Dans `OnModelCreating` :

```
// ----- Virement -----
modelBuilder.Entity<Virement>(b =>
{
    b.ToTable("Virements");
    b.HasKey(v => v.Id);
    b.Property(v => v.Reference).HasMaxLength(30).IsRequired();
    b.Property(v => v.Montant).HasColumnType("decimal(18,2)").IsRequired();
    b.Property(v => v.Devise).HasMaxLength(3).HasDefaultValue("BIF");
    b.Property(v => v.Statut).HasMaxLength(20).HasDefaultValue("EnAttente")
;
    b.Property(v => v.Motif).HasMaxLength(255);
    b.Property(v => v.ControleAML).HasDefaultValue(false);
    b.Property(v => v.DateVirement).HasDefaultValueSql("SYSDATETIME()");
```

```
    b.HasIndex(v => v.Reference).IsUnique();
    b.HasIndex(v => v.CompteSourceId);
    b.HasIndex(v => v.CompteDestinationId);
    b.HasIndex(v => v.Statut);

    b.HasOne(v => v.CompteSource)
     .WithMany()
     .HasForeignKey(v => v.CompteSourceId)
     .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(v => v.CompteDestination)
     .WithMany()
     .HasForeignKey(v => v.CompteDestinationId)
     .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(v => v.Utilisateur)
     .WithMany()
     .HasForeignKey(v => v.UtilisateurId)
     .OnDelete(DeleteBehavior.Restrict);
});

// ----- Operation -----
modelBuilder.Entity<Operation>(b =>
{
    b.ToTable("Operations");
    b.HasKey(o => o.Id);
    b.Property(o => o.Reference).HasMaxLength(30).IsRequired();
    b.Property(o => o.TypeOperation).HasMaxLength(30).IsRequired();
    b.Property(o => o.Montant).HasColumnType("decimal(18,2)").IsRequired();
    b.Property(o => o.Devise).HasMaxLength(3).HasDefaultValue("BIF");
    b.Property(o => o.Statut).HasMaxLength(20).HasDefaultValue("EnAttente")
;
    b.Property(o => o.Commentaire).HasMaxLength(255);
    b.Property(o => o.DateOperation).HasDefaultValueSql("SYSDATETIME()");
    b.Property(o => o.DateCreation).HasDefaultValueSql("SYSDATETIME()");

    b.HasIndex(o => o.Reference).IsUnique();
    b.HasIndex(o => o.CompteId);
    b.HasIndex(o => o.UtilisateurId);
    b.HasIndex(o => o.AgenceId);
    b.HasIndex(o => o.DateOperation);
    b.HasIndex(o => o.Statut);

    b.HasOne(o => o.Compte)
     .WithMany()
     .HasForeignKey(o => o.CompteId)
     .OnDelete(DeleteBehavior.Restrict);
```

```
    b.HasOne(o => o.Utilisateur)
     .WithMany()
     .HasForeignKey(o => o.UtilisateurId)
     .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(o => o.Agence)
     .WithMany()
     .HasForeignKey(o => o.AgenceId)
     .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(o => o.Virement)
     .WithMany()
     .HasForeignKey(o => o.VirementId)
     .OnDelete(DeleteBehavior.SetNull);

    b.HasOne(o => o.Pret)
     .WithMany()
     .HasForeignKey(o => o.PretId)
     .OnDelete(DeleteBehavior.SetNull);
});

// ----- Ecriture -----
modelBuilder.Entity<Ecriture>(b =>
{
    b.ToTable("Ecritures");
    b.HasKey(e => e.Id);
    b.Property(e => e.Sens).HasMaxLength(10).IsRequired();
    b.Property(e => e.Montant).HasColumnType("decimal(18,2)").IsRequired();
    b.Property(e => e.Devise).HasMaxLength(3).HasDefaultValue("BIF");
    b.Property(e => e.DateEcriture).HasDefaultValueSql("SYSDATETIME()");
    b.Property(e => e.Libelle).HasMaxLength(255);

    b.HasIndex(e => e.OperationId);
    b.HasIndex(e => e.CompteId);

    b.HasOne(e => e.Operation)
     .WithMany(o => o.Ecritures)
     .HasForeignKey(e => e.OperationId)
     .OnDelete(DeleteBehavior.Cascade);

    b.HasOne(e => e.Compte)
     .WithMany(c => c.Ecritures)
     .HasForeignKey(e => e.CompteId)
     .OnDelete(DeleteBehavior.Restrict);
});
```

```
// ----- AlerteAML -----
modelBuilder.Entity<AlerteAML>(b =>
{
    b.ToTable("AlertesAML");
    b.HasKey(a => a.Id);
    b.Property(a => a.TypeAlerte).HasMaxLength(50).IsRequired();
    b.Property(a => a.Niveau).HasMaxLength(20).IsRequired();
    b.Property(a => a.Description).HasMaxLength(500);
    b.Property(a => a.Statut).HasMaxLength(20).HasDefaultValue("Ouverte");
    b.Property(a => a.DateAlerte).HasDefaultValueSql("SYSDATETIME()");

    b.HasIndex(a => a.Statut);
    b.HasIndex(a => a.ClientId);

    b.HasOne(a => a.Client)
     .WithMany()
     .HasForeignKey(a => a.ClientId)
     .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(a => a.Operation)
     .WithMany(o => o.Alertes)
     .HasForeignKey(a => a.OperationId)
     .OnDelete(DeleteBehavior.SetNull);

    b.HasOne(a => a.TraitePar)
     .WithMany()
     .HasForeignKey(a => a.TraiteParId)
     .OnDelete(DeleteBehavior.SetNull);
});
```

Après modification, recréez la migration :

bash

```
rmdir /S /Q Migrations
sqlcmd -S localhost -E -Q "ALTER DATABASE AkarangaBank SET SINGLE_US
ER WITH ROLLBACK IMMEDIATE; DROP DATABASE AkarangaBank;"
dotnet ef migrations add InitialClean
dotnet ef database update
```

Vérifiez qu'il n'y a pas de colonnes `*1` :

bash

```
findstr /S "Id1" Migrations\*.cs
```

Aucune ligne ne doit apparaître.

8.5. ViewModels

Fichier à créer : `ViewModels/OperationViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class DepotViewModel {
    [Required]
    [Display(Name = "Compte")]
    public int CompteId { get; set; }

    [Required]
    [Range(0.01, double.MaxValue, ErrorMessage = "Le montant doit être positif.")]
    public decimal Montant { get; set; }

    [StringLength(255)]
    public string? Commentaire { get; set; }
}

public class RetraitViewModel
{
    [Required]
    [Display(Name = "Compte")]
    public int CompteId { get; set; }

    [Required]
    [Range(0.01, double.MaxValue, ErrorMessage = "Le montant doit être positif.")]
    public decimal Montant { get; set; }

    [StringLength(255)]
    public string? Commentaire { get; set; }
}

public class VirementViewModel
{
    [Required]
    [Display(Name = "Compte source")]
    public int CompteSourceId { get; set; }

    [Required]
    [Display(Name = "Compte destination")]
    public int CompteDestinationId { get; set; }

    [Required]
    [Range(0.01, double.MaxValue, ErrorMessage = "Le montant doit être positif.")]
    public decimal Montant { get; set; }

    [StringLength(255)]
    public string? Motif { get; set; }
}
```

Fichier à créer : `ViewModels/ReleveViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class releveViewModel
{
    public Compte Compte { get; set; } = null!;
    public decimal SoldeInitial { get; set; }
    public decimal SoldeFinal { get; set; }
    public IEnumerable<Ecriture> Ecritures { get; set; } = new List<Ecriture>();
}
```

Fichier à créer : `ViewModels/JournalViewModel.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.ViewModels;

public class JournalViewModel
{
    public IEnumerable<Operation> Operations { get; set; } = new List<Operation>()
;
    public DateTime? Du { get; set; }
    public DateTime? Au { get; set; }
    public string? TypeOperation { get; set; }
    public int Page { get; set; } = 1;
    public int TaillePage { get; set; } = 20;
    public int Total { get; set; }
    public int TotalPages => (int)Math.Ceiling((double)Total / TaillePage);
}
```

8.6. Modèles de conformité

Fichier à créer : `Models/Conformite/EnumsConformite.cs`

```
namespace AkarangaBank.Web.Models;

public enum NiveauAlerte {
    Faible,
    Moyen,
    Eleve,
    Critique
}

public enum NiveauRisqueClient {
    Inconnu,
    Faible,
    Moyen,
    Eleve
}
```

Fichier à créer : `Models/Conformite/ResultatConformite.cs`

```
namespace AkarangaBank.Web.Models;

public class ResultatConformite
{
    public int CompteId { get; set; }
    public decimal Montant { get; set; }
    public DateTime DateAnalyse { get; set; }
    public bool EstBloquee { get; set; }
    public bool NecessiteDeclaration { get; set; }
    public bool NecessiteVigilanceRenforcee { get; set; }
    public string? Motif { get; set; }
    public NiveauAlerte Niveau { get; set; } = NiveauAlerte.Faible;
}
```

Fichier à créer : `Models/Conformite/ResultatSurveillance.cs`

```
namespace AkarangaBank.Web.Models;

public class ResultatSurveillance
{
    public int CompteId { get; set; }
    public DateTime DateAnalyse { get; set; }
    public List<AlerteSurveillance> Alertes { get; set; } = new();
    public bool NecessiteDeclaration { get; set; }
}
```

```
public class AlerteSurveillance
{
    public string Regle { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public NiveauAlerte Niveau { get; set; }
}
```

8.7. Interfaces des services

Fichier à créer : `Services/IComptabiliteService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public interface IComptabiliteService
{
    Task<Operation> DeposerAsync(DepotViewModel vm, int utilisateurId);
    Task<Operation> RetirerAsync(RetraitViewModel vm, int utilisateurId);
    Task<Operation> VirerAsync(VirementViewModel vm, int utilisateurId);
```

```
    Task<JournalViewModel> GetJournalAsync(
        DateTime? du, DateTime? au, string? typeOperation, int page, int taille);
    Task<ReleveViewModel?> GetReleveAsync(int compteId);
    Task<decimal> GetSoldeAsync(int compteId);

    Task<bool> VerifierEquilibreGlobalAsync();
}
```

Fichier à créer : `Services/IConformiteService.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public interface IConformiteService
{
    Task<ResultatConformite> AnalyserTransactionAsync(
        int compteId, decimal montant, string typeOperation);

    Task<bool> DetecterFractionnementAsync(
        int compteId, decimal montant, DateTime date);

    Task<NiveauRisqueClient> EvaluerRisqueClientAsync(int clientId);
    Task<decimal> GetSeuilAMLAsync();
}
```

Fichier à créer : `Services/ISurveillanceService.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public interface ISurveillanceService {
    Task<ResultatSurveillance> AnalyserTransactionAsync(
        int compteId, decimal montant, string typeOperation, DateTime date);

    Task<NiveauRisqueClient> EvaluerRisqueClientAsync(int clientId);
}
```

Fichier à créer : `Services/IDeclarationService.cs`

```
using AkarangaBank.Web.Models;


namespace AkarangaBank.Web.Services;


public interface IDeclarationService {
    Task<AlerteAML> CreerAlerteAsync(
        int clientId, int? operationId, string typeAlerte,
        string niveau, string description);
```

```
    Task<bool> TraiterAlerteAsync(int alerteId, int traiteParId, string statut, st
ring? commentaire);
    Task<IEnumerable<AlerteAML>> GetAlertesAsync(string? statut, string? niveau);
}
```

Fichier à créer : `Services/ICurrentUserService.cs`

```
namespace AkarangaBank.Web.Services;

public interface ICurrentUserService {
    Task<int> GetUserIdAsync();
    Task<int> GetAgenceIdAsync();
}
```

8.8. Implémentation du service de comptabilité

Fichier à créer : `Services/ComptabiliteService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public class ComptabiliteService : IComptabiliteService {
    private readonly AkarangaBankContext _context;
    private readonly IConformiteService _conformite;
    private readonly ISurveillanceService _surveillance;
    private readonly IDeclarationService _declaration;

    public ComptabiliteService(
        AkarangaBankContext context,
        IConformiteService conformite,
        ISurveillanceService surveillance,
        IDeclarationService declaration) {
        _context = context;
        _conformite = conformite;
        _surveillance = surveillance;
        _declaration = declaration;
    }

    // ============================================================
```

```
    // DÉPÔT
    // ============================================================
    public async Task<Operation> DeposerAsync(DepotViewModel vm, int utilisateurId
) {
        var compte = await _context.Comptes.FindAsync(vm.CompteId)
            ?? throw new InvalidOperationException("Compte introuvable.");

        if (compte.Statut != "Actif")
            throw new InvalidOperationException($"Compte {compte.Statut}. Opératio
n impossible.");

        var analyse = await _conformite.AnalyserTransactionAsync(
            vm.CompteId, vm.Montant, "Depot");

        if (analyse.EstBloquee)
            throw new InvalidOperationException(analyse.Motif);

        var fractionnement = await _conformite.DetecterFractionnementAsync(
            vm.CompteId, vm.Montant, DateTime.UtcNow);

        var surveillance = await _surveillance.AnalyserTransactionAsync(
            vm.CompteId, vm.Montant, "Depot", DateTime.UtcNow);

        await using var transaction = await _context.Database.BeginTransactionAsyn
c();
        try {
            var operation = new Operation {
                Reference = GenererReference("AKR"),
                TypeOperation = "Depot",
                CompteId = vm.CompteId,
                UtilisateurId = utilisateurId,
                AgenceId = compte.AgenceId,
                Montant = vm.Montant,
                Devise = compte.Devise,
                Statut = "Valide",
                DateOperation = DateTime.UtcNow,
                DateValidation = DateTime.UtcNow,
                Commentaire = vm.Commentaire?.Trim()
            };
            _context.Operations.Add(operation);
            await _context.SaveChangesAsync();
```

```
            var ecriture = new Ecriture {
                OperationId = operation.Id,
                CompteId = vm.CompteId,
                Sens = "Credit",
                Montant = vm.Montant,
                Devise = compte.Devise,
                DateEcriture = DateTime.UtcNow,
                Libelle = $"Dépôt sur compte {compte.NumeroCompte}"
            };
            _context.Ecritures.Add(ecriture);

            compte.Solde += vm.Montant;
            compte.DateModification = DateTime.UtcNow;

            await _context.SaveChangesAsync();
            await transaction.CommitAsync();

            await CreerAlertesSiNecessaireAsync(
                compte.ClientId, operation.Id, analyse, fractionnement, surveillan
ce);

            return operation;
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
    }

    // ================================================================
    // RETRAIT
    // ================================================================
    public async Task<Operation> RetirerAsync(RetraitViewModel vm, int utilisateur
Id)  {
        var compte = await _context.Comptes.FindAsync(vm.CompteId)
            ?? throw new InvalidOperationException("Compte introuvable.");

        if (compte.Statut != "Actif")
            throw new InvalidOperationException($"Compte {compte.Statut}. Opératio
n impossible.");
```

```
        if (compte.Solde < vm.Montant)
            throw new InvalidOperationException("Solde insuffisant.");

        var analyse = await _conformite.AnalyserTransactionAsync(
            vm.CompteId, vm.Montant, "Retrait");

        if (analyse.EstBloquee)
            throw new InvalidOperationException(analyse.Motif);

        var fractionnement = await _conformite.DetecterFractionnementAsync(
            vm.CompteId, vm.Montant, DateTime.UtcNow);

        var surveillance = await _surveillance.AnalyserTransactionAsync(
            vm.CompteId, vm.Montant, "Retrait", DateTime.UtcNow);

        await using var transaction = await _context.Database.BeginTransactionAsyn
c();
        try {
            var operation = new Operation {
                Reference = GenererReference("AKR"),
                TypeOperation = "Retrait",
                CompteId = vm.CompteId,
                UtilisateurId = utilisateurId,
                AgenceId = compte.AgenceId,
                Montant = vm.Montant,
                Devise = compte.Devise,
                Statut = "Valide",
                DateOperation = DateTime.UtcNow,
                DateValidation = DateTime.UtcNow,
                Commentaire = vm.Commentaire?.Trim()
            };
            _context.Operations.Add(operation);
            await _context.SaveChangesAsync();

            var ecriture = new Ecriture {
                OperationId = operation.Id,
                CompteId = vm.CompteId,
                Sens = "Debit",
                Montant = vm.Montant,
                Devise = compte.Devise,
                DateEcriture = DateTime.UtcNow,
                Libelle = $"Retrait du compte {compte.NumeroCompte}"
```

```
            };
            _context.Ecritures.Add(ecriture);

            compte.Solde -= vm.Montant;
            compte.DateModification = DateTime.UtcNow;

            await _context.SaveChangesAsync();
            await transaction.CommitAsync();

            await CreerAlertesSiNecessaireAsync(
                compte.ClientId, operation.Id, analyse, fractionnement, surveillan
ce);

            return operation;
        }
        catch {
            await transaction.RollbackAsync();
            throw;
        }
    }

    // ================================================================
    // VIREMENT
    // ================================================================
    public async Task<Operation> VirerAsync(VirementViewModel vm, int utilisateurI
d)  {
        if (vm.CompteSourceId == vm.CompteDestinationId)
            throw new InvalidOperationException(
                "Les comptes source et destination doivent être différents.");

        var source = await _context.Comptes.FindAsync(vm.CompteSourceId)
            ?? throw new InvalidOperationException("Compte source introuvable.");
        var destination = await _context.Comptes.FindAsync(vm.CompteDestinationId)
            ?? throw new InvalidOperationException("Compte destination introuvable
.");

        if (source.Statut != "Actif" || destination.Statut != "Actif")
            throw new InvalidOperationException("Les deux comptes doivent être act
ifs.");

        if (source.Devise != destination.Devise)
```

```
            throw new InvalidOperationException("Les devises des deux comptes doiv
ent être identiques.");

        if (source.Solde < vm.Montant)
            throw new InvalidOperationException("Solde insuffisant sur le compte s
ource.");

        var analyse = await _conformite.AnalyserTransactionAsync(
            vm.CompteSourceId, vm.Montant, "Virement");

        if (analyse.EstBloquee)
            throw new InvalidOperationException(analyse.Motif);

        var fractionnement = await _conformite.DetecterFractionnementAsync(
            vm.CompteSourceId, vm.Montant, DateTime.UtcNow);

        var surveillance = await _surveillance.AnalyserTransactionAsync(
            vm.CompteSourceId, vm.Montant, "Virement", DateTime.UtcNow);

        await using var transaction = await _context.Database.BeginTransactionAsyn
c();
        try {
            // 1. Virement
            var virement = new Virement {
                Reference = GenererReference("VIR"),
                CompteSourceId = vm.CompteSourceId,
                CompteDestinationId = vm.CompteDestinationId,
                UtilisateurId = utilisateurId,
                Montant = vm.Montant,
                Devise = source.Devise,
                Statut = "Valide",
                Motif = vm.Motif?.Trim(),
                ControleAML = analyse.NecessiteVigilanceRenforcee || fractionnemen
t,
                DateVirement = DateTime.UtcNow,
                DateValidation = DateTime.UtcNow
            };
            _context.Virements.Add(virement);
            await _context.SaveChangesAsync();
            // 2. Opération métier
            var operation = new Operation {
                Reference = virement.Reference,
```

```
            TypeOperation = "Virement",
            CompteId = vm.CompteSourceId,
            UtilisateurId = utilisateurId,
            AgenceId = source.AgenceId,
            VirementId = virement.Id,
            Montant = vm.Montant,
            Devise = source.Devise,
            Statut = "Valide",
            DateOperation = DateTime.UtcNow,
            DateValidation = DateTime.UtcNow,
            Commentaire = vm.Motif?.Trim()
        };
        _context.Operations.Add(operation);
        await _context.SaveChangesAsync();
        // 3. Écritures (partie double)
        var debit = new Ecriture {
            OperationId = operation.Id,
            CompteId = vm.CompteSourceId,
            Sens = "Debit",
            Montant = vm.Montant,
            Devise = source.Devise,
            DateEcriture = DateTime.UtcNow,
            Libelle = $"Virement sortant vers {destination.NumeroCompte}"
        };
        var credit = new Ecriture
        {
            OperationId = operation.Id,
            CompteId = vm.CompteDestinationId,
            Sens = "Credit",
            Montant = vm.Montant,
            Devise = destination.Devise,
            DateEcriture = DateTime.UtcNow,
            Libelle = $"Virement entrant depuis {source.NumeroCompte}"
        };
        _context.Ecritures.Add(debit);
        _context.Ecritures.Add(credit);

        VerifierEquilibre(new[] { debit, credit });

        // 4. Mise à jour des soldes
        source.Solde -= vm.Montant;
        destination.Solde += vm.Montant;
```

```
            source.DateModification = DateTime.UtcNow;
            destination.DateModification = DateTime.UtcNow;

            await _context.SaveChangesAsync();
            await transaction.CommitAsync();

            await CreerAlertesSiNecessaireAsync(
                source.ClientId, operation.Id, analyse, fractionnement, surveillan
ce);

            return operation;
        }
        catch  {
            await transaction.RollbackAsync();
            throw;
        }
    }

    // ================================================================
    // CONSULTATION
    // ================================================================
    public async Task<JournalViewModel> GetJournalAsync(
        DateTime? du, DateTime? au, string? typeOperation, int page, int taille) {
        var query = _context.Operations
            .AsNoTracking()
            .Include(o => o.Compte)
            .Include(o => o.Utilisateur)
            .Include(o => o.Ecritures)
            .AsQueryable();

        if (du.HasValue)
            query = query.Where(o => o.DateOperation >= du.Value);
        if (au.HasValue)
            query = query.Where(o => o.DateOperation <= au.Value);
        if (!string.IsNullOrWhiteSpace(typeOperation))
            query = query.Where(o => o.TypeOperation == typeOperation);

        var total = await query.CountAsync();
        var operations = await query
            .OrderByDescending(o => o.DateOperation)
            .Skip((page - 1) * taille)
            .Take(taille)
```

```
            .ToListAsync();

        return new JournalViewModel {
            Operations = operations,
            Du = du,
            Au = au,
            TypeOperation = typeOperation,
            Page = page,
            TaillePage = taille,
            Total = total
        };
    }

    public async Task<ReleveViewModel?> GetReleveAsync(int compteId) {
        var compte = await _context.Comptes
            .AsNoTracking()
            .Include(c => c.Client)
            .FirstOrDefaultAsync(c => c.Id == compteId);

        if (compte is null) return null;

        var ecritures = await _context.Ecritures
            .AsNoTracking()
            .Include(e => e.Operation)
            .Where(e => e.CompteId == compteId)
            .OrderBy(e => e.DateEcriture)
            .ToListAsync();

        var totalCredits = ecritures.Where(e => e.Sens == "Credit").Sum(e => e.Mon
tant);
        var totalDebits = ecritures.Where(e => e.Sens == "Debit").Sum(e => e.Monta
nt);
        var soldeInitial = compte.Solde - totalCredits + totalDebits;

        return new releveViewModel {
            Compte = compte,
            SoldeInitial = soldeInitial,
            SoldeFinal = compte.Solde,
            Ecritures = ecritures
        };
    }
```

```
    public async Task<decimal> GetSoldeAsync(int compteId) {
        var compte = await _context.Comptes
            .AsNoTracking()
            .FirstOrDefaultAsync(c => c.Id == compteId)
            ?? throw new InvalidOperationException("Compte introuvable.");

        return compte.Solde;
    }

    public async Task<bool> VerifierEquilibreGlobalAsync() {
        var desequilibres = await _context.Ecritures
            .GroupBy(e => e.OperationId)
            .Select(g => new
            {
                OperationId = g.Key,
                Diff = g.Sum(e => e.Sens == "Debit" ? e.Montant : -e.Montant)
            })
            .Where(x => x.Diff != 0)
            .ToListAsync();

        return desequilibres.Count == 0;
    }
    // ================================================================
    // UTILITAIRES PRIVÉS
    // ================================================================
    private async Task CreerAlertesSiNecessaireAsync(
        int clientId, int operationId,
        ResultatConformite analyse, bool fractionnement, ResultatSurveillance surv
eillance) {
        if (analyse.NecessiteDeclaration) {
            await _declaration.CreerAlerteAsync(
                clientId, operationId,
                "DepassementSeuil",
                analyse.Niveau.ToString(),
                analyse.Motif ?? "Dépassement du seuil AML.");
        }

        if (fractionnement) {
            await _declaration.CreerAlerteAsync(
                clientId, operationId,
                "Fractionnement",
```

```
                "Eleve", "Suspicion de fractionnement (structuring) : plusieurs op
                 érations juste en dessous du seuil sur une courte période.");
        }

        foreach (var alerte in surveillance.Alertes) {
            await _declaration.CreerAlerteAsync(
                clientId, operationId,
                alerte.Regle,
                alerte.Niveau.ToString(),
                alerte.Description);
        }
    }

    private static string GenererReference(string prefix) {
        var date = DateTime.UtcNow.ToString("yyyyMMdd");
        var random = Guid.NewGuid().ToString("N")[..6].ToUpperInvariant();
        return $"{prefix}-{date}-{random}";
    }

    private static void VerifierEquilibre(IEnumerable<Ecriture> ecritures) {
        var totalDebit = ecritures.Where(e => e.Sens == "Debit").Sum(e => e.Montan
t);
        var totalCredit = ecritures.Where(e => e.Sens == "Credit").Sum(e => e.Mont
ant);

        if (totalDebit != totalCredit)
            throw new InvalidOperationException(
                $"Écriture déséquilibrée : débit={totalDebit:N2}, crédit={totalCre
dit:N2}.");
    }
}
```

8.9. Implémentation du service de conformité

Fichier à créer : `Services/ConformiteService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class ConformiteService : IConformiteService {
    private readonly AkarangaBankContext _context;

    private const decimal SeuilVigilanceRenforcee = 5_000_000m;
    private const decimal SeuilDeclarationEspece = 10_000_000m;
```

```
    public ConformiteService(AkarangaBankContext context) {
        _context = context;
    }

    public async Task<decimal> GetSeuilAMLAsync() {
        var param = await _context.Parametres
            .AsNoTracking()
            .FirstOrDefaultAsync(p => p.Cle == "SEUIL_AML");

        if (param is null) return SeuilVigilanceRenforcee;

        return decimal.TryParse(param.Valeur, out var seuil)
            ? seuil
            : SeuilVigilanceRenforcee;
    }

    public async Task<ResultatConformite> AnalyserTransactionAsync(
        int compteId, decimal montant, string typeOperation) {
        var compte = await _context.Comptes
            .Include(c => c.Client)
            .FirstOrDefaultAsync(c => c.Id == compteId)
            ?? throw new InvalidOperationException("Compte introuvable.");

        var seuilAML = await GetSeuilAMLAsync();

        var resultat = new ResultatConformite {
            Montant = montant,
            CompteId = compteId,
            DateAnalyse = DateTime.UtcNow
        };

        // Règle 1 : dépôt/retrait espèces au-dessus du seuil de déclaration
        if (typeOperation is "Depot" or "Retrait" && montant >= SeuilDeclarationEs
pece)
        {
            resultat.NecessiteDeclaration = true;
            resultat.Motif = $"Transaction en espèces ≥ {SeuilDeclarationEspece:N0
} BIF. " +
                             "Déclaration obligatoire à la CNRF.";
            resultat.Niveau = NiveauAlerte.Eleve;
        }

        // Règle 2 : vigilance renforcée
        if (montant >= seuilAML)
        {
            resultat.NecessiteVigilanceRenforcee = true;
            resultat.Motif ??= $"Montant ≥ {seuilAML:N0} BIF (seuil AML). " +
                               "Vérification de l'origine des fonds requise.";
            resultat.Niveau = NiveauAlerte.Moyen;
        }
```

```
        // Règle 3 : solde insuffisant
        if (typeOperation is "Retrait" or "Virement" && compte.Solde < montant)
        {
            resultat.EstBloquee = true;
            resultat.Motif = "Solde insuffisant.";
            resultat.Niveau = NiveauAlerte.Critique;
        }

        // Règle 4 : compte non actif
        if (compte.Statut != "Actif")
        {
            resultat.EstBloquee = true;
            resultat.Motif = $"Compte {compte.Statut}. Opération impossible.";
            resultat.Niveau = NiveauAlerte.Critique;
        }

        return resultat;
    }

    public async Task<bool> DetecterFractionnementAsync(
        int compteId, decimal montant, DateTime date)
    {
        var seuilAML = await GetSeuilAMLAsync();
        var debut = date.AddHours(-24);

        var totalRecent = await _context.Ecritures
            .Where(e => e.DateEcriture >= debut && e.DateEcriture <= date)
            .Where(e => e.CompteId == compteId)
            .Where(e => e.Montant < seuilAML)
            .SumAsync(e => (decimal?)e.Montant) ?? 0m;

        return totalRecent + montant >= seuilAML && montant < seuilAML;
    }

    // ============================================================
    // ÉVALUATION DU RISQUE CLIENT
    // ============================================================
    public async Task<NiveauRisqueClient> EvaluerRisqueClientAsync(int clientId)
    {
        var client = await _context.Clients
            .FirstOrDefaultAsync(c => c.Id == clientId);

        if (client is null) return NiveauRisqueClient.Inconnu;

        var score = 0;

        // Nombre de comptes (requête directe car Client.Comptes n'existe pas)
        var nbComptes = await _context.Comptes
            .CountAsync(c => c.ClientId == clientId);
```

```
        if (nbComptes > 3) score += 2;

        // Transactions élevées
        var aTransactionsElevees = await _context.Ecritures
            .Where(e => e.Compte.ClientId == clientId)
            .AnyAsync(e => e.Montant >= SeuilVigilanceRenforcee);

        if (aTransactionsElevees) score += 3;

        // Alertes AML
        var aAlertes = await _context.AlertesAML
            .AnyAsync(a => a.ClientId == clientId);
        if (aAlertes) score += 5;

        return score switch
        {
            >= 6 => NiveauRisqueClient.Eleve,
            >= 3 => NiveauRisqueClient.Moyen,
            _ => NiveauRisqueClient.Faible
        };
    }
}
```

8.10. Implémentation du service de surveillance

Fichier à créer : `Services/SurveillanceService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class SurveillanceService : ISurveillanceService
{
    private readonly AkarangaBankContext _context;
    private readonly IConformiteService _conformite;

    public SurveillanceService(
        AkarangaBankContext context,
        IConformiteService conformite)
    {
        _context = context;
        _conformite = conformite;
    }

    public async Task<ResultatSurveillance> AnalyserTransactionAsync(
        int compteId, decimal montant, string typeOperation, DateTime date)
    {
        var alertes = new List<AlerteSurveillance>();
        var compte = await _context.Comptes
            .Include(c => c.Client)
```

```
            .FirstOrDefaultAsync(c => c.Id == compteId);

        if (compte is null)
            return new ResultatSurveillance { Alertes = alertes };

        var seuilAML = await _conformite.GetSeuilAMLAsync();

        // Règle 1 : montant élevé
        if (montant >= seuilAML)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "MontantEleve",
                Description = $"Transaction de {montant:N0} BIF dépasse le seuil A
ML.",
                Niveau = NiveauAlerte.Moyen
            });
        }

        // Règle 2 : fréquence inhabituelle
        var debut = date.AddDays(-7);
        var nbTransactions = await _context.Operations
            .Where(o => o.DateOperation >= debut && o.DateOperation <= date)
            .Where(o => o.CompteId == compteId)
            .CountAsync();

        if (nbTransactions > 20)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "FrequenceElevee",
                Description = $"{nbTransactions} transactions sur ce compte en 7 j
ours.",
                Niveau = NiveauAlerte.Eleve
            });
        }

        // Règle 3 : client à risque
        var risqueClient = await _conformite.EvaluerRisqueClientAsync(compte.Clien
tId);
        if (risqueClient == NiveauRisqueClient.Eleve)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "ClientARisque",
                Description = "Le client associé est classé à risque élevé.",
                Niveau = NiveauAlerte.Eleve
            });
        }

        // Règle 4 : montant inhabituel
```

```
        var moyenne = await GetMoyenneTransactionsAsync(compteId);
        if (moyenne > 0 && montant > moyenne * 5)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "MontantInhabituel",
                Description = $"Montant {montant:N0} BIF > 5x la moyenne ({moyenne
:N0} BIF).",
                Niveau = NiveauAlerte.Moyen
            });
        }

        return new ResultatSurveillance
        {
            CompteId = compteId,
            DateAnalyse = DateTime.UtcNow,
            Alertes = alertes,
            NecessiteDeclaration = alertes.Any(a => a.Niveau >= NiveauAlerte.Eleve
)
        };
    }

    public async Task<NiveauRisqueClient> EvaluerRisqueClientAsync(int clientId) {
        return await _conformite.EvaluerRisqueClientAsync(clientId);
    }

    private async Task<decimal> GetMoyenneTransactionsAsync(int compteId) {
        var montants = await _context.Operations
            .Where(o => o.CompteId == compteId)
            .Select(o => o.Montant)
            .ToListAsync();

        return montants.Count > 0 ? montants.Average() : 0m;
    }
}
```

8.11. Implémentation du service de déclaration

Fichier à créer : `Services/DeclarationService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class DeclarationService : IDeclarationService {
    private readonly AkarangaBankContext _context;

    public DeclarationService(AkarangaBankContext context)  {
        _context = context;
    }
```

```
    public async Task<AlerteAML> CreerAlerteAsync(
        int clientId, int? operationId, string typeAlerte,
        string niveau, string description) {
        var alerte = new AlerteAML {
            ClientId = clientId,
            OperationId = operationId,
            TypeAlerte = typeAlerte,
            Niveau = niveau,
            Description = description,
            Statut = "Ouverte",
            DateAlerte = DateTime.UtcNow
        };

        _context.AlertesAML.Add(alerte);
        await _context.SaveChangesAsync();
        return alerte;
    }

    public async Task<bool> TraiterAlerteAsync(
        int alerteId, int traiteParId, string statut, string? commentaire){
        var alerte = await _context.AlertesAML.FindAsync(alerteId);
        if (alerte is null) return false;

        alerte.Statut = statut;
        alerte.TraiteParId = traiteParId;
        alerte.DateTraitement = DateTime.UtcNow;

        if (!string.IsNullOrWhiteSpace(commentaire))
            alerte.Description = (alerte.Description ?? "") + " | " + commentaire;

        await _context.SaveChangesAsync();
        return true;
    }
    public async Task<IEnumerable<AlerteAML>> GetAlertesAsync(string? statut, stri
ng? niveau) {
        var query = _context.AlertesAML
            .Include(a => a.Client)
            .Include(a => a.Operation)
            .Include(a => a.TraitePar)
            .AsNoTracking()
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(statut))
            query = query.Where(a => a.Statut == statut);

        if (!string.IsNullOrWhiteSpace(niveau))
            query = query.Where(a => a.Niveau == niveau);

        return await query.OrderByDescending(a => a.DateAlerte).ToListAsync();
    }
}
```

8.12. Service pour l'utilisateur courant

📄 **Fichier à créer :** `Services/CurrentUserService.cs`

```
using Microsoft.AspNetCore.Identity;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class CurrentUserService : ICurrentUserService {
    private readonly IHttpContextAccessor _accessor;
    private readonly UserManager<ApplicationUser> _userManager;

    public CurrentUserService(
        IHttpContextAccessor accessor,
        UserManager<ApplicationUser> userManager){
        _accessor = accessor;
        _userManager = userManager;
    }

    public async Task<int> GetUserIdAsync(){
        var user = await _userManager.GetUserAsync(_accessor.HttpContext!.User);
        return user?.Id ?? throw new InvalidOperationException("Utilisateur non co
nnecté.");
    }

    public async Task<int> GetAgenceIdAsync(){
        var user = await _userManager.GetUserAsync(_accessor.HttpContext!.User);
        return user?.AgenceId ?? throw new InvalidOperationException("Utilisateur
non connecté.");
    }
}
```

8.13. Enregistrement des services dans `Program.cs`

➖ **Fichier :** `Program.cs` — **ajouter dans la section « Services métier » :**

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUserService, CurrentUserService>();
builder.Services.AddScoped<IConformiteService, ConformiteService>();
builder.Services.AddScoped<ISurveillanceService, SurveillanceService>();
builder.Services.AddScoped<IDeclarationService, DeclarationService>();
builder.Services.AddScoped<IComptabiliteService, ComptabiliteService>();
```

Partie D — Contrôleurs et vues

8.14. Contrôleur des opérations

Fichier à créer : `Controllers/OperationsController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Controllers;

[Authorize]
public class OperationsController : Controller {
    private readonly IComptabiliteService _comptabilite;
    private readonly ICurrentUserService _currentUser;
    private readonly AkarangaBankContext _context;

    public OperationsController(
        IComptabiliteService comptabilite,
        ICurrentUserService currentUser,
        AkarangaBankContext context) {
        _comptabilite = comptabilite;
        _currentUser = currentUser;
        _context = context;
    }

    // ------------------------------------------------------------------
    // DÉPÔT
    // ------------------------------------------------------------------
    [HttpGet]
    [Authorize(Policy = "GuichetierOnly")]
    public async Task<IActionResult> Depot() {
        await ChargerComptesAsync();
        return View(new DepotViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "GuichetierOnly")]
    public async Task<IActionResult> Depot(DepotViewModel vm){
        if (!ModelState.IsValid) {
            await ChargerComptesAsync(vm.CompteId);
            return View(vm);
        }

        try {
            var userId = await _currentUser.GetUserIdAsync();
```

```
            var operation = await _comptabilite.DeposerAsync(vm, userId);
            TempData["Success"] = $"Dépôt enregistré (réf. {operation.Reference}).
";
            return RedirectToAction(nameof(Journal));
        }
        catch (InvalidOperationException ex) {
            ModelState.AddModelError("", ex.Message);
            await ChargerComptesAsync(vm.CompteId);
            return View(vm);
        }
    }

    // ---------------------------------------------------------------
    // RETRAIT
    // ---------------------------------------------------------------
    [HttpGet]
    [Authorize(Policy = "GuichetierOnly")]
    public async Task<IActionResult> Retrait() {
        await ChargerComptesAsync();
        return View(new RetraitViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "GuichetierOnly")]
    public async Task<IActionResult> Retrait(RetraitViewModel vm)
    {
        if (!ModelState.IsValid)
        {
            await ChargerComptesAsync(vm.CompteId);
            return View(vm);
        }

        try
        {
            var userId = await _currentUser.GetUserIdAsync();
            var operation = await _comptabilite.RetirerAsync(vm, userId);
            TempData["Success"] = $"Retrait enregistré (réf. {operation.Reference}
).";
            return RedirectToAction(nameof(Journal));
        }
        catch (InvalidOperationException ex)
        {
            ModelState.AddModelError("", ex.Message);
            await ChargerComptesAsync(vm.CompteId);
            return View(vm);
        }
    }

    // ---------------------------------------------------------------
    // VIREMENT
```

```
    // ------------------------------------------------------------
    [HttpGet]
    [Authorize(Policy = "GuichetierOnly")]
    public async Task<IActionResult> Virement()
    {
        await ChargerComptesAsync();
        return View(new VirementViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "GuichetierOnly")]
    public async Task<IActionResult> Virement(VirementViewModel vm)
    {
        if (!ModelState.IsValid)
        {
            await ChargerComptesAsync(vm.CompteSourceId, vm.CompteDestinationId);
            return View(vm);
        }

        try
        {
            var userId = await _currentUser.GetUserIdAsync();
            var operation = await _comptabilite.VirerAsync(vm, userId);
            TempData["Success"] = $"Virement enregistré (réf. {operation.Reference
}).";
            return RedirectToAction(nameof(Journal));
        }
        catch (InvalidOperationException ex)
        {
            ModelState.AddModelError("", ex.Message);
            await ChargerComptesAsync(vm.CompteSourceId, vm.CompteDestinationId);
            return View(vm);
        }
    }

    // ------------------------------------------------------------
    // JOURNAL
    // ------------------------------------------------------------
    [HttpGet]
    public async Task<IActionResult> Journal(
        DateTime? du, DateTime? au, string? typeOperation, int page = 1)
    {
        const int taille = 20;
        var vm = await _comptabilite.GetJournalAsync(du, au, typeOperation, page,
taille);
        return View(vm);
    }

    // ------------------------------------------------------------
    // RELEVÉ
```

```
    // ------------------------------------------------------------------
    [HttpGet]
    public async Task<IActionResult> Releve(int id)
    {
        var vm = await _comptabilite.GetReleveAsync(id);
        if (vm is null) return NotFound();
        return View(vm);
    }

    // ------------------------------------------------------------------
    // CONTRÔLE D'ÉQUILIBRE
    // ------------------------------------------------------------------
    [HttpGet]
    [Authorize(Policy = "AdminOnly")]
    public async Task<IActionResult> ControleEquilibre()
    {
        var ok = await _comptabilite.VerifierEquilibreGlobalAsync();
        ViewBag.Equilibre = ok;
        return View();
    }

    // ------------------------------------------------------------------
    // UTILITAIRES
    // ------------------------------------------------------------------
    private async Task ChargerComptesAsync(int? sourceId = null, int? destinationI
d = null)
    {
        var comptes = await _context.Comptes
            .Include(c => c.Client)
            .Include(c => c.Agence)
            .Where(c => c.Statut == "Actif")
            .OrderBy(c => c.NumeroCompte)
            .Select(c => new
            {
                c.Id,
                Libelle = $"{c.NumeroCompte} — {c.Client.Nom} {c.Client.Prenom} ({
c.Solde:N2} {c.Devise})"
            })
            .ToListAsync();

        ViewBag.Comptes = new SelectList(comptes, "Id", "Libelle", sourceId);
        ViewBag.ComptesDestination = new SelectList(comptes, "Id", "Libelle", dest
inationId);
    }
}
```

8.15. Contrôleur de conformité

Fichier à créer : `Controllers/ConformiteController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using AkarangaBank.Web.Services;

namespace AkarangaBank.Web.Controllers;

[Authorize(Policy = "AdminOnly")]
public class ConformiteController : Controller {
    private readonly IDeclarationService _declarationService;
    private readonly IConformiteService _conformiteService;
    private readonly ICurrentUserService _currentUser;

    public ConformiteController(
        IDeclarationService declarationService,
        IConformiteService conformiteService,
        ICurrentUserService currentUser) {
        _declarationService = declarationService;
        _conformiteService = conformiteService;
        _currentUser = currentUser;
    }

    public async Task<IActionResult> Alertes(string? statut, string? niveau) {
        var alertes = await _declarationService.GetAlertesAsync(statut, niveau);
        ViewBag.Statut = statut;
        ViewBag.Niveau = niveau;
        return View(alertes);
    }

    public async Task<IActionResult> Details(int id) {
        var alertes = await _declarationService.GetAlertesAsync(null, null);
        var alerte = alertes.FirstOrDefault(a => a.Id == id);
        if (alerte is null) return NotFound();
        return View(alerte);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Traiter(int id, string statut, string? commen
taire) {
        var userId = await _currentUser.GetUserIdAsync();
        var ok = await _declarationService.TraiterAlerteAsync(id, userId, statut,
commentaire);
        if (!ok) return NotFound();

        TempData["Success"] = "Alerte traitée.";
        return RedirectToAction(nameof(Alertes));
    }
```

```
    public async Task<IActionResult> RisqueClient(int id)
    {
        var risque = await _conformiteService.EvaluerRisqueClientAsync(id);
        ViewBag.ClientId = id;
        ViewBag.Risque = risque;
        return View();
    }
}
```

8.16. Vues

Views/Operations/Depot.cshtml

```
@model AkarangaBank.Web.ViewModels.DepotViewModel
@{ ViewData["Title"] = "Dépôt"; }

<h1>Dépôt</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<div class="row">
    <div class="col-md-6">
        <form asp-action="Depot" method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <div class="mb-3">
                <label asp-for="CompteId" class="form-label">Compte</label>
                <select asp-for="CompteId" asp-items="ViewBag.Comptes" class="form
-select">
                    <option value="">-- Sélectionner --</option>
                </select>
                <span asp-validation-for="CompteId" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Montant" class="form-label">Montant (BIF)</label>
                <input asp-for="Montant" class="form-control" type="number" step="
0.01" />
                <span asp-validation-for="Montant" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Commentaire" class="form-label">Commentaire</label
>
                <textarea asp-for="Commentaire" class="form-control" rows="2"></te
xtarea>
            </div>
```

```
            <button type="submit" class="btn btn-primary">Enregistrer le dépôt</bu
tton>
            <a asp-action="Journal" class="btn btn-secondary">Annuler</a>
        </form>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Views/Operations/Retrait.cshtml

```
@model AkarangaBank.Web.ViewModels.RetraitViewModel
@{ ViewData["Title"] = "Retrait"; }

<h1>Retrait</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<div class="row">
    <div class="col-md-6">
        <form asp-action="Retrait" method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <div class="mb-3">
                <label asp-for="CompteId" class="form-label">Compte</label>
                <select asp-for="CompteId" asp-items="ViewBag.Comptes" class="form
-select">
                    <option value="">-- Sélectionner --</option>
                </select>
                <span asp-validation-for="CompteId" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Montant" class="form-label">Montant (BIF)</label>
                <input asp-for="Montant" class="form-control" type="number" step="
0.01" />
                <span asp-validation-for="Montant" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Commentaire" class="form-label">Commentaire</label
>
                <textarea asp-for="Commentaire" class="form-control" rows="2"></te
xtarea>
            </div>
```

```
            <button type="submit" class="btn btn-primary">Enregistrer le retrait</
button>
            <a asp-action="Journal" class="btn btn-secondary">Annuler</a>
        </form>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Views/Operations/Virement.cshtml

```
@model AkarangaBank.Web.ViewModels.VirementViewModel
@{ ViewData["Title"] = "Virement"; }

<h1>Virement</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<div class="row">
    <div class="col-md-6">
        <form asp-action="Virement" method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <div class="mb-3">
                <label asp-for="CompteSourceId" class="form-label">Compte source</
label>
                <select asp-for="CompteSourceId" asp-items="ViewBag.Comptes" class
="form-select">
                    <option value="">-- Sélectionner --</option>
                </select>
                <span asp-validation-for="CompteSourceId" class="text-danger"></sp
an>
            </div>

            <div class="mb-3">
                <label asp-for="CompteDestinationId" class="form-label">Compte des
tination</label>
                <select asp-for="CompteDestinationId" asp-items="ViewBag.ComptesDe
stination" class="form-select">
                    <option value="">-- Sélectionner --</option>
                </select>
                <span asp-validation-for="CompteDestinationId" class="text-danger"
></span>
            </div>

            <div class="mb-3">
```

```
                <label asp-for="Montant" class="form-label">Montant (BIF)</label>
                <input asp-for="Montant" class="form-control" type="number" step="
0.01" />
                <span asp-validation-for="Montant" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Motif" class="form-label">Motif</label>
                <input asp-for="Motif" class="form-control" />
            </div>

            <button type="submit" class="btn btn-primary">Enregistrer le virement<
/button>
            <a asp-action="Journal" class="btn btn-secondary">Annuler</a>
        </form>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Views/Operations/Journal.cshtml

```
@model AkarangaBank.Web.ViewModels.JournalViewModel
@{ ViewData["Title"] = "Journal des opérations"; }

<h1>Journal des opérations</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<form method="get" class="row g-2 mb-3">
    <div class="col-md-2">
        <input type="date" name="du" value="@Model.Du?.ToString("yyyy-MM-dd")" cla
ss="form-control" />
    </div>
    <div class="col-md-2">
        <input type="date" name="au" value="@Model.Au?.ToString("yyyy-MM-dd")" cla
ss="form-control" />
    </div>
    <div class="col-md-2">
        <select name="typeOperation" class="form-select">
            <option value="">Tous les types</option>
```

```
            <option value="Depot" selected="@(Model.TypeOperation == "Depot")">Dép
ôt</option>
            <option value="Retrait" selected="@(Model.TypeOperation == "Retrait")"
>Retrait</option>
            <option value="Virement" selected="@(Model.TypeOperation == "Virement"
)">Virement</option>
        </select>
    </div>
    <div class="col-md-2">
        <button type="submit" class="btn btn-primary">Filtrer</button>
    </div>
</form>

<table class="table table-striped table-sm">
    <thead>
        <tr>
            <th>Date</th>
            <th>Référence</th>
            <th>Type</th>
            <th>Compte</th>
            <th class="text-end">Montant</th>
            <th>Statut</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
    @foreach (var op in Model.Operations)
    {
        <tr>
            <td>@op.DateOperation.ToString("dd/MM/yyyy HH:mm")</td>
            <td>@op.Reference</td>
            <td>@op.TypeOperation</td>
            <td>@op.Compte.NumeroCompte</td>
            <td class="text-end">@op.Montant.ToString("N2") @op.Devise</td>
            <td>
                <span class="badge @(op.Statut == "Valide" ? "bg-success" :
                                     op.Statut == "EnAttente" ? "bg-warning text-d
ark" : "bg-secondary")">
                    @op.Statut
                </span>
            </td>
            <td>
```

```
                <a asp-action="Releve" asp-route-id="@op.CompteId" class="btn btn-
sm btn-outline-primary">Relevé</a>
            </td>
        </tr>
    }
    </tbody>
</table>

@if (Model.TotalPages > 1)
{
    <nav>
        <ul class="pagination">
            @for (int i = 1; i <= Model.TotalPages; i++)
            {
                <li class="page-item @(i == Model.Page ? "active" : "")">
                    <a class="page-link" asp-action="Journal" asp-route-page="@i"
                       asp-route-du="@Model.Du?.ToString("yyyy-MM-dd")"
                       asp-route-au="@Model.Au?.ToString("yyyy-MM-dd")"
                       asp-route-typeOperation="@Model.TypeOperation">@i</a>
                </li>
            }
        </ul>
    </nav>
}
```

Views/Operations/Releve.cshtml

```
@model AkarangaBank.Web.ViewModels. releveViewModel
@{ ViewData["Title"] = "Relevé"; }

<h1>Relevé de compte</h1>

<div class="card mb-3">
    <div class="card-body">
        <h5>@Model.Compte.NumeroCompte</h5>
        <p>
            <strong>Solde initial :</strong> @Model.SoldeInitial.ToString("N2") @M
odel.Compte.Devise<br />
            <strong>Solde final :</strong> @Model.SoldeFinal.ToString("N2") @Model
.Compte.Devise
        </p>
    </div>
```

```
</div>

<table class="table table-striped table-sm">
    <thead>
        <tr>
            <th>Date</th>
            <th>Libellé</th>
            <th class="text-end">Débit</th>
            <th class="text-end">Crédit</th>
        </tr>
    </thead>
    <tbody>
    @foreach (var e in Model.Ecritures)
    {
        <tr>
            <td>@e.DateEcriture.ToString("dd/MM/yyyy HH:mm")</td>
            <td>@e.Libelle</td>
            <td class="text-end text-danger">@(e.Sens == "Debit" ? e.Montant.ToStr
ing("N2") : "")</td>
            <td class="text-end text-success">@(e.Sens == "Credit" ? e.Montant.ToS
tring("N2") : "")</td>
        </tr>
    }
    </tbody>
</table>

<a asp-action="Journal" class="btn btn-secondary">Retour</a>
```

Views/Operations/ControleEquilibre.cshtml

```
@{
    ViewData["Title"] = "Contrôle d'équilibre";
    var ok = (bool)ViewBag.Equilibre;
}

<h1>Contrôle d'équilibre comptable</h1>

@if (ok)
{
    <div class="alert alert-success">
        <h4>✅ La comptabilité est équilibrée.</h4>
    </div>
}
else
{
```

```
    <div class="alert alert-danger">
        <h4>❌ Déséquilibre détecté.</h4>
    </div>
}

<a asp-action="Journal" class="btn btn-secondary">Retour au journal</a>
```

📄 Views/Conformite/Alertes.cshtml

```
@model IEnumerable<AkarangaBank.Web.Models.AlerteAML>
@{ ViewData["Title"] = "Alertes AML"; }
<h1>Alertes AML</h1>
@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<table class="table table-striped">
    <thead>
        <tr>
            <th>Date</th>
            <th>Client</th>
            <th>Type</th>
            <th>Niveau</th>
            <th>Statut</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
    @foreach (var a in Model)
    {
        <tr>
            <td>@a.DateAlerte.ToString("dd/MM/yyyy HH:mm")</td>
            <td>@a.Client.Nom @a.Client.Prenom</td>
            <td>@a.TypeAlerte</td>
            <td>
                <span class="badge @(a.Niveau == "Critique" ? "bg-danger" :
                                     a.Niveau == "Eleve" ? "bg-warning text-dark"
: "bg-info text-dark")">
                    @a.Niveau
                </span>
            </td>
            <td>@a.Statut</td>
```

```
            <td><a asp-action="Details" asp-route-id="@a.Id" class="btn btn-sm btn
-primary">Détails</a></td>
        </tr>
    }
    </tbody>
</table>
```

Partie E — Tests manuels

Test 1 — Dépôt

1. Se connecter (admin ou gestionnaire).
2. Aller sur `/Operations/Depot`.
3. Compte BJM-0001-0001, montant 100 000.
4. Vérifier : 1 écriture Credit.

Test 2 — Retrait

1. Montant 50 000 sur le même compte.
2. Vérifier : 1 écriture `Debit`.

Test 3 — Retrait avec solde insuffisant

1. Montant > solde.
2. Message « Solde insuffisant. »

Test 4 — Virement

1. Source BJM-0001-0001 → destination BJM-0001-0002, montant 30 000.
2. Vérifier : 2 écritures symétriques (débit/crédit).
3. Total débit = total crédit.

Test 5 — Virement interdit

1. Source = destination.
2. Message « Les comptes source et destination doivent être différents. »

Test 6 — Dépôt sous le seuil AML

1. Dépôt de 1 000 000 BIF.
2. Aucune alerte.

Test 7 — Dépôt au-dessus du seuil AML

1. Dépôt de 6 000 000 BIF.
2. Alerte `DepassementSeuil`, niveau Moyen.

Test 8 — Fractionnement

1. 3 dépôts de 2 000 000 BIF dans la journée.
2. Alerte `Fractionnement`.

Test 9 — Contrôle d'équilibre

1. `/Operations/ControleEquilibre`.

2. Message « La comptabilité est équilibrée. »

Test 10 — Rôle insuffisant

1. Se connecter en client.
2. Tenter un dépôt → accès refusé.

Exercices

Exercice 1 — Ajouter les frais de retrait
1 % (min 100 BIF). Générer une opération `Frais` + écriture.

Exercice 2 — Export CSV des alertes
Bouton sur `/Conformite/Alertes`.

Exercice 3 — Journal d'audit
Enregistrer chaque création d'alerte dans AuditLog.

Exercice 4 — Rapport quotidien AML
Un `BackgroundService` qui génère chaque nuit un rapport des alertes.

Questions de compréhension

1. Qu'est-ce que la partie double ? Pourquoi est-elle essentielle ?
2. Différence entre Operation et Ecriture ?
3. Pourquoi une transaction EF Core est-elle nécessaire ?
4. Qu'est-ce que la LBC/FT ? Que fait la CNRF ?
5. Différence entre vigilance renforcée et déclaration obligatoire ?
6. Qu'est-ce que le fractionnement ? Comment le détecter ?
7. Comment évaluer le risque d'un client ?
8. Pourquoi ne jamais révéler au client qu'une alerte a été créée ?
9. Comment vérifier que la comptabilité est équilibrée ?
10. Pourquoi ApplicationUser doit-il hériter d'IdentityUser<int> ?

Mini-TP 8 — Cœur comptable et conformité

Objectif : implémenter les opérations avec partie double et les contrôles AML.

1. Créez les ViewModels.
2. Créez les services : `ComptabiliteService`, `ConformiteService`, `SurveillanceService`, `DeclarationService`, `CurrentUserService`.
3. Enregistrez tous les services dans Program.cs.
4. Créez `OperationsController` et `ConformiteController`.
5. Créez les vues.
6. Testez tous les cas de la Partie E.

CHAPITRE 9 — VIREMENTS INTERBANCAIRES, SEUILS AML AVANCÉS ET DÉCLARATIONS À LA CNRF

Objectifs du chapitre

- Distinguer virement **intra-agence**, **inter-agences** et **interbancaire**.
- Implémenter un virement **inter-agences** avec compensation.
- Mettre en place une **file d'attente** pour les virements différés.
- Générer et transmettre une **Déclaration d'Opération Suspecte (DOS)** à la CNRF.
- Configurer des **seuils AML multiples** (vigilance, déclaration, blocage).
- Détecter les **schémas de blanchiment** avancés (structuring, smurfing, layering).
- Produire un **rapport mensuel de conformité**.
- Gérer un **workflow de validation** à plusieurs niveaux.

Partie A — Typologie des virements

9.1. Trois niveaux de virements

Type	Source → Destination	Complexité	Compensation
Intra-agence	Compte A → Compte B, même agence	Simple	Aucune
Inter-agences	Compte A (BJM) → Compte B (GIT)	Moyenne	Oui
Interbancaire	Akaranga → Autre banque (via BRB)	Élevée	Oui (via BRB)

Règle comptable :

- **Intra-agence** : 2 écritures symétriques, pas de compte de passage.
- **Inter-agences** : 4 écritures (2 par agence + 2 sur le compte de compensation).
- **Interbancaire** : 4 écritures + 1 compte de règlement BRB.

9.1.1. Le compte de compensation

Pour les virements inter-agences, chaque agence possède un **compte de compensation** interne :

```
Agence AKR-BJM  : Compte 9000-0001 (Compensation BJM)
Agence AKR-GIT  : Compte 9000-0002 (Compensation GIT)
```

Principe pour un virement de 100 000 BIF de BJM vers GIT :

Agence BJM (source) :

- Débit : Compte client source (100 000)
- Crédit : Compte compensation BJM (100 000)

Agence GIT (destination) :

- Débit : Compte compensation GIT (100 000)
- Crédit : Compte client destination (100 000)

Total débit = Total crédit = 200 000 réparti sur 4 écritures.

Partie B — Modélisation étendue

9.2. Nouvelles entités

9.2.1. `ViremmentInterAgence`

Fichier à créer : `Models/VirementInterAgence.cs`

```
namespace AkarangaBank.Web.Models;

public class VirementInterAgence
{
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;

    public int VirementId { get; set; }
    public Virement Virement { get; set; } = null!;

    public int AgenceSourceId { get; set; }
    public Agence AgenceSource { get; set; } = null!;

    public int AgenceDestinationId { get; set; }
    public Agence AgenceDestination { get; set; } = null!;

    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";

    public string Statut { get; set; } = "EnAttente";
    // EnAttente, EnCompensation, Regle, Rejete

    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateCompensation { get; set; }
    public DateTime? DateReglement { get; set; }

    public string? MotifRejet { get; set; }
    public int? ValideParId { get; set; }
    public ApplicationUser? ValidePar { get; set; }
}
```

9.2.2. `CompensationAgence`

Fichier à créer : `Models/CompensationAgence.cs`

```
namespace AkarangaBank.Web.Models;

public class CompensationAgence
{
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;

    public int AgenceId { get; set; }
```

```
    public Agence Agence { get; set; } = null!;

    public int CompteCompensationId { get; set; }
    public Compte CompteCompensation { get; set; } = null!;

    public decimal TotalDebits { get; set; }
    public decimal TotalCredits { get; set; }
    public decimal Solde { get; set; }

    public DateTime DateCompensation { get; set; } = DateTime.UtcNow;
    public string Statut { get; set; } = "EnCours";
    // EnCours, Equilibre, Desequilibre, Regle
}
```

9.2.3. `DeclarationSoupcon` (DOS)

Fichier à créer : `Models/DeclarationSoupcon.cs`

```
namespace AkarangaBank.Web.Models;

public class DeclarationSoupcon
{
    public int Id { get; set; }
    public string Reference { get; set; } = string.Empty;

    public int AlerteId { get; set; }
    public AlerteAML Alerte { get; set; } = null!;

    public int ClientId { get; set; }
    public Client Client { get; set; } = null!;

    public string TypeDeclaration { get; set; } = "DOS";
    // DOS = Déclaration d'Opération Suspecte
    // DTSE = Déclaration de Transaction en Espèces

    public string Contenu { get; set; } = string.Empty;
    public string? MotifSuspicion { get; set; }

    public string Statut { get; set; } = "Brouillon";
    // Brouillon, Validee, Transmise, Accusee, Rejetee

    public DateTime DateCreation { get; set; } = DateTime.UtcNow;
    public DateTime? DateTransmission { get; set; }
    public DateTime? DateAccuseReception { get; set; }

    public int? RedigeParId { get; set; }
    public ApplicationUser? RedigePar { get; set; }

    public string? ReponseCNRF { get; set; }
}
```

9.2.4. `SeuilAML` (multi-niveaux)

📄 Fichier à créer : `Models/SeuilAML.cs`

```
namespace AkarangaBank.Web.Models;

public class SeuilAML
{
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    // VIGILANCE, DECLARATION, BLOCAGE

    public string Libelle { get; set; } = string.Empty;
    public decimal Montant { get; set; }
    public string Devise { get; set; } = "BIF";
    public string TypeOperation { get; set; } = "Toutes";
    public bool Actif { get; set; } = true;
    public DateTime DateModification { get; set; } = DateTime.UtcNow;
}
```

Données initiales :

```
INSERT INTO SeuilsAML (Code, Libelle, Montant, TypeOperation) VALUES
('VIGILANCE',   'Vigilance renforcée',        5000000,  'Toutes'),
('DECLARATION', 'Déclaration espèces CNRF',  10000000,  'Depot'),
('DECLARATION', 'Déclaration espèces CNRF',  10000000,  'Retrait'),
('BLOCAGE',     'Blocage automatique',       50000000,  'Toutes');
```

9.3. Configuration du DbContext

✏️ Fichier : `Data/AkarangaBankContext.cs` **— ajouter les DbSet :**

```
public DbSet<VirementInterAgence> VirementsInterAgences => Set<VirementInterAgence
>();
public DbSet<CompensationAgence> CompensationsAgences => Set<CompensationAgence>()
;
public DbSet<DeclarationSoupcon> DeclarationsSoupcon => Set<DeclarationSoupcon>();
public DbSet<SeuilAML> SeuilsAML => Set<SeuilAML>();
```

Puis, dans `OnModelCreating` :

```
// ----- VirementInterAgence -----
modelBuilder.Entity<VirementInterAgence>(b => {
    b.ToTable("VirementsInterAgences");
    b.HasKey(v => v.Id);
    b.Property(v => v.Reference).HasMaxLength(30).IsRequired();
```

```
    b.Property(v => v.Montant).HasColumnType("decimal(18,2)");
    b.Property(v => v.Devise).HasMaxLength(3).HasDefaultValue("BIF");
    b.Property(v => v.Statut).HasMaxLength(20).HasDefaultValue("EnAttente");
    b.Property(v => v.MotifRejet).HasMaxLength(500);

    b.HasIndex(v => v.Reference).IsUnique();
    b.HasIndex(v => v.Statut);

    b.HasOne(v => v.Virement).WithMany()
          .HasForeignKey(v => v.VirementId).OnDelete(DeleteBehavior.Restrict);

    b.HasOne(v => v.AgenceSource).WithMany()
          .HasForeignKey(v => v.AgenceSourceId).OnDelete(DeleteBehavior.Restrict);

    b.HasOne(v => v.AgenceDestination).WithMany()
          .HasForeignKey(v => v.AgenceDestinationId).OnDelete(DeleteBehavior.Restr
ict);

    b.HasOne(v => v.ValidePar).WithMany()
          .HasForeignKey(v => v.ValideParId).OnDelete(DeleteBehavior.SetNull);
});

// ----- CompensationAgence -----
modelBuilder.Entity<CompensationAgence>(b => {
    b.ToTable("CompensationsAgences");
    b.HasKey(c => c.Id);
    b.Property(c => c.Reference).HasMaxLength(30).IsRequired();
    b.Property(c => c.TotalDebits).HasColumnType("decimal(18,2)");
    b.Property(c => c.TotalCredits).HasColumnType("decimal(18,2)");
    b.Property(c => c.Solde).HasColumnType("decimal(18,2)");
    b.Property(c => c.Statut).HasMaxLength(20).HasDefaultValue("EnCours");

    b.HasIndex(c => c.DateCompensation);

    b.HasOne(c => c.Agence).WithMany()
          .HasForeignKey(c => c.AgenceId).OnDelete(DeleteBehavior.Restrict);

    b.HasOne(c => c.CompteCompensation).WithMany()
          .HasForeignKey(c => c.CompteCompensationId).OnDelete(DeleteBehavior.Rest
rict);
});

// ----- DeclarationSoupcon -----
modelBuilder.Entity<DeclarationSoupcon>(b => {
    b.ToTable("DeclarationsSoupcon");
    b.HasKey(d => d.Id);
    b.Property(d => d.Reference).HasMaxLength(30).IsRequired();
    b.Property(d => d.TypeDeclaration).HasMaxLength(10).IsRequired();
    b.Property(d => d.Contenu).HasColumnType("nvarchar(max)");
    b.Property(d => d.MotifSuspicion).HasMaxLength(1000);
    b.Property(d => d.Statut).HasMaxLength(20).HasDefaultValue("Brouillon");
```

```
    b.Property(d => d.ReponseCNRF).HasMaxLength(1000);

    b.HasIndex(d => d.Reference).IsUnique();

    b.HasOne(d => d.Alerte).WithMany()
          .HasForeignKey(d => d.AlerteId).OnDelete(DeleteBehavior.Restrict);

    b.HasOne(d => d.Client).WithMany()
          .HasForeignKey(d => d.ClientId).OnDelete(DeleteBehavior.Restrict);

    b.HasOne(d => d.RedigePar).WithMany()
          .HasForeignKey(d => d.RedigeParId).OnDelete(DeleteBehavior.SetNull);
});

// ----- SeuilAML -----
modelBuilder.Entity<SeuilAML>(b => {
    b.ToTable("SeuilsAML");
    b.HasKey(s => s.Id);
    b.Property(s => s.Code).HasMaxLength(20).IsRequired();
    b.Property(s => s.Libelle).HasMaxLength(100).IsRequired();
    b.Property(s => s.Montant).HasColumnType("decimal(18,2)");
    b.Property(s => s.Devise).HasMaxLength(3).HasDefaultValue("BIF");
    b.Property(s => s.TypeOperation).HasMaxLength(30).HasDefaultValue("Toutes");

    b.HasIndex(s => s.Code);
});
```

Générez la migration :

bash

```
dotnet ef migrations add AjoutChapitre9
dotnet ef database update
```

Vérifiez l'absence de shadow properties :

bash

```
findstr /S "Id1" Migrations\*.cs
```

Aucune ligne ne doit apparaître.

Partie C — Services

9.4. Service de compensation

📄 Fichier à créer : `Services/ICompensationService.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public interface ICompensationService {
    Task<CompensationAgence> CalculerCompensationAsync(int agenceId, DateTime date
);
```

```
    Task<bool> ReglerCompensationAsync(int compensationId, int valideParId);
    Task<IEnumerable<CompensationAgence>> GetCompensationsAsync(int? agenceId, Dat
eTime? date);
    Task<bool> VerifierEquilibreCompensationsAsync(DateTime date);
}
```

Fichier à créer : `Services/CompensationService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class CompensationService : ICompensationService
{
    private readonly AkarangaBankContext _context;

    public CompensationService(AkarangaBankContext context)
    {
        _context = context;
    }

    public async Task<CompensationAgence> CalculerCompensationAsync(int agenceId,
DateTime date)
    {
        var debut = date.Date;
        var fin = debut.AddDays(1);

        var virements = await _context.VirementsInterAgences
            .Where(v => v.DateCreation >= debut && v.DateCreation < fin)
            .Where(v => v.AgenceSourceId == agenceId || v.AgenceDestinationId == a
genceId)
            .Where(v => v.Statut == "EnCompensation")
            .ToListAsync();

        var totalCredits = virements
            .Where(v => v.AgenceDestinationId == agenceId)
            .Sum(v => v.Montant);

        var totalDebits = virements
            .Where(v => v.AgenceSourceId == agenceId)
            .Sum(v => v.Montant);

        var compteCompensation = await _context.Comptes
            .FirstOrDefaultAsync(c => c.NumeroCompte == $"9000-{agenceId:D4}")
            ?? throw new InvalidOperationException(
                $"Compte de compensation introuvable pour l'agence {agenceId}.");

        var compensation = new CompensationAgence
        {
```

```
            Reference = $"COMP-{date:yyyyMMdd}-{agenceId:D4}",
            AgenceId = agenceId,
            CompteCompensationId = compteCompensation.Id,
            TotalDebits = totalDebits,
            TotalCredits = totalCredits,
            Solde = totalCredits - totalDebits,
            DateCompensation = date,
            Statut = "EnCours"
        };

        _context.CompensationsAgences.Add(compensation);
        await _context.SaveChangesAsync();

        return compensation;
    }

    public async Task<bool> ReglerCompensationAsync(int compensationId, int valide
ParId)
    {
        var compensation = await _context.CompensationsAgences
            .Include(c => c.CompteCompensation)
            .FirstOrDefaultAsync(c => c.Id == compensationId);

        if (compensation is null) return false;
        if (compensation.Statut == "Regle") return true;

        await using var transaction = await _context.Database.BeginTransactionAsyn
c();
        try
        {
            if (compensation.Solde != 0)
            {
                var operationId = await GetOrCreateOperationReglementAsync(
                    compensation, valideParId);

                var ecriture = new Ecriture
                {
                    OperationId = operationId,
                    CompteId = compensation.CompteCompensationId,
                    Sens = compensation.Solde > 0 ? "Credit" : "Debit",
                    Montant = Math.Abs(compensation.Solde),
                    Devise = compensation.CompteCompensation.Devise,
                    DateEcriture = DateTime.UtcNow,
                    Libelle = $"Règlement compensation {compensation.Reference}"
                };
                _context.Ecritures.Add(ecriture);
            }

            compensation.Statut = "Regle";
            compensation.DateCompensation = DateTime.UtcNow;
```

```
            await _context.SaveChangesAsync();
            await transaction.CommitAsync();

            return true;
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
    }

    public async Task<IEnumerable<CompensationAgence>> GetCompensationsAsync(
        int? agenceId, DateTime? date)
    {
        var query = _context.CompensationsAgences
            .Include(c => c.Agence)
            .Include(c => c.CompteCompensation)
            .AsNoTracking()
            .AsQueryable();

        if (agenceId.HasValue)
            query = query.Where(c => c.AgenceId == agenceId.Value);

        if (date.HasValue)
            query = query.Where(c => c.DateCompensation.Date == date.Value.Date);

        return await query.OrderByDescending(c => c.DateCompensation).ToListAsync(
);
    }

    public async Task<bool> VerifierEquilibreCompensationsAsync(DateTime date)
    {
        var compensations = await _context.CompensationsAgences
            .Where(c => c.DateCompensation.Date == date.Date)
            .ToListAsync();

        var totalDebits = compensations.Sum(c => c.TotalDebits);
        var totalCredits = compensations.Sum(c => c.TotalCredits);

        return totalDebits == totalCredits;
    }

    private async Task<int> GetOrCreateOperationReglementAsync(
        CompensationAgence compensation, int valideParId)
    {
        var operation = new Operation
        {
            Reference = $"REG-{compensation.Reference}",
            TypeOperation = "ReglementCompensation",
            CompteId = compensation.CompteCompensationId,
```

```
            UtilisateurId = valideParId,
            AgenceId = compensation.AgenceId,
            Montant = Math.Abs(compensation.Solde),
            Devise = compensation.CompteCompensation.Devise,
            Statut = "Valide",
            DateOperation = DateTime.UtcNow,
            DateValidation = DateTime.UtcNow,
            Commentaire = $"Règlement compensation {compensation.Reference}"
        };

        _context.Operations.Add(operation);
        await _context.SaveChangesAsync();
        return operation.Id;
    }
}
```

9.5. Service de virements inter-agences

Fichier à créer : `Services/IViremmentInterAgenceService.cs`

```
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public interface IViremmentInterAgenceService {
    Task<ViremmentInterAgence> InitierAsync(
        ViremmentInterAgenceViewModel vm, int utilisateurId);

    Task<bool> ValiderAsync(int viremmentInterAgenceId, int valideParId);
    Task<bool> RejeterAsync(int viremmentInterAgenceId, int valideParId, string mot
if);

    Task<IEnumerable<ViremmentInterAgence>> GetEnAttenteAsync(int agenceId);
    Task<IEnumerable<ViremmentInterAgence>> GetHistoriqueAsync(
        int agenceId, DateTime? du, DateTime? au);
}
```

Fichier à créer : `Services/ViremmentInterAgenceService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.ViewModels;

namespace AkarangaBank.Web.Services;

public class ViremmentInterAgenceService : IViremmentInterAgenceService
```

```
{
    private readonly AkarangaBankContext _context;
    private readonly IConformiteService _conformite;

    public ViremmentInterAgenceService(
        AkarangaBankContext context,
        IConformiteService conformite)
    {
        _context = context;
        _conformite = conformite;
    }

    public async Task<ViremmentInterAgence> InitierAsync(
        ViremmentInterAgenceViewModel vm, int utilisateurId)
    {
        var source = await _context.Comptes
            .Include(c => c.Agence)
            .FirstOrDefaultAsync(c => c.Id == vm.CompteSourceId)
            ?? throw new InvalidOperationException("Compte source introuvable.");

        var destination = await _context.Comptes
            .Include(c => c.Agence)
            .FirstOrDefaultAsync(c => c.Id == vm.CompteDestinationId)
            ?? throw new InvalidOperationException("Compte destination introuvable
.");

        if (source.AgenceId == destination.AgenceId)
            throw new InvalidOperationException(
                "Pour un virement intra-agence, utilisez le virement standard.");

        if (source.Solde < vm.Montant)
            throw new InvalidOperationException("Solde insuffisant.");

        var analyse = await _conformite.AnalyserTransactionAsync(
            vm.CompteSourceId, vm.Montant, "ViremmentInterAgence");

        if (analyse.EstBloquee)
            throw new InvalidOperationException(analyse.Motif);

        await using var transaction = await _context.Database.BeginTransactionAsyn
c();
        try
        {
            // 1. Créer le virement standard
            var virement = new Virement
            {
                Reference = GenererReference("VIR"),
                CompteSourceId = vm.CompteSourceId,
                CompteDestinationId = vm.CompteDestinationId,
                UtilisateurId = utilisateurId,
                Montant = vm.Montant,
```

```
                Devise = source.Devise,
                Statut = "EnAttente",
                Motif = vm.Motif,
                ControleAML = analyse.NecessiteVigilanceRenforcee,
                DateVirement = DateTime.UtcNow
            };
            _context.Virements.Add(virement);
            await _context.SaveChangesAsync();

            // 2. Créer le virement inter-agence
            var virementInter = new VirementInterAgence
            {
                Reference = virement.Reference,
                VirementId = virement.Id,
                AgenceSourceId = source.AgenceId,
                AgenceDestinationId = destination.AgenceId,
                Montant = vm.Montant,
                Devise = source.Devise,
                Statut = "EnAttente",
                DateCreation = DateTime.UtcNow
            };
            _context.VirementsInterAgences.Add(virementInter);
            await _context.SaveChangesAsync();

            await transaction.CommitAsync();
            return virementInter;
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
    }

    public async Task<bool> ValiderAsync(int virementInterAgenceId, int valideParI
d)
    {
        var virementInter = await _context.VirementsInterAgences
            .Include(v => v.Virement)
                .ThenInclude(v => v.CompteSource)
            .Include(v => v.Virement)
                .ThenInclude(v => v.CompteDestination)
            .FirstOrDefaultAsync(v => v.Id == virementInterAgenceId);

        if (virementInter is null) return false;
        if (virementInter.Statut != "EnAttente") return false;

        var source = virementInter.Virement.CompteSource;
        var destination = virementInter.Virement.CompteDestination;

        if (source.Solde < virementInter.Montant)
```

```
            throw new InvalidOperationException("Solde insuffisant sur le compte s
ource.");

        await using var transaction = await _context.Database.BeginTransactionAsyn
c();
        try
        {
            // Créer l'opération métier
            var operation = new Operation
            {
                Reference = virementInter.Reference,
                TypeOperation = "VirementInterAgence",
                CompteId = source.Id,
                UtilisateurId = valideParId,
                AgenceId = source.AgenceId,
                VirementId = virementInter.VirementId,
                Montant = virementInter.Montant,
                Devise = virementInter.Devise,
                Statut = "Valide",
                DateOperation = DateTime.UtcNow,
                DateValidation = DateTime.UtcNow
            };
            _context.Operations.Add(operation);
            await _context.SaveChangesAsync();

            // Écritures : 4 lignes (2 par agence)
            var debitClient = new Ecriture
            {
                OperationId = operation.Id,
                CompteId = source.Id,
                Sens = "Debit",
                Montant = virementInter.Montant,
                Devise = source.Devise,
                Libelle = $"Virement inter-agence vers {destination.NumeroCompte}"
            };
            var creditCompSource = new Ecriture
            {
                OperationId = operation.Id,
                CompteId = await GetCompteCompensationIdAsync(source.AgenceId),
                Sens = "Credit",
                Montant = virementInter.Montant,
                Devise = source.Devise,
                Libelle = $"Compensation agence source {source.Agence.Code}"
            };
            var debitCompDestination = new Ecriture
            {
                OperationId = operation.Id,
                CompteId = await GetCompteCompensationIdAsync(destination.AgenceId
),
                Sens = "Debit",
                Montant = virementInter.Montant,
```

```
                Devise = destination.Devise,
                Libelle = $"Compensation agence destination {destination.Agence.Co
de}"
            };
            var creditClient = new Ecriture
            {
                OperationId = operation.Id,
                CompteId = destination.Id,
                Sens = "Credit",
                Montant = virementInter.Montant,
                Devise = destination.Devise,
                Libelle = $"Virement inter-agence depuis {source.NumeroCompte}"
            };

            _context.Ecritures.AddRange(
                debitClient, creditCompSource, debitCompDestination, creditClient)
;

            source.Solde -= virementInter.Montant;
            destination.Solde += virementInter.Montant;
            source.DateModification = DateTime.UtcNow;
            destination.DateModification = DateTime.UtcNow;

            virementInter.Statut = "EnCompensation";
            virementInter.ValideParId = valideParId;
            virementInter.DateCompensation = DateTime.UtcNow;

            await _context.SaveChangesAsync();
            await transaction.CommitAsync();

            return true;
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
    }

    public async Task<bool> RejeterAsync(
        int virementInterAgenceId, int valideParId, string motif)
    {
        var virementInter = await _context.VirementsInterAgences
            .Include(v => v.Virement)
            .FirstOrDefaultAsync(v => v.Id == virementInterAgenceId);

        if (virementInter is null) return false;

        virementInter.Statut = "Rejete";
        virementInter.MotifRejet = motif;
        virementInter.ValideParId = valideParId;
```

```
        virementInter.Virement.Statut = "Rejete";

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<IEnumerable<VirementInterAgence>> GetEnAttenteAsync(int agen
ceId)
    {
        return await _context.VirementsInterAgences
            .Include(v => v.AgenceSource)
            .Include(v => v.AgenceDestination)
            .Where(v => v.Statut == "EnAttente")
            .Where(v => v.AgenceDestinationId == agenceId)
            .OrderByDescending(v => v.DateCreation)
            .ToListAsync();
    }

    public async Task<IEnumerable<VirementInterAgence>> GetHistoriqueAsync(
        int agenceId, DateTime? du, DateTime? au)
    {
        var query = _context.VirementsInterAgences
            .Include(v => v.AgenceSource)
            .Include(v => v.AgenceDestination)
            .AsNoTracking()
            .AsQueryable();

        if (du.HasValue) query = query.Where(v => v.DateCreation >= du.Value);
        if (au.HasValue) query = query.Where(v => v.DateCreation <= au.Value);

        return await query
            .Where(v => v.AgenceSourceId == agenceId || v.AgenceDestinationId == a
genceId)
            .OrderByDescending(v => v.DateCreation)
            .ToListAsync();
    }

    private async Task<int> GetCompteCompensationIdAsync(int agenceId)
    {
        var compte = await _context.Comptes
            .FirstOrDefaultAsync(c => c.NumeroCompte == $"9000-{agenceId:D4}");

        if (compte is null)
        {
            compte = new Compte
            {
                NumeroCompte = $"9000-{agenceId:D4}",
                AgenceId = agenceId,
                Solde = 0,
                Devise = "BIF",
                Statut = "Actif",
```

```
                DateOuverture = DateTime.UtcNow
            };
            _context.Comptes.Add(compte);
            await _context.SaveChangesAsync();
        }

        return compte.Id;
    }

    private static string GenererReference(string prefix)
    {
        var date = DateTime.UtcNow.ToString("yyyyMMdd");
        var random = Guid.NewGuid().ToString("N")[..6].ToUpperInvariant();
        return $"{prefix}-{date}-{random}";
    }
}
```

9.6. Service de déclaration à la CNRF

Fichier à créer : `Services/ICNRFService.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public interface ICNRFService {
    Task<DeclarationSoupcon> GenererDOSAsync(int alerteId, int redigeParId);
    Task<bool> TransmettreAsync(int declarationId);
    Task<bool> EnregistrerAccuseAsync(int declarationId, string reponse);
    Task<IEnumerable<DeclarationSoupcon>> GetDeclarationsAsync(string? statut);
    Task<string> GenererContenuDOSAsync(AlerteAML alerte);
}
```

Fichier à créer : `Services/CNRFService.cs`

```
using System.Text;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class CNRFService : ICNRFService {
    private readonly AkarangaBankContext _context;

    public CNRFService(AkarangaBankContext context) {
        _context = context;
    }

    public async Task<DeclarationSoupcon> GenererDOSAsync(int alerteId, int redige
ParId) {
        var alerte = await _context.AlertesAML
            .Include(a => a.Client)
```

```
            .Include(a => a.Operation)
            .FirstOrDefaultAsync(a => a.Id == alerteId)
            ?? throw new InvalidOperationException("Alerte introuvable.");

        var contenu = await GenererContenuDOSAsync(alerte);

        var declaration = new DeclarationSoupcon
        {
            Reference = GenererReference("DOS"),
            AlerteId = alerteId,
            ClientId = alerte.ClientId,
            TypeDeclaration = "DOS",
            Contenu = contenu,
            MotifSuspicion = alerte.Description,
            Statut = "Brouillon",
            DateCreation = DateTime.UtcNow,
            RedigeParId = redigeParId
        };

        _context.DeclarationsSoupcon.Add(declaration);
        await _context.SaveChangesAsync();

        return declaration;
    }

    public async Task<bool> TransmettreAsync(int declarationId)
    {
        var declaration = await _context.DeclarationsSoupcon.FindAsync(declaration
Id);
        if (declaration is null) return false;

        if (declaration.Statut != "Validee" && declaration.Statut != "Brouillon")
            return false;

        declaration.Statut = "Transmise";
        declaration.DateTransmission = DateTime.UtcNow;

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<bool> EnregistrerAccuseAsync(int declarationId, string repon
se)
    {
        var declaration = await _context.DeclarationsSoupcon.FindAsync(declaration
Id);
        if (declaration is null) return false;

        declaration.Statut = "Accusee";
        declaration.DateAccuseReception = DateTime.UtcNow;
        declaration.ReponseCNRF = reponse;
```

```
        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<IEnumerable<DeclarationSoupcon>> GetDeclarationsAsync(string
? statut)
    {
        var query = _context.DeclarationsSoupcon
            .Include(d => d.Client)
            .Include(d => d.Alerte)
            .Include(d => d.RedigePar)
            .AsNoTracking()
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(statut))
            query = query.Where(d => d.Statut == statut);

        return await query.OrderByDescending(d => d.DateCreation).ToListAsync();
    }

    public Task<string> GenererContenuDOSAsync(AlerteAML alerte)
    {
        var sb = new StringBuilder();
        sb.AppendLine("======================================================");
        sb.AppendLine("DÉCLARATION D'OPÉRATION SUSPECTE (DOS)");
        sb.AppendLine("Akaranga Bank — Conformité LBC/FT");
        sb.AppendLine("======================================================");
        sb.AppendLine();
        sb.AppendLine($"Référence déclaration : {GenererReference("DOS")}");
        sb.AppendLine($"Date de génération     : {DateTime.UtcNow:dd/MM/yyyy HH:mm
}");
        sb.AppendLine();
        sb.AppendLine("--- IDENTIFICATION DU CLIENT ---");
        sb.AppendLine($"Nom complet      : {alerte.Client.Nom} {alerte.Client.Preno
m}");
        sb.AppendLine($"Numéro client    : {alerte.Client.NumeroClient}");
        sb.AppendLine($"Numéro CNI       : {alerte.Client.NumeroCNI}");
        sb.AppendLine($"Téléphone        : {alerte.Client.Telephone}");
        sb.AppendLine();
        sb.AppendLine("--- OPÉRATION CONCERNÉE ---");
        if (alerte.Operation is not null)
        {
            sb.AppendLine($"Référence opération : {alerte.Operation.Reference}");
            sb.AppendLine($"Type                : {alerte.Operation.TypeOperation}
");
            sb.AppendLine($"Montant             : {alerte.Operation.Montant:N2} {a
lerte.Operation.Devise}");
            sb.AppendLine($"Date                : {alerte.Operation.DateOperation:
dd/MM/yyyy HH:mm}");
        }
```

```
        else
        {
            sb.AppendLine("Aucune opération associée.");
        }
        sb.AppendLine();
        sb.AppendLine("--- MOTIF DE LA DÉCLARATION ---");
        sb.AppendLine($"Type d'alerte : {alerte.TypeAlerte}");
        sb.AppendLine($"Niveau        : {alerte.Niveau}");
        sb.AppendLine($"Description   : {alerte.Description}");
        sb.AppendLine();
        sb.AppendLine("--- AUTORITÉ DESTINATAIRE ---");
        sb.AppendLine("Cellule Nationale de Renseignement Financier (CNRF)");
        sb.AppendLine("Bujumbura, Burundi");
        sb.AppendLine();
        sb.AppendLine("--- MENTIONS LÉGALES ---");
        sb.AppendLine("Déclaration établie conformément à la loi n° 1/02 du 4 février 2008");
        sb.AppendLine("relative à la lutte contre le blanchiment de capitaux et le financement");
        sb.AppendLine("du terrorisme, modifiée par la loi n° 1/08 du 27 mars 2025.");
        sb.AppendLine("==================================================");

        return Task.FromResult(sb.ToString());
    }

    private static string GenererReference(string prefix)
    {
        var date = DateTime.UtcNow.ToString("yyyyMMdd");
        var random = Guid.NewGuid().ToString("N")[..6].ToUpperInvariant();
        return $"{prefix}-{date}-{random}";
    }
}
```

9.7. Service de seuils AML multiples

Fichier à créer : `Services/ISeuilAMLService.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public interface ISeuilAMLService {
    Task<SeuilAML?> GetSeuilAsync(string code, string typeOperation);
    Task<IEnumerable<SeuilAML>> GetAllAsync();
    Task<bool> MettreAJourAsync(int seuilId, decimal nouveauMontant, int utilisateurId);
    Task<decimal> GetSeuilVigilanceAsync();
    Task<decimal> GetSeuilDeclarationAsync();
    Task<decimal> GetSeuilBlocageAsync();
}
```

Fichier à créer : `Services/SeuilAMLService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class SeuilAMLService : ISeuilAMLService
{
    private readonly AkarangaBankContext _context;

    public SeuilAMLService(AkarangaBankContext context)
    {
        _context = context;
    }

    public async Task<SeuilAML?> GetSeuilAsync(string code, string typeOperation)
    {
        return await _context.SeuilsAML
            .AsNoTracking()
            .Where(s => s.Code == code && s.Actif)
            .Where(s => s.TypeOperation == typeOperation || s.TypeOperation == "To
utes")
            .OrderByDescending(s => s.TypeOperation == typeOperation)
            .FirstOrDefaultAsync();
    }

    public async Task<IEnumerable<SeuilAML>> GetAllAsync()
    {
        return await _context.SeuilsAML
            .AsNoTracking()
            .OrderBy(s => s.Code)
            .ThenBy(s => s.TypeOperation)
            .ToListAsync();
    }

    public async Task<bool> MettreAJourAsync(
        int seuilId, decimal nouveauMontant, int utilisateurId)
    {
        var seuil = await _context.SeuilsAML.FindAsync(seuilId);
        if (seuil is null) return false;

        var ancienMontant = seuil.Montant;
        seuil.Montant = nouveauMontant;
        seuil.DateModification = DateTime.UtcNow;

        // Journaliser dans AuditLog
        var audit = new AuditLog
        {
            DateEvenement = DateTime.UtcNow,
```

```
            UtilisateurId = utilisateurId,
            Action = "ModificationSeuilAML",
            TableConcernee = "SeuilsAML",
            EnregistrementId = seuilId,
            ValeurAvant = ancienMontant.ToString("N2"),
            ValeurApres = nouveauMontant.ToString("N2"),
            Details = $"Modification du seuil {seuil.Code} pour {seuil.TypeOperati
on}"
        };
        _context.AuditLogs.Add(audit);

        await _context.SaveChangesAsync();
        return true;
    }

    public async Task<decimal> GetSeuilVigilanceAsync()
    {
        var seuil = await GetSeuilAsync("VIGILANCE", "Toutes");
        return seuil?.Montant ?? 5_000_000m;
    }

    public async Task<decimal> GetSeuilDeclarationAsync()
    {
        var seuil = await GetSeuilAsync("DECLARATION", "Toutes");
        return seuil?.Montant ?? 10_000_000m;
    }

    public async Task<decimal> GetSeuilBlocageAsync()
    {
        var seuil = await GetSeuilAsync("BLOCAGE", "Toutes");
        return seuil?.Montant ?? 50_000_000m;
    }
}
```

9.8. Service de détection avancée

Fichier à créer : `Services/IDetectionAvanceeService.cs`

```
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public interface IDetectionAvanceeService {
    Task<ResultatSurveillance> DetecterStructuringAsync(int clientId, DateTime dat
e);
    Task<ResultatSurveillance> DetecterSmurfingAsync(int clientId, DateTime date);
    Task<ResultatSurveillance> DetecterLayeringAsync(int clientId, DateTime date);
    Task<ResultatSurveillance> DetecterVelociteAnormaleAsync(int clientId, DateTim
e date);
}
```

Fichier à créer : `Services/DetectionAvanceeService.cs`

```
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;

namespace AkarangaBank.Web.Services;

public class DetectionAvanceeService : IDetectionAvanceeService
{
    private readonly AkarangaBankContext _context;
    private readonly ISeuilAMLService _seuils;

    public DetectionAvanceeService(
        AkarangaBankContext context,
        ISeuilAMLService seuils)
    {
        _context = context;
        _seuils = seuils;
    }

    // Structuring : plusieurs opérations juste sous le seuil de déclaration
    public async Task<ResultatSurveillance> DetecterStructuringAsync(
        int clientId, DateTime date)
    {
        var alertes = new List<AlerteSurveillance>();
        var seuil = await _seuils.GetSeuilVigilanceAsync();
        var debut = date.AddDays(-30);

        var operations = await _context.Operations
            .Where(o => o.DateOperation >= debut && o.DateOperation <= date)
            .Where(o => o.Compte.ClientId == clientId)
            .Where(o => o.Montant >= seuil * 0.8m && o.Montant < seuil)
            .ToListAsync();

        if (operations.Count >= 3)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "Structuring",
                Description = $"{operations.Count} opérations entre " +
```

```
                                $"{seuil * 0.8m:N0} et {seuil:N0} BIF sur 30 jours.
" +
                                "Suspicion de fractionnement structuré.",
                Niveau = NiveauAlerte.Eleve
            });
        }

        return new ResultatSurveillance
        {
            CompteId = 0,
            DateAnalyse = DateTime.UtcNow,
            Alertes = alertes,
            NecessiteDeclaration = alertes.Any()
        };
    }

    // Smurfing : plusieurs comptes liés effectuent des dépôts sous le seuil
    public async Task<ResultatSurveillance> DetecterSmurfingAsync(
        int clientId, DateTime date)
    {
        var alertes = new List<AlerteSurveillance>();
        var seuil = await _seuils.GetSeuilVigilanceAsync();
        var debut = date.AddDays(-7);

        var comptes = await _context.Comptes
            .Where(c => c.ClientId == clientId)
            .Select(c => c.Id)
            .ToListAsync();

        if (comptes.Count < 2) return new ResultatSurveillance { Alertes = alertes
};

        var total = await _context.Operations
            .Where(o => o.DateOperation >= debut && o.DateOperation <= date)
            .Where(o => comptes.Contains(o.CompteId))
            .Where(o => o.Montant < seuil)
            .SumAsync(o => (decimal?)o.Montant) ?? 0m;

        if (total >= seuil * 3)
        {
            alertes.Add(new AlerteSurveillance
            {
```

```
                Regle = "Smurfing",
                Description = $"Total cumulé de {total:N0} BIF sur {comptes.Count}
comptes " +
                              "en 7 jours, chaque opération sous le seuil AML.",
                Niveau = NiveauAlerte.Critique
            });
        }

        return new ResultatSurveillance
        {
            CompteId = 0,
            DateAnalyse = DateTime.UtcNow,
            Alertes = alertes,
            NecessiteDeclaration = alertes.Any()
        };
    }

    // Layering : virements en cascade entre plusieurs comptes
    public async Task<ResultatSurveillance> DetecterLayeringAsync(
        int clientId, DateTime date)
    {
        var alertes = new List<AlerteSurveillance>();
        var debut = date.AddDays(-14);

        var virements = await _context.VirementsInterAgences
            .Where(v => v.DateCreation >= debut && v.DateCreation <= date)
            .Where(v => v.Virement.CompteSource.ClientId == clientId)
            .CountAsync();

        if (virements >= 5)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "Layering",
                Description = $"{virements} virements inter-agences en 14 jours. "
+
                              "Schéma possible de superposition.",
                Niveau = NiveauAlerte.Eleve
            });
        }

        return new ResultatSurveillance
```

```
        {
            CompteId = 0,
            DateAnalyse = DateTime.UtcNow,
            Alertes = alertes,
            NecessiteDeclaration = alertes.Any()
        };
    }

    // Vélocité anormale : beaucoup d'opérations en peu de temps
    public async Task<ResultatSurveillance> DetecterVelociteAnormaleAsync(
        int clientId, DateTime date)
    {
        var alertes = new List<AlerteSurveillance>();
        var debut = date.AddHours(-1);

        var nbOperations = await _context.Operations
            .Where(o => o.DateOperation >= debut && o.DateOperation <= date)
            .Where(o => o.Compte.ClientId == clientId)
            .CountAsync();

        if (nbOperations >= 10)
        {
            alertes.Add(new AlerteSurveillance
            {
                Regle = "VelociteAnormale",
                Description = $"{nbOperations} opérations en 1 heure. " +
                              "Comportement inhabituel.",
                Niveau = NiveauAlerte.Eleve
            });
        }

        return new ResultatSurveillance
        {
            CompteId = 0,
            DateAnalyse = DateTime.UtcNow,
            Alertes = alertes,
            NecessiteDeclaration = alertes.Any()
        };
    }
}
```

9.9. Enregistrement des services dans `Program.cs`

➖ Fichier : `Program.cs` — ajouter dans la section « Services métier » :

```
builder.Services.AddScoped<ICompensationService, CompensationService>();
builder.Services.AddScoped<IViremmentInterAgenceService, ViremmentInterAgenceService
>();
builder.Services.AddScoped<ICNRFService, CNRFService>();
builder.Services.AddScoped<ISeuilAMLService, SeuilAMLService>();
builder.Services.AddScoped<IDetectionAvanceeService, DetectionAvanceeService>();
```

Partie D — Contrôleurs et vues

9.10. ViewModel

📄 Fichier à créer : `ViewModels/ViremmentInterAgenceViewModel.cs`

```
using System.ComponentModel.DataAnnotations;

namespace AkarangaBank.Web.ViewModels;

public class ViremmentInterAgenceViewModel
{
    [Required]
    [Display(Name = "Compte source")]
    public int CompteSourceId { get; set; }

    [Required]
    [Display(Name = "Compte destination")]
    public int CompteDestinationId { get; set; }

    [Required]
    [Range(0.01, double.MaxValue)]
    public decimal Montant { get; set; }

    [StringLength(500)]
    public string? Motif { get; set; }
}
```

9.11. Contrôleur des virements inter-agences

📄 Fichier à créer : `Controllers/ViremmentsInterAgencesController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;
```

```
namespace AkarangaBank.Web.Controllers;

[Authorize(Policy = "GuichetierOnly")]
public class VirementsInterAgencesController : Controller
{
    private readonly IVirementInterAgenceService _service;
    private readonly ICurrentUserService _currentUser;
    private readonly AkarangaBankContext _context;

    public VirementsInterAgencesController(
        IVirementInterAgenceService service,
        ICurrentUserService currentUser,
        AkarangaBankContext context)
    {
        _service = service;
        _currentUser = currentUser;
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        var agenceId = await _currentUser.GetAgenceIdAsync();
        var virements = await _service.GetEnAttenteAsync(agenceId);
        return View(virements);
    }

    public async Task<IActionResult> Initier()
    {
        await ChargerComptesAsync();
        return View(new VirementInterAgenceViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Initier(VirementInterAgenceViewModel vm)
    {
        if (!ModelState.IsValid)
        {
            await ChargerComptesAsync(vm.CompteSourceId, vm.CompteDestinationId);
            return View(vm);
        }
```

```
        try
        {
            var userId = await _currentUser.GetUserIdAsync();
            var virement = await _service.InitierAsync(vm, userId);
            TempData["Success"] = $"Virement inter-agence initié (réf. {virement.R
eference}).";
            return RedirectToAction(nameof(Index));
        }
        catch (InvalidOperationException ex)
        {
            ModelState.AddModelError("", ex.Message);
            await ChargerComptesAsync(vm.CompteSourceId, vm.CompteDestinationId);
            return View(vm);
        }
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "AdminOnly")]
    public async Task<IActionResult> Valider(int id)
    {
        try
        {
            var userId = await _currentUser.GetUserIdAsync();
            await _service.ValiderAsync(id, userId);
            TempData["Success"] = "Virement validé.";
        }
        catch (InvalidOperationException ex)
        {
            TempData["Error"] = ex.Message;
        }

        return RedirectToAction(nameof(Index));
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "AdminOnly")]
    public async Task<IActionResult> Rejeter(int id, string motif)
    {
        var userId = await _currentUser.GetUserIdAsync();
        await _service.RejeterAsync(id, userId, motif);
```

```
        TempData["Success"] = "Virement rejeté.";
        return RedirectToAction(nameof(Index));
    }

    public async Task<IActionResult> Historique(DateTime? du, DateTime? au)
    {
        var agenceId = await _currentUser.GetAgenceIdAsync();
        var virements = await _service.GetHistoriqueAsync(agenceId, du, au);
        ViewBag.Du = du;
        ViewBag.Au = au;
        return View(virements);
    }

    private async Task ChargerComptesAsync(int? sourceId = null, int? destinationI
d = null)
    {
        var comptes = await _context.Comptes
            .Include(c => c.Client)
            .Include(c => c.Agence)
            .Where(c => c.Statut == "Actif")
            .Where(c => !c.NumeroCompte.StartsWith("9000"))
            .OrderBy(c => c.NumeroCompte)
            .Select(c => new
            {
                c.Id,
                Libelle = $"{c.NumeroCompte} — {c.Client.Nom} {c.Client.Prenom} ({
c.Agence.Code})"
            })
            .ToListAsync();

        ViewBag.ComptesSource = new SelectList(comptes, "Id", "Libelle", sourceId)
;
        ViewBag.ComptesDestination = new SelectList(comptes, "Id", "Libelle", dest
inationId);
    }
}
```

9.12. Contrôleur CNRF

Fichier à créer : `Controllers/CNRFController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using AkarangaBank.Web.Services;
```

```
namespace AkarangaBank.Web.Controllers;

[Authorize(Policy = "AdminOnly")]
public class CNRFController : Controller
{
    private readonly ICNRFService _service;
    private readonly ICurrentUserService _currentUser;

    public CNRFController(ICNRFService service, ICurrentUserService currentUser)
    {
        _service = service;
        _currentUser = currentUser;
    }

    public async Task<IActionResult> Index(string? statut)
    {
        var declarations = await _service.GetDeclarationsAsync(statut);
        ViewBag.Statut = statut;
        return View(declarations);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Generer(int alerteId)
    {
        var userId = await _currentUser.GetUserIdAsync();
        var declaration = await _service.GenererDOSAsync(alerteId, userId);
        TempData["Success"] = $"DOS générée : {declaration.Reference}";
        return RedirectToAction(nameof(Index));
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Transmettre(int id)
    {
        await _service.TransmettreAsync(id);
        TempData["Success"] = "Déclaration transmise à la CNRF.";
        return RedirectToAction(nameof(Index));
    }

    public async Task<IActionResult> Details(int id)
    {
        var declarations = await _service.GetDeclarationsAsync(null);
        var declaration = declarations.FirstOrDefault(d => d.Id == id);
        if (declaration is null) return NotFound();
        return View(declaration);
    }
}
```

9.13. Vues

Views/VirementsInterAgences/Initier.cshtml

```
@model AkarangaBank.Web.ViewModels.VirementInterAgenceViewModel
@{ ViewData["Title"] = "Virement inter-agences"; }

<h1>Virement inter-agences</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<div class="row">
    <div class="col-md-6">
        <form asp-action="Initier" method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <div class="mb-3">
                <label asp-for="CompteSourceId" class="form-label">Compte source</
label>
                <select asp-for="CompteSourceId" asp-items="ViewBag.ComptesSource"
                        class="form-select">
                    <option value="">-- Sélectionner --</option>
                </select>
            </div>

            <div class="mb-3">
                <label asp-for="CompteDestinationId" class="form-label">Compte des
tination</label>
                <select asp-for="CompteDestinationId" asp-items="ViewBag.ComptesDe
stination"
                        class="form-select">
                    <option value="">-- Sélectionner --</option>
                </select>
            </div>

            <div class="mb-3">
                <label asp-for="Montant" class="form-label">Montant (BIF)</label>
                <input asp-for="Montant" class="form-control" type="number" step="
0.01" />
            </div>

            <div class="mb-3">
                <label asp-for="Motif" class="form-label">Motif</label>
                <input asp-for="Motif" class="form-control" />
            </div>

            <button type="submit" class="btn btn-primary">Initier le virement</but
ton>
```

```
            <a asp-action="Index" class="btn btn-secondary">Annuler</a>
        </form>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Views/VirementsInterAgences/Index.cshtml

```
@model IEnumerable<AkarangaBank.Web.Models.VirementInterAgence>
@{ ViewData["Title"] = "Virements inter-agences en attente"; }

<h1>Virements inter-agences en attente</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}
@if (TempData["Error"] is string err)
{
    <div class="alert alert-danger">@err</div>
}

<table class="table table-striped">
    <thead>
        <tr>
            <th>Date</th>
            <th>Référence</th>
            <th>Agence source</th>
            <th>Agence destination</th>
            <th class="text-end">Montant</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
    @foreach (var v in Model)
    {
        <tr>
            <td>@v.DateCreation.ToString("dd/MM/yyyy HH:mm")</td>
            <td>@v.Reference</td>
            <td>@v.AgenceSource.Code</td>
            <td>@v.AgenceDestination.Code</td>
            <td class="text-end">@v.Montant.ToString("N2") @v.Devise</td>
            <td>
                @if (User.IsInRole("admin"))
                {
                    <form asp-action="Valider" asp-route-id="@v.Id" method="post"
class="d-inline">
```

```
                        <button type="submit" class="btn btn-sm btn-success">Valid
er</button>
                    </form>
                    <form asp-action="Rejeter" asp-route-id="@v.Id" method="post"
class="d-inline">
                        <input type="hidden" name="motif" value="Rejeté par admin"
/>
                        <button type="submit" class="btn btn-sm btn-danger">Rejete
r</button>
                    </form>
                }
            </td>
        </tr>
    }
    </tbody>
</table>

<a asp-action="Initier" class="btn btn-primary">+ Nouveau virement inter-agences</
a>
```

Views/CNRF/Index.cshtml

```
@model IEnumerable<AkarangaBank.Web.Models.DeclarationSoupcon>
@{ ViewData["Title"] = "Déclarations CNRF"; }

<h1>Déclarations d'opérations suspectes (DOS)</h1>

@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}

<table class="table table-striped">
    <thead>
        <tr>
            <th>Date</th>
            <th>Référence</th>
            <th>Client</th>
            <th>Type</th>
            <th>Statut</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
    @foreach (var d in Model)
    {
        <tr>
            <td>@d.DateCreation.ToString("dd/MM/yyyy HH:mm")</td>
            <td>@d.Reference</td>
            <td>@d.Client.Nom @d.Client.Prenom</td>
            <td>@d.TypeDeclaration</td>
            <td>
                <span class="badge @(d.Statut == "Transmise" ? "bg-success" :
                                     d.Statut == "Accusee" ? "bg-info" : "bg-secon
dary")">
```

```
                    @d.Statut
                </span>
            </td>
            <td>
                <a asp-action="Details" asp-route-id="@d.Id" class="btn btn-sm btn
-primary">
                    Détails
                </a>
                @if (d.Statut == "Brouillon" || d.Statut == "Validee")
                {
                    <form asp-action="Transmettre" asp-route-id="@d.Id"
                          method="post" class="d-inline">
                        <button type="submit" class="btn btn-sm btn-success">
                            Transmettre
                        </button>
                    </form>
                }
            </td>
        </tr>
    }
    </tbody>
</table>
```

Partie E — Tests manuels

Test 1 — Virement inter-agences

1. Initier un virement de BJM vers GIT.
2. Vérifier qu'il apparaît dans `/ViremensInterAgences`.
3. Valider avec un admin.
4. Vérifier les 4 écritures dans le journal.

Test 2 — Compensation

1. Effectuer 5 virements BJM → GIT.
2. Lancer le calcul de compensation pour GIT.
3. Vérifier que les totaux débits/crédits sont cohérents.

Test 3 — Structuring

1. Effectuer 3 dépôts de 4 500 000 BIF sur le même compte.
2. Vérifier la création d'une alerte Structuring.

Test 4 — Smurfing

1. Créer plusieurs comptes pour un même client.
2. Effectuer des dépôts sous le seuil sur chaque compte.
3. Vérifier la création d'une alerte `Smurfing`.

Test 5 — Génération de DOS

1. Aller sur `/CNRF`.
2. Cliquer sur **Générer** pour une alerte.
3. Vérifier le contenu de la DOS.

Test 6 — Transmission à la CNRF

1. Depuis `/CNRF`, cliquer sur **Transmettre**.
2. Vérifier que le statut passe à `Transmise`.

Test 7 — Seuils AML multiples

1. Aller sur `/SeuilsAML`.
2. Modifier le seuil de vigilance à 3 000 000 BIF.
3. Vérifier qu'une entrée AuditLog est créée.

Test 8 — Vélocité anormale

1. Effectuer 10 opérations en moins d'une heure.
2. Vérifier la création d'une alerte `VelociteAnormale`.

Test 9 — Équilibre des compensations

1. Lancer `VerifierEquilibreCompensationsAsync`.
2. Vérifier que la somme des débits = somme des crédits.

Test 10 — Rôle insuffisant

1. Se connecter en client.
2. Tenter d'accéder à `/CNRF` → accès refusé.

Exercices

Exercice 1 — Rapport mensuel AML
Générer un PDF mensuel avec :

- Nombre d'alertes par niveau
- Nombre de DOS transmises
- Montant total des opérations suspectes
- Top 10 des clients à risque

Exercice 2 — Notification email au conformité
À chaque alerte `Critique`, envoyer un email au responsable conformité.

Exercice 3 — Workflow de validation à 3 niveaux

- Niveau 1 : guichetier initie
- Niveau 2 : gestionnaire valide
- Niveau 3 : admin confirme

Exercice 4 — Intégration API CNRF
Simuler une API REST qui reçoit les DOS et renvoie un accusé de réception.

Exercice 5 — Tableau de bord conformité
Dashboard avec indicateurs en temps réel :

- Alertes ouvertes
- DOS en attente
- Seuils actuels
- Clients à risque

Questions de compréhension

1. Quelle est la différence entre un virement intra-agence et inter-agences ?
2. Pourquoi un virement inter-agences génère-t-il 4 écritures ?
3. Qu'est-ce qu'un compte de compensation ?
4. Quelle est la différence entre structuring et smurfing ?
5. Qu'est-ce que le layering ? Comment le détecter ?
6. Pourquoi la vélocité des opérations est-elle un indicateur de blanchiment ?
7. Quelles sont les mentions légales obligatoires dans une DOS ?
8. Pourquoi les seuils AML doivent-ils être configurables ?
9. Comment garantir l'intégrité des déclarations transmises à la CNRF ?
10. Pourquoi la journalisation (AuditLog) est-elle critique pour la conformité ?

Mini-TP 9 — Virements interbancaires et conformité avancée

Objectif : implémenter les virements inter-agences et les déclarations CNRF.

1. Créez les entités : `ViremmentInterAgence`, `CompensationAgence`, `DeclarationSoupcon`, `SeuilAML`.
2. Créez les services : `CompensationService`, `ViremmentInterAgenceService`, `CNRFService`, `SeuilAMLService`, `DetectionAvanceeService`.
3. Enregistrez les services dans `Program.cs`.
4. Créez les contrôleurs et vues.
5. Testez tous les cas de la Partie E.

CHAPITRE 10 — INTRODUCTION AUX API POUR APPLICATION MOBILE

Objectifs du chapitre

- Comprendre ce qu'est une **API** et pourquoi elle est essentielle pour une application mobile.
- Distinguer les grands **styles d'API** : REST, SOAP, GraphQL, gRPC.
- Comprendre les enjeux de **sécurité** propres aux API (JWT, OAuth 2.0, rate limiting).
- Situer l'API d'Akaranga Bank dans une **architecture globale**.
- Identifier les **bonnes pratiques** de conception d'API.
- Préparer le terrain pour les cours spécialisés **API REST** et **API SOAP** de l'année prochaine.

Note importante : ce chapitre est **informatif**. Il ne remplace pas les cours spécialisés à venir. L'implémentation détaillée d'une API REST et d'un service SOAP sera traitée l'année prochaine.

Partie A — Qu'est-ce qu'une API ?

10.1. Définition

API = *Application Programming Interface* (interface de programmation applicative).

C'est un **contrat** qui permet à deux logiciels de communiquer entre eux, sans que l'un ait besoin de connaître les détails internes de l'autre.

Analogie : une API est comme un **guichet de banque**. Le client (application mobile) présente une demande (requête), le guichetier (serveur) traite la demande selon des règles précises, et retourne une réponse.

10.1.1. Pourquoi une API pour Akaranga Bank ?

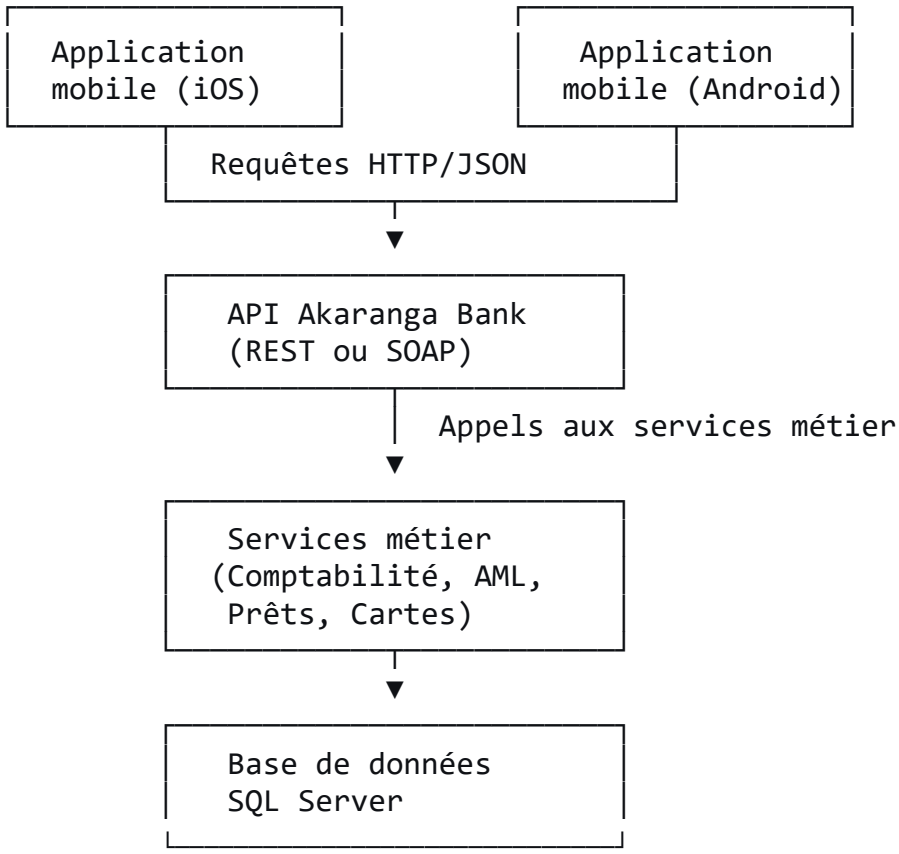
Aujourd'hui, Akaranga Bank a :

- Des **pages web** (Razor Pages, MVC) accessibles depuis un navigateur.
- Une **base de données** SQL Server.
- Des **services métier** (comptabilité, conformité, prêts, cartes).

Demain, elle veut offrir une **application mobile** (iOS / Android) aux clients. Cette application ne peut pas afficher du HTML généré par Razor Pages : elle a besoin de **données brutes** (JSON, XML) qu'elle mettra en forme elle-même.

C'est là qu'intervient l'API.

10.1.2. Schéma conceptuel



10.2. Les grands styles d'API

Il existe plusieurs **styles** d'API. Chacun a ses forces et ses faiblesses.

Style	Format	Transport	Avantages	Inconvénients
REST	JSON, XML	HTTP	Simple, scalable, standard	Pas de contrat fort
SOAP	XML	HTTP, SMTP, TCP	Contrat fort (WSDL), transactions	Verbeux, complexe
GraphQL	JSON	HTTP	Requêtes flexibles	Complexité serveur
gRPC	Protobuf	HTTP/2	Performant, binaire	Moins lisible

10.2.1. REST — *Representational State Transfer*

Principe : exposer des **ressources** (comptes, clients, virements) identifiées par des **URL**, manipulées via les **verbes HTTP**.

Exemples d'endpoints :

```
GET    /api/v1/comptes              → Liste des comptes
GET    /api/v1/comptes/1            → Détail du compte 1
POST   /api/v1/comptes              → Créer un compte
PUT    /api/v1/comptes/1            → Modifier le compte 1
DELETE /api/v1/comptes/1            → Supprimer le compte 1
GET    /api/v1/comptes/1/solde      → Solde du compte 1
POST   /api/v1/virements            → Initier un virement
```

Format d'échange : JSON le plus souvent.

Exemple de requête :

http

```
GET /api/v1/comptes/1/solde HTTP/1.1
Host: api.akaranga.bi
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Exemple de réponse :

```
json
{
  "compteId": 1,
  "numeroCompte": "BJM-0001-0001",
  "solde": 150000.00,
  "devise": "BIF",
  "dateConsultation": "2026-10-15T14:30:00Z"
}
```

REST est aujourd'hui le style dominant pour les applications mobiles et les services web grand public.

10.2.2. SOAP — *Simple Object Access Protocol*

Principe : échanger des **messages XML** structurés selon un **contrat strict** (WSDL).

Exemple de requête SOAP :

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/
">
  <soap:Body>
    <ConsulterSolde xmlns="http://akaranga.bi/api">
      <NumeroCompte>BJM-0001-0001</NumeroCompte>
    </ConsulterSolde>
  </soap:Body>
</soap:Envelope>
```

Exemple de réponse SOAP :

```xml
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ConsulterSoldeResponse xmlns="http://akaranga.bi/api">
      <Solde>150000.00</Solde>
      <Devise>BIF</Devise>
    </ConsulterSoldeResponse>
  </soap:Body>
</soap:Envelope>
```

SOAP est encore utilisé dans :

- Les **banques** (échanges interbancaires, SWIFT).
- Les **administrations** (services fiscaux, douanes).
- Les **systèmes critiques** (santé, défense).

Raison : SOAP offre un **contrat fort** (WSDL) qui garantit la compatibilité entre systèmes hétérogènes.

10.2.3. REST vs SOAP — Comparaison

Critère	REST	SOAP
Format	JSON, XML	XML uniquement
Contrat	Souple (OpenAPI optionnel)	Strict (WSDL obligatoire)
Verbes	HTTP (GET, POST, etc.)	POST uniquement
Transactions	Non standardisées	WS-Transaction
Sécurité	HTTPS, JWT, OAuth 2.0	WS-Security
Performance	Rapide	Plus lent (XML verbeux)
Cas d'usage	Mobile, web, startups	Banques, administrations
Courbe d'apprentissage	Faible	Élevée

En résumé : REST est **simple et moderne**, SOAP est **rigoureux et ancien**. Les deux ont leur place.

Partie B — Sécurité des API

10.3. Pourquoi la sécurité est critique

Une API bancaire est une **cible privilégiée** :

- Les données financières sont sensibles.
- Les transactions sont irréversibles.
- Les attaques peuvent être automatisées (bots).

Une API mal sécurisée peut entraîner :

- Vol de données clients.
- Fraude financière.

- Sanctions réglementaires (BRB, CNRF).
- Perte de confiance.

10.3.1. Les menaces principales

Menace	Description	Contre-mesure
Interception	Écoute du trafic réseau	HTTPS obligatoire
Usurpation	Faux utilisateur	Authentification forte (2FA)
Injection	Code malveillant dans les requêtes	Validation stricte
Déni de service	Saturation du serveur	Rate limiting
Rejeu	Rejouer une requête capturée	Nonce, timestamp, JWT
Élévation de privilèges	Accès à des ressources interdites	Autorisation par rôle

10.3.2. Les mécanismes de sécurité

A. HTTPS obligatoire

Tout échange doit être **chiffré** via TLS 1.3.

Configuration :

csharp

```
app.UseHttpsRedirection();
app.UseHsts();
```

B. Authentification par JWT

JWT = *JSON Web Token*. C'est un jeton signé qui contient l'identité de l'utilisateur.

Structure :

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Décomposition :

- **Header** : algorithme de signature
- **Payload** : claims (identité, rôle, expiration)
- **Signature** : garantit l'intégrité

Cycle de vie :

1. Client envoie email + mot de passe.
2. Serveur vérifie et génère un **access token** (courte durée) + **refresh token** (longue durée).
3. Client utilise l'access token dans l'en-tête `Authorization: Bearer ...`.
4. À expiration, client utilise le refresh token pour obtenir un nouveau access token.

C. OAuth 2.0

OAuth 2.0 est un **cadre d'autorisation** qui permet à une application tierce d'accéder à des ressources sans connaître le mot de passe.

Exemple : une application mobile Akaranga Bank pourrait permettre à un client de se connecter via Google ou Facebook (OAuth 2.0 avec Google/Facebook comme fournisseur d'identité).

Flux simplifié :

1. L'application redirige vers le fournisseur d'identité.
2. L'utilisateur s'authentifie.
3. Le fournisseur retourne un **code d'autorisation**.
4. L'application échange le code contre un **access token**.

D. Rate limiting

Rate limiting = limiter le nombre de requêtes par client sur une période donnée.

Pourquoi ?

- Protéger contre les attaques par force brute.
- Éviter la saturation du serveur.
- Réduire les coûts d'infrastructure.

Exemple :

- 60 requêtes par minute pour un utilisateur normal.
- 5 tentatives de connexion par 5 minutes.

E. Validation des entrées

Ne jamais faire confiance aux données reçues.

Règles :

- Valider les types (int, decimal, string).
- Valider les longueurs (max 100 caractères).
- Valider les formats (email, téléphone).
- Valider les plages (montant > 0).
- Rejeter les caractères dangereux (SQL, HTML).

F. Journalisation

Chaque appel API doit être **tracé** :

- Date et heure
- Utilisateur
- Endpoint appelé
- Paramètres
- Résultat (succès, erreur)
- Adresse IP

Utilité : détection d'anomalies, audit, débogage.

Partie C — Cas d'usage pour Akaranga Bank

10.4. API REST pour l'application mobile

Endpoints principaux :

Méthode	Endpoint	Description
POST	`/api/v1/auth/login`	Connexion
POST	`/api/v1/auth/refresh`	Rafraîchir le token
POST	`/api/v1/auth/logout`	Déconnexion
GET	`/api/v1/comptes`	Liste des comptes
GET	`/api/v1/comptes/{id}/solde`	Solde d'un compte
GET	`/api/v1/comptes/{id}/releve`	Relevé d'un compte
POST	`/api/v1/virements`	Initier un virement
GET	`/api/v1/virements`	Historique des virements
GET	`/api/v1/prets`	Liste des prêts
GET	`/api/v1/cartes`	Liste des cartes
POST	`/api/v1/cartes/{id}/opposition`	Faire opposition

10.5. API SOAP pour les partenaires

Cas d'usage : une autre banque veut échanger des virements avec Akaranga Bank.

Service SOAP :

```xml
<soap:Envelope>
  <soap:Body>
    <InitierVirementInterbancaire>
      <BanqueSource>AKR</BanqueSource>
      <BanqueDestination>BANQUE_X</BanqueDestination>
      <CompteSource>BJM-0001-0001</CompteSource>
      <CompteDestination>XXXX-XXXX</CompteDestination>
      <Montant>1000000</Montant>
      <Devise>BIF</Devise>
      <Reference>AKR-20261015-000001</Reference>
    </InitierVirementInterbancaire>
  </soap:Body>
</soap:Envelope>
```

Réponse :

```xml
<soap:Envelope>
  <soap:Body>
    <InitierVirementInterbancaireResponse>
      <Statut>Accepte</Statut>
      <ReferenceExterne>BANQUE_X-20261015-000042</ReferenceX>
      <DateAcceptation>2026-10-15T14:30:00Z</DateAcc>
    </InitierVirementInterbancaireResponse>
  </soap:Body>
</soap:Envelope>
```

Pourquoi SOAP ici ?

- Les échanges interbancaires nécessitent un **contrat strict**.
- Le format XML est **universel**.
- WS-Security garantit la **non-répudiation**.

CHAPITRE 11 — TESTS AUTOMATISÉS, PISTE D'AUDIT ET DÉPLOIEMENT

Objectifs du chapitre

- Écrire des **tests unitaires** avec xUnit et Moq.
- Écrire des **tests d'intégration** avec `WebApplicationFactory`.
- Tester la **logique comptable** (équilibre débit/crédit, fractionnement).
- Mettre en place une **piste d'audit** complète et immuable.
- Utiliser un `SaveChangesInterceptor` pour tracer automatiquement les modifications.
- Consulter l'audit via une **interface dédiée** (auditeur).
- **Déployer** l'application (IIS, Docker, Azure).
- Configurer un **pipeline CI/CD** (GitHub Actions).
- Mettre en place des **health checks**.
- Configurer l'application pour la **production**.

Partie A — Pourquoi tester ?

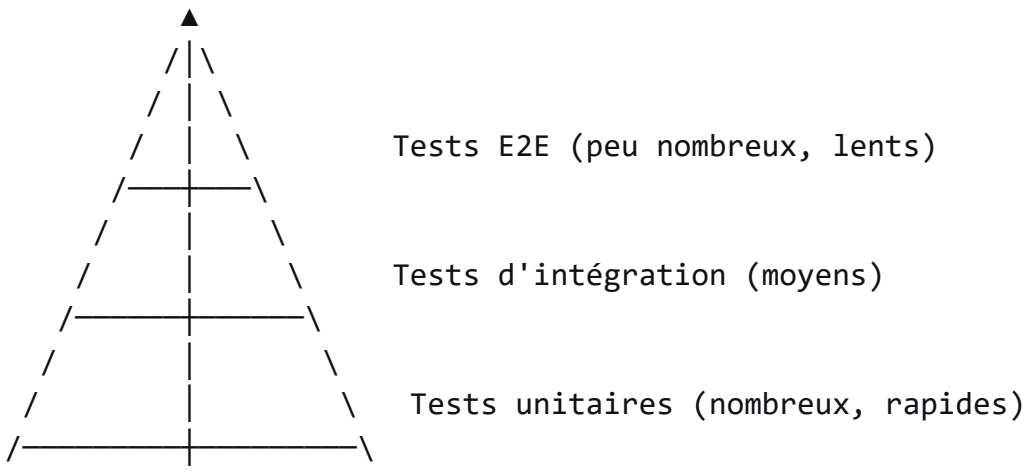
11.1. Le contexte

Akaranga Bank manipule de l'**argent réel**. Une erreur de calcul peut :

- Créer de l'argent à partir de rien.
- Faire disparaître des fonds.
- Fausser la comptabilité.
- Entraîner des sanctions réglementaires (BRB, CNRF).

Les tests automatisés sont la seule défense fiable contre ces risques.

11.1.1. Pyramide des tests



11.1.2. Ce qu'on teste en priorité

Priorité	Élément	Pourquoi
Critique	`ComptabiliteService.VirerAsync`	Manipule de l'argent
Critique	`ConformiteService.AnalyserTransactionAsync`	Conformité LBC/FT
Critique	`VerifierEquilibre`	Intégrité comptable
Important	`AuthService` (à venir, chapitre API)	Sécurité
Important	`CompensationService`	Virements inter-agences
Utile	Contrôleurs MVC	Comportement HTTP

Partie B — Tests unitaires avec xUnit et Moq

11.2. Créer le projet de tests

bash

```
cd E:\COURS1\CSHARP\Projets\AkarangaBank
dotnet new xunit -o AkarangaBank.Tests
cd AkarangaBank.Tests
dotnet add reference ../AkarangaBank.Web/AkarangaBank.Web.csproj
```

Installez les packages :

bash

```
dotnet add package Moq
dotnet add package FluentAssertions
dotnet add package Microsoft.EntityFrameworkCore.InMemory
dotnet add package Microsoft.AspNetCore.Mvc.Testing
dotnet add package coverlet.collector
```

Package	Rôle
`xunit`	Framework de tests
`Moq`	Simulation des dépendances
`FluentAssertions`	Assertions lisibles
`EF Core InMemory`	Base de données en mémoire
`Mvc.Testing`	Tests d'intégration

`coverlet.collector`	Couverture de code

11.3. Structure du projet de tests

```
AkarangaBank.Tests/
├── UnitTests/
│   ├── Services/
│   │   ├── ComptabiliteServiceTests.cs
│   │   ├── ConformiteServiceTests.cs
│   │   ├── CompensationServiceTests.cs
│   │   └── DetectionAvanceeServiceTests.cs
│   └── Helpers/
│       └── TestDbContextFactory.cs
├── IntegrationTests/
│   └── CustomWebApplicationFactory.cs
└── AkarangaBank.Tests.csproj
```

11.4. Helper : base de données en mémoire

Fichier à créer : `UnitTests/Helpers/TestDbContextFactory.cs`

```
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using Microsoft.EntityFrameworkCore;

namespace AkarangaBank.Tests.UnitTests.Helpers;

public static class TestDbContextFactory
{
    public static AkarangaBankContext CreateInMemoryContext(string d
bName = "")
    {
        var options = new DbContextOptionsBuilder<AkarangaBankContex
t>()
            .UseInMemoryDatabase(databaseName: string.IsNullOrEmpty(
dbName)
                ? Guid.NewGuid().ToString()
                : dbName)
            .Options;

        var context = new AkarangaBankContext(options);
        context.Database.EnsureCreated();
        return context;
    }

    // Créer une agence de test
```

```
    public static async Task<Agence> CreerAgenceAsync(
        AkarangaBankContext context,
        string code = "AKR-BJM",
        bool estSiege = true)
    {
        var agence = new Agence
        {
            Code = code,
            Nom = $"Akaranga Bank {code}",
            Ville = "Bujumbura",
            EstSiege = estSiege
        };
        context.Agences.Add(agence);
        await context.SaveChangesAsync();
        return agence;
    }

    // Créer un client de test
    public static async Task<Client> CreerClientAsync(
        AkarangaBankContext context,
        int agenceId,
        string numeroClient = "CLI-000001")
    {
        var client = new Client
        {
            NumeroClient = numeroClient,
            Nom = "Nshimirimana",
            Prenom = "Emmanuel",
            DateNaissance = new DateTime(1985, 3, 12),
            NumeroCNI = $"BJM-{Guid.NewGuid():N}"[..12],
            Telephone = "+25779123456",
            Email = "test@akaranga.bi",
            AgenceId = agenceId,
            Statut = "Actif"
        };
        context.Clients.Add(client);
        await context.SaveChangesAsync();
        return client;
    }

    // Créer un compte de test
    public static async Task<Compte> CreerCompteAsync(
        AkarangaBankContext context,
```

```
        int clientId,
        int agenceId,
        string numero,
        decimal soldeInitial = 0m,
        string devise = "BIF")
    {
        var typeCompte = await context.TypesCompte
            .FirstOrDefaultAsync(t => t.Code == "COURANT");

        if (typeCompte is null)
        {
            typeCompte = new TypeCompte
            {
                Code = "COURANT",
                Libelle = "Compte Courant",
                TauxInteret = 0
            };
            context.TypesCompte.Add(typeCompte);
            await context.SaveChangesAsync();
        }

        var compte = new Compte
        {
            NumeroCompte = numero,
            ClientId = clientId,
            TypeCompteId = typeCompte.Id,
            AgenceId = agenceId,
            Solde = soldeInitial,
            Devise = devise,
            Statut = "Actif",
            DateOuverture = DateTime.UtcNow
        };
        context.Comptes.Add(compte);
        await context.SaveChangesAsync();
        return compte;
    }
}
```

11.5. Test — ComptabiliteService (dépôt)

Fichier à créer : `UnitTests/Services/ComptabiliteServiceTests.cs`

```
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.Services;
using AkarangaBank.Web.ViewModels;
using AkarangaBank.Tests.UnitTests.Helpers;
using FluentAssertions;
using Moq;
using Xunit;

namespace AkarangaBank.Tests.UnitTests.Services;

public class ComptabiliteServiceTests
{
    private readonly AkarangaBankContext _context;
    private readonly Mock<IConformiteService> _conformiteMock;
    private readonly Mock<ISurveillanceService> _surveillanceMock;
    private readonly Mock<IDeclarationService> _declarationMock;
    private readonly ComptabiliteService _service;

    public ComptabiliteServiceTests()
    {
        _context = TestDbContextFactory.CreateInMemoryContext();

        _conformiteMock = new Mock<IConformiteService>();
        _surveillanceMock = new Mock<ISurveillanceService>();
        _declarationMock = new Mock<IDeclarationService>();

        // Par défaut : aucune alerte, aucun blocage
        _conformiteMock
            .Setup(c => c.AnalyserTransactionAsync(
                It.IsAny<int>(), It.IsAny<decimal>(), It.IsAny<strin
g>()))
            .ReturnsAsync(new ResultatConformite { EstBloquee = fals
e });

        _conformiteMock
            .Setup(c => c.DetecterFractionnementAsync(
                It.IsAny<int>(), It.IsAny<decimal>(), It.IsAny<DateT
ime>()))
            .ReturnsAsync(false);
```

```
        _surveillanceMock
            .Setup(s => s.AnalyserTransactionAsync(
                It.IsAny<int>(), It.IsAny<decimal>(),
                It.IsAny<string>(), It.IsAny<DateTime>()))
            .ReturnsAsync(new ResultatSurveillance());

        _service = new ComptabiliteService(
            _context,
            _conformiteMock.Object,
            _surveillanceMock.Object,
            _declarationMock.Object);
    }

    // ================================================================
    // DÉPÔT
    // ================================================================

    [Fact]
    public async Task DeposerAsync_CompteValide_CreeOperationEtCredi
t()
    {
        // Arrange
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 100_000m);

        var vm = new DepotViewModel { CompteId = compte.Id, Montant
= 50_000m };

        // Act
        var operation = await _service.DeposerAsync(vm, utilisateurI
d: 1);

        // Assert
        operation.Should().NotBeNull();
        operation.TypeOperation.Should().Be("Depot");
        operation.Montant.Should().Be(50_000m);
        operation.Statut.Should().Be("Valide");
```

```
        var compteMisAJour = await _context.Comptes.FindAsync(compte
.Id);
        compteMisAJour!.Solde.Should().Be(150_000m);

        var ecritures = _context.Ecritures
            .Where(e => e.OperationId == operation.Id)
            .ToList();

        ecritures.Should().HaveCount(1);
        ecritures[0].Sens.Should().Be("Credit");
        ecritures[0].Montant.Should().Be(50_000m);
    }

    [Fact]
    public async Task DeposerAsync_CompteInexistant_LeveException()
    {
        var vm = new DepotViewModel { CompteId = 999, Montant = 50_0
00m };

        var act = async () => await _service.DeposerAsync(vm, 1);

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*introuvable*");
    }

    [Fact]
    public async Task DeposerAsync_CompteBloque_LeveException()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001");

        compte.Statut = "Bloque";
        await _context.SaveChangesAsync();

        var vm = new DepotViewModel { CompteId = compte.Id, Montant
= 50_000m };

        var act = async () => await _service.DeposerAsync(vm, 1);
```

```
        await act.Should().ThrowAsync<InvalidOperationException>();
    }

    // ============================================================
    // RETRAIT
    // ============================================================

    [Fact]
    public async Task RetirerAsync_SoldeSuffisant_CreeOperationEtDeb
it()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 100_000m);

        var vm = new RetraitViewModel { CompteId = compte.Id, Montan
t = 30_000m };

        var operation = await _service.RetirerAsync(vm, 1);

        operation.TypeOperation.Should().Be("Retrait");

        var compteMisAJour = await _context.Comptes.FindAsync(compte
.Id);
        compteMisAJour!.Solde.Should().Be(70_000m);

        var ecriture = _context.Ecritures
            .First(e => e.OperationId == operation.Id);
        ecriture.Sens.Should().Be("Debit");
    }

    [Fact]
    public async Task RetirerAsync_SoldeInsuffisant_LeveException()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
```

```
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 10_000m);

        var vm = new RetraitViewModel { CompteId = compte.Id, Montan
t = 50_000m };

        var act = async () => await _service.RetirerAsync(vm, 1);

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*Solde insuffisant*");
    }

    // ═══════════════════════════════════════════════════════════
    // VIREMENT
    // ═══════════════════════════════════════════════════════════

    [Fact]
    public async Task VirerAsync_ComptesValides_CreeDeuxEcrituresEqu
ilibrees()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);

        var source = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 100_000m);
        var destination = await TestDbContextFactory.CreerCompteAsyn
c(
            _context, client.Id, agence.Id, "BJM-0001-0002", soldeIn
itial: 50_000m);

        var vm = new VirementViewModel
        {
            CompteSourceId = source.Id,
            CompteDestinationId = destination.Id,
            Montant = 30_000m,
            Motif = "Test"
        };

        var operation = await _service.VirerAsync(vm, 1);
```

```
        operation.TypeOperation.Should().Be("Virement");

        var sourceMisAJour = await _context.Comptes.FindAsync(source
.Id);
        var destinationMisAJour = await _context.Comptes.FindAsync(d
estination.Id);

        sourceMisAJour!.Solde.Should().Be(70_000m);
        destinationMisAJour!.Solde.Should().Be(80_000m);

        var ecritures = _context.Ecritures
            .Where(e => e.OperationId == operation.Id)
            .ToList();

        ecritures.Should().HaveCount(2);

        var totalDebit = ecritures.Where(e => e.Sens == "Debit").Sum
(e => e.Montant);
        var totalCredit = ecritures.Where(e => e.Sens == "Credit").S
um(e => e.Montant);

        totalDebit.Should().Be(totalCredit);
        totalDebit.Should().Be(30_000m);
    }

    [Fact]
    public async Task VirerAsync_MemeCompte_LeveException()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 100_000m);

        var vm = new VirementViewModel
        {
            CompteSourceId = compte.Id,
            CompteDestinationId = compte.Id,
            Montant = 10_000m
        };
```

```
        var act = async () => await _service.VirerAsync(vm, 1);

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*différents*");
    }

    [Fact]
    public async Task VirerAsync_SoldeInsuffisant_LeveException()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);

        var source = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 5_000m);
        var destination = await TestDbContextFactory.CreerCompteAsyn
c(
            _context, client.Id, agence.Id, "BJM-0001-0002");

        var vm = new ViremmentViewModel
        {
            CompteSourceId = source.Id,
            CompteDestinationId = destination.Id,
            Montant = 50_000m
        };

        var act = async () => await _service.VirerAsync(vm, 1);

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*Solde insuffisant*");
    }

    [Fact]
    public async Task VirerAsync_DevisesDifferentes_LeveException()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
```

```
        var source = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001",
            soldeInitial: 100_000m, devise: "BIF");
        var destination = await TestDbContextFactory.CreerCompteAsyn
c(
            _context, client.Id, agence.Id, "BJM-0001-0002", devise:
"USD");

        var vm = new VirementViewModel
        {
            CompteSourceId = source.Id,
            CompteDestinationId = destination.Id,
            Montant = 10_000m
        };

        var act = async () => await _service.VirerAsync(vm, 1);

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*devises*");
    }

    // ═══════════════════════════════════════════════════════════
    // ÉQUILIBRE
    // ═══════════════════════════════════════════════════════════

    [Fact]
    public async Task VerifierEquilibreGlobalAsync_ApresPlusieursOpe
rations_RetourneVrai()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);

        var source = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 1_000_000m);
        var destination = await TestDbContextFactory.CreerCompteAsyn
c(
            _context, client.Id, agence.Id, "BJM-0001-0002", soldeIn
itial: 500_000m);

        for (int i = 0; i < 3; i++)
```

```
        {
            await _service.VirerAsync(new VirementViewModel
            {
                CompteSourceId = source.Id,
                CompteDestinationId = destination.Id,
                Montant = 10_000m
            }, 1);
        }

        var equilibre = await _service.VerifierEquilibreGlobalAsync(
);
        equilibre.Should().BeTrue();
    }
}
```

11.6. Test — ConformiteService

Fichier à créer : `UnitTests/Services/ConformiteServiceTests.cs`

```
using AkarangaBank.Web.Data;
using AkarangaBank.Web.Models;
using AkarangaBank.Web.Services;
using AkarangaBank.Tests.UnitTests.Helpers;
using FluentAssertions;
using Xunit;

namespace AkarangaBank.Tests.UnitTests.Services;

public class ConformiteServiceTests
{
    private readonly AkarangaBankContext _context;
    private readonly ConformiteService _service;

    public ConformiteServiceTests()
    {
        _context = TestDbContextFactory.CreateInMemoryContext();
        _service = new ConformiteService(_context);
    }

    [Fact]
    public async Task AnalyserTransactionAsync_MontantSousSeuil_PasD
eVigilance()
    {
```

```
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 10_000_000m);

        var resultat = await _service.AnalyserTransactionAsync(
            compte.Id, 1_000_000m, "Depot");

        resultat.EstBloquee.Should().BeFalse();
        resultat.NecessiteVigilanceRenforcee.Should().BeFalse();
        resultat.NecessiteDeclaration.Should().BeFalse();
    }

    [Fact]
    public async Task AnalyserTransactionAsync_MontantAuDessusSeuilA
ML_VigilanceRenforcee()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 10_000_000m);

        var resultat = await _service.AnalyserTransactionAsync(
            compte.Id, 6_000_000m, "Depot");

        resultat.NecessiteVigilanceRenforcee.Should().BeTrue();
        resultat.Niveau.Should().Be(NiveauAlerte.Moyen);
    }

    [Fact]
    public async Task AnalyserTransactionAsync_DepotEspeces12M_Decla
rationObligatoire()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
```

```
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 10_000_000m);

        var resultat = await _service.AnalyserTransactionAsync(
            compte.Id, 12_000_000m, "Depot");

        resultat.NecessiteDeclaration.Should().BeTrue();
        resultat.Niveau.Should().Be(NiveauAlerte.Eleve);
    }

    [Fact]
    public async Task AnalyserTransactionAsync_RetraitSoldeInsuffisa
nt_Bloque()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);
        var compte = await TestDbContextFactory.CreerCompteAsync(
            _context, client.Id, agence.Id, "BJM-0001-0001", soldeIn
itial: 100_000m);

        var resultat = await _service.AnalyserTransactionAsync(
            compte.Id, 500_000m, "Retrait");

        resultat.EstBloquee.Should().BeTrue();
        resultat.Motif.Should().Contain("Solde insuffisant");
    }

    [Fact]
    public async Task EvaluerRisqueClientAsync_ClientAvecAlerte_Reto
urneEleve()
    {
        var agence = await TestDbContextFactory.CreerAgenceAsync(_co
ntext);
        var client = await TestDbContextFactory.CreerClientAsync(_co
ntext, agence.Id);

        // Créer une alerte pour ce client
        _context.AlertesAML.Add(new AlerteAML
        {
            ClientId = client.Id,
```

```
            TypeAlerte = "DepassementSeuil",
            Niveau = "Eleve",
            Statut = "Ouverte"
        });
        await _context.SaveChangesAsync();

        var risque = await _service.EvaluerRisqueClientAsync(client.
Id);

        risque.Should().Be(NiveauRisqueClient.Eleve);
    }
}
```

11.7. Lancer les tests

bash

```
cd AkarangaBank.Tests
dotnet test
```

Résultat attendu :

```
Passed!  - Failed: 0, Passed: 12, Skipped: 0, Total: 12
```

Partie C — Tests d'intégration

11.8. Configuration

Fichier à créer : `IntegrationTests/CustomWebApplicationFactory.cs`

```
using AkarangaBank.Web.Data;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace AkarangaBank.Tests.IntegrationTests;

public class CustomWebApplicationFactory : WebApplicationFactory<Pro
gram>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder
)
    {
        builder.ConfigureServices(services =>
        {
```

```
            // Supprimer le DbContext réel
            var descriptor = services.SingleOrDefault(
                d => d.ServiceType == typeof(DbContextOptions<Akaran
gaBankContext>));
            if (descriptor != null) services.Remove(descriptor);

            // Ajouter un DbContext en mémoire
            services.AddDbContext<AkarangaBankContext>(options =>
            {
                options.UseInMemoryDatabase("TestDb_" + Guid.NewGuid
());
            });

            // Créer la base
            var sp = services.BuildServiceProvider();
            using var scope = sp.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<Akaran
gaBankContext>();
            db.Database.EnsureCreated();
        });
    }
}
```

⚠ **Important :** pour que `WebApplicationFactory<Program>` fonctionne, ajoutez à la fin de `Program.cs` :

csharp

```
public partial class Program { }
```

11.9. Test d'intégration — Accès non authentifié

📄 **Fichier à créer :** `IntegrationTests/GlobalProtectionTests.cs`

```
using System.Net;
using FluentAssertions;
using Xunit;

namespace AkarangaBank.Tests.IntegrationTests;

public class GlobalProtectionTests : IClassFixture<CustomWebApplicat
ionFactory>
{
    private readonly HttpClient _client;
```

```
    public GlobalProtectionTests(CustomWebApplicationFactory factory
)
    {
        // Ne pas suivre les redirections (pour tester le 302)
        _client = factory.CreateClient(new()
        {
            AllowAutoRedirect = false
        });
    }

    [Theory]
    [InlineData("/Clients")]
    [InlineData("/Comptes")]
    [InlineData("/Prets")]
    [InlineData("/Cartes")]
    [InlineData("/Operations/Journal")]
    [InlineData("/Conformite/Alertes")]
    public async Task PageProtegee_SansAuthentification_RedirigeVers
Login(string url)
    {
        // Act
        var response = await _client.GetAsync(url);

        // Assert
        response.StatusCode.Should().Be(HttpStatusCode.Redirect);
        response.Headers.Location!.ToString()
            .Should().Contain("/Identity/Account/Login");
    }

    [Fact]
    public async Task PageAccueil_AccessibleSansAuthentification()
    {
        var response = await _client.GetAsync("/");
        response.StatusCode.Should().Be(HttpStatusCode.OK);
    }
}
```

11.10. Lancer les tests d'intégration

bash

```
dotnet test --filter "FullyQualifiedName~IntegrationTests"
```

Partie D — Piste d'audit

11.11. Pourquoi une piste d'audit ?

Exigence réglementaire : la loi burundaise impose la conservation des données pendant **5 à 10 ans**.

Exigence métier : en cas de litige, il faut pouvoir **prouver** qui a fait quoi, quand et comment.

Caractéristiques :

- **Immuable** : on ne modifie jamais une entrée d'audit.
- **Complète** : toute action sensible est enregistrée.
- **Consultable** : l'auditeur peut la parcourir.

11.12. Interception automatique des modifications

L'idée : **chaque fois que `SaveChangesAsync` est appelé, on enregistre les modifications** dans `AuditLog`.

Fichier à créer : `Data/Interceptors/AuditInterceptor.cs`

```
using System.Text.Json;
using AkarangaBank.Web.Models;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.EntityFrameworkCore.Diagnostics;

namespace AkarangaBank.Web.Data.Interceptors;

public class AuditInterceptor : SaveChangesInterceptor
{
    private readonly IHttpContextAccessor _accessor;

    public AuditInterceptor(IHttpContextAccessor accessor)
    {
        _accessor = accessor;
    }

    public override async ValueTask<InterceptionResult<int>> SavingC
hangesAsync(
        DbContextEventData eventData,
        InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        var context = eventData.Context;
        if (context is null) return result;
```

```
        var auditEntries = new List<AuditLog>();

        foreach (var entry in context.ChangeTracker.Entries())
        {
            if (entry.Entity is AuditLog) continue;

            if (entry.State == EntityState.Added ||
                entry.State == EntityState.Modified ||
                entry.State == EntityState.Deleted)
            {
                auditEntries.Add(CreateAuditEntry(entry));
            }
        }

        if (auditEntries.Any())
        {
            context.Set<AuditLog>().AddRange(auditEntries);
        }

        return await base.SavingChangesAsync(eventData, result, canc
ellationToken);
    }

    private AuditLog CreateAuditEntry(EntityEntry entry)
    {
        var httpContext = _accessor.HttpContext;
        var userId = GetUserId(httpContext);

        var entityName = entry.Entity.GetType().Name;
        var action = entry.State switch
        {
            EntityState.Added => "Creation",
            EntityState.Modified => "Modification",
            EntityState.Deleted => "Suppression",
            _ => "Inconnu"
        };

        return new AuditLog
        {
            DateEvenement = DateTime.UtcNow,
            UtilisateurId = userId,
            Action = $"{action}_{entityName}",
            TableConcernee = entityName,
```

```
            EnregistrementId = GetEntityId(entry),
            ValeurAvant = entry.State == EntityState.Modified
                ? GetOldValues(entry) : null,
            ValeurApres = entry.State != EntityState.Deleted
                ? GetNewValues(entry) : null,
            AdresseIP = httpContext?.Connection.RemoteIpAddress?.ToS
tring(),
            UserAgent = httpContext?.Request.Headers["User-Agent"].T
oString(),
            Details = $"Entité {entityName} — {action}"
        };
    }

    private int? GetUserId(HttpContext? httpContext)
    {
        if (httpContext?.User?.Identity?.IsAuthenticated != true) re
turn null;

        var idClaim = httpContext.User.FindFirst(
            System.Security.Claims.ClaimTypes.NameIdentifier)
            ?? httpContext.User.FindFirst("sub");

        return int.TryParse(idClaim?.Value, out var id) ? id : null;
    }

    private int? GetEntityId(EntityEntry entry)
    {
        var idProp = entry.Properties.FirstOrDefault(p => p.Metadata
.Name == "Id");
        return idProp?.CurrentValue as int?;
    }

    private static string GetOldValues(EntityEntry entry)
    {
        var dict = entry.Properties
            .Where(p => p.IsModified)
            .ToDictionary(p => p.Metadata.Name, p => p.OriginalValue
);
        return JsonSerializer.Serialize(dict);
    }

    private static string GetNewValues(EntityEntry entry)
    {
```

```
        var dict = entry.Properties
            .Where(p => p.IsModified || entry.State == EntityState.A
dded)
            .ToDictionary(p => p.Metadata.Name, p => p.CurrentValue)
;
        return JsonSerializer.Serialize(dict);
    }
}
```

11.13. Enregistrement de l'intercepteur

➖ **Fichier :** `Program.cs` **— modifier** `AddDbContext` **:**

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<AuditInterceptor>();

builder.Services.AddDbContext<AkarangaBankContext>((sp, options) =>
{
    var interceptor = sp.GetRequiredService<AuditInterceptor>();
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("AkarangaBank"))
        .AddInterceptors(interceptor);
});
```

11.14. Consultation de l'audit

📄 **Fichier à créer :** `Controllers/AuditController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AkarangaBank.Web.Data;

namespace AkarangaBank.Web.Controllers;

[Authorize(Policy = "AuditeurOnly")]
public class AuditController : Controller
{
    private readonly AkarangaBankContext _context;

    public AuditController(AkarangaBankContext context)
    {
        _context = context;
    }
```

```
    public async Task<IActionResult> Index(
        string? action, string? table, DateTime? du, DateTime? au, i
nt page = 1)
    {
        const int taille = 50;

        var query = _context.AuditLogs
            .Include(a => a.Utilisateur)
            .Include(a => a.Agence)
            .AsNoTracking()
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(action))
            query = query.Where(a => a.Action.Contains(action));

        if (!string.IsNullOrWhiteSpace(table))
            query = query.Where(a => a.TableConcernee == table);

        if (du.HasValue)
            query = query.Where(a => a.DateEvenement >= du.Value);

        if (au.HasValue)
            query = query.Where(a => a.DateEvenement <= au.Value);

        var total = await query.CountAsync();
        var logs = await query
            .OrderByDescending(a => a.DateEvenement)
            .Skip((page - 1) * taille)
            .Take(taille)
            .ToListAsync();

        ViewBag.Action = action;
        ViewBag.Table = table;
        ViewBag.Du = du;
        ViewBag.Au = au;
        ViewBag.Page = page;
        ViewBag.TotalPages = (int)Math.Ceiling((double)total / taill
e);

        return View(logs);
    }
```

```
    public async Task<IActionResult> Details(int id)
    {
        var log = await _context.AuditLogs
            .Include(a => a.Utilisateur)
            .Include(a => a.Agence)
            .FirstOrDefaultAsync(a => a.Id == id);

        if (log is null) return NotFound();
        return View(log);
    }
}
```

Fichier à créer : `Views/Audit/Index.cshtml`

```
@model IEnumerable<AkarangaBank.Web.Models.AuditLog>
@{ ViewData["Title"] = "Piste d'audit"; }

<h1>Piste d'audit</h1>

<form method="get" class="row g-2 mb-3">
    <div class="col-md-3">
        <input type="text" name="action" value="@ViewBag.Action"
               class="form-control" placeholder="Action..." />
    </div>
    <div class="col-md-3">
        <input type="text" name="table" value="@ViewBag.Table"
               class="form-control" placeholder="Table..." />
    </div>
    <div class="col-md-2">
        <input type="date" name="du" value="@ViewBag.Du?.ToString("y
yyy-MM-dd")"
               class="form-control" />
    </div>
    <div class="col-md-2">
        <input type="date" name="au" value="@ViewBag.Au?.ToString("y
yyy-MM-dd")"
               class="form-control" />
    </div>
    <div class="col-md-2">
        <button type="submit" class="btn btn-primary">Filtrer</butto
n>
    </div>
</form>
```

```
<table class="table table-striped table-sm">
    <thead>
        <tr>
            <th>Date</th>
            <th>Utilisateur</th>
            <th>Action</th>
            <th>Table</th>
            <th>IP</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
    @foreach (var log in Model)
    {
        <tr>
            <td>@log.DateEvenement.ToString("dd/MM/yyyy HH:mm:ss")</
td>
            <td>@(log.Utilisateur?.Email ?? "—")</td>
            <td>@log.Action</td>
            <td>@log.TableConcernee</td>
            <td>@log.AdresseIP</td>
            <td>
                <a asp-action="Details" asp-route-id="@log.Id"
                   class="btn btn-sm btn-primary">Détails</a>
            </td>
        </tr>
    }
    </tbody>
</table>
```

11.15. Test — Vérifier que l'audit fonctionne

1. Connectez-vous.
2. Effectuez une opération (dépôt, virement).
3. Allez sur `/Audit` (connecté en auditeur ou admin).
4. Vérifiez que l'opération apparaît dans la liste.

Partie E — Déploiement

11.16. Configuration pour la production

Fichier à créer : `appsettings.Production.json`

```
json
{
  "ConnectionStrings": {
    "AkarangaBank": "Server=prod-server;Database=AkarangaBank;User I
d=app_user;Password=***;TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Warning",
      "Microsoft.AspNetCore": "Error"
    }
  },
  "AllowedHosts": "akaranga.bi,www.akaranga.bi"
}
```

⚠ En production :

- **Ne jamais** committer les secrets dans le code.
- Utiliser des **variables d'environnement** ou un **coffre-fort** (Azure Key Vault).

11.17. Déploiement sur IIS

Étape 1 — Installer le bundle d'hébergement ASP.NET Core

Télécharger : https://dotnet.microsoft.com/download/dotnet/8.0

Étape 2 — Publier l'application

bash

```
dotnet publish AkarangaBank.Web/AkarangaBank.Web.csproj \
    -c Release \
    -o ./publish
```

Étape 3 — Créer un site IIS

1. Ouvrir IIS Manager.
2. Clic droit sur **Sites** → **Add Website**.
3. Nom : `AkarangaBank`.
4. Physical path : `C:\inetpub\AkarangaBank`.
5. Binding : `https`, port `443`, certificat SSL.

Étape 4 — Configurer les permissions

Le compte `IIS AppPool\AkarangaBank` doit avoir accès au dossier.

11.18. Déploiement avec Docker

Fichier à créer : `AkarangaBank.Web/Dockerfile`

```
dockerfile
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 80
EXPOSE 443

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["AkarangaBank.Web/AkarangaBank.Web.csproj", "AkarangaBank.Web/
"]
RUN dotnet restore "AkarangaBank.Web/AkarangaBank.Web.csproj"
COPY . .
WORKDIR "/src/AkarangaBank.Web"
RUN dotnet build "AkarangaBank.Web.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "AkarangaBank.Web.csproj" -c Release -o /app/publ
ish

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "AkarangaBank.Web.dll"]
```

Construire et lancer :

bash

```
docker build -t akaranga-bank:1.0 .
docker run -d -p 8080:80 \
  -e "ConnectionStrings__AkarangaBank=Server=db;Database=AkarangaBan
k;..." \
  --name akaranga-web \
  akaranga-bank:1.0
```

Fichier à créer : `docker-compose.yml`

yaml

```yaml
version: '3.8'

services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      SA_PASSWORD: "Akaranga@2025!"
      ACCEPT_EULA: "Y"
    ports:
      - "1433:1433"
    volumes:
      - mssql_data:/var/opt/mssql

  web:
    build: .
    depends_on:
      - db
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
      - ConnectionStrings__AkarangaBank=Server=db;Database=AkarangaB
ank;User Id=sa;Password=Akaranga@2025!;TrustServerCertificate=True;
    ports:
      - "8080:80"

volumes:
  mssql_data:
```

Lancer :

bash

```bash
docker-compose up -d
```

11.19. Déploiement sur Azure

Étape 1 — Créer une Web App

bash

```bash
az webapp create --resource-group AkarangaBankRG \
    --plan AkarangaBankPlan \
    --name akaranga-bank-api \
    --runtime "DOTNET|8.0"
```

Étape 2 — Configurer les variables

bash

```
az webapp config appsettings set \
    --resource-group AkarangaBankRG \
    --name akaranga-bank-api \
    --settings \
    "ConnectionStrings__AkarangaBank=Server=..." \
    "ASPNETCORE_ENVIRONMENT=Production"
```

Étape 3 — Déployer

bash

```
az webapp deploy --resource-group AkarangaBankRG \
    --name akaranga-bank-api \
    --src-path ./publish.zip
```

Partie F — CI/CD avec GitHub Actions

11.20. Configuration du pipeline

Fichier à créer : `.github/workflows/ci.yml`

yaml

```yaml
name: CI/CD AkarangaBank

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup .NET
      uses: actions/setup-dotnet@v4
      with:
```

```yaml
        dotnet-version: '8.0.x'

    - name: Restore
      run: dotnet restore

    - name: Build
      run: dotnet build --no-restore --configuration Release

    - name: Test
      run: dotnet test --no-build --configuration Release \
           --logger "trx;LogFileName=test-results.trx" \
           --collect:"XPlat Code Coverage"

    - name: Upload test results
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: test-results
        path: '**/test-results.trx'

  deploy:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest

    steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup .NET
      uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '8.0.x'

    - name: Publish
      run: dotnet publish AkarangaBank.Web/AkarangaBank.Web.csproj \
           -c Release -o ./publish

    - name: Deploy to Azure
      uses: azure/webapps-deploy@v3
      with:
        app-name: akaranga-bank-api
        publish-profile: ${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
```

```
        package: ./publish
```

Partie G — Health checks

11.21. Configuration

➖ Fichier : `Program.cs` — ajouter :

csharp

```csharp
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AkarangaBankContext>("database");

// ...

app.MapHealthChecks("/health");
app.MapHealthChecks("/health/ready");
```

Test :

bash

```bash
curl https://localhost:5001/health
```

Réponse : `Healthy`

Partie H — Tests manuels

Test 1 — Lancer tous les tests

bash

```bash
cd AkarangaBank.Tests
dotnet test
```

Résultat : tous les tests passent.

Test 2 — Couverture de code

bash

```bash
dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=opencover
```

Test 3 — Audit des modifications

1. Modifier un client.
2. Vérifier dans /Audit qu'une entrée `Modification_Client` apparaît.

Test 4 — Déploiement Docker

bash

```bash
docker-compose up -d
```

Accédez à `http://localhost:8080`.

Test 5 — Health check

```bash
curl http://localhost:8080/health
```

Test 6 — Pipeline CI

1. Poussez un commit sur GitHub.
2. Vérifiez que le pipeline s'exécute et passe.

Exercices

Exercice 1 — Test de la détection de smurfing
Écrire un test unitaire pour `DetecterSmurfingAsync` avec plusieurs comptes.

Exercice 2 — Test d'intégration du virement
Écrire un test qui :
1. Se connecte (mock).
2. Initie un virement.
3. Vérifie le `302` (redirection).

Exercice 3 — Test du contrôle d'équilibre
Créer une opération déséquilibrée et vérifier que `VerifierEquilibre` lève une exception.

Exercice 4 — Audit des accès non autorisés
Étendre l'intercepteur pour tracer les tentatives d'accès refusées.

Exercice 5 — Rapport d'audit mensuel
Générer un PDF mensuel avec les actions sensibles.

Questions de compréhension

1. Quelle est la différence entre un test unitaire et un test d'intégration ?
2. Pourquoi utiliser `Moq` plutôt que des vraies dépendances ?
3. Pourquoi une base de données en mémoire est-elle utile pour les tests ?
4. Qu'est-ce qu'une piste d'audit ? Pourquoi doit-elle être immuable ?
5. Comment un `SaveChangesInterceptor` fonctionne-t-il ?
6. Pourquoi ne jamais committer la clé JWT dans le code ?
7. Quelle est la différence entre IIS, Docker et Azure pour le déploiement ?
8. Qu'est-ce qu'un pipeline CI/CD ? Quel est son intérêt ?
9. Pourquoi les health checks sont-ils critiques en production ?
10. Comment garantir la traçabilité des actions sensibles en production ?

Mini-TP 11 — Tests, audit et déploiement

Objectif : rendre Akaranga Bank testable, auditable et déployable.

1. Créez le projet `AkarangaBank`.Tests.
2. Écrivez des tests unitaires pour `ComptabiliteService` et `ConformiteService`.
3. Écrivez des tests d'intégration pour la protection globale.
4. Implémentez `AuditInterceptor`.
5. Créez l'interface de consultation de l'audit (`/Audit`).
6. Configurez `appsettings.Production.json.`
7. Créez un `Dockerfile` et un docker-`compose.yml.`
8. Configurez GitHub Actions.
9. Ajoutez les health checks.
10. Déployez en local (Docker) et testez.

CHAPITRE 12 — PRÊTS, REMBOURSEMENTS ET CARTES BANCAIRES

Énoncé du travail à réaliser

1. Objectif du chapitre

Implémenter le **cycle de vie complet d'un prêt** (demande → validation → déblocage → remboursement) et la **gestion des cartes bancaires** (émission, plafonds, opposition, transactions), en respectant les principes comptables (partie double) et réglementaires (AML).

2. Contexte

Akaranga Bank dispose déjà de :

- Une base de données complète (chapitres 6-9).
- Un système d'authentification (chapitre 7).
- Une logique comptable (chapitre 8).
- Un dispositif AML (chapitres 8-9).

Il manque :

- La gestion des **prêts** (produit bancaire).
- La gestion des **cartes bancaires** (produit attaché au compte).

3. Livrables attendus

L'étudiant doit produire :

Livrable	Format
Entités de modèle	Fichiers `.cs` dans `Models/`
Configuration EF Core	Dans `AkarangaBankContext.cs`
Migrations	Dossier `Migrations/`
ViewModels	Fichiers `.cs` dans `ViewModels/`
Services	Interfaces + implémentations dans `Services/`
Contrôleurs	Fichiers `.cs` dans `Controllers/`
Vues	Fichiers `.cshtml` dans `Views/`
Tests unitaires	Projet `AkarangaBank.Tests`
Documentation	Rapport avec captures d'écran

4. Exigences fonctionnelles

4.1. Gestion des prêts

EF-12.1 — Le système doit permettre de créer une **demande de prêt** pour un client existant.

EF-12.2 — La demande doit contenir :

- Client (obligatoire)
- Compte de débit (obligatoire)
- Type de prêt (obligatoire)
- Montant (entre `MontantMin` et `MontantMax` du type)

- Durée en mois (entre `DureeMinMois` et `DureeMaxMois` du type)
- Commentaire (optionnel)

EF-12.3 — Le système doit calculer automatiquement :

- Le **taux d'intérêt** appliqué (moyenne du taux min et max du type)
- La **mensualité** selon la formule d'amortissement constant
- Le **statut initial** : `EnAttente`

EF-12.4 — Un administrateur ou gestionnaire doit pouvoir :

- **Valider** un prêt (avec un taux personnalisé)
- **Rejeter** un prêt (avec un motif obligatoire)

EF-12.5 — La validation d'un prêt doit permettre son **déblocage** :

- Création d'une opération `DeblocagePret`
- Crédit du compte client du montant principal
- Changement du statut à `EnCours`
- Génération automatique de l'**échéancier**

EF-12.6 — L'échéancier doit contenir une ligne par mensualité avec :

- Numéro d'échéance
- Part de capital
- Part d'intérêts
- Montant total
- Solde restant après paiement
- Date d'échéance

EF-12.7 — Le remboursement d'une échéance doit :

- Débiter le compte client du montant total
- Marquer l'échéance `Payee`
- Mettre à jour le prêt (montant remboursé, solde restant)
- Si toutes les échéances sont payées, passer le prêt à `Solde`

EF-12.8 — Le système doit détecter les **retards** :

- Une échéance non payée après sa date passe à `EnRetard`
- Une pénalité est calculée (0,5 % par mois de retard, minimum 500 BIF)
- Un prêt avec ≥ 3 échéances en retard passe à `Impaye`

4.2. Gestion des cartes bancaires

EF-12.9 — Le système doit permettre d'**émettre une carte** pour un compte actif.

EF-12.10 — La carte doit contenir :

- Numéro (16 chiffres, préfixe selon type : Visa = 4, MasterCard = 5)
- Numéro masqué (`**** **** **** 1234`)
- Type (`Visa`, `MasterCard`, `Locale`)
- Date d'expiration (3 ans)
- Plafonds (retrait, paiement, paiement en ligne)
- Statut (`Active`)

EF-12.11 — Le système doit permettre de **bloquer** une carte avec un motif :

- `Perte, Vol, Fraude, Autre`
- Le statut devient `Perdue, Volee, Opposition` ou `Bloquee`

EF-12.12 — Le système doit permettre de **renouveler** une carte :

- L'ancienne passe à `Expiree`
- Une nouvelle carte est créée avec les mêmes plafonds

EF-12.13 — Le système doit **autoriser ou refuser** une transaction carte :

- Vérifier le statut de la carte (`Active`)
- Vérifier la date d'expiration
- Vérifier le solde du compte
- Vérifier les **plafonds journaliers**
- Vérifier la **conformité AML**
- Si tout est OK : créer une opération + une écriture `Debit`
- Sinon : enregistrer une transaction `Refusee` avec motif

EF-12.14 — Le système doit permettre de consulter l'**historique des transactions** d'une carte.

5. Exigences non-fonctionnelles

ENF-12.1 — Toute opération financière (déblocage, remboursement, transaction carte) doit être **atomique** (transaction EF Core).

ENF-12.2 — Les montants doivent être stockés en `decimal(18,2)` (jamais `float` ou `double`).

ENF-12.3 — Toutes les relations doivent utiliser `HasOne(x => x.Navigation)` (pas de shadow properties).

ENF-12.4 — Les contrôleurs doivent être protégés par des politiques :

- `GuichetierOnly` pour la création de demande
- `AdminOnly` pour la validation, le déblocage, l'émission de carte

ENF-12.5 — Les calculs de mensualité doivent être **exacts** (arrondi au centime, ajustement sur la dernière échéance).

ENF-12.6 — Les transactions carte doivent respecter les plafonds (retrait, paiement, en ligne).

ENF-12.7 — Les tests unitaires doivent couvrir :

- Calcul de mensualité (au moins 5 cas)
- Génération d'échéancier (vérification de la somme = montant principal)
- Autorisation de transaction (solde, plafond, statut carte)
- Détection de retard

6. Tests à réaliser

6.1. Tests unitaires

TU-12.1 — `CalculerMensualite` avec un taux de 0 % :
P = 1 200 000, taux = 0, durée = 12
M attendu = 100 000

TU-12.2 — `CalculerMensualite` avec un taux de 12 % :
P = 1 000 000, taux = 12, durée = 12
M attendu ≈ 88 849

TU-12.3 — `GenererEcheancier` : la somme des capitaux doit être égale au montant principal.
TU-12.4 — `GenererEcheancier` : le solde restant après la dernière échéance doit être 0.
TU-12.5 — `DebloquerAsync` : le compte client doit être crédité du montant principal.
TU-12.6 — `RembourserAsync` : le compte client doit être débité du montant total + pénalité.
TU-12.7 — `MettreAJourRetardsAsync` : une échéance dépassée passe à `EnRetard`.
TU-12.8 — `AutoriserTransactionAsync` : une carte bloquée → transaction `Refusee`.
TU-12.9 — `AutoriserTransactionAsync` : un montant > plafond → transaction `Refusee`.
TU-12.10 — `AutoriserTransactionAsync` : un montant valide → opération créée + écriture `Debit`.

6.2. Tests d'intégration

TI-12.1 — Un utilisateur non authentifié ne peut pas accéder à /Prets.
TI-12.2 — Un utilisateur avec le rôle client ne peut pas valider un prêt.
TI-12.3 — Un admin peut créer, valider, débloquer et rembourser un prêt.

6.3. Tests manuels

TM-12.1 — Créer une demande de prêt de 500 000 BIF sur 12 mois.
TM-12.2 — Vérifier le calcul de la mensualité.
TM-12.3 — Valider le prêt avec un taux de 12 %.
TM-12.4 — Débloquer le prêt : vérifier le crédit du compte client et la génération de l'échéancier.
TM-12.5 — Rembourser la première échéance : vérifier le débit du compte.

TM-12.6 — Attendre la date d'échéance et lancer `MettreAJourRetardsAsync` : vérifier l'échéance `EnRetard` et la pénalité.
TM-12.7 — Émettre une carte Visa pour un compte actif.
TM-12.8 — Effectuer un achat de 50 000 BIF : vérifier le débit et l'enregistrement.
TM-12.9 — Tenter un achat > plafond : vérifier le refus.
TM-12.10 — Bloquer la carte avec motif « Perte ».

CHAPITRE 13 — ACCESSIBILITÉ ET INTERNATIONALISATION

Énoncé du travail à réaliser

1. Objectif du chapitre

Rendre l'application Akaranga Bank **accessible** à tous les utilisateurs (y compris en situation de handicap) et **internationalisable** (kirundi, français, anglais, swahili), conformément aux standards **WCAG 2.1 AA**.

2. Contexte

Akaranga Bank opère au Burundi, pays multilingue :

- **Kirundi** : langue nationale
- **Français** : langue officielle
- **Anglais** : langue des affaires
- **Swahili** : langue commerciale

De plus, la banque doit être accessible aux personnes en situation de handicap (visuel, moteur, cognitif).

3. Livrables attendus

Livrable	Format
Fichiers de ressources	`.resx` dans `Resources/`
Service de localisation	`Services/LocalizationService.cs`
Contrôleur de culture	`Controllers/CultureController.cs`
Sélecteur de langue	`_LanguageSelector.cshtml`
CSS accessible	`wwwroot/css/site.css`
Layout accessible	`_Layout.cshtml`
Rapport d'audit d'accessibilité	Document avec captures

4. Exigences fonctionnelles

4.1. Internationalisation

EF-13.1 — L'application doit supporter **4 cultures** :

- `fr-BI` (français, Burundi) — culture par défaut
- `rn-BI` (kirundi)
- `en-US` (anglais)
- `sw-KE` (swahili)

EF-13.2 — L'utilisateur doit pouvoir **changer de langue** via un sélecteur dans le menu.
EF-13.3 — Le choix de la langue doit être **persistant** (cookie, 1 an).
EF-13.4 — Les éléments suivants doivent être **traduits** :

- Titres des pages
- Libellés des menus
- Messages d'erreur
- Messages de succès
- Libellés des formulaires
- En-têtes de tableaux

EF-13.5 — Les **dates** doivent être affichées selon la culture :

- `fr-BI : 15/10/2026`
- `en-US : 10/15/2026`
- **EF-13.6** — Les **montants** doivent être affichés selon la culture :
- `fr-BI : 1 500 000,00 BIF`
- `en-US : 1,500,000.00 BIF`

4.2. Accessibilité

EF-13.7 — Toutes les images doivent avoir un attribut `alt`.
EF-13.8 — Tous les formulaires doivent avoir des **labels associés** (`<label for="...">`).
EF-13.9 — Les **messages d'erreur** doivent être annoncés aux lecteurs d'écran (`role="alert"`).
EF-13.10 — La navigation **au clavier** doit être complète :

- Tab : naviguer entre les éléments
- Entrée : activer
- Échap : fermer

EF-13.11 — Le **focus** doit être **visible** (contour jaune).
EF-13.12 — Un **skip link** doit permettre d'aller directement au contenu.
EF-13.13 — Les **contrastes** doivent respecter WCAG AA :

- Texte normal : ratio ≥ 4,5:1
- Texte large : ratio ≥ 3:1

EF-13.14 — Les **tableaux** doivent avoir :

- `scope="col"` sur les en-têtes
- `role="table"` et `aria-label`

EF-13.15 — Les **boutons** doivent avoir un `aria-label` si leur contenu est une icône.

5. Exigences non-fonctionnelles

ENF-13.1 — Les traductions doivent être **complètes** (pas de texte en dur).
ENF-13.2 — Le changement de langue doit être **immédiat** (rechargement de la page).
ENF-13.3 — Les tests d'accessibilité doivent être validés avec **axe-core** ou **Lighthouse**.
ENF-13.4 — Le score Lighthouse Accessibility doit être **≥ 90**.
ENF-13.5 — Le zoom à 200 % doit être supporté sans perte de contenu.

6. Contraintes techniques

CT-13.1 — Utiliser `IStringLocalizer<SharedResource>` pour les traductions.
CT-13.2 — Utiliser `RequestLocalizationOptions` pour la configuration.
CT-13.3 — Respecter les conventions ARIA 1.2.

CT-13.4 — Ne pas utiliser de tableaux HTML pour la mise en page.
CT-13.5 — Les couleurs doivent respecter la charte Akaranga Bank :

- Bleu marine : `#1F3864`
- Or : `#C9A227`

7. Tests à réaliser

7.1. Tests manuels

TM-13.1 — Changer la langue en kirundi : vérifier que le menu et les titres changent.
TM-13.2 — Naviguer uniquement au clavier : tous les éléments doivent être accessibles.
TM-13.3 — Vérifier le focus sur les boutons : contour jaune visible.
TM-13.4 — Utiliser un lecteur d'écran (NVDA) : le contenu doit être annoncé correctement.
TM-13.5 — Tester le contraste avec un outil (WebAIM Contrast Checker) : ratios respectés.
TM-13.6 — Lancer Lighthouse sur /Clients : score Accessibility ≥ 90.
TM-13.7 — Zoomer à 200 % : le contenu reste lisible.
TM-13.8 — Vérifier les dates en `fr-BI` et `en-US`.
TM-13.9 — Vérifier les montants `en fr-BI` et `en-US`.
TM-13.10 — Vérifier le skip link : il doit apparaître au focus.

7.2. Tests automatisés

TA-13.1 — Test d'intégration : `GET /?culture=rn-BI` doit retourner du kirundi.
TA-13.2 — Test d'intégration : le cookie de culture doit être défini après `POST /Culture/SetLanguage`.
TA-13.3 — Test d'accessibilité avec `AngleSharp` : toutes les images ont un alt.

CHAPITRE 14 — GESTION DOCUMENTAIRE ET NOTIFICATIONS

Énoncé du travail à réaliser

1. Objectif du chapitre

Mettre en place un **système de gestion documentaire** sécurisé (upload, stockage, versioning, purge) et un **système de notifications** multicanal (email, SMS, push, interne).

2. Contexte

Akaranga Bank doit :

- Conserver les **pièces justificatives** (CNI, justificatif de domicile, contrats) pendant 5 à 10 ans (obligation LBC/FT).
- **Notifier** les clients et le personnel lors d'événements sensibles (virement reçu, échéance de prêt, alerte AML).

3. Livrables attendus

Livrable	Format
Entités	`Document, DocumentAcces, Notification, NotificationTemplate`
Configuration EF Core	Dans `AkarangaBankContext.cs`
Service documentaire	`IDocumentService` + implémentation
Service de templates	`INotificationTemplateService` + implémentation
Service email	`IEmailService` + implémentation
Service SMS	`ISmsService` + implémentation
Service push	`IPushNotificationService` + implémentation
Service central	`INotificationService` + implémentation
Hub SignalR	`NotificationHub`
Tâches Hangfire	`NotificationBackgroundJob`
Contrôleurs	`DocumentsController, NotificationsController`
Vues	Upload, liste, détails

4. Exigences fonctionnelles

4.1. Gestion documentaire

EF-14.1 — Le système doit permettre d'**uploader** un document :
- Formats acceptés : PDF, JPG, PNG, DOCX, XLSX
- Taille max : 10 Mo
- Catégories : KYC, Contrat, Comptable, AML, RH, Audit

EF-14.2 — Le système doit calculer le **hash SHA-256** de chaque document.
EF-14.3 — Le système doit stocker le fichier sur disque avec un **nom unique** (GUID).
EF-14.4 — Le système doit permettre de **télécharger** un document et **tracer l'accès** dans `DocumentAcces`.
EF-14.5 — Le système doit permettre d'**archiver** un document (statut `Archive`).

EF-14.6 — Le système doit permettre de **supprimer** un document (suppression logique, statut Supprime).
EF-14.7 — Une tâche planifiée doit **purger** les documents supprimés depuis plus de N jours.

4.2. Notifications

EF-14.8 — Le système doit gérer des **templates** multilingues :

- Code (ex : VIREMENT_RECU)
- Type (Email, SMS, Push)
- Langue
- Sujet
- Contenu HTML et texte

EF-14.9 — Le système doit **rendre** un template avec des variables ({{NomClient}}, {{Montant}}, etc.).
EF-14.10 — Le système doit envoyer des **emails** (MailKit, SMTP).
EF-14.11 — Le système doit envoyer des **SMS** (Twilio).
EF-14.12 — Le système doit envoyer des **notifications push** (Firebase Cloud Messaging).
EF-14.13 — Le système doit envoyer des **notifications internes** temps réel (SignalR).
EF-14.14 — Le système doit **tracer** chaque notification dans la table Notifications.
EF-14.15 — Le système doit **réessayer** les notifications échouées (Hangfire, 3 tentatives max).
EF-14.16 — Une tâche planifiée doit envoyer un **rappel d'échéance** 3 jours avant la date.

4.3. Intégration avec les services existants

EF-14.17 — Un virement reçu doit générer une notification email au destinataire.
EF-14.18 — Une validation de prêt doit générer une notification email au client.
EF-14.19 — Une alerte AML critique doit générer une notification email aux responsables conformité.

5. Exigences non-fonctionnelles

ENF-14.1 — Les documents doivent être stockés **hors du dossier web** (sécurité).
ENF-14.2 — Les accès aux documents doivent être **tracés** (qui, quand, quelle action).
ENF-14.3 — Les emails doivent être envoyés de manière **asynchrone** (ne pas bloquer l'utilisateur).
ENF-14.4 — Les notifications échouées doivent être **relancées** automatiquement.
ENF-14.5 — Les templates doivent être **versionnables**.
ENF-14.6 — Les mots de passe SMTP/Twilio doivent être dans user-secrets (jamais dans le code).

6. Contraintes techniques

CT-14.1 — Utiliser IFormFile pour l'upload.
CT-14.2 — Utiliser MailKit pour les emails.
CT-14.3 — Utiliser Twilio pour les SMS.
CT-14.4 — Utiliser FirebaseAdmin pour les push.
CT-14.5 — Utiliser Hangfire pour les tâches planifiées.
CT-14.6 — Utiliser SignalR pour le temps réel.

7. Tests à réaliser

7.1. Tests unitaires

TU-14.1 — UploadAsync avec un fichier PDF valide → document créé.
TU-14.2 — UploadAsync avec un fichier trop volumineux → exception.
TU-14.3 — UploadAsync avec une extension interdite → exception.
TU-14.4 — RenderAsync remplace correctement les variables.
TU-14.5 — CreerEtEnvoyerAsync avec email → notification Envoye.
TU-14.6 — CreerEtEnvoyerAsync sans email → notification Echec.
TU-14.7 — ReessayerAsync avec 3 tentatives → notification Echec définitive.

7.2. Tests d'intégration

TI-14.1 — POST /Documents/Upload avec un fichier → 302 (redirection).
TI-14.2 — GET /Documents/Telecharger/1 → fichier retourné.
TI-14.3 — Un client ne peut pas accéder aux documents d'un autre client.

7.3. Tests manuels

TM-14.1 — Uploader une CNI (PDF).
TM-14.2 — Télécharger la CNI et vérifier le contenu.
TM-14.3 — Vérifier dans DocumentAcces que l'accès est tracé.
TM-14.4 — Effectuer un virement : vérifier que l'email est reçu.
TM-14.5 — Vérifier la notification dans la table Notifications.
TM-14.6 — Attendre l'échéance d'un prêt : vérifier le SMS de rappel.
TM-14.7 — Forcer l'échec d'un email : vérifier le réessai automatique.
TM-14.8 — Vérifier la notification SignalR en temps réel.
TM-14.9 — Vérifier le dashboard Hangfire (/hangfire).
TM-14.10 — Lancer la purge manuellement : vérifier la suppression.

CHAPITRE 15 — GÉNÉRATION DE DOCUMENTS ET REPORTING

Énoncé du travail à réaliser

1. Objectif du chapitre

Produire les **documents officiels** d'Akaranga Bank (relevés, attestations, contrats, déclarations CNRF) et les **rapports de gestion** (journal, balance, compte de résultat, bilan, rapport AML, tableau de bord).

2. Contexte

Akaranga Bank doit fournir :

- Aux clients : relevés de compte, attestations de solde, reçus.
- À la BRB : états comptables.
- À la CNRF : déclarations d'opérations suspectes.
- Aux dirigeants : tableaux de bord et rapports.

3. Livrables attendus

Livrable	Format
Service PDF	`IPdfService` + implémentation
Service Excel	`IExcelService` + implémentation
Service CSV	`ICsvService` + implémentation
Service reporting	`IReportService` + implémentation
Contrôleurs	`ReportingController, DocumentsPdfController`
Vues	Tableau de bord, journal, balance, rapports
Layout d'impression	`_LayoutPrint.cshtml`

4. Exigences fonctionnelles

4.1. Génération de documents PDF

EF-15.1 — Le système doit générer un **relevé de compte** PDF contenant :

- En-tête Akaranga Bank
- Informations du compte
- Tableau des écritures
- Solde initial et final
- Pied de page avec numérotation

EF-15.2 — Le système doit générer une **attestation de solde** PDF avec :

- En-tête officiel
- Montant en lettres
- Date
- Signature

EF-15.3 — Le système doit générer un **contrat de prêt** PDF avec :

- Parties (Banque, Emprunteur)
- Articles (objet, durée, taux, remboursement, garanties, défaut)
- Signatures

EF-15.4 — Le système doit générer une **déclaration CNRF** PDF avec :

- En-tête CNRF
- Identification du client
- Opération concernée
- Motif de la déclaration
- Mentions légales

EF-15.5 — Le système doit générer un **reçu d'opération** PDF (format A5).

4.2. Reporting

EF-15.6 — Le système doit produire un **journal comptable** :

- Filtres : date, type d'opération
- Pagination

EF-15.7 — Le système doit produire une **balance générale** :

- Par compte : total débits, total crédits, solde
- Vérification de l'équilibre

EF-15.8 — Le système doit produire un **compte de résultat** :

- Produits
- Charges
- Résultat net

EF-15.9 — Le système doit produire un **bilan** :

- Actif
- Passif
- Capitaux propres

EF-15.10 — Le système doit produire un **rapport AML mensuel** :

- Nombre d'alertes par niveau
- Nombre de DOS transmises
- Montant total des opérations suspectes
- Top 10 des clients à risque

EF-15.11 — Le système doit produire un **tableau de bord** avec :

- Nombre de clients
- Nombre de comptes
- Solde total
- Opérations du jour
- Prêts en cours
- Alertes AML

4.3. Export

EF-15.12 — Le système doit exporter en **Excel** :

- Journal comptable
- Balance
- Alertes AML
- Portefeuille de prêts

EF-15.13 — Le système doit exporter en **CSV** le journal.

EF-15.14 — Le système doit permettre l'**impression** des rapports (`window.print()`).

5. Exigences non-fonctionnelles

ENF-15.1 — Les PDF doivent respecter la charte Akaranga Bank (bleu marine, or).
ENF-15.2 — Les PDF doivent être **vectoriels** (pas d'images bitmap).
ENF-15.3 — Les exports Excel doivent être **formatés** (en-têtes en gras, colonnes ajustées).
ENF-15.4 — Les rapports doivent être **rapides** (< 5 secondes pour 1000 lignes).
ENF-15.5 — Les documents doivent être **traçables** dans AuditLog.
ENF-15.6 — Les vues d'impression doivent être **optimisées** pour A4.

6. Contraintes techniques

CT-15.1 — Utiliser **QuestPDF** pour les PDF.
CT-15.2 — Utiliser **ClosedXML** pour Excel.
CT-15.3 — Utiliser **CsvHelper** pour CSV.
CT-15.4 — Le tableau de bord doit utiliser **Bootstrap 5**.
CT-15.5 — Le layout d'impression doit masquer les éléments no-print.

7. Tests à réaliser

7.1. Tests unitaires

TU-15.1 — `GenererReleve` retourne un PDF non vide.
TU-15.2 — `GenererAttestationSolde` contient le montant.
TU-15.3 — `GenererContratPret` contient les clauses obligatoires.
TU-15.4 — `GetBalanceAsync` vérifie l'équilibre (débits = crédits).
TU-15.5 — `GetCompteResultatAsync` : Résultat = Produits - Charges.
TU-15.6 — `GetRapportAMLAsync` compte correctement les alertes.

7.2. Tests manuels

TM-15.1 — Générer un relevé de compte PDF : vérifier les écritures et le solde.
TM-15.2 — Générer une attestation de solde : vérifier le montant.
TM-15.3 — Générer un contrat de prêt : vérifier les articles.
TM-15.4 — Générer une DOS CNRF : vérifier les mentions légales.
TM-15.5 — Afficher le tableau de bord : vérifier les indicateurs.
TM-15.6 — Générer une balance : vérifier l'équilibre débit/crédit.
TM-15.7 — Générer un compte de résultat : vérifier le résultat net.
TM-15.8 — Générer un bilan : vérifier Actif = Passif + Capitaux propres.
TM-15.9 — Exporter le journal en Excel : ouvrir et vérifier.
TM-15.10 — Exporter le journal en CSV : vérifier le séparateur.
TM-15.11 — Imprimer le journal (Ctrl+P) : vérifier la mise en page.
TM-15.12 — Vérifier la traçabilité dans AuditLog.

CHAPITRE 16 — PROJET FINAL : SYNTHÈSE ET SOUTENANCE

Énoncé du projet intégrateur

1. Objectif du chapitre

Ce chapitre constitue le **projet final** du cours. Il vise à :

- **Intégrer** toutes les compétences acquises (chapitres 1 à 15).
- **Démontrer** la maîtrise technique et méthodologique.
- **Produire** une application Akaranga Bank complète, fonctionnelle et documentée.

Le projet final n'est **pas** un nouveau chapitre technique. C'est une **mise en situation professionnelle** : vous êtes développeur dans une équipe qui doit livrer un système bancaire.

2. Contexte du projet

Akaranga Bank est une banque commerciale fictive basée à Bujumbura. Après plusieurs mois de développement, la direction souhaite une **application complète** couvrant :

- L'authentification sécurisée du personnel.
- La gestion des clients (KYC).
- La gestion des comptes et des opérations.
- La conformité LBC/FT.
- Les produits bancaires (prêts, cartes).
- Le reporting réglementaire.
- Une API pour l'application mobile (conceptuelle).

Vous êtes le/la développeur(se) en charge de la livraison finale.

3. Périmètre du projet final

3.1. Ce qui doit être livré

Domaine	Fonctionnalités	Chapitres concernés
Environnement	Projet .NET 8+ compilable	1
Langage	C# moderne, decimal pour les montants	2
Razor	Pages d'affichage	3
MVC	Contrôleurs, vues, modèles	4-5
Base de données	SQL Server + EF Core	6
Sécurité	Identity, 2FA, rôles	7
Comptabilité	Partie double, dépôt/retrait/virement	8
Conformité	AML, seuils, CNRF	8-9
Virements inter-agences	Compensation	9
API	Documentation conceptuelle	10
Tests	Unitaires + intégration	11
Audit	Piste immuable	11
Déploiement	Docker ou Azure	11
Prêts	Cycle complet	12

Cartes	Émission, transactions	12
Accessibilité	WCAG AA	13
i18n	4 langues	13
Documents	Upload, stockage, purge	14
Notifications	Email, SMS, push	14
Reporting	PDF, Excel, CSV	15

3.2. Ce qui est optionnel (bonus)

- API REST implémentée (chapitre 10 avancé, vu l'année prochaine).
- Service SOAP (idem).
- Intelligence artificielle pour la détection de fraude.
- Application mobile native.

4. Livrables attendus

#	Livrable	Format	Obligatoire
1	Code source complet	Dépôt Git	✅
2	Base de données	Script SQL + migrations	✅
3	Projet de tests	`AkarangaBank.Tests`	✅
4	Fichier `.csproj`	Compilable	✅
5	`README.md`	Documentation technique	✅
6	`CHANGELOG.md`	Historique des versions	✅
7	`Dockerfile` + `docker-compose.yml`	Déploiement	⚠ Recommandé
8	Fichier `.gitignore`	Propre	✅
9	Pipeline CI/CD	GitHub Actions	⚠ Recommandé

5. Exigences fonctionnelles du projet final

5.1. Authentification et sécurité

EF-16.1 — Le système doit permettre la connexion via email + mot de passe.
EF-16.2 — Le système doit supporter la **2FA TOTP** (Google Authenticator).
EF-16.3 — Le système doit gérer **5 rôles** : admin, gestionnaire, guichetier, auditeur, client.
EF-16.4 — Le système doit interdire l'inscription publique.
EF-16.5 — Un administrateur doit pouvoir créer/désactiver des utilisateurs.
EF-16.6 — Les mots de passe doivent respecter une politique stricte (12 caractères, majuscule, minuscule, chiffre, caractère spécial).
EF-16.7 — Le système doit verrouiller un compte après 5 tentatives échouées.

5.2. Gestion des clients

EF-16.8 — Le système doit permettre de créer/modifier/supprimer des clients.
EF-16.9 — Chaque client doit avoir un **numéro unique** (`CLI-000001`).
EF-16.10 — Chaque client doit avoir un **numéro CNI unique**.

EF-16.11 — Le système doit gérer les **vérifications KYC**.

5.3. Gestion des comptes

EF-16.12 — Le système doit permettre d'ouvrir un compte pour un client.
EF-16.13 — Le système doit gérer **3 types de comptes** : COURANT, EPARGNE, CREDIT.
EF-16.14 — Le système doit afficher le **solde** d'un compte.
EF-16.15 — Le système doit générer un **relevé de compte**.

5.4. Opérations bancaires

EF-16.16 — Le système doit permettre un **dépôt** (crédit).
EF-16.17 — Le système doit permettre un **retrait** (débit, vérification du solde).
EF-16.18 — Le système doit permettre un **virement intra-agence** (2 écritures).
EF-16.19 — Le système doit permettre un **virement inter-agences** (4 écritures).
EF-16.20 — Le système doit vérifier l'**équilibre débit/crédit** pour chaque opération.

5.5. Conformité LBC/FT

EF-16.21 — Le système doit implémenter **3 seuils AML** : VIGILANCE (5M), DECLARATION (10M), BLOCAGE (50M).
EF-16.22 — Le système doit détecter le **fractionnement** (structuring).
EF-16.23 — Le système doit détecter le **smurfing**.
EF-16.24 — Le système doit détecter le **layering**.
EF-16.25 — Le système doit détecter la **vélocité anormale**.
EF-16.26 — Le système doit générer une **Déclaration d'Opération Suspecte (DOS)**.
EF-16.27 — Le système doit permettre de **traiter** une alerte AML (statuts).

5.6. Produits bancaires

EF-16.28 — Le système doit gérer les **prêts** (demande, validation, déblocage, remboursement).
EF-16.29 — Le système doit calculer la **mensualité** selon la formule d'amortissement.
EF-16.30 — Le système doit générer un **échéancier**.
EF-16.31 — Le système doit gérer les **cartes bancaires** (émission, plafonds, opposition).
EF-16.32 — Le système doit autoriser/refuser les **transactions carte**.

5.7. Gestion documentaire

EF-16.33 — Le système doit permettre d'**uploader** des documents (PDF, JPG, PNG, DOCX, XLSX).
EF-16.34 — Le système doit stocker les documents hors de wwwroot.
EF-16.35 — Le système doit calculer le **hash SHA-256** de chaque document.
EF-16.36 — Le système doit tracer les **accès** aux documents.

5.8. Notifications

EF-16.37 — Le système doit envoyer des **emails** (MailKit).
EF-16.38 — Le système doit envoyer des **SMS** (Twilio).
EF-16.39 — Le système doit envoyer des **notifications push** (Firebase).
EF-16.40 — Le système doit envoyer des **notifications internes** (SignalR).

5.9. Reporting

EF-16.41 — Le système doit générer des **relevés PDF**.
EF-16.42 — Le système doit générer des **attestations de solde PDF**.
EF-16.43 — Le système doit générer des **contrats de prêt PDF**.
EF-16.44 — Le système doit produire un **journal comptable**.
EF-16.45 — Le système doit produire une **balance générale**.
EF-16.46 — Le système doit produire un **compte de résultat**.
EF-16.47 — Le système doit produire un **bilan**.
EF-16.48 — Le système doit produire un **rapport AML mensuel**.
EF-16.49 — Le système doit afficher un **tableau de bord**.
EF-16.50 — Le système doit exporter en **Excel** et **CSV**.

5.10. Accessibilité et i18n

EF-16.51 — L'application doit supporter **4 langues** : fr-BI, rn-BI, en-US, sw-KE.
EF-16.52 — L'application doit être **accessible** (WCAG 2.1 AA).
EF-16.53 — Le score **Lighthouse Accessibility** doit être ≥ 90.
EF-16.54 — La navigation **au clavier** doit être complète.

6. Exigences non-fonctionnelles

6.1. Performance

ENF-16.1 — Les pages doivent se charger en **moins de 2 secondes** (hors connexion lente).
ENF-16.2 — Les requêtes EF Core doivent utiliser `AsNoTracking()` pour les lectures.
ENF-16.3 — Les listes doivent être **paginées** (max 50 lignes/page).
ENF-16.4 — Les index doivent être créés sur les colonnes fréquemment filtrées.

6.2. Sécurité

ENF-16.5 — Toutes les communications doivent être en **HTTPS**.
ENF-16.6 — Les données sensibles doivent être **chiffrées au repos**.
ENF-16.7 — Les mots de passe doivent être **hachés** (BCrypt, Argon2).
ENF-16.8 — Les API (futures) doivent utiliser **JWT**.
ENF-16.9 — Le **rate limiting** doit être configuré.
ENF-16.10 — Les secrets doivent être dans `user-secrets` ou un coffre-fort.

6.3. Maintenabilité

ENF-16.11 — Le code doit respecter les **conventions C#** (PascalCase, camelCase).
ENF-16.12 — Les méthodes doivent être **courtes** (< 50 lignes).
ENF-16.13 — Les classes doivent avoir une **responsabilité unique**.
ENF-16.14 — Les commentaires doivent expliquer le **pourquoi**, pas le **quoi**.
ENF-16.15 — Aucun code mort ne doit subsister.

6.4. Qualité

ENF-16.16 — Le projet doit compiler **sans erreur**.
ENF-16.17 — Aucun avertissement `shadow state` dans les logs.
ENF-16.18 — Aucune shadow property dans les migrations (`Id1`).

ENF-16.19 — Les tests unitaires doivent **tous passer**.
ENF-16.20 — La couverture de code doit être ≥ **60 %**.

6.5. Conformité réglementaire

ENF-16.21 — Le système doit respecter la **loi burundaise 1/02 de 2008** (LBC/FT).
ENF-16.22 — Le système doit conserver les données **5 à 10 ans**.
ENF-16.23 — Le système doit générer les **DOS** au format CNRF.
ENF-16.24 — Le système doit tracer toutes les actions sensibles dans AuditLog.

7. Tests à réaliser

7.1. Tests unitaires (obligatoires)

TU-16.1 — CalculerMensualite avec 5 cas différents.
TU-16.2 — GenererEcheancier : somme des capitaux = montant principal.
TU-16.3 — VirerAsync : 2 écritures équilibrées.
TU-16.4 — AnalyserTransactionAsync : dépôt > 5M → vigilance renforcée.
TU-16.5 — DetecterFractionnementAsync : 3 dépôts < seuil → vrai.
TU-16.6 — AutoriserTransactionAsync : carte bloquée → refus.
TU-16.7 — VerifierEquilibreGlobalAsync : après plusieurs virements → vrai.
TU-16.8 — EvaluerRisqueClientAsync : client avec alerte → élevé.
TU-16.9 — GenererReference : format unique AKR-YYYYMMDD-XXXXXX.
TU-16.10 — RenderAsync : template avec variables → rendu correct.

7.2. Tests d'intégration (obligatoires)

TI-16.1 — Sans authentification, /Clients → redirection vers Login.
TI-16.2 — Avec rôle client, /AdminUtilisateurs → 403.
TI-16.3 — Avec admin, /AdminUtilisateurs → 200.
TI-16.4 — Postuler à /Operations/Depot avec compte invalide → 400.
TI-16.5 — POST /Culture/SetLanguage → cookie défini.

7.3. Tests manuels (obligatoires)

TM-16.1 — Créer un admin, activer la 2FA, se reconnecter.
TM-16.2 — Créer un client, ouvrir un compte, effectuer un dépôt.
TM-16.3 — Effectuer un virement intra-agence et inter-agences.
TM-16.4 — Déclencher une alerte AML (dépôt > 5M).
TM-16.5 — Générer une DOS et la transmettre à la CNRF.
TM-16.6 — Créer un prêt, le valider, le débloquer, rembourser.
TM-16.7 — Émettre une carte, effectuer une transaction, bloquer.
TM-16.8 — Uploader une CNI, la télécharger, vérifier l'audit.
TM-16.9 — Effectuer un virement, vérifier l'email reçu.
TM-16.10 — Générer un relevé PDF, une balance, un rapport AML.
TM-16.11 — Changer la langue en kirundi, vérifier l'interface.
TM-16.12 — Lancer Lighthouse sur 5 pages → score ≥ 90.
TM-16.13 — Déployer avec Docker et tester.

TM-16.14 — Vérifier la piste d'audit (/Audit).
TM-16.15 — Vérifier le health check (/health).

8. Documentation attendue

8.1. README.md

Doit contenir :

- Nom du projet et description
- Prérequis (SDK, SQL Server)
- Instructions d'installation
- Instructions de lancement
- Structure du projet
- Technologies utilisées
- Licence
- Auteurs

9. Bonus possibles

9.1. Bonus techniques (+2 points)

- **API REST implémentée** : endpoints fonctionnels avec Swagger.
- **Service SOAP** : contrat WSDL + test avec SoapUI.
- **Intelligence artificielle** : détection de fraude par ML.NET
- **Déploiement cloud** : Azure App Service + SQL Azure.
- **Application mobile** : Xamarin ou .NET MAUI.
- **Blockchain** : registre distribué pour la comptabilité.

9.2. Bonus fonctionnels (+1 point)

- **Module de recouvrement** : relances automatiques des impayés.
- **Module d'épargne programmée** : objectifs d'épargne.
- **Module d'assurance** : produits d'assurance liés aux prêts.
- **Module de change** : conversion de devises.

9.3. Bonus qualité (+1 point)

- **Couverture de tests > 80 %**.
- **Score Lighthouse > 95**.
- **Documentation exceptionnelle** (vidéos, tutoriels).
- **Présentation remarquable** (animation, storytelling).

10. Conclusion du cours

Ce projet final marque la **fin du cours Développement Web avec ASP.NET Core — Akaranga Bank**.

En 16 chapitres, vous avez parcouru l'ensemble du cycle de vie d'une application bancaire :

- **Chapitres 1-2** : les fondations (environnement, langage).
- **Chapitres 3-6** : le web (Razor, MVC, EF Core).
- **Chapitres 7-9** : le cœur métier (Identity, opérations, conformité).
- **Chapitre 10** : l'ouverture (API, conceptuel).

- **Chapitres 11** : la qualité (tests, audit, déploiement).
- **Chapitres 12-15** : les produits (prêts, cartes, documents, reporting).
- **Chapitre 16** : la synthèse (projet final).

Vous êtes désormais **capable de** :

- Concevoir une application d'entreprise complète.
- Manipuler des données financières avec rigueur.
- Respecter les exigences réglementaires (LBC/FT).
- Tester, auditer et déployer une application.
- Travailler en autonomie sur un projet complexe.